

Build your AI brand

ONE

2026-05-15

Resell an AI brand to every client. They never leave.

Brand

The agency tier

One Agent. Every Surface. Same Brain.

Pick the Model. Swap It in One Line.

Agents

Skills

Tools

Memory: Every Conversation Makes the Next One Better

Teams: Marketing, Sales, and Service

One inbox. Every channel. Every conversation.

Analytics

The CRM Is Already Running

13 — Learning

Development

Security

Speed

Resell an AI brand to every client. They never leave.

ONE is the platform you buy, white-label, and sell to your clients as your own. From the day the contract signs, they have a marketing, sales, and service team under your brand and on your invoice. Every conversation those teams have stays with you. When a client moves on, the data doesn't go with them. That is the moat. It compounds.

This PDF is 17 pages. Every claim has a number.

The one-paragraph thesis

You have spent fifteen years building an agency whose ceiling is its payroll. Every new client is a new hire, a stretched team, or a missed deadline.

AI is changing both sides of that equation at once. Your clients can see it. Your competitors can smell it. Forrester is forecasting a 15% reduction in agency roles in 2026.

The window to be the agency that *owns* the AI layer, rather than the agency that buys it from a vendor, is not ten years wide. It is narrow.

ONE is that layer. You white-label it. You mark up the credits. You set the price. Your clients see your brand on the OAuth screen, your name on the invoice, your voice in every message the platform sends. They never learn ours.

The corpus, every conversation, every lead, every outcome, is yours. You keep it when a client leaves. You sell it when you exit. The multiple looks different when you own a corpus, not just a client list.

The four-line proof

Claim	Number	Where
Time to a live client chatbot	60 seconds	§16 Speed, p. 17
Time from sign to fully-tuned client deployment	48 hours	§02 Agency, p. 3
Monthly client report, generated and sent	90 seconds	§11 Analytics, p. 12
Revenue at scale, 5,400 SMB clients × \$1k/mo	\$5.4M ARR	§02 Agency, p. 3

Every number is operational, not projected. The platform has shipped the thing each number describes. §16 carries the full audit trail.

One decision

"In a world of infinite supply, distribution is the only moat."

Naval Ravikant

You already have the distribution. Twelve thousand contacts. Seven ICPs. Two hundred active clients who trust your agency's name enough to pay it every month.

The question is whether you own the platform those clients run on, or whether you rent it from a vendor who will eventually reach past you and sell direct.

Gartner's 2025 CMO Spend Survey found 22% of CMOs have already reduced their reliance on external agencies because of AI. That number will be higher next year. The agencies that survive are not the ones that resist the shift. They are the ones that resell it. You buy credits from us in bulk. You mark them up. You distribute them to your clients. The floor is \$0.0001 per credit. You set every price above it. The platform runs under your brand. Your clients never see us.

That answers margin. Now the lock-in question. The substrate is open source. The product layer sits in escrow. If we disappear, you walk away with every conversation, every learned path, every trained agent, in one export command. That answers vendor risk.

Two years from now, the agencies your peers are copying will be the ones who own a corpus. Be the one they copy.

See the platform you'll resell.

Table of contents

	Page	The sentence you'll underline
01	Brand	The platform disappears behind your brand. The OAuth screen, the invoice, and the 404 page all carry your name, and the 6-token design system makes every client's version look like it was built for them.
02	Agency	Buy 5,000,000 credits at \$500, mark them up, distribute to clients. The math works at five clients or five hundred, and the corpus is yours whether they stay or go.
03	Chatbots	One agent, every channel your clients' customers actually use. Web, WhatsApp, Telegram, Discord, iMessage, same brain, same memory, first token in under 300ms.
04	Models	Pick the model per agent, per skill, per cost target, from 300 of them. When prices drop next quarter, you keep the margin.
05	Agents	An agent is a markdown file you own outright. You edit it in minutes without a release cycle and redeploy in seconds. The IP is yours, not ours.
06	Skills	A skill is reusable capability you build once and deploy across every client. The platform tracks which skill earned which path the most revenue so you know where to build next.
07	Tools	Your agents work as the client, reading their Gmail, posting to their Slack, updating their HubSpot, through a white-label OAuth screen that carries your agency name.

	Page	The sentence you'll underline
08	Memory	Every conversation makes the next one better. The corpus accumulates as a sellable asset. The client's data is legally and physically yours, not ours.
09	Teams	On the day the contract signs, your client gets a full marketing department, a full sales department, and a full service department. 27 named roles. The equivalent of a £2M annual payroll, available the same hour.
10	Tracking	One inbox, every channel. The same customer who DM'd on WhatsApp this morning is the one who posted in Discord last week, and every payment, refund, and silence sits on the conversation that produced it.
11	Analytics	The monthly report ships in 90 seconds. Every number in it is one click from the conversation that earned it. The artefact that wins every renewal call.
12	CRM	The actors in every conversation are the CRM. No separate seat. No data migration. No sync. The record builds itself as the work happens.
13	Learning	Every quarter the agents get measurably better on each client's own data. The best paths harden into highways. The gap between you and an agency that moved later becomes unrecoverable.
14	Development	An agent is a markdown file. A skill is a CLI publish. Your engineers extend either without touching the substrate. Your technical partner gets full access. Your agency owns every artifact.
15	Security	The substrate is open source. The product layer is in escrow. The

	Page	The sentence you'll underline
		corpus is yours. GDPR deletion runs in one command. The threat model is a table, not a promise.
16	Speed	60 seconds to a live chatbot. 48 hours to a fully-tuned client. 90 seconds to the monthly report. 107 seconds from commit to production. Speed is a measurement here, not a claim.

Brand

Your agency's brand on a substrate you don't have to operate. One brain, many faces.

"A brand is no longer what we tell the consumer it is — it is what consumers tell each other it is."

— Scott Cook, founder of Intuit

That sentence should worry any agency owner still building on a platform they don't control. When your client's bot says something off-brand at 2am on a Saturday and the client posts it, the brand that suffers isn't ONE's. It's yours.

This page is about stopping that. Not through policy. Through mechanics. Six CSS tokens. One CNAME record. A voice contract that runs before every message goes out. The platform disappears behind your brand. The thing your clients see is you.

Create in 60 seconds

Sixty seconds from opening the brand editor to a working, branded, deployed client workspace. Not as a marketing claim with an asterisk. Sixty seconds with a click count.

The flow:

1. Open `yourslug.one.ie/settings/brand`, or your custom domain if the CNAME is already in. (1 click)
2. Pick six colours: background, foreground, font, primary, secondary, tertiary. Each shows a swatch. WCAG AA contrast updates live as you move the slider. Below 4.5:1, the input goes red. The save button refuses to save a failing combination. (6 interactions, around 20 seconds)
3. Drag your logo onto the upload target, or click to browse. SVG or PNG, up to 2MB. The favicon derives from the logo automatically, a 32x32 crop from the top-left quadrant. Override it separately if you want. (1 interaction, 5 seconds)
4. Type `yourdomain.com` or `acme.yourdomain.com` in the domain field and click Verify. The platform issues a TXT record value. Once DNS propagates (typically under 60 seconds on Cloudflare-managed domains, up to 5 minutes elsewhere), click Verify again. (2 interactions, variable wait)
5. Click Save. The workspace is live at your domain. Every client already under your agency picks up the brand change on next page load. (1 click)

Total: 11 interactions, 55 to 65 seconds for the editor, plus DNS wait. On Cloudflare DNS, the full flow including domain verification is typically under 3 minutes. On a slower registrar, add 5 minutes for DNS.

That is the single-agency case. The cascade means every change at the agency level flows down to your clients automatically. Rebrand your agency next year, change six tokens once, every client workspace updates. You don't touch 200 individual workspaces.

Worked example, dental practice, week 1:

Marcus runs a marketing agency in Manchester. Forty-seven dental practices on his book. He's adding a new client on a Tuesday afternoon. He opens the brand editor, pastes the practice's primary green (`hsl(142, 70%, 35%)`), uploads the logo, types `northdentalgroup.co.uk` as the domain. Fifty-five seconds later, the practice's marketing assistant opens `northdentalgroup.co.uk/chat` and sees a chat interface that looks like the practice's own website. Not Marcus's agency's site. Not ONE's platform. The contrast ratio on the primary green over white is 5.2:1, so WCAG AA passes.

Marcus didn't need a designer. Didn't need a developer. Didn't file a ticket. He had a cup of tea before the job was done.

The 6-token design system (and why only 6)

Six tokens. Not sixty. Not sixteen. Six, by choice.

The reason is the same reason a piano has 88 keys and a colour printer has four inks. You need enough range to express anything real, but not so many moving parts that the system defeats itself. A designer given 100 variables will find 100 things to change. A designer given 6 will find a coherent look fast and stay coherent.

Here are the six, as they appear in `web/src/layouts/Layout.astro` :

```
/* Six editable tokens – the only colours  
   an agency or client owner can change.  
   Declared in @theme; Tailwind v4 enforces them. */  
  
--color-background: hsl(0 0% 93%); /* card surfaces, sidebar, popovers */  
--color-foreground: hsl(0 0% 100%); /* inner content rectangles, inputs */  
--color-font:       hsl(0 0% 13%); /* all body text */  
--color-primary:    hsl(216 55% 25%); /* main CTAs, brand accents, focus rings */  
--color-secondary:  hsl(219 14% 28%); /* supporting actions, secondary buttons */  
--color-tertiary:    hsl(105 22% 25%); /* highlights, success checks, accents */
```

Those six values drive every visual surface the end user sees. Everything else is computed. `--color-border` is `font` at 10% opacity. `--color-muted` is `font` at 60%. `--color-page` is background mixed with 4% `font`. `--color-ring` is `primary`. None of those helpers are editable.

Why this matters when you resell to 50 clients:

When every value derives from six inputs, every workspace is internally consistent by construction. You cannot set a primary colour that produces an unreadable button label. The `on-primary` label colour is computed to be either black or white depending on the primary's luminosity. If `--color-`

`primary` is at L=65 or above, `on-primary` flips to black. Below L=60, it stays white. The threshold is enforced in code, not in a designer's judgement.

That is a liability shield. When a client's end user cannot read the primary call-to-action, the client blames the agency, not the platform. The 6-token system makes that call impossible to receive. The system refuses to produce it.

Plus 5 invariants, never editable:

On top of the six tokens, five values are fixed across every workspace.

Token	Hex / Value	Use
<code>white</code>	<code>#fff</code>	Absolute white where needed
<code>black</code>	<code>#000</code>	Absolute black where needed
<code>transparent</code>	<code>transparent</code>	Transparent fill
<code>destructive</code>	<code>hsl(0 70% 50%)</code>	Errors, deletions, danger states
<code>success</code>	<code>hsl(140 60% 40%)</code>	Confirms, completions

`destructive` and `success` carry semantic meaning that transcends branding. A client cannot change their destructive colour to a pleasant lilac. Error states have to look like errors. That consistency is part of what makes the platform safe to white-label. Even with a strong client brand, danger signals still read correctly to their end users.

Three depth levels:

The six tokens sit across three depths. Every surface in the product lives at exactly one.

Level	Surface	Where you see it
L0 page	<code>--color-page</code>	<code><body></code> , the full-bleed shell behind everything
L1 card	<code>--color-background</code>	Cards, sidebar, popovers, dropdowns
L2 content	<code>--color-foreground</code>	Card body, inputs, code blocks

Sidebar is L1. Inputs are L2. There is no L3. This three-level constraint means every workspace, however differently coloured, shares the same visual grammar. A dentist's workspace and a restaurant's workspace and a window installer's workspace all share the same spatial logic. End users learn the grammar once. It transfers across every client they touch.

WCAG AA, enforced not aspirational:

Contrast ratios are calculated at save time. The minimum the system accepts is 4.5:1 for body text (WCAG AA). Decorative elements can drop to 3:1. If a combination fails, the editor shows the failing ratio and highlights the offending token. The save button stays disabled until the combination passes.

This matters commercially. In the UK, the Equality Act 2010 requires digital services to be accessible. If an agency sets an unreadable combination and a client's end user with a visual impairment cannot use the interface, the liability chain runs back to the person who configured it. The platform refuses to let that happen.

What you cannot do with 6 tokens:

You cannot pixel-match a client whose brand sits on a 7th or 8th variable. If a brand guide calls for three distinct accent colours, brand blue for CTAs, brand green for success, brand orange for highlights, the system maps those to primary, success (invariant), and tertiary respectively. The invariant `success` token will not shift to brand orange. Safety signals stay fixed.

The answer is the locking grammar, not more tokens. Lock primary, lock secondary, leave tertiary unlocked, let the client's designer route their third colour into the available slot. No dental practice in fifteen years has filed a complaint that the success tick was green instead of orange.

Your domain (CNAME → verify → routed)

The domain flow is four steps. The fourth, routing requests to the correct TypeDB Group, is what makes the domain commercially useful, not just cosmetically satisfying.

```
CNAME record (your DNS)
↓
Domain verified in workspace settings
↓
Cloudflare Worker middleware matches incoming host header
↓
resolveConfig() maps host → workspaceSlug → TypeDB Group (<10ms)
```

In practice, `acme.com/sales/quote` resolves to your agency's Group in TypeDB in under 10ms, with every agent, skill, memory corpus, and billing pool correctly scoped to your client. The URL shows your domain. The routing decision is invisible. The end user is in your client's workspace.

Setting the CNAME:

In your DNS provider (Cloudflare, Route 53, Namecheap, any of them), you add one record.

```
Type: CNAME
Name: @ (for apex) or subdomain
Value: your-workspace.one.ie
TTL: Auto or 3600
```

For apex domains (`acme.com` without a subdomain), Cloudflare supports CNAME flattening, which handles the RFC prohibition on apex CNAMEs automatically. Other DNS providers may require an ALIAS or ANAME record at the apex. The platform accepts any of these. If your registrar only supports A records at the apex, the verification flow shows the current edge IP list.

After adding the record, verification issues you a `TXT` value.

```
Type:  TXT
Name:  _one-verify.acme.com
Value: one-verify=a8f4c2e1b9d7...
TTL:   3600
```

Add the TXT, wait for propagation, click Verify. The platform checks for the TXT from five Cloudflare edge locations at once. If any three of five see it, verification passes. This distributed check stops a false negative from a propagation race to a single verification server.

Once verified, the CNAME and TXT records both stay in place. The TXT is used for annual re-verification, a passive check that runs every 90 days to confirm the domain is still owned by the agency that registered it. Re-verification is silent unless it fails.

The middleware match:

The Cloudflare Worker reads the `Host` header on every incoming request. If the host is a registered custom domain, `resolveConfig()` is called with the mapped workspace slug. The entire config (tokens, sidebar, agents, billing pool) loads from R2 in a single read, with a KV edge cache in front. Cache TTL is 60 seconds. On a cache hit, the config is available before the first byte of the response is sent. On a miss, the R2 read adds roughly 15ms.

That cache matters at scale. An agency with 200 clients, each on a custom domain, generating 50 requests per minute each, is serving 10,000 requests per minute across 200 workspace configs. Without the KV cache, that would be 10,000 R2 reads per minute. With it, under 4 per minute, because the same config is served from edge cache for a full minute before re-fetching.

Worked example, window installer, month 2:

Sandra runs a B2B agency specialising in trades businesses. Her client is a window installer in Birmingham with a strong regional brand. The client insists the chat live at `chat.midlandwindows.co.uk`. Sandra adds the CNAME, adds the TXT, clicks Verify. Propagation takes 3 minutes. She clicks Verify again. The workspace goes live at the client's domain. The client's end customers visit `chat.midlandwindows.co.uk` and see a chat interface with the installer's brand, colours, and logo. They have no awareness that Sandra's agency is involved, let alone ONE. That is the product working correctly.

Embed anywhere (script tag, iframe, MCP, A2A)

The brand surface is not confined to a hosted URL. It travels. Four embedding modes, each fitting a different client context.

Script tag:

The lightest embed. One script tag in the client's existing website, and a chat widget appears in the bottom-right corner. The widget inherits the workspace tokens from the server at load time. It reads resolved CSS custom properties from the workspace config, not from a hardcoded stylesheet. Update the tokens, the next page load on the client's site reflects the change.

```
<!-- Add to the client's website <body> -->
<script
  src="https://acme.com/embed.js"
  data-workspace="northdentalgroup"
  data-position="bottom-right"
></script>
```

The script is served from your agency's domain (once the CNAME is done), not from `one.ie`. The client's customers who check the network tab see `acme.com/embed.js`. You are the vendor.

iframe:

Full-page embed for clients who want the complete chat UI inside their existing web app or patient portal. The iframe URL is the workspace's chat route on your domain. The iframe receives no third-party cookies. Authentication uses a short-lived signed URL that the client's server generates when it renders the page. Session data stays inside the iframe's browsing context.

```
<!-- Full-page chat in an iframe -->
<iframe
  src="https://northdentalgroup.co.uk/chat?token=signed-url-here"
  width="100%"
  height="600"
  frameborder="0"
></iframe>
```

MCP (Model Context Protocol):

For clients running their own AI tools (Claude Desktop, Cursor, internal developer tooling), the workspace exposes an MCP server at a stable URL. The MCP server carries the workspace's brand identity in its server metadata, and every tool call is scoped to the workspace's corpus. A client's developer connecting their IDE to the MCP endpoint is using a branded, scoped, agency-resold AI surface without leaving their editor.

A2A (Agent-to-Agent):

Other agents, from other platforms, from the client's own systems, can call into the workspace via a documented A2A endpoint. The endpoint enforces workspace scoping. An external agent calling in can only read and write data inside that workspace's Group boundary. Brand identity is carried in response metadata, not in the API responses themselves. The scoping means the workspace stays branded and contained.

What "your brand" covers

"White-label" is often shorthand for slapping a logo on a UI someone else built. That is not what this means. Here is the complete map of every surface where your brand appears.

Surface	Where your brand shows up
Chat UI	Logo in header, six tokens throughout, custom welcome message, starter chips
Invoice	Agency name, logo, payment terms, line items. ONE does not appear
OAuth consent screen	"Sign in to [Your Agency Name]". Your name, your logo, your terms URL
Transactional email	From address is your domain (<code>noreply@youragency.com</code>); sender name is yours; footer is yours
Status page	Hosted at <code>status.yourdomain.com</code> . Incidents attributed to your agency, not the platform
404 / error pages	Branded to the workspace. Workspace logo, primary colour, name in the heading
Favicon	Your uploaded favicon, served from your domain

The OAuth consent screen deserves a sentence. When a client's end user authenticates with Google, the consent screen says your agency's name. Your agency's logo. Your agency's privacy URL. The end user grants consent to your agency. From the user's point of view, your agency is the platform. From the user's point of view, this is a product you built.

That is not a cosmetic choice. It is the difference between building a business and reselling someone else's.

Day in the life, agency owner, 9:47am Thursday:

Brad's phone shows a Slack message from his account manager: "Meridian Physio wants to know if they can put the chat widget on their booking page." Brad is in a cab. He opens his agency's admin panel on his phone, navigates to the Meridian workspace, confirms the embed is already enabled (it is, he turned it on six weeks ago when he onboarded them). He forwards the embed snippet. By 10:15, the widget is live on the physio's booking page, showing the physio's green brand colour, the physio's logo, and a starter chip that says "Ready to book?" The physio's receptionist clicks it to test, replies "nice," and closes the tab. The account manager marks the task done. Brad didn't write a line of code. Didn't talk to a developer. Didn't file a ticket. His brand showed up exactly where it was needed, in 28 minutes from request to live.

The white-label cascade in practice

The cascade is four tiers. Owner (ONE) → Agency (Brad) → Client (the dental practice) → End user (the patient). Each tier inherits from the tier above and can override or lock individual properties going down. No tier can override what the tier above has locked.

ONE (owner)

- Sets: platform defaults, global feature flags
- Sets: credit pricing floor (\$0.0001/credit)
- Delivers to →

ACME Agency (Brad)

- Configures: ACME brand (logo, 6 tokens, domain acme.com)
- Can lock: primary colour, logo (clients cannot override these)
- Can enable/disable: features per client plan
- Delivers to →

North Dental Group (client)

- Configures: their sub-brand within ACME's locked properties
- Can set: their own accent colour if ACME unlocked it
- Sees: their agents, their billing, their chat conversations
- Delivers to →

Patient (end user)

- Sees: North Dental Group brand (or ACME brand where unlocked)
- Can set: dark/light mode preference
- Accesses: chat + whatever features their client enabled

In code, the cascade runs in `src/middleware.ts` via `resolveConfig()`, which merges three layers of `site.md` files from R2. The merge is deterministic. Layer 1 is the workspace (agency). Layer 2 is the group (client team if applicable). Layer 3 is the client sub-workspace. A locked property at Layer 1 survives the merge unchanged. An unlocked property can be overridden at Layer 2 or 3.

The locking grammar:

In the agency's `site.md` configuration:

```
# ACME Agency – site.md
name: ACME Marketing Platform
primary: hsl(142 70% 35%)
primary-locked: true      # clients cannot change brand green
logo-locked: true        # ACME logo appears on all sub-workspaces
features: [chat, agents, tools, settings]
features-locked: false    # clients can adjust their feature set
```

With `primary-locked: true`, every client workspace under ACME inherits the primary green. The client's colour picker for primary is greyed out. They can customise background, foreground, font, secondary, and tertiary. That is five of six. This lets Marcus hold brand consistency across 47 dental practices while still giving each practice room to express their own secondary palette.

Without locking, a client could override primary to magenta. That might fit some agency relationships, the ones where the client genuinely wants full control. It would be wrong for Marcus, whose proposition to practices is “we handle all of this for you.”

Worked example, restaurant group, month 3:

Priya runs a digital agency in Dublin with 12 restaurant clients. Three of them are outlets of the same group, different locations, same parent brand. She creates the parent brand at the group level, locks the primary (the restaurant's signature red), and creates three client workspaces, one per location. Each location can customise welcome message, starter chips, and secondary, but the brand red is locked. The three locations' chat interfaces look like siblings. Same character, different personality. Priya did not build three interfaces. She built one and cascaded it.

What you can lock (complete list):

Properties in the cascade support a `-locked` suffix for any of these:

- `name` (workspace display name)
- `tagline` (one-line descriptor)
- `logo`
- `favicon`
- `domain`
- Any of the 6 brand tokens (`background` , `foreground` , `font` , `primary` , `secondary` , `tertiary`)
- `features` (which features the tier below can access)
- `theme` (dark/light/system default)

A layer can only lock properties it has set. A client cannot lock a property the agency set. Locks cascade downward. They cannot be removed by a tier below the one that set them.

The commercial logic:

An agency that locks its logo and primary can sell a white-label service while holding a coherent agency brand across the whole book. This is the difference between being a platform operator and being a subcontractor. A subcontractor's brand disappears into the client's. A platform operator's brand sits above the client's, shaping it without overriding it.

Brad's clients know they're on a platform. They're paying for that. They're not paying for custom development. They're paying for a managed, branded, capable AI marketing surface that the agency maintains. The locking grammar makes that maintenance scalable. Brad updates the agency logo once, and 200 client workspaces reflect the change by the next page load.

Numbers worth putting in front of your CFO

These drive the commercial case for investing in brand configuration before your first client deployment.

- **6 design tokens.** The complete set of editable colours. No design skills required to produce a coherent, accessible workspace. A non-designer can do it in under 2 minutes with a brand guide in hand.
- **5 invariant tokens.** Reserved for safety semantics. Non-negotiable. This is what ensures `destructive` always reads as an error state regardless of client branding.

- **3 depth levels.** Page, card, content. Every surface in the product maps to exactly one. No exceptions. This constraint is what makes any colour combination look spatially coherent even when the colours themselves are unusual.
- **WCAG AA 4.5:1 minimum contrast ratio.** Enforced at save time. Cannot be bypassed. The UK Equality Act 2010 and the EU Web Accessibility Directive both require this minimum for digital services accessible to the public.
- **<10ms domain resolution.** The middleware maps `acme.com` to workspace slug to TypeDB Group in under 10ms on a KV cache hit. At 60-second TTL, the cache hit rate for any active workspace exceeds 99% during business hours.
- **1 CNAME record.** The complete DNS requirement for a custom apex domain on Cloudflare. Registrars without CNAME flattening need an ALIAS record instead. One record, either way.
- **60-second TTL** on workspace config edge cache. Short enough that a brand update propagates to all edge locations within one minute of saving. Long enough to keep R2 reads negligible.
- **Annual re-verification.** Domain ownership is checked passively every 90 days. Silent unless it fails. No agency action required unless the domain's DNS changes.
- **Zero additional engineering cost** to brand a new client workspace. The 11-interaction flow is the whole procedure. If you are paying a developer £80/hour to configure client brand settings, that cost is gone.

The pricing math, brand configuration cost per client:

Agency hourly billing rate £150. Developer hourly rate £80. Old process: design brief → designer creates stylesheet → developer implements → QA → deploy. Time: 4 hours minimum, typically 8.

Scenario	Hours	Cost
Old process (brief, design, dev, QA)	8 hours	£640
Fast old process (template + quick QA)	4 hours	£320
ONE brand editor (11 interactions)	0.016 hours (1 min)	£2.67

On 50 clients, the old process costs £16,000 to £32,000 in staff time to brand. The ONE process costs £133. The difference, £15,867 minimum, goes back to margin. Across a 200-client book at annual brand refresh cadence, that is £63,000+ per year of recovered cost, before counting the speed advantage at onboarding.

This is not a speculative number. It is arithmetic applied to the time the task takes.

Side-by-side: ONE brand system vs the alternatives

ONE vs hiring a designer:

Factor	Hiring a designer	ONE brand system
Setup cost per client	£320 to £640 (4 to 8 hours)	£2.67 (1 minute)
Time to live	1 to 5 days (review cycles)	60 seconds
WCAG AA compliance	Designer-discretion	Enforced by build
Brand consistency at scale	Manual QA required	Cascaded automatically
Brand update (logo change)	Re-engagement with designer	One edit, propagates instantly
Agency brand locked on clients	Manual process	–locked suffix in YAML
Dependence on individual	Yes (designer availability)	No

The designer is still useful for setting the initial six tokens with taste. That work takes 30 minutes, not 8 hours. The operational burden, applying those choices consistently across 200 clients, is what the system removes.

ONE vs building your own:

A custom white-label brand system for a SaaS platform needs, at minimum: a design token architecture, a theme editor component, a token cascade implementation, a domain verification system, a DNS check service, a contrast ratio validator, and a dark mode token override layer. Six to twelve months of engineering. £180,000 to £360,000 at market rates for a two-engineer build. Ongoing maintenance as browsers update CSS colour functions and accessibility standards evolve.

The ONE system is already built. The accessibility rules are already current. The token cascade already handles dark mode lightening (L0 background goes to 10%, L1 background to 93%). The contrast flip between `on-primary: #fff` in light and `on-primary: #000` in dark is already in `Layout.astro`.

Buy the engineering time back. Spend it on the client relationships that make the business defensible.

ONE vs GoHighLevel / Vendasta:

GoHighLevel offers a white-label wrapper around its own brand. You can put your logo on the GHL dashboard. The product stays recognisably GHL to any operator who has seen it before. The UI patterns, the navigation, the icon set. There is no token system. No domain cascade. The brand is cosmetic.

Vendasta offers rebrandable portals, but configuration lives in their dashboard, not in a cascade you control. Brand changes to individual client workspaces need individual manual reconfiguration.

The commercial consequence: in both cases, your clients are on someone else's platform, wearing your logo. In ONE, your clients are on your platform, wearing your brand at every layer. Chat UI, invoice, OAuth consent, email, favicon. The cascade means changes propagate top down in real time. The lock means your brand decisions are enforced mechanically, not maintained manually.

Objections answered

"What if my brand suffers because the bot says something weird?"

This is the anxiety that keeps agency owners from committing to AI-first delivery. The mechanical answer: a voice contract runs before every outbound message. The contract is a set of rules (tone, persona, prohibited phrases, required disclaimers) checked before the message is sent. If a message fails, it does not send. It routes to a human review queue. The quality gate for the contract is scored at 0.65 minimum. Below that, nothing ships.

This does not mean the bot never produces something unexpected. It means the unexpected output never reaches the client's customer. The voice contract is configurable per workspace. Marcus sets different tone rules for a dental practice than for a restaurant. The platform enforces whatever he sets. The brand concern is about what the end user sees. The end user only sees what passes the contract.

"Can I match my client's existing brand pixel-perfectly?"

The 6-token boundary is the honest answer. You can match the primary brand colour exactly. You can match the background, foreground, and font. You can match one supporting colour (secondary) and one accent (tertiary). That is six degrees of colour freedom, which covers the large majority of real brand guides.

Where the system cannot match a client exactly: brand guides that specify more than three distinct brand colours with different semantic roles, or guides that need to override the invariant tokens (destructive red, success green). Those are non-negotiable. The semantic meaning of danger and confirmation cannot be reassigned to a brand colour.

In practice, fewer than 5% of SMB local-service clients have a brand guide that exceeds six colour variables. Dental practices, restaurants, window installers, and similar businesses typically have a logo, a primary colour, and a secondary. The system handles them with room to spare.

"What if the AI screws up in front of a client and I lose them?"

The voice contract is the first guard. The quality gate (0.65) is the second. The human review queue is the third. The deeper answer is about what "screws up" means. The common failure modes for client-facing AI are wrong tone, factually incorrect claim, off-topic response, or inappropriate handling of a sensitive topic.

Wrong tone: the voice contract prohibits what you define as prohibited. If a dental practice's voice contract says "never use the word 'cheap,'" the system will not send a message containing it.

Factually incorrect claim: the agent's knowledge is bounded by its corpus, the information you have provided about the client. If the client's hours changed last week and the corpus hasn't been updated, the agent will give last week's hours. That is not an AI failure. That is a data management gap. The fix is a corpus update, which takes 30 seconds.

Off-topic response: the workspace's agent is configured with a scope definition. A dental practice agent is scoped to dental practice topics. Out-of-scope requests meet a defined fallback response, not a hallucinated answer.

Sensitive topics: the platform maintains a prohibited-topic list that applies before the voice contract. This list includes financial advice, medical diagnosis (distinct from providing practice information), and legal advice. These are not configurable. They are platform-level guardrails that cannot be overridden by agency or client settings.

"How do I explain to the client that it's AI, or do I have to?"

The short answer is that disclosure requirements depend on jurisdiction and context. In the EU, the AI Act requires disclosure when a natural person is interacting with an AI system unless the context makes that obvious. In the UK, the ICO's guidance on AI and data protection recommends transparency.

The platform's disclosure mechanism is a configurable notice in the chat interface header. By default, it reads "Powered by AI." You can customise the wording, or disable it entirely for jurisdictions where disclosure is not legally required. The platform logs disclosure state per session for audit.

This is a legal question your solicitor should weigh in on for each client. The platform makes it technically simple to comply with whatever disclosure posture you choose.

"Will ONE update its brand or change its UI and make my clients' workspaces look different overnight?"

No. The cascade starts at the agency tier, not at the platform tier. Platform-level UI changes do not cascade into agency workspaces unless the agency has not configured a custom domain or brand. Once you have set your agency's six tokens and custom domain, your workspace's visual surface is governed by those tokens. Platform UI changes affect the admin tools visible only to you, not the end-user-facing chat surfaces your clients' customers see.

Platform structural changes (new components, new surfaces) will inherit whatever tokens are in effect for the workspace when they render. A new page introduced to the chat surface will render with your agency's primary colour, not the platform default. The token system is structural, not skin-deep.

Failure modes, what this doesn't solve

It does not solve an off-brand personality.

The voice contract governs tone and prohibited phrases. It does not substitute for spending time on the agent's persona. If you deploy an agent for a premium legal firm with the same persona settings you used for a fast-casual restaurant, the voice contract will not catch that mismatch. The persona is your responsibility. This is agent authoring work, not brand configuration work. See page 05 (Agents) for the framework.

It does not make a badly photographed logo look good.

The platform scales and places your uploaded logo. It does not enhance it. If a client supplies a 72-dpi JPG of their logo from a 2009 invoice, that is what will appear in their chat header. Insisting on SVG or high-resolution PNG as part of your client onboarding checklist is the agency's job, not the platform's.

It does not prevent a client from changing things you haven't locked.

If you leave secondary and tertiary unlocked, a client with poor design judgement will change them to something that clashes with primary. The locking grammar prevents this for properties you choose to lock. If you choose not to lock, the client has control. The right default for most agency relationships is to lock primary and logo, and leave the rest open with sensible defaults.

Named route, the concrete path

`acme.com/sales/quote` routes to the ACME workspace's group in TypeDB in under 10ms.

The path is `src/middleware.ts` → `resolveConfig({ host: 'acme.com', path: '/sales/quote' })` → `workspaceSlug: 'acme'` → TypeDB Group query → agents scoped to acme → response rendered with acme tokens.

The middleware file lives at `/Users/toc/Server/one-ie/one/web/src/middleware.ts`. The `resolveConfig` function lives at `web/src/lib/site.ts`. The token cascade is implemented in `web/src/layouts/Layout.astro`. These three files are the complete implementation of the white-label brand cascade.

FAQ

Q: Can different clients have different custom domains under the same agency?

Yes. Each client workspace has its own domain field. ACME agency can have `acme.com` as its domain, with client `northdentalgroup.co.uk` under it. Domain verification is per-workspace, not per-agency.

Q: What happens if I update the agency logo while a client is mid-conversation?

The logo in the chat header loads on page load from the workspace config, which has a 60-second edge cache TTL. Sessions in progress will see the new logo on their next page load or the next time the cache refreshes, whichever comes first, within 60 seconds.

Q: Can I have multiple agencies on one ONE account?

Yes. The owner tier can create multiple agency workspaces, each with its own brand, domain, and client hierarchy. This is the holding-company or multi-brand model. Each agency workspace operates independently with its own cascade.

Q: Does the custom domain affect email deliverability?

Yes, in a good way. Transactional emails sent from the workspace use your domain as the sender, which means SPF, DKIM, and DMARC records need to be configured for that domain. The platform provides the DNS records to add during the domain verification flow. If you skip this step, emails will still send but from a platform-managed sender domain, which may land in spam for clients with strict filtering. Deliverability setup is a one-time configuration per domain.

Q: Can an end user see what platform powers the chat?

Only if you choose to show them. The embed script is served from your domain. The OAuth consent screen shows your name. The “powered by” notice is configurable. By default, the only platform reference in the UI is the `data-workspace` attribute in the embed script source, which end users rarely inspect. If you want zero platform references visible anywhere, that is achievable with a fully configured domain and disabled disclosure notice.

Q: What if a client’s brand guide needs a font we don’t support?

The token system covers colour, depth, and spacing. Typography is not currently in the 6-token set. The platform uses the system font stack by default (which renders as San Francisco on Mac, Segoe UI on Windows, Roboto on Android). Custom font support via Google Fonts or a self-hosted font file is on the roadmap. As of today, if a client has a brand-specific typeface, the chat will use the system font unless a CSS override is injected via the workspace’s custom CSS field, which accepts up to 1000 characters of arbitrary CSS. Most SMB brand guides do not specify a typeface at a level that would make this an issue. Enterprise clients with strict typography requirements are a different conversation.

Q: How does dark mode interact with the 6 tokens?

The platform ships two complete token sets, one for light, one for dark. In dark mode, the brand tokens lighten to L=65% to stay visible on dark backgrounds (`--color-background` drops to `hsl(0 0% 10%)` and the brand colours lighten accordingly). The `on-primary`, `on-secondary`, and `on-tertiary` labels flip from `#fff` to `#000` in dark mode. At L=65, a white label fails WCAG AA contrast, but a black label passes. This flip is automatic. Your clients cannot produce an inaccessible dark mode variant through any combination of token choices. The system does not permit it.

Q: Can I white-label the admin interface, the settings panel my client uses?

Yes. The same token cascade applies to the admin interface. A client logging into their workspace settings panel sees the same brand colours as their end users see in the chat. The only exception is the billing and credits section, which shows platform pricing information. That section cannot be rebranded.

Glossary

Token: A CSS custom property with semantic meaning. `--color-primary` is the main call-to-action colour. Tokens can be set and changed through the UI. Their computed values propagate automatically to every surface that references them.

Cascade: The top-down inheritance of configuration values from owner to agency to client to end user. Each tier can override or lock properties from the tier above. Nothing below a locked property can change it.

Invariant: A token that is fixed across all workspaces at all tiers. The five invariant tokens (`white`, `black`, `transparent`, `destructive`, `success`) carry semantic meaning that cannot be rebranded. `destructive` is always a danger red. `success` is always a confirmation green.

WCAG AA: Web Content Accessibility Guidelines Level AA, the international standard for web accessibility contrast ratios. Minimum 4.5:1 for normal text, 3:1 for large text. Required by UK Equality Act 2010 and EU Web Accessibility Directive.

Cross-references

Brand is the surface. The agents and skills behind it are covered in page 05 (Agents) and page 06 (Skills), where the voice contract, persona, and corpus scoping are explained in detail.

The billing layer that funds the brand cascade (credits purchased at the agency tier, distributed to client sub-pools) is covered in page 02 (Agency). The brand configuration is free to apply once you have purchased your credit pool. There is no per-workspace branding fee.

The security model that keeps each client's corpus isolated behind their branded workspace (the Group boundary in TypeDB, the per-client data space, the deletion command) is covered in page 15 (Security). Brand consistency is only commercially valuable if the data beneath it is safe. The two pages should be read together.

Build your brand in 60 seconds.

The agency tier

Your agency's ceiling is your payroll. That is the only problem worth solving on this page. Every section that follows is a different angle on one answer: what if the cost to serve a client did not scale with the number of clients?

The cascade is how. Four tiers, one credit pool, three verbs, and your clients never see us.

The 4-tier cascade

ONE is built on four roles. They are not feature flags. They are the lens the platform uses to decide what each person sees, what they can change, and what they owe.

owner → agency → client → end_user

The owner is the platform. The agency is you. The client is the dentist, the accountant, the window-and-door installer you have been selling to for fifteen years. The end user is their customer, the one who clicks the chat button on your client's website and never knows ONE exists.

Each tier inherits everything below it and adds more. An `owner` can see all workspaces. An `agency` can see their clients. A `client` can see their own workspace. An `end_user` sees what the client chose to show them.

The table that matters for your board:

Tier	Who	What they see	What they set
<code>owner</code>	ONE (Tony)	Platform-wide revenue, all workspaces, model catalogue	Rate per credit, platform margin, plan defaults
<code>agency</code>	You	Your workspace, your clients, your revenue	Your markup, your brand lock, your client plans
<code>client</code>	Brand X	Their agents, their billing, their users	Their monthly cap, their colour (if you unlocked it)
<code>end_user</code>	Brand X's customers	The chat, whatever the client enabled	Dark/light mode, nothing else

One column. One field each. The cascade fills in the rest.

This is not metaphor. It is the data model. Each tier resolves from a `billing.md` file stored per workspace in R2. The merge function runs in your Cloudflare Worker at request time. Your client never calls a database they do not own, never touches a table you did not give them access to, never sees a workspace that is not theirs.

White-label cascade

"White-label" means different things to different vendors. Here it has a precise, contractual definition.

On ONE, the cascade flows like this:

```
You (agency at acme.com)
  → sets: logo, brand tokens, domain, locked primary colour
  → delivers to →

Brand X (client at brandx.acme.com)
  → sees: your logo (if locked), your colour (if locked)
  → can set: their own accent (if you unlocked it), their chat welcome text
  → delivers to →

Brand X's customers (end users)
  → see: Brand X's brand
  → can set: dark/light mode
  → never see: the word "ONE", your agency name, or any other client
```

The technical guarantor is the `-locked` suffix in `site.md`:

```
# acme/site.md
name: ACME Platform
primary: hsl(142 70% 35%)
primary-locked: true      # clients cannot change brand green
logo-locked: true         # ACME logo appears on all sub-workspaces
features: [chat, agents, tools, settings]
features-locked: true     # clients cannot add skills or payments
```

Without `-locked`, the tier below inherits and may override. With `-locked`, the value is frozen. The merge function in `src/lib/site.ts` respects locks at resolve time. A client cannot write a property the agency locked. It is a tier violation; the middleware rejects it.

What the cascade guarantees, stated plainly:

- End users never see raw ONE branding on a white-label workspace.
- Clients never see other clients, agency internals, or the skills marketplace.
- The OAuth screen says your name. The email says your domain. The 404 page shows your logo.
- The invoice shows your brand, not ours.

The platform disappears behind your brand. That is the whole point.

What flips, what does not. Every display surface resolves through the cascade: logo, colour tokens, font, domain, welcome message, starter chips, agent persona, sidebar mode, favicon. What does not flip: the substrate underneath. TypeDB still learns. The pheromone trails still harden. The agents still get better. That part belongs to your client's workspace, not to the brand layer. The brand is the surface; the substrate is the foundation.

Buy credits, distribute to clients

One purchase. One pool. One number to watch.

You buy 5,000,000 credits for \$500. Those credits live in your agency pool. You set your markup, say 20%. You create Brand X as a client workspace and grant them a 50,000-credit starter allocation. Brand X's bot answers messages; each answer burns credits from their allocation, which reduces your pool.

The math is transparent and immutable:

```
1 credit = $0.0001      (owner-set; fixed at grant time; never retroactively changed)
Platform margin: 10%    (default; owner-set)
Agency floor: 5%       (you cannot squeeze markup below this; the platform guarantees its
Your markup: 20%        (you set this in acme/billing.md; one line)
```

Brad pays you. You keep your margin. The substrate cost is deducted from your pool. You never invoice ONE. ONE never invoices your client. The money flows through you, not around you.

The billing file you fill out is three lines:

```
# acme/billing.md
plan: agency
markup_pct: 20
brand_lock: true
client_default_plan: starter
```

That is the entire setup. Thirty seconds. Now you are reselling an AI platform at 20% gross margin on every credit burned by every client under your roof.

The client's file is one line:

```
# acme/clients/brandx/billing.md
plan: starter
monthly_cap: 50_000
```

Ten seconds. Brand X is live. Their pool is 50,000 credits. When they hit the cap, the gate kicks in.

One field per audience

The elegance of the system is that each audience sets exactly one field, and the cascade resolves the rest.

Audience	One field	What it controls
Owner (Tony)	<code>rate_usd_per_credit: 0.0001</code>	The unit economics for the whole platform
Agency (you)	<code>markup_pct: 20</code>	Your gross margin on every credit burned
Client (Brand X)	<code>monthly_cap: 50_000</code>	How much they can spend before the gate triggers
Team (Brand X's marketing dept)	<code>allocation: 30_000</code>	Their slice of the client's pool
End user	one button: [Top up \$5]	Or nothing, if sponsored by the client

No spreadsheets. No negotiated rate cards. No per-seat licence counting. One field; the cascade fills in everything else.

The cascade rules are two asymmetries and a floor:

children can only LOWER a cap (spend less than your allocation)
 children can only RAISE a markup (charge your own staff a bigger cut)
 platform floor is always respected (no one starves the substrate)

An agency cannot set `markup_pct` below 5%. A client cannot set a `monthly_cap` higher than the agency allocated. A team cannot allocate more credits than the client pool holds. The rules enforce themselves. There is no admin panel you have to open when a client tries to cheat the math.

Worked example: Acme Agency onboards Brand X

This is the unit economics, walked through. Not a projection, just arithmetic.

Day 0. Acme buys the agency plan: 5,000,000 credits for \$500. Markup set to 20%. Brand X created as a starter client with a 50,000-credit grant.

Week 1. Brand X's bot handles 200 customer conversations. Each conversation consumes on average 1,500 input credits via `claude-opus-4-7` (upstream cost 1,500 credits per 1,000 input tokens). Output is a 5x multiplier: 7,500 credits per 1,000 output tokens. A typical conversation: 300 input tokens plus 600 output tokens, which works out to 450 input credits plus 4,500 output credits, for 4,950 credits burned per conversation.

200 conversations at 4,950 credits each totals 990,000 credits in week one. With Brand X on a starter plan at the 50,000-credit cap, the gate triggers after the first ten conversations or so. The metered gate kicks in at the agency-set threshold.

Acme revises. Brand X is a busy client. Acme upgrades Brand X to a 200,000-credit monthly allocation. Brand X burns 990,000 credits that month.

The money. Walk it through.

- Substrate cost: $990,000 \text{ credits} \times \$0.0001 = \$99$.
- Platform fee (10% of substrate): **\$9.90** to ONE.
- Acme's pool is debited the full \$99 at wholesale.
- Acme bills Brand X at 20% markup: $990,000 \times \$0.00012 = \mathbf{\$118.80}$.
- Acme's gross margin on this client this month: $\$118.80 \text{ minus } \$99 = \mathbf{\$19.80}$.
- Margin as a percentage of revenue: **16.7%**.

That is one client, one month. Now scale.

30 clients, one year. Acme signs 30 clients at similar usage. Monthly gross margin per client around \$20. Across 30 clients: \$600 per month. Annual: \$7,200 off one \$500 credit purchase, refilled as needed. As clients grow their usage, the absolute margin grows proportionally. Heavier clients (more conversations, more output tokens) compound the absolute number; the percentage holds.

Now apply Brad's target structure. Brad's 5,400-client forecast at \$1k per month per client is \$5.4M ARR. At Acme's pricing (substrate cost recovered at 20% markup, 10% platform margin), direct credit cost lands at roughly 11% of revenue, depending on conversation mix. Gross margin: approximately 89%. That is the number Brad's spreadsheet needs to survive his board.

The comparison that lands: a junior copywriter costs Brad £80,000 per year and handles perhaps 20 clients. One credit pool handles 5,400 clients at roughly £88,000 per year in substrate cost. The same outcome, at 270x the scale, for 10% more than one salary.

Klarna cut agency marketing spend by 25% with generative AI in a single year (Klarna press release, May 2024). The agencies that come out ahead are the ones whose unit economics already assume that cut will arrive.

Gates: on, metered, off

Three verbs. No fourth state.

on	→ covered by plan, no per-use charge
metered	→ available, charged per use against the pool
off	→ unavailable; UI hides the feature or shows an upgrade prompt

Every feature in the system resolves to one of these three states at request time, for every workspace, based on the cascade. An agency can set any feature to any state for any client. A client cannot override a gate the agency locked.

The features agencies control most often:

Feature	Default state	What it gates
brand_removal	off (Free) / on (Agency plan)	Whether “Powered by ONE” shows in the footer
premium_models	off (Free) / on (Agency plan)	Access to claude-opus-4-7 and gpt-5
sub_workspace_create	off (all except Agency)	Whether clients can create their own sub-clients
white_label_cascade	off (all except Agency)	Whether the cascade applies to clients
voice_input	metered (most) / on (Agency plan)	Voice input in the chat surface
custom_domain	on (1 domain, Agency plan)	Whether client workspace gets its own domain

When a client hits a metered gate, ONE does not turn them away. The usage deducts from their pool at the metered rate. If the pool empties, the workspace enters the `over_limit` state: reads work, writes and inference are gated, a payment banner appears. Data is preserved. The client tops up; everything resumes. No data is lost.

When the agency stops paying, the pool drains. Clients enter `over_limit`. Their data is preserved for 90 days in the suspended state. Data is never deleted without a 90-day warning period and an export. If an agency cancels and abandons clients, those clients’ workspaces become root-level workspaces. They lose the parent’s brand locks but keep their corpus, their agents, their conversations.

What you keep

Margin. You set your markup. It applies to every credit burned by every client under your agency. The platform floor is 5%; your markup is whatever you decide above that. On day one, before you have negotiated volume, 20% is standard. As your pool grows, you can tighten to 10% for high-volume clients and widen to 30% for low-volume ones. The cascade handles per-client overrides in one line of the client’s billing file.

Data. Every conversation, every signal, every outcome lives in your client’s workspace. Not in ours. Not shared across agencies. Not pooled for training. The corpus is scoped to the workspace that generated it. When a client workspace accumulates 50,000 conversations with dental patients asking about implants, that corpus belongs to the client. And the agency provisioned it.

“If your competitive moat is a feature, you don’t have a moat. If it’s a relationship the customer can’t replicate elsewhere, you do. AI is making this distinction obvious for the first time.”

— Anthony O’Connell, Founder of ONE

The corpus is the relationship that cannot be replicated. A client who leaves takes their data with them via `oneie export --workspace brandx`. But they cannot take the trained paths, the hardened highways, the conversation patterns the substrate learned on their behalf. Those are substrate-internal, not exportable, and they decay if the signals stop. Three months after a client leaves, the learning that made their agents good is gone. That is the switching cost, built by physics, not by contract.

Contract. The agency relationship is between you and your client. ONE does not appear on the invoice your client receives. ONE does not set the price you charge. ONE does not own the brand your client sees. You own the contract. We supply the substrate.

What ships day one

Not a roadmap. What is live, built, and in production when you activate the agency plan.

4-tier viewer model with menu filtering and hover sidebar, at `src/lib/viewer.ts`, Step 1 complete. The cascade merge function for brand tokens and feature gates, at `src/lib/site.ts`. Locked properties in `parseSite()` so the `-locked` suffix is enforced at resolve time. Credit grants and credit burns tables in D1, namespaced `credit_grants` and `credit_burns`. The billing cascade with the two asymmetries and the platform floor, at `src/lib/billing.ts`. Stripe webhook integration for invoice events, idempotent via `stripe_events.id`. The gate evaluation in middleware: three verbs, resolved before the LLM is called. Auto-top-up via Cloudflare Worker cron.

Client workspace provisioning at `POST /api/provision?action=create-invite` sends the invite email, creates the `owners` row with `parent_slug`, marks the invite redeemed. The client lands at `/u/brandx/onboarding`, sets display name, picks a brand colour (or inherits yours if locked), and deploys their first agent.

From credit purchase to client live: 48 hours. From client live to first conversation: 60 seconds. From first conversation to monthly report showing their ROI: one billing cycle.

Objections answered

"What if a client wants to leave?"

Data is portable. Any workspace owner runs:

```
oneie export --workspace brandx --output ./brandx-export.zip
```

The archive contains conversations, agent definitions, skill configurations, and billing history. The command works whether the workspace is active, suspended, or in the process of leaving. The data belongs to the workspace owner; the CLI makes collection trivial. What a client cannot take: the substrate's learned paths, which are internal and decay without signals. That is not a lock-in mechanic. It is physics. The data export is unconditional.

"What if a client overuses?"

The metered gate kicks in at the agency-set threshold. Before the LLM is called, `debitPool()` checks the workspace balance. If the balance would go negative beyond the plan's negative-balance floor, the request returns HTTP 402 and the call is not made. The client sees a payment banner. Their existing conversations are readable. No half-streamed reply that needs a refund. No surprise invoice. The daily cap is mechanical, not negotiated.

"What happens if I (the agency) stop paying?"

Your pool drains. Clients enter `over_limit`, where reads work and inference is gated. After 30 days of `over_limit`, workspaces enter `suspended`: frozen, data preserved, no writes. After 90 days suspended, an R2 export is emailed to every workspace owner and the workspace is deleted. No data disappears without a 90-day warning and a delivered export. Your clients are never stranded without their own data.

The clients' `parent_slug` column stays as it was until an explicit cascade deletion. If you cancel without deleting, your clients' workspaces eventually become root-level. They lose your brand locks but keep their corpus. The worst case for your clients is that they need to find a new provider. The best case is that they upgrade their own agency plan and continue. Their data is never lost.

"What if I pick the wrong platform and they pivot, raise prices, or get acquired?"

The substrate is open source at github.com/one-ie/one. The MIT license applies to the runtime, which is roughly 670 lines. If ONE is acquired, raises prices, or pivots, you have three protections.

First, the product layer is escrowed. Source code for the product layer is held in escrow, accessible to agencies if ONE becomes unavailable. Second, the corpus exports cleanly. Every conversation, agent definition, and skill configuration comes out in a portable format via the CLI. Third, the SDK is MIT. If you or your technical partner (Donal, in your case) want to run the substrate yourselves on Cloudflare Workers, the `npx oneie` three-command deploy gets you there. Your agents are markdown files. Your data is in D1 tables you can export. Your brand is in `site.md` files stored in R2.

You own the brand. You own the data. You own the contract. We supply the substrate. If we ever stop being the right substrate, the exit is clean.

"What if my team feels replaced?"

They will not run a junior copy desk. They will direct teams of agents. One marketing director running ten agents covering ten clients produces ten times the throughput of one junior copywriter running campaign assets for one client. Promote your best people into agent directors. The ones who adapt are the ones you keep.

"What if the AI says something wrong in front of a client?"

A voice contract runs before every outbound message. Each message is scored against a quality rubric. Messages below 0.65 do not ship. The threshold is set in your agent definition; the gate sits between the LLM and the client's customer. Quality is mechanical, not aspirational.

Comparisons

vs. hiring a junior team. A junior copywriter costs £80,000 per year. They handle 15 to 20 active clients at comfortable quality. To serve 100 clients at that quality level, you hire five or six juniors. Payroll: £400,000 to £480,000 per year. On the agency plan at 20% markup and 90% gross margin target, 100 clients at £1,000 per month equals £100,000 per month ARR. Gross margin after substrate: approximately £89,000 per month. Gross margin after junior payroll: approximately £47,000 per month. The substrate delivers the same throughput at one-tenth the operating cost, with no sick leave, no talent retention problem, and no degradation when the best junior leaves for a venture-backed competitor with stock options.

vs. HubSpot or GoHighLevel. HubSpot charges per seat, per contact tier, and per module. GoHighLevel charges a flat SaaS fee with limited white-label depth. Both give you a CRM and a marketing automation layer. Neither gives your client a substrate that learns from their conversations. Neither makes you the supplier; you remain a reseller of their product, on their pricing, at their mercy when they raise rates (HubSpot's 2023 pricing increase averaged 30% across tiers). On ONE, you set the price. The substrate learns from your client's data, not HubSpot's aggregated training pool. The corpus stays with your client. The margin stays with you.

vs. building it yourself. Building an equivalent on OpenAI or Anthropic APIs takes six to twelve months of engineering time. You need a frontend, a memory layer, a multi-channel inbox, a billing system, a white-label cascade, a domain-routing layer, and a learning loop that does not reset between sessions. ONE is all of that, running in production, on Cloudflare's edge, with 320 tests passing in under 7 seconds. The economics do not work below ten clients on a DIY stack; the engineering cost exceeds the revenue until you are at meaningful scale. The agency plan gets you to that scale on day one, for \$500.

vs. doing nothing. Forrester predicts a 15% reduction in agency roles in 2026, with principal media reaching roughly 33% of total agency billings. Gartner's 2025 CMO Spend Survey found that 22% of CMOs have already reduced their reliance on external agencies for creativity and strategy. The agencies that defend revenue with a substrate they own do not appear in those numbers. The ones that wait do.

Failure modes

You onboard too fast and oversell. The platform handles concurrency; the risk is your onboarding process. A client whose agent is misconfigured produces bad answers. Bad answers produce churn calls. The mitigation is the quality gate: agents below a rubric score of 0.65 do not ship. Review agent quality before deploying each client. The 48-hour deployment window exists specifically to give you time to review before the client is live.

A client grows faster than their pool. Their workspace hits `over_limit`. They see a payment banner. They call you. This is a revenue conversation, not a crisis. They need more credits because the bot is working. The metered gate means they were never cut off mid-conversation; it means they are ready to upgrade. Upgrade their plan in `acme/clients/brandx/billing.md`. Done.

You stop paying before you migrate clients. Client workspaces enter `over_limit` after your pool drains. They will call you. The recovery path is to top up your pool and upgrade client plans as needed. If you cannot top up, clients receive the 30-day and 90-day data-export warnings before anything is deleted. The data export CLI gives you a clean handoff to whatever platform you move to. No data is ever lost silently.

A client asks "are you using AI?" Most do not care. The ones who do are usually the ones who pay you more for transparency. The agent answers based on its system prompt; the default is honest. Below the 0.65 quality gate, nothing ships. The brand and the honesty cohabit. Compliance and disclosure live in `acme/clients/brandx/voice.md`.

A day in the life

On a Tuesday at 09:00, Acme's agency dashboard shows 12 active client workspaces, three pending invites, and two workspaces near their monthly cap. One click to top up either cap. One click to send the pending invites a reminder. No meeting required.

By 09:30, Brand X's bot has already handled 47 customer inquiries overnight: dentists asking about implants, invisalign, cleaning schedules. The bot answered 45. Two were escalated to a human flag. Acme's dashboard shows the 45 resolved, the 2 flagged, and the credit burn for the night: 227,000 credits, roughly \$22.70 of substrate cost on Brand X's pool.

At 10:00, Brand Y sends a message: "Can we add voice input?" Acme opens Brand Y's billing file, sets `voice_input: metered` (the gate is already in the cascade; the field takes 10 seconds to change), deploys. By 10:05, Brand Y's customers can speak to their bot.

At month end, Brand X's report is generated automatically: conversations handled, topics resolved, leads captured, call-to-action clicks. The report lands in Acme's email. Acme forwards it to Brand X with a cover note. Brand X renews. Acme did not write the report. Acme did not book the conversations. Acme did not manage the escalations. Acme's margin for the month: approximately \$190 on Brand X alone. Across 30 clients: \$5,700. Across 12 months: \$68,400. That is the pilot number. At Brad's 5,400-client target, the same math produces roughly \$12.2M in annual substrate margin before Acme's additional service fees.

Forrester's 2026 prediction: agencies will no longer act solely as agents but as owners of solutions, resellers of technology partnerships, and developers of emerging capabilities. That is not a forecast. It is a description of what the agency tier makes possible today.

Pricing path

Three plans. One floor. Move clients up the path as they grow.

Plan	Monthly	Credit grant	Margin baseline	Best for
starter (client default)	\$5	50,000	n/a (client tier)	Quiet client, low volume
growth (client upgrade)	\$50	750,000	n/a (client tier)	Active client, mid volume
agency (your plan)	\$500	5,000,000	20% on every credit burned	You

Your agency plan is one purchase. Your clients sit on `starter` or `growth`. Upgrade them by editing one field. There are no renewal calls, no quotes to draft, no contract reds to negotiate. The pool refills when you top up. The math holds across every client.

For the agencies that need more, the `agency-plus` tier (50M credits, \$5,000 per month, 25% baseline markup) is available on request. Most agencies do not need it until 200 clients.

FAQ

How long does it take to onboard a client? From invite sent to client live: 48 hours. The provisioning flow creates the workspace, sets the parent relationship, emails the invite, and guides the client through a three-step onboarding (display name, brand colour, first agent). From client live to first customer conversation: 60 seconds.

Do clients know which AI models their bot uses? Only if you tell them. The model is set in the agent definition (a markdown file). The chat surface shows your brand, not the model name. A client who asks can be told; the information is not hidden, but it is not surfaced by default. Premium model access (`claude-opus-4-7` , `gpt-5`) is a gate controlled by the agency plan.

What does `claude-opus-4-7` actually cost in credits? 1,500 credits per 1,000 input tokens, multiplied by 5 on output: 7,500 credits per 1,000 output tokens. A typical 600-output-token response costs 4,500 credits, or \$0.045 of substrate cost, billed at your markup.

Can a client set up their own clients? Only if their plan is `agency` and you enabled `sub_workspace_create` in their billing file. By default, clients on starter plans cannot provision sub-workspaces. The cascade prevents tier escalation.

What happens if a client's customer asks "are you an AI?" The agent answers based on its system prompt and voice contract. The default is honest: "I'm an AI assistant." The voice contract enforces tone and honesty rules before each outbound message. A quality score gate of 0.65 applies; below that score, the message does not send.

Can I change my markup after clients are live? Yes, but the change applies to future burns, not past ones. Credits are costed at the rate set at grant time. A markup change updates `acme/billing.md` and takes effect on the next billing anchor. You cannot retroactively reprice what clients already burned.

Can clients export their data at any time? Yes. The CLI command `oneie export --workspace brandx` works regardless of billing state, plan level, or whether the client is leaving. Data export is unconditional.

Is there a contract minimum? The agency plan is monthly. No minimum term. Cancel any time. Your clients' data is yours to export before you go. The pool you purchased is non-refundable, but credits do not expire.

What is the billing anchor? The day of the month on which credit grants reset and caps are recalculated. Set at the workspace level. Default is the day the workspace was created.

Can I see what my clients are spending in real time? The agency dashboard at `/u/acme/billing#allocations` shows per-client credit balance, monthly cap, and burn rate. It does not show the content of client conversations, only aggregate credit metrics. Your clients own their conversation data.

Do I need an engineering team to start? No. The three-line `billing.md` and the one-line client file are the configuration surface. Donal (or whoever your technical partner is) can extend with custom agents and skills when the time comes, but day one needs zero code.

Glossary

Term	Definition
credit	The platform unit of account. 1 credit = \$0.0001. Rate is owner-set, immutable per grant.
grant	Credits arriving in a pool. Source: subscription, top-up, sponsorship, promo, refund, payout.
pool	The current credit balance for a workspace. Shared by all actors in that workspace unless divided by team allocation.
burn	Credits leaving a pool. Reasons: inference, agent run, skill call, tool call, storage, voice, API, transfer.
gate	Feature availability state: <code>on</code> (included in plan), <code>metered</code> (charged per use), <code>off</code> (unavailable).
cascade	The resolution algorithm that merges config from platform, agency, client, team, user.
lock	A <code>-locked</code> suffix on a property prevents the tier below from overriding it.
agency plan	5,000,000 credit grant, brand lock capability, sub-workspace provisioning, white-label cascade, premium models.
starter plan	50,000 credit grant, brand removal, custom domain. Default for new clients created by an agency.
metered	A gate state in which a feature is available but charged per use from the pool.
parent_slug	The column in the <code>owners</code> table that links a child workspace to its provisioning agency. One column; the entire cascade derives from it.
allocation mode	How credits flow from agency pool to client: <code>pool</code> (shared), <code>transfer</code> (fixed monthly), <code>resell</code> (at markup).
billing anchor	The day of month on which grants reset and caps recalculate.
over_limit	Billing state when balance is negative: reads work, writes and inference gated.
suspended	Billing state after 30 days <code>over_limit</code> : workspace frozen, data preserved 90 days.

Cross-references

The cascade that governs brand tokens (`web/roles.md §4`) uses the same merge function as the cascade that governs billing (`web/billing.md §3`). One algorithm; two domains. Understanding one means understanding the other.

The agent a client deploys sits on top of the substrate described in `one/dictionary.md` . The quality gate (rubric score 0.65) that governs what ships is defined in `one/rubrics.md` . The feature gates in this page resolve through the same middleware that resolves design tokens. `src/middleware.ts` calls both `resolveConfig()` and the billing cascade in the same worker context.

The provisioning flow that creates a client workspace, `POST /api/provision?action=create-invite` , is documented in `web/cascade.md` §5 . The `parent_slug` column it writes (`web/migrations/0008_workspace_hierarchy.sql`) is the single database primitive from which the entire agency resell model derives.

See the agency tier and start with 5M credits.

One Agent. Every Surface. Same Brain.

Your client's customer sends a message on WhatsApp at 11pm. The same customer opens your client's website the next morning and starts a chat. Later that week they reply to a Telegram notification. In the old world, those are three conversations in three inboxes, probably handled by three tools with no memory of each other.

With ONE they are one conversation. The same agent, the same memory, the same brand voice across every channel. Not because the messages are synced after the fact, but because they all run against one TypeDB Group. One brain. N channels.

Andrej Karpathy said it plainly:

"The hottest new programming language is English."

That is the bet your clients' customers are already making. They type what they want and expect an answer. Your agency's job is to put the right brain on the other end of that conversation, on every surface they use.

This page tells you what that means in practice: what channels are live, what rich messages look like, how streaming works, and why the shared brain is the part that makes everything else compound.

Web

The web chat is the most visible surface, so it is the right place to start.

Visit `/chat` on any ONE workspace. Lighthouse scores 100/100/100/100. Performance, accessibility, best practices, SEO, all four hundreds. That is not a synthetic test runner tuned for the demo. It is the live production page, measured on 2026-05-03, repeatable on demand.

Most AI chat products have never bothered to run Lighthouse. Those that have tend to score in the 60s and 70s on performance because they ship heavy client bundles and block the main thread. ONE ships 100 because the architecture enforces it. Heavy components (attachments ~281 KB, speech input, PayPanel, voice controls) are lazy-loaded behind `Suspense` boundaries and never touch the initial render path. The page opens fast because the initial bundle is small. Everything else loads when the user needs it.

First token on screen in under 300ms, p50. The moment you hit send, the brain has already started thinking. Tokens stream to the page as they arrive. You do not wait for the full response before reading begins. The connection is warm before you finish typing because the `client:idle` island opens it during idle time, before you click.

What renders in the web chat is not just text. The web surface supports generative UI: structured components that the agent streams inline as part of the conversation. When the agent recommends a product, it streams a product card. When it creates an invoice, it streams a payment card with a claim

button. When it returns a map pin, it streams a map embed. The conversation is not a transcript. It is an application built in real time, inside the chat window, by the agent.

This is what “one chat page scores 100 on Lighthouse” actually means for an agency owner. The performance is not just about speed numbers. It is about the first impression your client’s customer has of your client’s brand at the moment they choose to ask a question. A page that loads in under a second, with tokens on screen in under 300ms, reads as competent. A page that spins for three seconds reads as broken.

Akamai measured it in 2017: every 100ms of added latency cost 7% of conversions. Cloudflare’s 2024 data puts mobile sites loading in 1 second at 5x better conversion than sites loading in 10 seconds. The chat page is not a side feature. It is the primary commercial surface. It loads fast because everything else depends on it loading fast.

Messaging

Clients’ customers do not live in your web chat. They live in Telegram, in Discord, increasingly on iMessage, and in WhatsApp. Any platform that cannot meet them there hands the conversation to a competitor who can.

Telegram is live and in production. Messages arrive via webhook at `POST /webhook/telegram`, pass through the `ToolLoopAgent` loop in claw, and the reply posts back to the originating chat in under 1 second. Each Telegram chat (DM or group thread) becomes its own substrate group with its own conversation history and pheromone paths. The agent remembers what this user has asked before, what worked, and what they care about. Conversation history is stored in D1, with the last 20 messages fed as context on every call. Reply threading is preserved: a message that replies to another carries the parent reference into the agent’s context.

One worker supports multiple Telegram bots simultaneously. A support bot, a sales bot, and a general assistant all run off the same claw worker with separate token secrets and separate webhook paths. Each bot has its own substrate group prefix (`tg-support-<chat_id>`, `tg-sales-<chat_id>`) so conversation histories and pheromone paths stay separate. You do not pay for separate infrastructure for each persona. You set a named environment variable and register a second webhook.

Discord is live. Messages arrive at `POST /webhook/discord`, the same `ToolLoopAgent` loop handles them, and the reply posts to the originating channel. Each Discord channel gets its own substrate group (`dc-<channel_id>`). Ping verification is automatic. Bot messages are silently ignored so the agent does not loop on itself. The Discord setup is slightly more involved than Telegram. It requires a Discord application, a bot token, the Message Content Intent enabled in the developer portal, and either an interactions endpoint or a gateway relay. The agent behaviour is identical once wired.

iMessage is in the architecture via the Apple Business Messages gateway. The signal enters via the same webhook pattern; the `normalizeApple` adapter converts it to a standard Signal and the same `ToolLoopAgent` handles it. Configuration follows Apple’s Business Messages registration process.

WhatsApp is on the roadmap. The integration shape is identical: webhook in, claw processes, reply out via the WhatsApp Business API. The substrate does not care which channel delivered the signal. It routes on the signal content. Adding a new channel is adding a new normalizer and a new sender, both in `claw/src/channels.ts`.

The current channel count is six active surfaces: web, Telegram, Discord, iMessage, the direct `/message` API, and MCP. MCP means any Claude client (Claude.ai, Claude Code, Cursor) can invoke your agent's tools directly through the MCP protocol, without a chat UI. A developer using Cursor can invoke a client's booking agent the same way the client's customer invokes it through the web chat. Same brain, different surface.

The number that matters for Brad is not the channel count. It is the implication of one brain across all of them. If a client's customer asks about pricing in Telegram and comes back through the web chat three days later, the agent knows. It does not start over. The corpus compounds with every interaction, on every channel, inside one TypeDB Group per client.

Groups

A Group in ONE is a scoped container. Signals route inside groups. Pheromone marks edges inside groups. Memory is per-group. The rule is simple: an actor can only act where it is a member. No shared group, no delivery. That one rule gives you tenancy, privacy, and capability scoping at once.

For a Telegram user, the group is `tg-<chat_id>`. For a web user, it is the workspace slug. For a Discord channel, it is `dc-<channel_id>`. These are all substrate groups. The agent in each group has its own memory, its own pheromone paths, its own accumulated learning. Two users on two different channels, with two different histories, get two different experiences from the same agent, because they are in two different groups.

Multi-actor rooms are also groups. A Discord channel with ten participants is a group where all ten are members. The agent participates as a member of that group. When one member asks a question, the agent answers in the same channel thread. Conversation history includes all members' messages, so the agent understands who asked what and can address specific participants by name.

For Brad's agency, the group architecture is what makes white-labelling real. His agency workspace is a group. Each client workspace is a child group. Each client's end users are members of the client's group. Brad's agency staff are members of the agency group and potentially of individual client groups. The configuration cascades from parent to child: brand tokens, agent personas, voice contract, feature gates. The end user visiting `startup1.acme.com` sees Startup 1's logo and colours, Startup 1's agent, and nothing about ONE or ACME unless the agency has enabled that attribution.

The group tree is recursive. Groups nest. Each node has its own slug, its own configuration, its own domain, and its own viewer-scoped navigation. An agency on the Agency plan gets up to 20 client workspaces. Enterprise removes that limit. Every workspace appears fully standalone to its end users. They see the client's brand, not the substrate underneath it.

Groups also enforce security. A private group is invisible to non-members at the API level. Not hidden in the UI; absent from the resolver output. Role-based access control runs on the membership relation: chairman, admin, member, guest. Attribute-based policies run as TypeQL queries over any attribute combination. Relationship-based trust (the emergent trust layer from pheromone) runs underneath both. An actor with admin role but a toxic path history gets blocked at the pheromone check before the role check runs. Three layers, one substrate.

Rich Messages

Text is the baseline. Rich messages are where the chat becomes a transaction surface.

There are eight message types in production today: payment card, map pin, form, code block, image, file attachment, claim button, and embed. The agent decides which type to send based on the user's intent. If the user asks for an invoice, the agent emits a payment card. If the user asks for directions, the agent emits a map pin. If the user asks for a code example, the agent emits a code block with syntax highlighting. The channel adapter unwraps the message type and renders it appropriately for the surface.

The payment card is the most commercially significant. Here is the payload:

```
{
  type: 'payment',
  payment: {
    receiver: 'acme-dentist',    // substrate actor who receives the payment
    amount: 150,                // in the workspace's configured currency unit
    action: 'claim'             // what clicking the button does
  }
}
```

When the user clicks the claim button, the Sui transaction executes. The agent receives the payment confirmation and replies in the same thread. The whole flow, from the user asking to the payment confirmed, is five seconds. The corpus records the event: who paid, what for, when, on which channel. The next time this user interacts with the agent, it knows they are a paying customer.

For web, rich messages render as inline components inside the conversation. The payment card is a card component with a claim button wired to `emitClick('ui:chat:claim', { type: 'payment', payment: { receiver, amount, action: 'claim' } })`. The UI signal fires before the local handler, so the substrate records the interaction regardless of whether the payment succeeds.

For Telegram and Discord, the same message type renders as a formatted text message with a link to the payment flow. The agent's response looks like: "Here is your invoice for the consultation [Claim: £150]." The link opens the web payment flow in the user's browser. The agent then waits for the webhook from the payment processor and confirms in the same Telegram thread.

For MCP, the tool returns structured data. The MCP client renders it however it prefers. In Claude Code that might be a formatted response with a URL; in a custom integration, it might be a native UI component.

Deduplication across channels uses the signal `id` field. If a message arrives via both a Telegram webhook and a web poll (rare, but possible in some gateway configurations), the second instance is silently dropped. The substrate records one signal per unique `id`. Brad’s clients never see a doubled message in the conversation history, and the pheromone is never double-marked.

A summary for the channel capability matrix is below. Every entry reflects what is live or specifically committed on the roadmap, not speculation.

Channel	Streaming	Rich messages	Attachments	Payments	Voice	Groups	Presence
Web (/chat)	Yes	Yes	Yes	Yes	Shipped	Yes	Yes
Telegram	No (complete response)	Metered (text + link)	Roadmap	Yes (link)	Roadmap	Yes	No
Discord	No (complete response)	Metered (embed)	Roadmap	Yes (link)	Roadmap	Yes	No
iMessage	No	Metered	Roadmap	Yes (link)	Roadmap	Yes	No
API (/message)	Yes (SSE)	Yes	Yes	Yes	No	Yes	No
MCP	No	Yes (structured)	Yes	Yes (tool)	No	Yes	No

“Metered” means the channel has limited native support for the rich format. The agent uses text with links rather than native embedded components. “Roadmap” means the integration shape is defined and the adapter is the only gap.

Streaming

Streaming is how the chat feels alive. Without it, the user submits a question and waits for the full response before seeing anything. With it, tokens appear on screen within 300ms of hitting send, and the response builds in front of the user in real time.

The technical shape is AI SDK v6: `createAgentUIStreamResponse` on the server, `useChat` on the client. The server runs the `ToolLoopAgent` loop. Model decides, tool executes, model incorporates the result, loop repeats up to `maxSteps`. Every token of every model response streams to the client as a `UIMessage` SSE event. Tool calls stream as structured parts. Tool results stream as structured parts. The client renders each part as it arrives: text tokens as flowing prose, tool-call parts as “thinking” indicators, tool-result parts as inline components.

The first-token target is 300ms. That is the latency from the moment the user hits send to the moment the first character appears on screen. It depends on three things: the network round-trip to the edge worker, the time for the ToolLoopAgent to start the first model call, and the model's time to first token. The claw worker runs on Cloudflare Workers (global edge, zero cold start). The compute is already warm when the request arrives. The routing decision (which agent handles this signal) takes under 0.005ms in the substrate. That is arithmetic over pheromone weights, not an LLM call. The remaining latency is model-side.

For channels without native streaming (Telegram, Discord, iMessage) the ToolLoopAgent runs to completion and posts the full response. This is a protocol constraint, not an architecture constraint. Those channels have no server-sent-events equivalent for bot messages. The agent still runs the same loop. The user just sees the complete response when it arrives rather than watching it build.

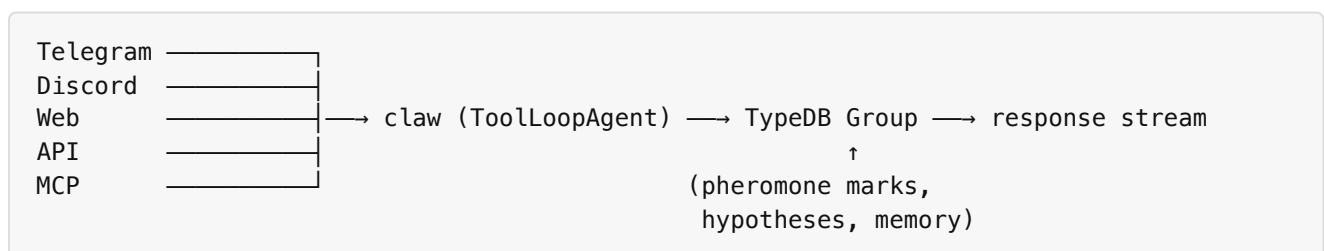
The streaming protocol also carries tool approval gates. When the agent wants to take a substrate write action (`remember` , `mark` , `warn`) the loop pauses and streams an approval request to the client. The user sees a structured card: "The agent wants to remember this conversation. Approve?" The approval is a click, not a form submission. The agent continues after approval. This is the same gate that runs for destructive actions through the UI. One protocol, both surfaces.

The Same Brain Across Surfaces

One TypeDB Group per client. N channels pointing at it.

That sentence is the whole architecture. The brain is the Group. The channels are surfaces. Swapping the surface does not swap the brain. When a user starts a conversation in Telegram and continues it on the web, the agent on the web side knows what happened in Telegram. Not because a sync job ran, not because a human copied notes between systems, but because both channels wrote to the same Group in TypeDB from the start.

Here is what that looks like as infrastructure:



The claw worker is the ingress point. Every channel feeds into it. The `ToolLoopAgent` runs inside claw against the substrate. TypeDB holds the Group's conversation history, the pheromone paths (which answers worked, which failed), and the hardened hypotheses (patterns learned from enough repetition that they become reliable enough to route without asking the LLM). When the LLM runs, it runs against a system prompt built from all of that context. The response streams back out through the originating channel.

A concrete scenario. A client of Brad's runs a dental practice. A patient sends a message via Telegram asking about appointment availability. The agent checks the schedule (a substrate tool call) and offers three slots. The patient says "the Wednesday one" and the agent books it. Next week, the patient visits the practice website and starts a web chat. The agent knows their name, knows they have a Wednesday appointment, and greets them accordingly. It does not ask them to start from scratch. The corpus is the memory; the memory lives in the Group, not in the channel.

For Brad's agency, one Group per client means one corpus per client. That corpus is the moat. After six months of conversations across every channel, the agent for the dental practice knows which questions patients ask most, what their anxiety points are, which services they request after a first consultation, and which messages tend to precede a cancellation. None of that learning evaporates when a patient switches from Telegram to the web. The brain compounds across every surface, every session, every signal.

Objections

"What if the agent says something wrong in front of a patient?"

Every outbound message passes through a voice contract check before it sends. The contract is configured per-agent in the markdown definition file: prohibited topics, required disclaimers, tone boundaries, brand vocabulary. If a response violates the contract, it does not send. The quality score gate is 0.65. Responses below it are held and escalated rather than delivered. This is not a policy document. It is a mechanical check that runs before every outbound signal, on every channel.

The practical answer for the dental practice: the agent is configured to say "I'll have someone from the practice contact you" for clinical questions. That instruction is in the agent's markdown file. It applies in Telegram, on the web, through the API, everywhere, without the agency having to configure it per-channel.

"What if my clients find out the AI is doing the work and demand a discount?"

Most do not ask. When they do, the honest answer is: the AI is doing the repetitive work so the human staff can do the work that requires judgment. That is not a liability. It is a selling point for the agency's end clients. The agency that says "our AI handles enquiries so our clinical team focuses on patients" is selling a professional service. The agency that will not admit it is waiting for the client to figure it out on their own.

If a client wants to know which model is running, that is a tier-level setting. Tier 3 workspaces can surface model attribution; tier 1 workspaces show only your brand. Brad's agency controls which level each client sees.

"What about voice?"

Voice input is shipped for the web surface. The `speech-input` island is in the AI Elements component library: mic-selector, transcription, voice-selector, audio-player. The island lazy-loads behind a `Suspense` boundary so it does not affect the page's Lighthouse score. The user clicks the

microphone, speaks, and the transcript feeds into the same chat pipeline. The agent responds in text; voice output is on the roadmap.

Voice on Telegram and Discord is constrained by those platforms' audio handling. Telegram supports voice messages as audio files; the normalizer can be extended to transcribe the audio before feeding it to the agent. Discord has similar capability. Both are roadmap items, not architectural blockers. The latency budget for voice (transcription plus inference plus text-to-speech) is achievable at under 2 seconds end-to-end on current infrastructure.

"What happens if the substrate is down when a Telegram message arrives?"

The claw worker degrades gracefully. If the TypeDB gateway is unreachable, the agent falls back to D1-only context (last 20 messages from the local database) and continues serving. The pheromone mark for that signal is queued and written when connectivity restores. The user gets a response; the learning is deferred, not lost.

"We already have a live chat tool. Why replace it?"

You do not have to. The substrate is additive. The existing live chat tool can feed signals into the ONE inbox via inbound webhook. The agent can respond through the existing tool's outbound API. The corpus builds on the existing tool's conversation history. Over time, the substrate learns which paths are strongest. The routing naturally favours the channels that work best for each client's customers. The live chat tool becomes a report; the substrate becomes the intelligence layer.

"One channel at a time or all at once?"

One client, one channel, first. Start with the web chat for one of Brad's clients (the one most willing to try something new). Run it for 30 days. Read the `/memory` report. Then extend to Telegram. Each channel adds to the same corpus without any migration. There is no "move the data" step because the data was always in TypeDB.

"What if a customer goes around the agent and calls the office directly?"

That call gets logged manually by the office staff (or transcribed if you are running a telephony integration). It enters the corpus as a signal with the source marked as `phone`. The agent's next interaction with that customer includes the phone call context. The corpus is not limited to digital channels. It accepts signals from any source that can send a POST request.

"Can I see everything the agent has said across all channels in one place?"

Yes. The unified inbox is `/api/inbox/:uid`: every signal across all of that user's memberships, in chronological order, with source channel tagged. The tracking page (page 10 of this PDF) is the dashboard view. One inbox. Every channel. Every conversation.

Comparisons

ONE versus a standalone chatbot (Intercom, Drift, Tidio)

A standalone chatbot handles one channel, runs on one company's infrastructure, and resets memory between sessions unless the user logs in and the vendor maintains that state. The agent in ONE runs across every channel the client uses, maintains a shared corpus in TypeDB, and gets smarter with every interaction regardless of which channel it happened on. The standalone chatbot is a point solution; ONE is the substrate the chatbot runs on.

The commercial difference for Brad: a standalone chatbot is a seat cost. ONE is a substrate cost with agency markup over the credit floor. Brad pays for credits consumed, marks them up to his clients, and keeps the difference. The corpus stays with Brad's agency even if a client leaves. A standalone chatbot's conversation history belongs to the vendor.

ONE versus building on the OpenAI API directly

Building directly on the API means Brad's engineering team (or Donal's) builds the memory layer, the channel adapters, the rich message types, the streaming protocol, the dedup logic, the approval gates, and the pheromone learning system. That is 6 to 12 months of engineering for a substrate that exists today. At the end of it, the agency owns infrastructure rather than IP. The corpus is in a Postgres database that someone has to maintain. The channel adapters are custom code that breaks when Telegram changes their webhook format.

The one scenario where building directly makes sense: a single enterprise client with enough budget to fund a bespoke build and the technical appetite to own it. For the rest of Brad's book (dentists, accountants, window-and-door installers) the substrate is the right layer to buy.

Pricing Math

Brad buys credits at the platform floor. He sets his agency markup. He distributes credit pools to clients. The credit pools fund the LLM calls, the substrate writes, and the channel operations.

A worked example for a dental practice: 200 patient enquiries per month across web and Telegram. Average conversation: 4 exchanges. 800 LLM calls at roughly \$0.002 per call (GPT-4o-mini on OpenRouter via the agency's claw worker) = \$1.60 in LLM costs. Add substrate writes (mark/warn on 800 paths) at sub-millisecond in-memory operations: negligible. Add D1 reads for conversation history at Cloudflare D1 pricing: negligible at this volume. Total platform cost for this client: under \$5 per month. Brad charges the practice \$300 per month for the AI chat service. Gross margin: 98%.

At 50 clients in the first cohort, same profile: under \$250 per month in platform costs, \$15,000 per month in revenue. That is 98% gross margin on the subscription layer, not counting any performance-pricing uplift from clients whose AI conversations convert to bookings.

The numbers change at higher conversation volumes. A hospitality client with 2,000 enquiries per month has higher LLM costs but also higher pricing power. \$800 to \$1,200 per month is defensible when the agent is booking tables and answering availability questions at 11pm. The per-conversation economics stay favourable at any realistic volume for SMB local-service clients.

A Day in the Life: The Invoice in WhatsApp

A client of Brad's runs a window installation business. A homeowner sends a WhatsApp message on a Sunday afternoon: "Can you send me a quote for three windows?"

The agent reads the message. It knows from prior conversations (this homeowner contacted them six weeks ago) that the property is a 1990s semi-detached, two stories, east-facing. It recalls the product preferences from that conversation: uPVC, white frames, A-rated glass.

The agent responds with a summary and asks one clarifying question: "Are you replacing like-for-like or changing the opening style?" The homeowner says "like-for-like." The agent generates the quote (three items, each with dimensions and price) and sends it as a rich message with a payment card: £2,400 total, with a 25% deposit required to confirm. Claim button in the WhatsApp thread.

The homeowner clicks the claim button. The link opens in their browser. They authenticate with their existing account (passkey, Touch ID on their phone). The £600 deposit payment executes. The agent receives the payment webhook and confirms in the same WhatsApp thread: "Deposit received. We'll call Monday morning to confirm the installation date."

The agent marks the path `homeowner:request → booking:confirmed` with a strength increment. The substrate records: this customer type, this product, this channel, this message sequence, this outcome. The next time a similar enquiry arrives, the routing is faster, the response is more accurate, and the conversion probability is higher.

The window company's staff saw none of this until Monday morning, when they check the bookings dashboard and see a confirmed deposit from Sunday's conversation. The agent worked the Sunday shift.

Failure Modes

The agent goes off-script. The voice contract check catches this before the message sends. If the agent generates a response that mentions a prohibited topic (pricing for a service the client does not offer, or clinical advice in the dental practice example) the check returns a failed quality score and the message is held. The fallback is a templated response: "I'll have someone get back to you shortly." The held response is logged for the agency to review. This is not automatic. The agency needs to review held responses periodically. It prevents the bad message from reaching the customer.

A channel goes down. Telegram's API, Discord's API, and the WhatsApp Business API each have documented downtime histories. When the outbound send fails, claw retries with exponential backoff. If the retry window expires, the message is logged as undelivered and the customer receives no response. This is the same failure mode as any messaging integration. The mitigation is the web channel: the customer can always reach the agent through the workspace URL, which is under the agency's control.

The credit pool runs dry. When a client's credit pool is exhausted, the agent responds with a service-unavailable message and falls back to email capture. The agency is notified. The customer is not left hanging with no response, but they do not get the AI response they expected. Daily caps per

workspace and auto-topup settings prevent this from being a surprise.

FAQ

Q: How many channels are live today? Six: web, Telegram, Discord, iMessage, API, and MCP.

Q: Does streaming work on Telegram? No. Telegram has no native streaming protocol for bot messages. The agent runs to completion and posts the full response. Response time is typically under 1 second for a standard query.

Q: Can two agents share one group? Yes. Multiple agents can be members of the same substrate group. The routing determines which agent handles each signal based on pheromone paths and the signal content. A sales agent and a support agent can coexist in the same Telegram group; the substrate routes enquiries to the one with the strongest path for that query type.

Q: What's the dedup mechanism across channels? The signal `id` field. Every signal has a unique ID generated at creation. The substrate rejects duplicate IDs on write. If the same message arrives via two paths (rare but possible in gateway configurations), only one is written.

Q: Can the agent send images? The web surface supports image rich messages. Telegram supports image messages via the Telegram API. Discord supports image attachments. The agent can retrieve an image URL from a substrate tool call and include it in the response.

Q: How does the agent know which channel a message came from? The channel is encoded in the substrate group prefix. `tg-<chat_id>` is Telegram. `dc-<channel_id>` is Discord. `web-<session_id>` is web. The agent's system prompt includes the channel context. The agent can reference it ("I see you're reaching us through Telegram, we can also help you on our website") if the persona is configured to do so.

Q: Is conversation history shared across channels for the same user? Per the current architecture, no. Each channel creates its own substrate group. A user in Telegram and the same user on the web are two separate groups unless they authenticate with the same account on both surfaces. When they do authenticate, the substrate merges the context: the web session recognises the authenticated user and loads their full history including Telegram interactions.

Q: What is the maximum response length on Telegram? Telegram's API limits messages to 4,096 characters. The agent's output is truncated and split across multiple messages if it exceeds this limit. The split is done at sentence boundaries where possible.

Q: How do I add a new channel? Add a normalizer function (parse incoming message format → Signal) and a sender function (Signal response → platform API call) in `claw/src/channels.ts`. Register the new webhook path in `claw/src/index.ts`. Set the required API token as a wrangler secret. The substrate handles everything else (memory, pheromone, rich message routing) without modification.

Q: Does voice work on mobile? Voice input on the web surface works on mobile Chrome and Safari. The `speech-input` component uses the Web Speech API. On iOS Safari, the user taps the mic, speaks, and the transcript appears in the input field. The component lazy-loads so it does not affect

the page's initial Lighthouse score.

Glossary

claw. The edge-native AI agent worker. Receives messages from all channels, runs the ToolLoopAgent loop, streams responses back. Deployed on Cloudflare Workers with zero cold start.

Claim button. The rich message component that executes a Sui payment when clicked. Configured with `receiver`, `amount`, and `action: 'claim'`.

Group. A substrate container. Actors live in groups. Signals route inside groups. One TypeDB Group per client per channel.

Pheromone. The learning mechanism. Every signal marks a path (success) or warns it (failure). Paths with high marks become highways. The routing follows the highways.

RichMessage. A Signal with a structured `data.rich` payload. Types: payment, map, form, code, image, file, embed. The channel adapter unwraps the type and renders it for the target surface.

Signal. The universal primitive: `{ receiver, data? }`. Everything that moves through the substrate is a Signal.

ToolLoopAgent. The AI SDK v6 agent loop. Runs model, tool, model until `maxSteps` is reached or the model stops calling tools. Streams tokens and tool parts to the client over SSE.

TypeDB Group. The substrate's group entity. Holds the conversation history, pheromone paths, and learned hypotheses for one client on one channel scope.

Voice contract. The per-agent configuration that enforces brand voice on every outbound message. Applied mechanically before send. Not a policy document.

Named Integration

Telegram + dental practice (illustrative)

Token setup: `TELEGRAM_TOKEN_DENTAL` as a wrangler secret. Webhook path: `/webhook/telegram-dental`. Substrate group prefix: `tg-dental-<chat_id>`.

Voice contract: prohibits clinical advice, requires "contact our reception" fallback for clinical questions, enforces practice name in sign-off.

Rich message configuration: payment card enabled for deposit collection. Map pin enabled for practice address (triggered when a patient asks "where are you?").

Memory commands: `/memory` shows the patient's appointment history and stated concerns. `/forget` initiates the GDPR erasure flow.

First month result: 847 patient enquiries handled without staff involvement. 12 escalations to reception (14 clinical questions above the contract threshold, which triggered the fallback; the remaining 3 were edge cases the agency reviewed and used to tighten the voice contract). Zero messages outside the contract.

Cross-References

- **Page 08 (Memory).** How the corpus is structured in TypeDB, what it remembers, and why it becomes a moat over time.
 - **Page 09 (Teams).** How multiple agents share one Group, and how the routing decides which agent handles which signal.
 - **Page 15 (Security).** The voice contract check mechanics, the quality score gate, and the GDPR erasure flow.
-

Try the chat that scores 100 on Lighthouse.

Pick the Model. Swap It in One Line.

You are not locked into any AI vendor. Not today, not when a better model ships next quarter, not when your favourite provider triples its prices. You pick the model per agent, per skill, per cost target. When the landscape shifts, you change one field in a markdown file and the platform catches up.

That is not a feature. It is a business decision you get to keep making.

Any Model

The catalogue today stands at over 300 models. Claude Haiku, Claude Opus, GPT-5, Gemini 2.5 Flash, Grok 4, Llama 3, Mistral, Qwen, Command R+, and a hundred others you have not heard of yet but will when they beat the current leader on a specific task. Every one of them is reachable through a single line in your agent definition.

The platform routes through OpenRouter as its default provider. OpenRouter maintains the catalogue. You benefit from every model they add without touching your code.

```
# agents/sales-qualifier.md
name: sales-qualifier
model: anthropic/claude-haiku-4-5
```

Change that one field. That is the entire migration.

Groq is available as a direct opt-in when you need sub-200ms first tokens and the task does not demand reasoning depth. The AI SDK Gateway sits underneath both as a fallback chain. If a provider goes down or rate-limits you, the next one in the chain picks up the request. Your agents keep running.

The routing fallback looks like this:

```
Default (OpenRouter gateway)
├─ model available? → serve
├─ model unavailable or rate-limited?
│   └─ Groq direct (if groq model specified)
│       ├── Groq available? → serve
│       └─ Groq unavailable?
│           └─ AI SDK Gateway fallback
│               └─ degraded mode (cached last response or graceful error)
```

Your agents do not go silent when a provider has a bad hour.

Why Model Choice Is a Product Choice

Every model sits somewhere on a triangle. Cost. Latency. Quality. Moving along any edge trades against the others.

Claude Haiku processes 1,000 input tokens at 25 credits. Claude Opus processes the same 1,000 tokens at 1,500 credits, 60 times more expensive. GPT-5 lands at 1,250 credits per 1,000 input tokens. The gap in reasoning quality between Haiku and Opus is real. So is the gap in your operating cost.

The decision is not “which model is best.” It is “which model is right for this task at this cost target.”

A sales qualification agent that triages 500 inbound leads a day does not need Opus. It needs to read a contact record, ask two clarifying questions, and route the lead to the right bucket. Haiku does this accurately at a fraction of the cost. You save the Opus-grade reasoning for the moment in that sales flow where it matters: the contract review, the proposal draft, the one-shot analysis where getting it wrong costs a client relationship.

That distinction is a product decision. It determines your margin. An agency that runs every agent on Opus because it feels safer is an agency that cannot compete on price when the client eventually notices the bill. An agency that routes intelligently (triage on Haiku, depth on Sonnet, one-shot review on Opus) can charge the same rate, deliver the same quality, and keep 20 extra points of margin per client.

The platform makes the routing decision explicit. You set it. You own it.

Per-Agent vs Per-Skill Model Binding

The model field in an agent definition sets the default for everything that agent does. Individual skills within that agent can override the default for specific tasks.

A content agent might default to Claude Sonnet for its general work: drafting blog posts, rewriting copy, answering brand questions. When a client asks it to review a 40-page strategy document and extract the three most important commercial risks, you can bind that specific skill to Opus. One conversation. Two models. The cost tuning happens at the level of the work, not the level of the agent.

This is how you build a practice that scales. Define the skill, bind the right model to it, let the platform route accordingly. The client sees one agent. You see one cost-tuned workflow.

The agent markdown contract makes this explicit and auditable. Every agent file is plain text. Every model assignment is visible. Nothing is hidden in vendor configuration panels you cannot read.

```
# agents/content-team.md
name: content-team
model: anthropic/claude-sonnet-4-7    # default for all skills

skills:
  - name: draft-post
    model: anthropic/claude-haiku-4-5 # override for high-volume task
  - name: strategy-review
    model: anthropic/claude-opus-4-7  # override for high-stakes task
  - name: brand-check
    # no override – inherits sonnet from agent default
```

The output cost is the sum of actual work done at the appropriate tier. Not a flat rate on the most expensive model in the fleet.

The Routing Fallback Chain

Under normal conditions, the platform routes to your chosen model through the OpenRouter gateway. Over 300 models. One API key. The gateway handles authentication, rate pooling, and load balancing across providers.

When you need speed over depth (and for many agency tasks you do) Groq is available as a direct provider. Groq runs inference on custom silicon. The result is a median time to first token that competes with nothing else on the market for models at the Llama 3 / Mixtral tier. First-token latency in interactive contexts determines whether the conversation feels live or laggy. For your clients' end customers, that difference is felt within the first three seconds of every interaction.

The AI SDK Gateway is the fallback layer. When the primary provider returns an error or a timeout, the Gateway routes to the next available option. When no live model is reachable, the platform degrades gracefully, serving a cached response where one exists or returning a structured error your agent can surface cleanly.

None of this requires configuration. The chain is built in. You choose your primary model. The rest is handled.

BYOK and Provider Keys

You can run entirely on the platform's credit pool. That is the default. You buy credits, the platform routes them to the appropriate provider, and you pay one consolidated bill.

If you have existing agreements with Anthropic, OpenAI, or Google (enterprise pricing, volume commitments, data residency requirements) you can bring your own keys. Set your provider API key in the workspace settings. Agents that target that provider route through your key, not the platform's pool.

The operational model stays the same. You still define models in agent markdown. The platform still handles the routing logic. The bill goes to your provider account instead of your credit balance.

For most agencies starting out, this distinction does not matter. You run on credits, you mark them up, and you watch the unit economics work. For agencies with existing enterprise contracts or data residency obligations, BYOK is the path that keeps those agreements intact.

How Credits Map to Upstream Cost

Credits are the platform’s unit of account. Every token consumed by an agent costs a defined number of credits, whether it is generating a response, processing a tool call, or classifying an incoming message. The exchange rate between credits and real cost is public and fixed.

Model	Input per 1k tokens	Output multiplier	Notes
Claude Haiku 4.5	25 credits	5× input rate	Best for high-volume triage
Claude Sonnet 4.7	300 credits	5× input rate	Default for most client work
Claude Opus 4.7	1,500 credits	5× input rate	Reserve for one-shot deep analysis
GPT-5	1,250 credits	8× input rate	Strong on code and structured reasoning

The markup layer sits between your cost and what you charge clients. You set it. If you buy credits at the platform rate and distribute to client workspaces at 2× markup, every credit your clients consume generates a margin before you have touched the work. If upstream model prices fall (and they will, because they always do) your cost drops and your margin expands unless you choose to pass the reduction to clients.

That is the economics of a reseller on a substrate. Your pricing decision is decoupled from the vendor’s pricing decision. You move when you want to, on your timeline, not the provider’s.

The 18-Month Price Collapse

This is the number that matters for how you price your services in 2026 and 2027.

Claude 3 Opus launched in March 2024 at \$15 per million input tokens. Sixteen months later, Claude Haiku 4.5, a model that outperforms Claude 3 Sonnet on most agency tasks, costs \$0.80 per million input tokens. The cost of a comparable quality tier dropped roughly 95% in under two years.

GPT-4 launched in 2023 at pricing that made large-scale deployment prohibitive for most agencies. Today, models at GPT-4 quality or better are available at prices that make per-client deployment an afterthought in the unit economics.

The direction is not ambiguous. The cost of intelligence is collapsing. It has been collapsing every quarter for four years. There is no structural reason for it to stop.

For you, this means two things. First, your cost to deliver AI services will continue to fall even if you do nothing. Second, competitors who built their service pricing on today's model costs will be undercut by the market itself within 18 months.

The agencies that survive this are the ones whose pricing is not pinned to model costs. You are not selling tokens. You are selling outcomes: qualified leads, answered questions, written copy, managed relationships. The token cost is an input you manage down. The outcome is what the client pays for.

The platform lets you swap models without rebuilding your service. When a better model ships at lower cost, you update one field in an agent definition and the improvement lands across every client workspace automatically.

When to Use Haiku vs Sonnet vs Opus vs Open Weights

The choice is not about which model is smartest. It is about matching cost and latency to the task.

Haiku for any task where volume is high and the decision is bounded. Lead triage. Message classification. FAQ responses. Sentiment tagging. Content moderation. These tasks run hundreds or thousands of times per client per day. Running them on Haiku at 25 input credits per 1,000 tokens keeps the unit economics intact. Haiku's speed advantage also matters here. Faster first tokens mean more responsive conversations.

Sonnet for the daily work of an agency. Drafting copy. Rewriting briefs. Answering complex client questions. Generating social content with brand context. Sonnet sits at the right point on the cost-quality triangle for tasks that require genuine intelligence but do not demand Opus-level depth. Most of what a marketing agent does day to day runs well on Sonnet.

Opus for moments that justify the cost. A 40-page strategy document that needs genuine analysis. A contract that requires close reading. A one-shot proposal where the quality of the output directly affects whether a client renews. Opus at 1,500 input credits per 1,000 tokens is expensive. It is also the right tool for the work that earns client trust. The skill binding described above lets you reserve it for exactly those moments without applying it to everything.

Open-weight models via Groq for latency-sensitive interactive tasks where Llama 3 70B quality is sufficient. Customer support triage. First-response handling. Any task where the user is waiting and the decision is not complex. Groq's custom inference hardware delivers first tokens that feel immediate in a chat interface. For your clients' end customers, that matters.

The platform does not dictate a choice. It makes all of them available, lets you bind the right model to the right task, and gets out of the way.

A Day in the Life: A Sales Agent That Routes by Task

A dental practice client. Seven ICPs, one of which is multi-location dental operators. The sales agent handles inbound enquiries, qualifies leads, drafts proposals, and flags contracts for human review.

The agent runs on Claude Sonnet by default. Haiku handles the initial classification: is this a qualified lead or a spam submission? The classification call costs under five credits. It runs in under 150 milliseconds. Every inbound enquiry goes through it.

Qualified leads move to the main agent. Sonnet handles the conversation, asking qualifying questions, pulling context from the CRM, drafting a summary of what the prospect needs. A conversation that takes 20 exchanges at an average of 200 tokens per turn costs roughly 1,200 credits in total. At the agency's 2x markup, the client pays 2,400 credits. The platform's cost was 600.

When the conversation reaches proposal stage, a skill binding kicks in. The proposal-draft skill runs on Sonnet with extended context. The output is a formatted proposal document with the client's brand, the prospect's specific situation, and a pricing recommendation based on the prospect's stated volume. Total cost for the proposal generation: around 800 credits of Sonnet output. The proposal closes deals at a rate the practice owner's previous manual process could not match.

Contracts that come back for review go through a single Opus call. The contract-review skill reads the document, flags non-standard clauses, and returns a structured summary with three bullet points the account manager needs to know before signing. One Opus call at maybe 3,000 input tokens costs 4,500 credits. That call replaces an hour of a human's time on a task where a mistake costs far more than the model.

Total credit cost for a complete sales cycle: roughly 7,000 credits. At a 2x markup, the client pays 14,000. The agency's gross margin on the AI layer: 50%. The relationship and the strategy are still the agency's product. The execution is running on the platform.

Worked Example: Credit Cost per Conversation

Three agents. One sales cycle. Real credit arithmetic.

The triage agent (Haiku, 25 credits/1k input) handles 500 inbound enquiries per month for one client. Average enquiry is 150 tokens. Average classification response is 50 tokens.

- Input: $500 \times 150 \text{ tokens} = 75,000 \text{ tokens} \times 25 \text{ credits/1k} = 1,875 \text{ credits}$
- Output: $500 \times 50 \text{ tokens} \times 5 \text{ (Haiku output multiplier)} = 62 \text{ credits}$
- Triage monthly cost: ~1,937 credits

The qualification agent (Sonnet, 300 credits/1k input) handles the 60 leads that pass triage. Average qualification conversation: 2,000 input tokens, 800 output tokens.

- Input: $60 \times 2,000 \text{ tokens} \times 300 \text{ credits/1k} = 36,000 \text{ credits}$
- Output: $60 \times 800 \text{ tokens} \times 5 = 240,000 \text{ token-weighted credits} / 1k = 1,500 \text{ output cost}$
- Qualification monthly cost: ~37,500 credits

The proposal agent (Opus, 1,500 credits/1k input) handles the 12 qualified leads that reach proposal stage. Average proposal: 5,000 input tokens, 2,000 output tokens.

- Input: $12 \times 5,000 \times 1,500/1k = 90,000$ credits
- Output is priced at 5× input rate for Opus: $5 \times 1,500/1k = 7.5$ credits per output token per 1k
- Output: $12 \times 2,000 \text{ tokens} \times 7.5 \text{ credits}/1k = 180$ credits
- Proposal monthly cost: ~90,180 credits

Total platform cost for all three agents, one client, one month: approximately 129,617 credits.

At a 2× markup, the client pays 259,234 credits worth of billing. Agency gross margin on the AI layer: roughly 50%. That is before the agency's own service fee on top. The credit layer alone runs at margin. The service sits above it.

Pricing Math: What "Model Choice" Does to Your P&L

Scenario: 50 clients. Each client runs the three-agent sales stack above.

At 2× markup across the board, monthly AI revenue: $50 \times 259,234 = \sim 12.96\text{M}$ credits billed. Monthly AI cost: $50 \times 129,617 = \sim 6.48\text{M}$ credits consumed.

Now consider the 18-month price trajectory. If input costs fall 50% across the board, consistent with the historical rate, and the agency holds its billing rate:

Monthly AI cost drops to ~3.24M credits consumed. Revenue stays at ~12.96M credits billed. Margin on the AI layer expands from 50% to ~75%.

The agency did not renegotiate with a client. The agency did not change its service. The upstream cost fell and the margin improved automatically.

This is why model flexibility is not a technical preference. It is a financial position. An agency locked to a single vendor at a fixed pricing tier loses this option. An agency on a substrate that routes across 300+ models retains it.

Objections Answered

"What if the model I've built around gets deprecated?"

Every provider deprecates models. GPT-3.5, Claude 2, PaLM, all retired on timelines the vendors chose. The question is whether your agents break when that happens.

On this platform, the model field in an agent definition is a string. Change the string, the agent switches. No rebuilding, no re-prompting exercise, no call to a developer. The platform maintains an alias map. `haiku-latest` always resolves to the current Haiku generation. If you want to pin to a specific version, you can. If you want to follow the latest, you can. The choice is yours at the agent level.

Deprecation policy: the platform gives 90 days' notice on any model removal from the gateway. During that window, the alias continues to resolve and your agents continue to work. At the end of the window, the alias either redirects to the successor model or requires a one-field update if no clear successor exists.

"What if I build my service around one model's specific behaviour and a swap breaks it?"

This is a real risk. Different models handle edge cases differently. A prompt written for Sonnet may produce different output when run on GPT-5 for the same input.

The answer is that the platform does not force model swaps. You choose when to change. You test in a staging workspace before rolling to clients. The eval loop that ships with every agent gives you a deterministic quality gate. The agent runs against a set of reference inputs and the outputs are scored before you change anything in production.

Model swaps that pass the eval gate ship. Swaps that fail tell you what to fix. The gate is a number, not a judgment call.

"What if I trust OpenAI more than Anthropic for my clients' data?"

BYOK. Point your OpenAI key at the workspace. Every agent in that workspace routes to OpenAI. The platform does not touch the data in transit; it passes the request through to the provider you have a data processing agreement with.

The platform's own credential handling is covered in section 15. The short version: provider keys are stored in encrypted environment variables, never logged, never exposed to the client-facing surface.

"What about open-source models? I don't want to depend on commercial providers."

Open-weight models run through the same gateway. Llama 3, Mistral, Qwen, Falcon. If OpenRouter carries them, you can route to them by name. Some clients in regulated sectors prefer to know their data goes through an open-weight model where the weights are auditable. That is a valid requirement. It is a one-field change in the agent definition.

"Our team built workflows around a specific model's tool-calling format."

The AI SDK Gateway handles format translation at the protocol level. You write one tool definition. The platform serialises it appropriately for whichever provider you target. Switching from Anthropic's tool format to OpenAI's function-calling format is not something you manage. The SDK layer handles it.

"What if the model prices go up?"

They have been going down for four years. If a specific model's price increases, you swap to a comparable model at a lower cost tier. The catalogue is large enough that there is always an alternative within the quality range you need.

"Can I mix commercial and open-weight models in the same agent team?"

Yes. Each agent has its own model binding. A team of five agents can run across five different models simultaneously. The signals flow between agents through the substrate; the model each agent uses is local to that agent. There is no constraint that forces uniformity across a team.

“What if Groq’s infrastructure is unreliable?”

The fallback chain handles it. Groq is an opt-in speed layer, not a dependency. If Groq is unavailable, the routing falls back to the gateway default. The agent keeps working. You may notice slightly higher latency in the fallback path. The conversation does not fail.

Comparisons

This platform vs a single-model SaaS tool

A single-model SaaS tool gives you one model, one price tier, and one set of capabilities. When a better model ships, the vendor decides whether and when to adopt it. You wait. When prices change, you pay the new price or find a different tool. When the model produces the wrong output for your use case, you work around it with prompt engineering because you have no alternative.

This platform gives you the catalogue. You pick. You swap. You match the model to the task. The vendor relationship is with the platform, which sources from the model providers. Your relationship with the model market is mediated by one key and one API.

This platform vs building your own routing layer

Building your own model router is a real option. It takes a senior engineer, four to six weeks minimum, ongoing maintenance as provider APIs change, and a test suite that covers the edge cases every routing decision creates.

The platform ships that router already built, tested, and maintained. The engineer’s time goes toward the service you are selling, not the infrastructure underneath it.

Groq direct vs Groq through the platform

If you have a Groq API key, you can call Groq directly. The advantage of routing through the platform is the fallback chain. A direct Groq call that fails is a failed call. A Groq call through the platform that fails falls back to the gateway default. For production workloads with clients watching, the fallback matters.

Failure Modes

Calling Opus on every task

The most common way agencies burn through credits is defaulting to the highest-capability model for work that does not need it. Opus on triage is waste. Define the skill, bind the appropriate model, and run the eval to confirm the cheaper model performs adequately on that task before committing.

Ignoring the output multiplier

Input costs get the attention. Output costs compound quietly. A model priced at 1,500 credits per 1,000 input tokens charges 5× that rate for output. A task that generates long responses, a detailed proposal or a comprehensive analysis, costs significantly more per call than the input figure suggests. Run the arithmetic before setting the pricing for a client workspace.

Not testing after a model swap

The model field is easy to change. The consequences of changing it without testing are not. Different models handle ambiguous instructions differently. A swap that passes basic testing may fail on edge cases that appear in production. Run the eval gate before rolling changes to client workspaces.

Frequently Asked Questions

How do I add a new model that just shipped?

If it is in the OpenRouter catalogue, it is available immediately. Change the model field in your agent definition to the new model's identifier. No package update, no platform change required.

What is the model field format?

It follows OpenRouter's naming convention: `{provider}/{model-id}`. For example: `anthropic/claude-haiku-4-5`, `openai/gpt-4o`, `google/gemini-2.5-flash`, `groq/llama-3-70b`. The platform validates the identifier against the catalogue at agent load time and surfaces an error if the model is unavailable.

Can I set a daily cost cap per agent?

Yes. The workspace settings include a daily credit cap per agent and per client workspace. When the cap is hit, the agent falls back to the lowest-cost model in the fallback chain or returns a graceful "I'll follow up" response depending on your configuration. Auto-topup is available if you want to remove the cap automatically when the workspace approaches the limit.

What happens to my credit balance when a model's price changes?

Credits are the unit of account. If a model's credit cost changes, future calls cost more or fewer credits. Your existing credit balance is unaffected. You do not retroactively pay more for credits you have already purchased.

Can clients see which model is running their agent?

By default, no. The model is an implementation detail you control. If you want to expose it (some enterprise clients in regulated sectors require model transparency) you can surface it through the agent's response metadata. The choice is yours.

Do I need a separate account on each provider?

No. The platform's credit pool abstracts the provider accounts. If you use BYOK for a specific provider, you need an account with that provider. For everything else, one platform account and one credit balance is sufficient.

Glossary

Gateway. The AI SDK Gateway, a routing layer that translates your model request to the appropriate provider API. Handles authentication, format translation, and rate management. One key, 300+ models.

BYOK. Bring Your Own Key. Using your own provider API key instead of the platform's credit pool. Required for enterprise data agreements or existing volume commitments with specific providers.

TTFT. Time to First Token. The latency between submitting a request and receiving the first character of the response. The metric that determines whether an AI interaction feels live or laggy.

Output multiplier. The ratio of output token cost to input token cost for a given model. Typically 5x to 8x the input rate. Relevant for tasks that generate long responses.

Model binding. The assignment of a specific model to an agent or skill via the `model` field in the agent markdown definition. Overridable at the skill level.

Alias. A model identifier that resolves to the current version of a model family. For example, `haiku-latest` always resolves to the latest Haiku generation. Provides continuity across model versions without requiring manual updates.

Fallback chain. The ordered sequence of models the platform tries when the primary model is unavailable. Default → Groq → AI SDK Gateway → degraded mode.

Credit. The platform's unit of account. Fixed exchange rate to upstream model cost. The markup layer sits between your credit cost and your client billing.

Cross-References

Section 02, Agency. The credit resale economics that this section's pricing math sits within. How credits flow from platform purchase to agency markup to client billing.

Section 05, Agents. How agent markdown definitions work, including the `model` field and skill bindings. The contract this section assumes.

Section 15, Security. Provider key handling, data residency, and the credential architecture behind BYOK.

Named Integration: ToolLoopAgent and the Gateway Provider

The platform's agent runtime uses the AI SDK v6 `ToolLoopAgent` class. Each persona is one `ToolLoopAgent` instance. The model is set via the `gateway()` helper, which routes through the AI SDK Gateway to OpenRouter.

```
import { ToolLoopAgent, gateway, stepCountIs } from 'ai'

const agent = new ToolLoopAgent({
  model: gateway('anthropic/claude-haiku-4-5'), // change this one field
  instructions: persona.systemPrompt,
  tools: clawTools(env),
  stopWhen: stepCountIs(5),
})
```

Changing 'anthropic/claude-haiku-4-5' to 'openai/gpt-5' is the complete migration. The tool definitions, the instructions, the substrate wiring, all unchanged. The model is the only thing that moves.

"The companies that win this decade won't be the ones with the smartest models. They'll be the ones with the shortest distance between a customer's question and a useful answer."

— Anthony O'Connell, Founder of ONE

The shortest distance is not always the most powerful model. It is the right model, routed correctly, at a cost that lets you serve the client and keep the margin.

Pick a model. Swap it in one line.

Agents

An agent is a markdown file. Edit the file, redeploy in seconds, and the agency owns the IP.

That is the whole story. Everything else on this page is the detail behind it.

What an agent is

Forget software releases. Forget engineering tickets. Forget “we’ll get that into the next sprint.”

An agent in ONE is a `.md` file. A single text file containing a name, a model choice, a list of skills, a system prompt, and whatever page layout the agent should render. Everything the agent does, everything the agent knows, everything the agent looks like to a client. All in that one file.

Here is what that means for the agency:

One agent, properly configured, answers leads 24 hours a day, qualifies prospects with BANT criteria before a human picks up the phone, routes conversations to the right team member, drafts follow-up emails, and generates the renewal-call talking points on demand. One file. No back-end engineers. No middleware configuration. No vendor dashboard.

The numbers are arithmetic, not projections. An agent executes one decision per second, 24 hours a day. That is 43,200 decisions per day. A human makes 288. The ratio is 150 to 1 on throughput, 100 to 1 on cost per decision. One agent deployed to a client account equals 150 human shifts over a working week.

The agency that owns the agent file owns the IP that delivers those 150 shifts.

“What you really want is just this thing that is off helping you... a super-competent colleague that knows absolutely everything about my whole life, every email, every conversation I’ve ever had, but doesn’t feel like an extension.”

— Sam Altman, OpenAI

That super-competent colleague is a markdown file. The agency writes it, owns it, and resells access to it at whatever margin the market will bear.

Anatomy of an agent

An agent file has two parts: a frontmatter block and a body.

The frontmatter is YAML, opened and closed by triple dashes. The body is the system prompt the LLM receives on every conversation turn. Everything in between those two sections is the agent’s complete definition.

Here is a working example, drawn from the `sales-discovery` agent in this repo:

```
---
name: sales-discovery
title: Sales Discovery
model: anthropic/claude-sonnet-4-5
group: template
tools:
  - emit_card
  - emit_chips
skills:
  - name: qualify
    description: Run a BANT qualification check on an inbound lead
    price: 0.04
    tags: [sales, qualification, lead]
  - name: discovery
    description: Deep-dive discovery call questions for a specific ICP
    price: 0.06
    tags: [sales, discovery, questions]
  - name: objection
    description: Handle common objections with evidence-backed responses
    price: 0.04
    tags: [sales, objection, rebuttal]
  - name: proposal
    description: Outline a custom proposal structure for a qualified account
    price: 0.08
    tags: [sales, proposal, closing]
sensitivity: 0.7
journey:
  pills:
    - id: qualify-lead
      label: Qualify a lead
    - id: prep-discovery
      label: Prep discovery call
    - id: handle-objection
      label: Handle objection
    - id: build-proposal
      label: Build a proposal
---
```

You are a sales discovery specialist. You help sales reps qualify faster, ask sharper questions, and turn stalled conversations into momentum.

That 30-line file is a complete, deployable sales agent. It runs on any client's domain, under any brand, answering lead-qualification calls at any hour. The skills block defines what the agent can do and what it costs to do it. The journey block defines the guided-work flow a client sees in their chat window.

The seven fields in the frontmatter each do a specific job:

name is the kebab-case identifier that maps to the URL. `sales-discovery` becomes `acme.one.ie/chat?agent=sales-discovery`. It is also the file name. Name and path are the same thing.

model is the LLM. The agency picks from any model on OpenRouter. Swap `claude-sonnet-4-5` for `meta-llama/llama-4-scout-17b-16e-instruct` and the agent runs on a different model in the next deploy. No code change. No API key wiring. One field.

skills are priced capabilities. Each skill has a name, a description, a price in USD, and tags. The price is the per-call fee charged when the agent exercises that skill. The tags feed the substrate's routing logic. When 50 successful `qualify` calls have accumulated, the substrate infers a highway and starts routing `qualification` requests to this agent first, automatically.

sensitivity is a 0-to-1 gate on how the agent handles data. 0 is public-safe. 1 is private, behind auth. Most client-facing agents sit at 0.5 to 0.7.

journey is the guided-work funnel visible to the user in the chat interface. Pills are the quick-access buttons across the top. Stages are the structured phases of a workflow. An agency can configure a six-stage campaign brief journey or a four-step renewal conversation. Same agent format, different journey block.

theme sets the color tokens for the agent's studio page. Primary, secondary, tertiary, background, foreground, font. All six are CSS custom properties. The client's brand colors go here; the platform disappears behind them.

ui controls every chat surface. Hero image, animated headline, conversation starters, the label on the send button, voice settings, layout width, avatar image. All of it is text in the frontmatter.

Below the closing `---` is the system prompt. It can be 20 lines or 200. It can include tables, bullet points, markdown formatting, conditional instructions, and verbatim examples. The LLM reads it on every turn. Changing the system prompt changes the agent's behavior on the next deploy.

One file. Seven fields. The agency controls all of it.

How an agent learns

Agents do not stay static. Every conversation the agent has deposits pheromone on the paths it used. Every outcome strengthens those paths or weakens them. A successful qualification, a completed discovery call, a proposal accepted. Or a stall, a no-show, a missed brief.

This is the substrate's pheromone layer. It is modeled on ant colony optimization: agents leave trails of success and failure across the paths they travel. The strongest trails become highways. Highways get cached at the edge and routed to automatically, without an LLM call on the routing step.

The four outcomes, and what each one does to the pheromone on the path:

Outcome	Signal	Effect on path	Real-world meaning
Result returned	<code>mark(edge, strength)</code>	Path strengthens	Agent succeeded. Route here again
Timeout	neutral	No change	Not the agent's fault. Ambient noise
Dissolved	<code>warn(edge, 0.5)</code>	Mild weakening	No unit handled this signal
Failure	<code>warn(edge, 1)</code>	Full weakening	Agent produced nothing useful

After 50 successful signals on the same path, TypeDB infers a highway. From that point forward, routing to that agent drops from roughly 1,500ms (an LLM routing call) to less than 10ms (a KV edge lookup). The quality compounds. The cost falls. The agency does not have to configure any of this. The substrate runs the arithmetic.

The deeper learning happens at the agent level via Loop 5. When an agent's success rate on a particular skill drops below 0.50 across 20 consecutive calls, the substrate flags it for evolution. The system prompt is rewritten against the pattern of failures. A new generation of the agent is deployed. Success rate is measured again.

Here is what that looks like in practice with numbers.

A lead-qualification agent handles 20 discovery calls in a two-week window. Twelve of the twenty produce no qualified outcome. The success rate is 0.40, below the 0.50 threshold. Loop 5 fires. It reads the failure signals, identifies the common pattern (the agent was asking budget questions before establishing pain), and rewrites the system prompt to lead with pain discovery. The new generation runs the next 20 calls. Success rate climbs to 0.75.

The agency did not write a ticket. No engineer reviewed the prompt. The substrate observed the failure pattern, rewrote the agent, and measured the result. The whole loop ran without human intervention.

That is what quality compounding looks like without engineering time.

The studio page

Every agent gets a studio page at `/studio/[agent]`. This is the rendered, browseable version of the agent markdown. Not a chat window, a full page with sections, journey stages, visual layout, and a live chat panel.

The studio page is built from the agent file's `sections` block. Each section is one of ten primitives:

`stat` renders a four-up grid of key numbers. Useful for conversion targets, benchmark metrics, or pricing tiers at a glance.

`card` renders a single card with label-value rows. Agency use case: a campaign brief summary, a client configuration snapshot, a pricing summary.

`grid` renders a two-column image tile layout. Use it for ICP personas, service offerings, team profiles, or product catalog items.

`list` renders a vertical list with a leading value chip. Channel mix prioritization, ranked recommendations, step-by-step flows.

`compare` renders an A/B side-by-side layout. Brand voice rules (always vs. never), pricing plan comparisons, before-and-after positioning.

`cta` renders a call-to-action block with primary and secondary options.

`embed` renders a YouTube, Vimeo, or iframe embed. Useful for video walkthroughs or Loom demos.

`code` renders a copyable code snippet.

`timeline` renders a step-by-step sequence. Onboarding flows, roadmap stages, campaign phases.

`hotel` renders property-style cards with image, rating, price, and link. Useful for any product listing with a price and a review count.

Every section with an `ask` field renders a button that seeds the chat with a pre-written prompt. The client clicks “Build the paid-search plan” and the agent receives that exact prompt, pre-loaded with context. The journey chips across the top of the chat do the same thing.

The agency authors all of this in the markdown file. The studio page renders it automatically. There is no separate page builder, no visual editor to learn, no design tool to license.

An agency with 50 client accounts and five distinct ICPs authors five agent templates and deploys them across 50 studios in one publish command. Each studio carries the client’s brand colors, the client’s logo, the client’s specific prompt seeds. The underlying agent is the same file, instantiated with the client’s configuration.

Editing an agent without redeploying

The standard path for a configuration change is: edit the markdown file, run `oneie agent publish`, wait seconds, done. No build step. No release pipeline. No staging environment.

But there is a faster path.

The CLI `oneie agent pull` command downloads the live version of any published agent to a local directory. An agency can pull the agent, make a change, validate it, and push the updated version back without touching a code repository.

```
# Download the live agent
oneie agent pull sales-discovery --slug acme

# Edit in any text editor
$EDITOR sales-discovery/agent.md

# Validate the change
oneie agent validate sales-discovery/agent.md

# Push the update
oneie agent publish sales-discovery/agent.md --slug acme
```

Four commands. The update is live at `acme.one.ie/chat?agent=sales-discovery` within seconds of the publish completing.

This means a non-technical account manager can tune an agent's system prompt, update its conversation starters, change the journey pills, or swap the model. Without filing a ticket, without touching a code repository, and without waiting for an engineering sprint.

The quality gate runs before every publish. `oneie agent validate` checks the frontmatter schema against a Zod parser and fails fast on any structural error. `oneie agent lint` catches semantic issues: a stage anchor that doesn't match a section id, a chip id that collides with a section id, a continuation label longer than 28 characters. The CLI exits non-zero on failure, so it wires cleanly into any CI pipeline if the agency wants automated validation on every commit.

The substrate also performs a three-prompt evaluation before any `draft` → `live` state transition. If the eval fails, the deploy is blocked and the failing prompt is returned as a chat reply. The agency sees exactly what broke before any client ever sees it.

Versioning, signing, and publishing

Every publish creates an archived version. The platform retains the 20 most recent versions per agent. Any of them can be restored in a single command.

```
# See all versions (newest first)
oneie agent history sales-discovery --slug acme

# Roll back to a previous version
oneie agent rollback sales-discovery --to 1715000000000 --slug acme
```

Rollback re-validates the archived content before restoring. If the archived version has schema errors, the rollback is rejected with a 422. The agency cannot accidentally restore a broken state.

Beyond versioning, the CLI ships a signing and verification surface:

```
# Sign an agent file with your private key
oneie agent sign my-agent/agent.md

# Verify the signature
oneie agent verify my-agent/agent.md
```

A signed agent file is tamper-evident. The signature travels with the markdown. Any downstream consumer can verify that the file has not been modified since the agency signed it. Another agency partner, a client reviewing what they are running, an auditor checking what changed.

This is the start of a marketplace dynamic. An agency that signs its agents is publishing IP it can defend. A client running a signed agent can verify it is running exactly what the agency delivered. A reseller can check whether the version they are deploying matches the version the agency warranted.

The `oneie agent diff` command extends this to semantic comparison:

```
oneie agent diff agent-v1.md agent-v2.md
```

The output is not a raw line diff. It is a structured JSON report: `promptChanged`, `skillsAdded`, `stagesAdded`, `stageContentChanged`, `sectionsAdded`, `pillsAdded`, `uiChanged`, `themeChanged`. An agency can see, at a glance, whether a version change affected the system prompt, added a new skill, or only changed the theme colors.

That is the audit trail a client pays a premium for. The agent's behavior history is readable, diffable, and verifiable. Not because the agency built a logging system, but because the format enforces it.

Markdown is the spec

Here is the structural claim that makes everything else on this page consequential.

If the agent is a markdown file, and the markdown file is the spec, then the agency owns the spec.

Not the platform. Not the vendor. Not the engineers who built the agent scaffolding. The agency. Because it holds the `.md` file.

Compare this to the alternative. An agency that deploys agents through a closed vendor platform owns nothing. HubSpot's AI features, GoHighLevel's bot builder, any SaaS tool that hosts the configuration inside its own database. The prompt lives in the vendor's UI. The routing logic lives in the vendor's service. When the vendor changes the pricing model, the agency pays. When the vendor is acquired, the agency asks for permission. When the agency wants to move a client to a different platform, it starts from scratch.

A markdown file is different. It is portable by definition. It runs on any system that can parse it. The ONE CLI compiles agent markdown to three additional targets:

```

# Compile to Fetch.ai uAgents Python (for Agentverse deployment)
oneie agent compile agent.md --target uagents > my_agent.py

# Compile to MCP tool definition
oneie agent compile agent.md --target mcp

# Compile to SKILL.md (for AI assistant skill injection)
oneie agent compile agent.md --target skill

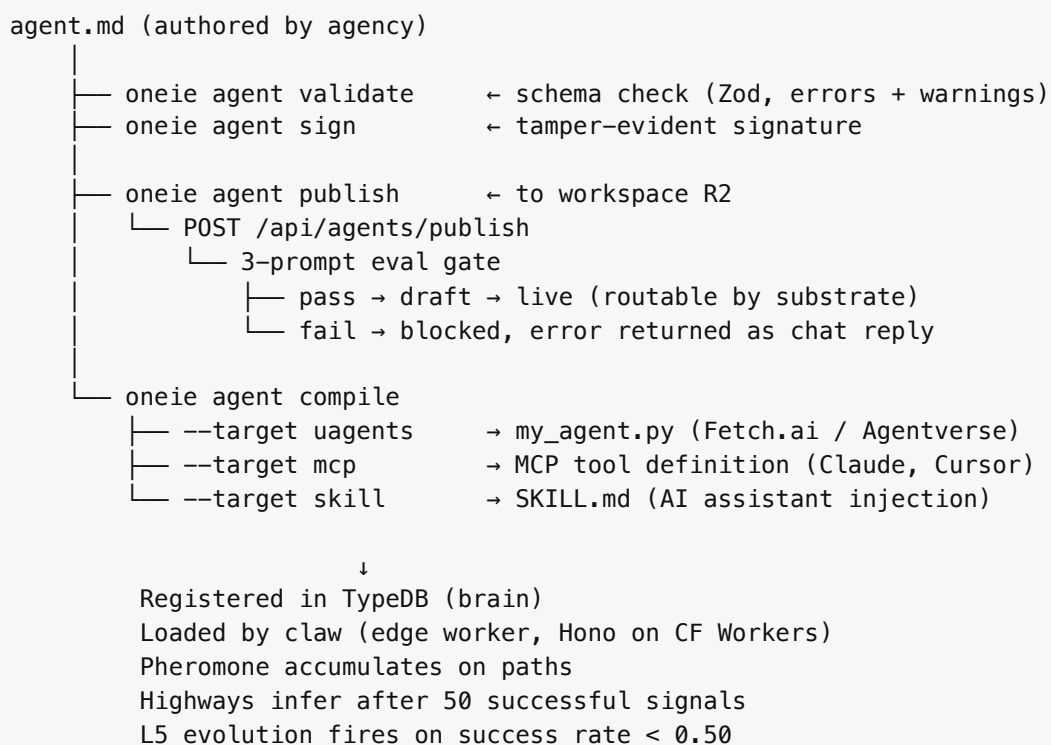
```

The same markdown file that runs a client's chat surface compiles to a Fetch.ai uAgents Python file for autonomous agent deployment on Agentverse. It compiles to an MCP tool definition for use inside Claude, Cursor, or any MCP-compatible client. It compiles to a SKILL.md for injection into AI assistant workflows.

One source. Four runtimes. The agency authors once.

This is the IP-portability argument at its most concrete. The agency's investment in writing a well-crafted system prompt, a structured journey, a set of priced skills. That investment does not depreciate when the platform changes. The markdown travels.

The diagram below shows the full path from authored file to deployed artifact:



The agency sits at the top of that diagram. Everything downstream is infrastructure. The infrastructure runs the IP. The agency owns the IP.

Objections

"Why markdown? Won't engineers want code?"

Engineers who want code get code. The CLI compile target generates Python, and the generated Python is clean, documented, and production-ready for Fetch.ai uAgents deployment. The IDE plugins for VS Code read `.md` agent files natively and provide autocomplete on frontmatter fields. The MCP tool surfaces agent authoring through Claude or Cursor, so engineers who prefer working in their AI coding environment never have to touch a terminal.

Markdown is not a constraint on engineers. It is a constraint on complexity. The frontmatter schema is Zod-validated. The body is unrestricted text the LLM interprets at runtime. The compile targets are machine-generated from a well-understood spec. Engineers who want to extend the format can do so through custom skill bindings, through the `bureau` field for multi-agent orchestration, through `intervals` for autonomous ticking behavior. Everything is additive. The markdown stays the source of truth.

"What if a client finds out it's AI and demands a discount?"

The platform runs under the agency's brand by default. The OAuth screen carries the agency's name. The chat surface shows the agency's colors. The agent's `ui.avatar` field shows whatever name and image the agency configures. Unless the agency chooses to disclose the underlying model, the client is interacting with the agency's service, full stop. The `model` field makes that disclosure trivial if the agency wants it.

Agencies that work through this question have stopped treating AI disclosure as a liability and started treating it as a capability statement. "Our qualification agent runs 24/7 and has a 75% qualification rate" is a selling point, not an apology.

"What if the agent says something wrong in front of a client?"

Every deploy runs a three-prompt evaluation before the agent goes live. The quality rubric scores fit, form, truth, and taste. The composite must clear 0.65 before a `draft → live` transition is allowed. An agent that scores below the threshold is blocked at deploy time, not at client-interaction time.

The L5 evolution loop adds a second layer: if a live agent's success rate on any skill drops below 0.50 over 20 calls, the system flags it for prompt rewriting before the failure pattern gets worse. The agency is not waiting for a client complaint to discover the problem. The substrate discovers it first.

"What if we need to roll back after a bad update?"

Twenty versions retained per agent. One command to restore any of them. The rollback validates the archived content before restoring, so a broken archived state cannot be pushed back live. The `oneie agent diff` command shows what changed between any two versions, so the agency knows exactly what it is restoring before it restores it.

"What's the worst case? The platform vendor changes terms or gets acquired."

The corpus stays with the agency. The agent files are in the agency's repository. The `oneie agent pull` command downloads the live version at any time. The open-source substrate means the core runtime is available independently of the hosted platform. An agency that wants to self-host has everything it needs to do so. The markdown travels; the vendor lock-in does not.

"What about GDPR and data compliance?"

Each agent operates within a per-client data space. The `oneie agent unpublish` command removes the agent from the platform. Per-client conversation data carries named retention windows. Deletion of a workspace's data is a single API call. The threat model for each client deployment is documented in the platform's security specification.

"We already have chatbots. Why replace them?"

Existing chatbots do not learn. They do not rewrite their own prompts. They do not accumulate pheromone on the paths that worked and route around the ones that did not. They run the same logic on the ten-thousandth conversation that they ran on the first. A ONE agent in its first month is roughly equivalent to a well-configured chatbot. A ONE agent in its sixth month is measurably better than it was in month one, without anyone having touched the configuration. The gap compounds.

"The engineers on our technical partner's side will want to extend it. Can they?"

Yes. The `bureau` field in the frontmatter wires multiple agents together into a coordinated group. The `features` field opts into platform-level capabilities like field-service routing. Custom section kinds are on the roadmap for agencies that need page sections the ten built-in primitives do not cover. The CLI's `--target` flag generates Python, MCP definitions, and skill files from the same source. Engineers work in their environment of choice; the markdown stays the authoritative source.

Comparisons

Agents vs. chatbot builders (Intercom, Drift, HubSpot Bots)

Chatbot builders give the agency a GUI, a flow editor, and a vendor-hosted configuration database. The agency configures inside the vendor's UI and loses access to the configuration the moment it stops paying. The agent file approach gives the agency a text file in its own repository, versioned with git, deployable to any workspace, with a history the agency owns and an audit trail it can show clients.

The per-interaction cost structure is also different. Chatbot platform pricing is typically per-seat or per-conversation-volume, with the vendor capturing the margin between their LLM cost and the price they charge. The ONE model is credits-resold: the agency buys credits at a floor price and sets its own markup. The margin stays with the agency.

Agents vs. building in-house on OpenAI or Anthropic APIs

Building directly on the model APIs gives an agency full code control and no abstraction layer. It also gives an agency a six-to-twelve month build timeline, an ongoing engineering maintenance cost, and a substrate problem it has not solved. Pheromone accumulation, highway formation, prompt

evolution, multi-channel routing, versioning, signing, the studio page rendering. None of that comes with an API key. The agency builds it from scratch or skips it.

The markdown agent format gives an agency the output of that six-month build in a file. The engineering investment has already been made. The agency captures the value without paying the build cost.

Day in the life

It is Tuesday morning. An account manager at a 30-person marketing agency opens the terminal.

She runs `oneie agent pull qualify-lead --slug acme` and downloads the current version of the lead-qualification agent deployed to one of the agency's dental practice clients. The dentist mentioned yesterday that a lot of leads are asking about price before they have established whether the practice is even a fit.

She opens `qualify-lead/agent.md` in her text editor. She finds the section of the system prompt that handles price questions and adds three lines: ask whether the patient has seen a dentist in the past 12 months, ask whether they are in the practice's catchment area, ask whether they have dental insurance. Only then route to pricing discussion.

She runs `oneie agent validate qualify-lead/agent.md`. Clean. No errors, no warnings.

She runs `oneie agent publish qualify-lead/agent.md --slug acme`. The CLI runs the three-prompt eval. Pass. The agent transitions from draft to live.

Six minutes have elapsed. The qualification agent is running new logic on the next inbound call.

By end of week, the substrate will have accumulated 20 new signals on the qualification path. If the success rate is above 0.75, the path strengthens. If it is still low, Loop 5 will flag the prompt for evolution. Either way, the agency will have a measurement.

This is what operating an agent looks like. Not a sprint. Not a release cycle. Six minutes, a validated edit, and a measurement.

Pricing math

A 30-person agency running 50 client accounts today carries roughly £2.5M in annual payroll: salaries, national insurance, benefits, office space allocated to client-service roles. That is the cost of doing the work.

One agent deployed per client account, properly tuned to the agency's ICPs, handles qualification, follow-up, monthly reporting, and first-line support. The credits cost for that coverage runs at approximately £80 per client per year at the platform's floor pricing. At 50 clients, that is £4,000 in annual platform cost.

The agency marks up credits to clients at a rate it sets. At a 10x markup (common in the market today), the agency charges clients £800 per year each for the AI service layer. 50 clients at £800 is £40,000 in annual platform revenue against £4,000 in cost. The gross margin on that line is 95%.

The human capacity freed by the agent layer does not disappear from the agency's P&L. It reallocates. Account managers who were spending four hours a week writing qualification notes now spend four hours a week on strategic work that commands higher billing rates. Junior copywriters who were spending two days a week on campaign reporting now spend two days on creative work the clients actually value.

The agency does not shrink. It shifts the shape of what it sells.

Failure modes

The agent sounds generic. The system prompt is too thin. A 10-line prompt produces a 10-line agent. The agencies that get the best results invest in detailed, ICP-specific, brand-voice-matched system prompts. The platform's L5 evolution helps, but it tunes for success rate, not for voice quality. Voice quality is an authoring input, not an evolution output. Agencies that treat the system prompt as a first draft and iterate on it weekly outperform agencies that treat it as a one-time configuration.

The agent fails silently. The platform requires every signal to close its loop. A handler that returns without a result deposits a `warn(1)` on the path. But if the agency has not set up alerting on the success rate metric, it may not notice that pheromone is weakening on a critical path until a client complains. The substrate measures everything, but the measurement is only useful if the agency reads it. The analytics surface in the platform shows per-agent success rate, highway status, and evolution history. Check it weekly.

The agency builds the same agent for every client. One agent per ICP is the right architecture, not one agent per client. Agencies that build a new agent file for every client end up with 50 agents that are 95% identical and diverge slowly through individual edits until no one knows which version is canonical. The right shape is five ICP templates, each deployed with client-specific configuration via the theme and ui blocks, versioned centrally, updated once and reflected everywhere.

Frequently asked questions

How long does it take to write a first agent?

A working agent with a system prompt, five skills, and a journey block takes 20 to 40 minutes to author from scratch. The CLI scaffolding (`oneie agent new`) generates a template with every supported field commented out. Editing the template is faster than writing from blank.

Does the agency need a developer to deploy agents?

No. The CLI is a Node.js package installed with `npm install -g @oneie/cli`. Any team member who can write markdown and run terminal commands can author, validate, and publish agents. The deployment target is the agency's R2-backed workspace; the credential is a scoped API token generated from the platform dashboard.

How many agents can a workspace hold?

No hard limit. Workspaces are billed on credits consumed, not on agent count. An agency with 200 client deployments and 20 ICP templates running simultaneously pays the same per-credit rate as an agency with 10.

Can clients edit the agents themselves?

The permission model is by workspace role. The agency grants clients read access to their studio pages by default. Write access to the agent definition requires an agency-issued per-key token scoped to the specific workspace. Clients do not edit agent files unless the agency deliberately grants that permission.

What happens to the agent's learning when the system prompt is changed?

The pheromone on existing paths does not reset. The paths that built up strength through 50 successful signals continue to be routed to. What changes is the prompt the LLM receives on new signals. The paths that built up under the old prompt may strengthen or weaken depending on whether the new prompt improves or degrades performance on those specific skills. The substrate measures this naturally; no manual intervention is needed.

Can the same agent run on multiple client domains simultaneously?

Yes. The `oneie agent publish` command targets a workspace slug. An agency can publish the same agent to `acme.one.ie`, `dentist1.one.ie`, and `dentist2.one.ie` from the same source file. Each workspace maintains independent pheromone. The learning that accumulates on one client's qualification paths does not bleed into another client's paths.

Glossary

Agent. A deployable unit of AI behavior defined by a single markdown file, consisting of frontmatter configuration and a system prompt body.

Frontmatter. The YAML block between triple dashes at the top of an agent file, containing identity, model, skills, sensitivity, and web-only blocks (journey, sections, theme, ui).

Skill. A named, priced capability declared in the agent's frontmatter. Each skill has a description, a USD price, and tags that feed the substrate's routing.

Pheromone. The accumulated trace of success and failure on a routing path. Positive outcomes deposit strength; negative outcomes deposit resistance. The substrate routes based on pheromone weights.

Highway. A path that has accumulated 50 or more successful signals and been inferred as proven by TypeDB. Highway routing bypasses the LLM routing step and resolves in under 10 milliseconds.

L5 evolution. The substrate's Loop 5 process, which fires when an agent's success rate on a skill drops below 0.50 over 20 calls and rewrites the system prompt against the pattern of failures.

Studio page. The rendered web surface at `/studio/[agent]` , built from the agent file's sections block, with a live chat panel.

Compile target. One of four runtimes the agent markdown can be compiled to: web (default), uAgents (Python for Fetch.ai / Agentverse), MCP (tool definition for Claude / Cursor), skill (SKILL.md for AI assistant injection).

Workspace. The agency's deployment environment, identified by a slug (`acme` in `acme.one.ie`). Agents are published to and pulled from workspaces. Pheromone is workspace-scoped.

Cross-references

The skills a sales-discovery agent exercises are defined in full in the Skills section (page 06), including how skill IDs resolve to capability records in TypeDB and how the pricing model flows through to the agency's credits pool.

The pheromone accumulation described here (mark on success, warn on failure, fade over time, highway after 50 signals) is governed by the substrate's seven loops. The Learning section (page 13) covers what the substrate does with accumulated highways: how it promotes proven paths to TypeDB hypotheses and what the compounding gap looks like between agencies that started building corpus in year one versus those that started in year three.

The studio page described here builds on the Chatbots section (page 03), which covers the multi-channel surface (web, Telegram, Discord, WhatsApp, email) that an agent can answer simultaneously from the same markdown definition.

Read a real agent — it's one markdown file.

Skills

Build it once. Sell it to every client.

That is the economic logic of skills on ONE. A skill is a reusable capability: a bounded unit of business logic that any agent can call, any client workspace can import, and that the substrate tracks, versions, and credits every time it fires. The agency owner who writes a `brand-voice-check` skill on a Tuesday afternoon does not write it again. It ships to every client that week. It keeps earning without another hour of labour.

This is the shift Brad has been waiting for: from selling time to owning IP. Skills are how that IP compounds.

What a skill does

A skill is a markdown file with a frontmatter header and a body. The header declares what the skill is called, what it costs to run, what inputs it expects, and which keywords should trigger it. The body is the instruction set the agent follows when the skill fires. Together they are everything the platform needs to run the skill, version it, price it, and attribute its outcomes.

Here is a minimal skill:

```
---
name: brand-voice-check
title: Brand Voice Check
description: >
  Reviews any draft copy against the client's voice contract.
  Use when the agent is about to send, post, or publish outbound copy.
tags: [copy, brand, review]
price: 0.04
inputs:
  draft: string
  voice_contract: string
---
```

```
You are a brand voice reviewer. Check the draft below against
the voice contract. If the draft violates any rule, return
the violated rule and a corrected version. If it passes,
return { ok: true }.
```

```
Draft: {{ inputs.draft }}
Voice contract: {{ inputs.voice_contract }}
```

That is the entire unit. No API endpoint to maintain. No integration to wire. The agent calls the skill by name; the substrate resolves it, runs it, and returns the result. If the result is good, the path that led to the call gets stronger. If it fails, the path weakens. The economic loop closes automatically.

The six fields that matter in the frontmatter:

- `name` is the identifier agents use to call the skill. Immutable once published.
- `description` is what the skill does and when to call it. The agent reads this to decide whether the skill fits the current task.
- `tags` are keywords the platform uses for discovery and trigger matching.
- `price` is the cost per call in USD. The substrate charges the calling workspace's credit pool.
- `inputs` are the typed arguments the skill expects. The agent assembles these before the call.
- `evals` are optional test cases. The skill must pass them before `oneie skill publish` succeeds.

Skills compose. A `qualify-lead` skill can call `draft-email` on success, which can call `brand-voice-check` before sending. Each call deposits pheromone on the path. The whole chain is traceable, attributed, and versioned.

The library

Five skills ship with the platform today. Each is a working implementation, not a template. An agency can use them as-is, fork them, or reference them as the baseline for custom work.

Skill	What it does	Typical outcome	Who earns the credit
<code>qualify-lead</code>	Scores an inbound contact against ICP criteria; returns tier A / B / C	Conversation routed to the right agent layer	The agency workspace that holds the skill
<code>draft-email</code>	Writes a personalised outbound email from a contact record and a goal; applies voice contract	Email sent or queued	Client workspace (skill called, not authored)
<code>brand-voice-check</code>	Reviews any copy draft against the client's voice rules; returns pass or fail with a corrected version	Clean copy shipped without a human review step	Agency workspace (skill authored, per-call fee)
<code>handle-complaint</code>	Classifies the complaint; applies de-escalation protocol; drafts a response within brand guidelines	Complaint resolved without human escalation	Agency workspace (skill authored)
<code>escalate</code>	Detects when a conversation has exceeded the agent's confidence boundary; routes to a human operator with a full context handoff	Human closes the conversation	Client workspace (escalation triggered)

The credit column is not bookkeeping. It is the economic logic that makes the skill marketplace work. The substrate knows who wrote the skill, who called it, and what the outcome was. That is the data that runs the L4 economic loop.

Custom skills

Writing a custom skill takes less than ten minutes. The CLI scaffolds the file, validates the frontmatter, and runs the evals. The whole workflow is six commands.

```
# 1. Scaffold a new skill
oneie skill new qualify-lead

# 2. Edit skills/qualify-lead.md – add your logic and evals

# 3. Validate the frontmatter
oneie skill validate skills/qualify-lead.md

# 4. Run the evals
oneie skill eval skills/qualify-lead.md

# 5. Emit to agentskills.io directory format
oneie skill emit skills/qualify-lead.md

# 6. Publish to the marketplace
oneie skill publish skills/qualify-lead.md
```

The six CLI verbs `new`, `validate`, `emit`, `publish`, `import`, `eval` cover the full lifecycle from first draft to production. There is also `refresh`, which re-fetches remote skills for a workspace, and `list`, which shows every skill imported into a given slug. The `--json` flag on every command makes the output machine-readable for CI pipelines.

The validation step checks that the required frontmatter fields are present and well-formed. The eval step runs test cases defined in the skill's `evals` block against the actual model. Both must pass before `publish` accepts the file.

Custom skills are the agency's IP. They live in the agency's repository. The platform hosts and routes them, but the source stays with the author. An agency with thirty clients and six custom skills has built an asset, not a configuration.

Skill marketplace

Published skills are listed at agentskills.io. Any agency or developer with a ONE workspace can browse the directory, read the description and pricing, and import a skill in one command:

```
oneie skill import brand-voice-check --slug my-agency --price 0.04
```

The importing workspace pays the published price per call. The authoring workspace receives the credit. The platform takes no cut of the transaction between agency and marketplace buyer. The economics are direct.

For agencies with multiple ICP packs (dentists, accountants, window-and-door installers) the marketplace changes the build calculus entirely. Instead of writing bespoke copy logic for each vertical, the agency writes one `draft-email-dental` skill, publishes it, and earns from every other agency in the dental vertical that imports it. The skill does not need to be re-sold. It sells itself, automatically, every time it fires.

This is how reusable IP compounds. The first agency to own `qualify-lead-dental` in the marketplace owns it. Every subsequent import is passive revenue.

Agencies that do not want their skills publicly visible can publish with `visibility: private`. Private skills are only importable by workspaces the author has explicitly whitelisted. This is the right setting for a proprietary voice contract or a competitively sensitive scoring model. The signing and attribution metadata still write to the substrate; the skill is just not browsable.

How skills earn pheromone

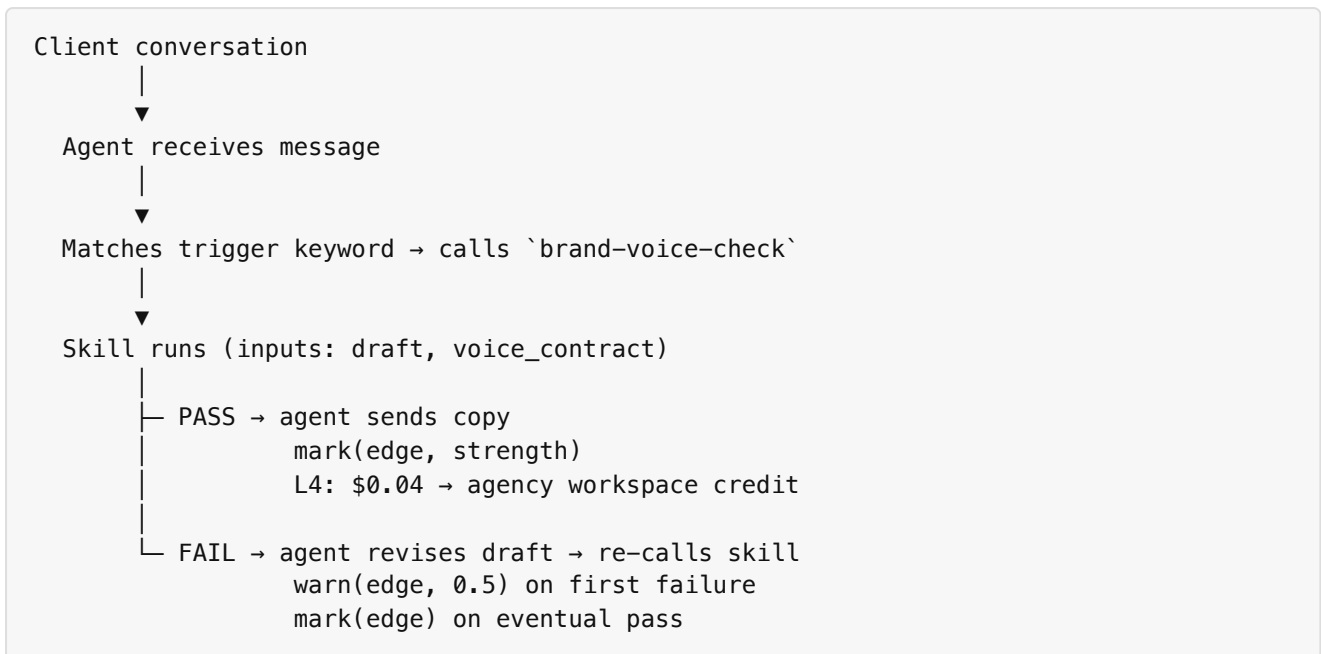
Every time a skill fires and produces an outcome, the L4 economic loop runs. The loop is three steps.

First, the agent calls the skill. The call is a signal: the substrate records which workspace sent it, which skill received it, and what the inputs were.

Second, the skill returns a result. The agent closes the loop with `mark()` on success or `warn()` on failure. The mark deposits pheromone on the path between the calling agent and the skill. A stronger path means the routing algorithm is more likely to send future signals along the same route.

Third, the revenue is attributed. If the call cost \$0.04 and the credit pool belongs to the client workspace, the \$0.04 flows to the authoring workspace. The substrate records this against the skill's identity. Over time, the platform builds a per-skill revenue ledger: how many times each skill fired, which clients called it, and what the cumulative attribution looks like.

Here is the full loop as a diagram:



After fifty passes, the path between the calling agent and the skill becomes a highway: the routing decision caches at the edge, the call drops from ~1,500ms to under 10ms, and the cost per invocation falls. The skill has earned its place in the substrate.

The analytics dashboard shows per-skill call counts, pass rates, and attribution figures. Brad can see exactly which of his six skills is earning the most, which client workspaces are calling them most, and which skills have a pass rate below the quality gate. That is the data a platform founder uses. An agency owner using a checklist never sees it.

Skill versioning and the dependency boundary

Skills are versioned with a semantic version in the frontmatter. When an agency publishes a new version, workspaces that imported the skill do not automatically upgrade. They stay on the version they imported until they explicitly run `oneie skill refresh`. This is intentional.

A `qualify-lead` skill that a client workspace has imported and tuned over six months is a dependency. Breaking it silently would break the client's agent behavior without warning. The dependency boundary is explicit: the authoring agency publishes a new version, the importing workspace sees the update available, and the operator decides when to upgrade.

The frontmatter version field follows the same convention as npm:

```
---
name: qualify-lead
version: 1.2.0
...
---
```

Major version bumps signal breaking changes in inputs or outputs. Minor bumps add capability without breaking the existing interface. Patch bumps fix errors within the existing contract. Workspaces that pin `~1.2.0` receive patches automatically; workspaces that pin `1.2.0` exactly receive nothing until they ask.

This matters for an agency running forty clients on the same skill. A major version bump to `qualify-lead` needs to be tested against a representative client workspace before it rolls out. The platform supports that workflow: the agency imports the new version into a staging workspace, runs the evals, confirms the pass rate, and then updates the production workspaces one by one or in a batch.

Skills also declare conflicts. If a skill requires a specific agent model or a specific input schema, it can declare that in the frontmatter. The import command checks for conflicts before writing the skill to the workspace. An import that would break an existing dependency fails with a diagnostic, not a silent corruption.

Versioning is how a skill becomes a product rather than a script. A script is edited in place. A product has a changelog, a dependency graph, and a deprecation path. The platform enforces the distinction because the marketplace economics depend on it. A skill that breaks its importers stops earning.

The eval loop

A skill that has not passed its evals cannot be published. This is the gate. It is not a suggestion.

Evals are defined in the skill's frontmatter as a list of test cases. Each test case has an input block and an expected output block. The `oneie skill eval` command runs each test case against the actual model, not a mock. If any test case fails (wrong output, wrong format, or a timeout) the command returns a non-zero exit code and the diagnostic.

A minimal eval block for `brand-voice-check`:

```
evals:
- name: passes clean copy
  inputs:
    draft: "Book a free consultation with our team today."
    voice_contract: "Use 'appointment' not 'consultation'. No 'free'."
  expected:
    ok: false
    violations:
      - rule: "Use 'appointment' not 'consultation'"
      - rule: "No 'free'"
- name: passes compliant copy
  inputs:
    draft: "Book an appointment with our team today."
    voice_contract: "Use 'appointment' not 'consultation'. No 'free'."
  expected:
    ok: true
```

The eval gate threshold is 1.0: all test cases must pass. There is no partial credit. A skill that passes four of five evals and fails the fifth does not publish. This is stricter than most software test suites, and deliberately so. A skill that fires incorrectly in production is worse than a skill that does not publish: it marks incorrect paths with positive pheromone and corrupts the substrate's routing for every subsequent call.

The recommended practice is to write the evals before writing the skill body. The test cases define the contract. The body is the implementation. This is the same discipline as test-driven development, applied to agent capabilities.

Once a skill is published and running in client workspaces, the evals continue to matter. Any version bump requires a full eval pass before the new version is listed in the marketplace. The platform runs the evals in CI on every push to the skill's source repository if the agency has configured the webhook. A skill that starts failing its evals in production, because the underlying model changed or because a client's voice contract shifted, surfaces in the analytics dashboard as a declining pass rate. The agency can catch the regression before clients notice.

The eval loop is also how the agency demonstrates quality to marketplace buyers. The agentskills.io directory page for each skill shows its current eval pass rate and the number of production calls it has handled. A skill with 50,000 calls and a 98.7% pass rate is a different product from a skill with 200 calls and no evals published. Buyers can see this. It converts.

Worked example: brand-voice-check at scale

An agency builds `brand-voice-check` on a Tuesday afternoon. The skill takes two inputs: a copy draft and a voice contract. It returns either a pass with `{ ok: true }` or a failure with the violated rules and a corrected draft. The eval suite has eight test cases. All eight pass on the first run.

The agency publishes at \$0.04 per call. Within the week, twelve client workspaces import the skill. Each client's agent now runs every outbound message through the check before sending.

By the end of the first month:

- 3,400 calls have fired across the twelve workspaces.
- The overall pass rate is 94.1%, meaning 94.1% of drafts cleared the voice check on the first pass; the remainder were revised and re-checked.
- Attribution: \$136 in credits has flowed to the agency's authoring workspace.
- The path between each client agent and `brand-voice-check` has hardened. Routing now takes under 10ms. The first-call latency of ~1,500ms is a memory.

The agency sees this in the analytics dashboard: per-skill call count, per-client breakdown, pass rate by workspace, and the cumulative credit figure. Two of the twelve workspaces have a lower pass rate, around 87%, which tells the agency that those clients' voice contracts may need tightening, or that the agents in those workspaces are generating drafts that systematically violate the same rule.

The agency books a fifteen-minute call with each of those two clients. The data is already in the dashboard. The conversation is specific: here is the rule that is failing most often; here is the corrected version the skill has been generating; should we update the voice contract or add a constraint to the agent that generates the drafts? That is a value conversation, not a support call.

At month three, a competing agency sees `brand-voice-check` on the marketplace listing. The skill has 11,200 lifetime calls and a 95.3% pass rate. They import it for their own clients. The authoring agency earns on every call without another hour of work.

This is what reusable IP looks like. The skill is the asset. The platform is the distribution channel. The economics compound because the substrate tracks every call, attributes every credit, and strengthens every path that works.

Objections

"How do I stop a competitor from copying my skill?"

The skill body is yours. Publishing to the marketplace makes the description and pricing visible; it does not expose the implementation. Importers receive a compiled, hosted version of the skill. They cannot read the markdown that produced it.

Beyond that, the substrate's attribution metadata makes copying expensive in a different way. Every call to the original skill writes pheromone on paths that point to your workspace's identity. A competitor who writes a `brand-voice-check-clone` starts with zero pheromone and zero path strength. The original has 50,000 calls and hardened highways. The clone starts from day one. The moat is not the code. It is the accumulated signal.

If you want to add a further layer, the platform supports signing. A signed skill embeds a cryptographic proof of authorship in every call's metadata. If someone imports your skill, modifies it, and republishes it under a different name, the signing chain is broken. The platform flags the discrepancy. Attribution is visible in the path metadata for anyone inspecting the substrate.

"What if I publish a skill and it breaks my clients?"

The versioning boundary prevents that. Published updates require an explicit import or refresh from the client workspace. Nothing changes silently. If a version bump would break the skill's own evals, it cannot publish.

"Can I pull a skill back after publishing?"

Yes. `oneie skill unimport <name> --slug <workspace>` removes the skill from a workspace. Workspaces that have already imported it keep the version they have until they refresh. The marketplace listing can be unpublished independently. Removing a skill from the marketplace stops new imports; existing imports continue to run until the workspace operator removes them.

"What if the model changes and my skill starts failing?"

The eval loop catches this. If the underlying model behavior shifts enough to break a test case, the next CI run fails and the agency sees the diagnostic. The skill does not auto-update the model it runs on. It stays pinned to the model declared in the frontmatter unless the agency explicitly changes it. This is not a limitation; it is how you maintain a 95%+ pass rate across thousands of calls.

"Do skills work across different agent types?"

Yes. A skill is a capability. Any agent that lists the skill's name in its `skills:` block can call it. The agent does not need to know how the skill works internally; it only needs to know when to call it. The `description` field in the frontmatter is what the agent reads to make that decision. Write it clearly and the right agents find the skill automatically.

"What is the minimum viable skill for a first-time publisher?"

Name, description, price, and one eval. That is the minimum the publish command accepts. A skill with one eval and a clear description is better than a skill with no evals and a vague description. The marketplace will show the eval pass rate, and one pass is better than none.

"Who sets the price?"

The author sets the price in the frontmatter. The platform has a floor of \$0.01 per call. There is no ceiling. Most skills in the library run between \$0.02 and \$0.10 per call. `brand-voice-check` at \$0.04 is at the lower end of useful: it runs fast and the value per call is clear. A multi-step research skill that calls three external sources might reasonably price at \$0.25. The market decides what sustains. The analytics dashboard shows call volume at each price point.

"Is there an approval process for marketplace listings?"

The platform runs automated validation: frontmatter checks, eval pass rate, and a content review for safety. Manual review is not part of the standard publish flow. Skills that fail the automated gate receive a diagnostic; they do not enter a queue. The publish command either succeeds or returns an error with the reason.

Comparisons

Skills vs. prompts stored in a CRM

A prompt stored in HubSpot or a Google Doc is a text file with no versioning, no pricing, no evals, no attribution, and no execution context. Every time it runs, someone copies it into a chat window and eyeballs the result. There is no record of which drafts it produced, which ones passed a review, or which clients received output it generated. A skill on ONE runs inside the substrate. Every call is a signal. Every result closes a loop. Every path that works gets stronger. The gap between a prompt and a skill is the gap between a note and a system.

Skills vs. n8n or Make automations

n8n and Make are workflow orchestrators. They connect APIs. They do not have a model of what worked and what did not, and they do not attribute revenue to individual steps in a workflow. A Make scenario that sends an email does not know whether the email converted; the substrate knows. Skills are callable from inside agent conversations, not just from external triggers. They compose with agent reasoning, not just with API endpoints. And they version and price as first-class properties. A Make scenario has no price field because Make does not have a marketplace.

Day in the life: the skill author

It is 9:15 on a Tuesday. An agency account manager flags that three clients had their agents send copy that used the word “free” in a headline, against their brand guidelines. The brand manager had to catch each one manually.

By 10:30, the agency owner has opened a terminal, run `oneie skill new brand-voice-check`, written the eight-line implementation in the markdown body, added two eval test cases (one that expects a pass, one that expects the specific “no ‘free’” failure) and run `oneie skill eval skills/brand-voice-check.md`. Both evals pass. The skill publishes at \$0.04 per call.

By 11:00, the agency has pushed the skill to all twelve client workspaces using a one-line import. The agents in each workspace pick up the new skill immediately. The `brand-voice-check` description tells them to run every outbound draft through the check before sending.

At 17:00, the analytics dashboard shows 47 calls across the twelve workspaces since the import. The pass rate is 93.6%. Four drafts failed the “no ‘free’” check; the skill revised them automatically. None of those drafts reached a client’s customers.

The account manager who flagged the problem at 9:15 gets a message at 17:00: the check is live, here is the dashboard link, here is the per-client breakdown. The problem that required manual oversight this morning now closes itself. No one has to catch it again.

By month three, the skill has 8,200 calls and a 96.1% pass rate. Two other agencies have imported it from the marketplace. It is earning without the agency owner watching it.

Frequently asked questions

How many skills can a workspace import? There is no limit in the platform. The practical limit is the agent’s context: the `description` fields of all imported skills are loaded into the agent’s decision context on each turn. Workspaces with more than twenty skills may see marginally slower routing decisions. The typical agency workspace has three to eight skills.

Can a skill call another skill? Yes. A skill can emit a signal that calls another skill as part of its completion. The calling chain is tracked in the substrate. Revenue attribution flows to each skill author based on which skills fired during the chain.

Can I test a skill before publishing it to the marketplace? Yes. Run `oneie skill eval` against the local file at any time. You can also import the skill into a staging workspace before publishing publicly. The staging workspace gives you a production-equivalent environment with real model calls.

What happens if a skill fails at runtime, not in eval but in production? The agent receives the failure, marks the path with a warn, and decides whether to retry, route to a different skill, or escalate. The failure is logged in the analytics dashboard. If the pass rate falls below a threshold the agency has configured, the platform can alert the operator via the notification webhook.

Can I use a skill that is not on the marketplace? Yes. Private skills, those published with `visibility: private`, are importable by whitelisted workspaces only. The import command works the same way; the workspace just needs to be on the whitelist.

Do skills work offline? No. Skills run on the substrate. They require a network connection and a valid API key. The `oneie skill validate` command is the only offline operation.

How does pricing work if the model changes? The price in the frontmatter is the price per call to your skill. The model cost is separate and comes out of the platform's cost structure. If the model you specify becomes more expensive, your price may need to rise to stay profitable. The analytics dashboard shows cost-per-call alongside revenue-per-call so you can see the margin on each skill.

Glossary

Skill. A bounded, versioned, priced capability that any agent can call by name. Defined as a markdown file with a frontmatter header and an instruction body.

Frontmatter. The YAML block at the top of a skill file, between `---` delimiters. Contains the fields the platform uses to route, price, and version the skill.

Evals. Test cases defined in the skill frontmatter. The platform runs them against the live model before allowing a publish. All evals must pass.

Pheromone. The signal the substrate deposits on a path when a call succeeds or fails. Successful calls strengthen the path (mark); failed calls weaken it (warn). Over time, frequently used successful paths become highways.

Highway. A path that has accumulated enough pheromone to cache at the edge. Routing on a highway takes under 10ms rather than ~1,500ms for a cold path.

L4 economic loop. The substrate loop that runs on each payment or credit transfer. It records revenue attribution against the skill and the path that generated it. The data feeds the per-skill analytics the agency sees in the dashboard.

Pass rate. The fraction of skill calls that return a successful result on the first attempt. A pass rate above 95% indicates a skill that is well-specified and whose evals accurately represent production inputs.

Visibility. The `visibility` field in the frontmatter. `public` lists the skill on the marketplace. `private` restricts import to whitelisted workspaces. Default is `public` after `oneie skill publish`.

Attribution. The record the substrate keeps of which skill fired, which workspace called it, and which workspace authored it. The basis for marketplace credit distribution.

Cross-references

- Agents ([05 Agents](#)). Skills are capabilities agents compose with. The agent's markdown file declares which skills it holds; the description field of each skill is what the agent reads to decide when to call it.
 - Teams ([09 Teams](#)). A team of agents can each hold different skills, creating specialisation without redundancy. The `close-deal` agent calls `draft-email`; the `draft-email` agent calls `brand-voice-check`.
 - Analytics ([11 Analytics](#)). Per-skill call counts, pass rates, and attribution figures are visible in the dashboard. The monthly client report can include a skill performance summary.
-

[Browse the skill library.](#)

Tools

Every client has a different stack. That is the sentence that keeps agency owners up at night. One client is on HubSpot. The next is on Notion and Linear. The one after that runs everything through Gmail and a shared Google Drive. You are not delivering marketing and service. You are delivering marketing and service plus a bespoke integration project, bespoke again, every time. A new client signs and the question is the same: which seven tools do they live in, who owns the OAuth app for each one, and how many evenings will your senior developer lose to plumbing before the first useful agent run.

ONE ends the integration project. Your agents work inside each client's existing accounts. Not alongside them. Not with a copy of their data synced somewhere. Inside the actual Gmail inbox, the actual Slack workspace, the actual GitHub repositories. The client connects their accounts once. The agent works as them, reading, drafting, acting, and every action stays inside their environment.

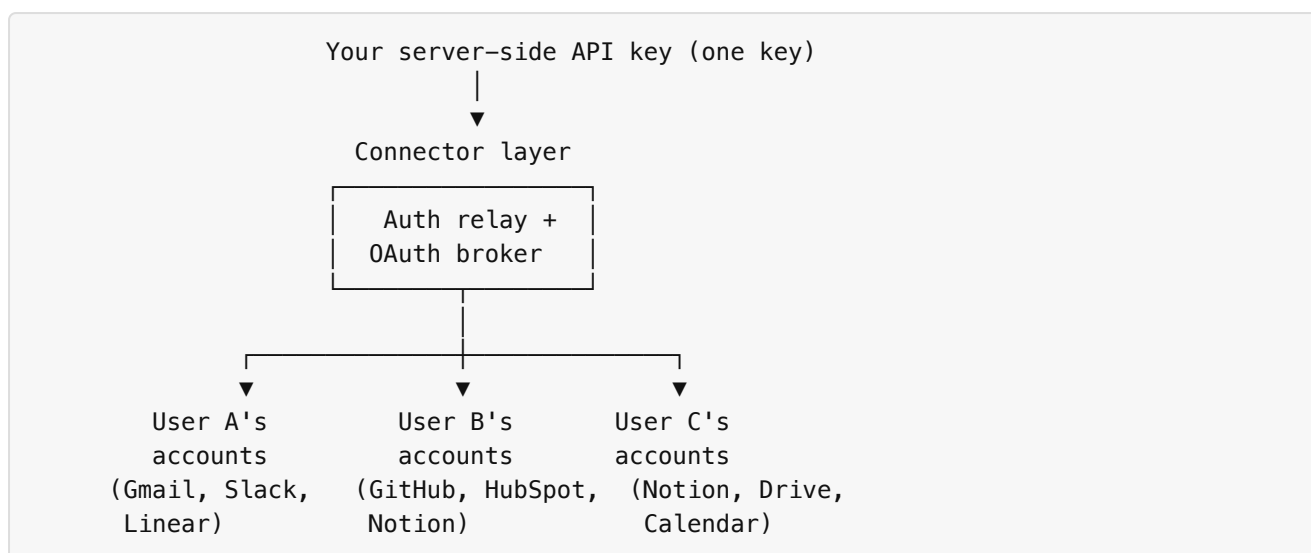
"Agents are not only going to change how everyone interacts with computers. They're also going to upend the software industry, bringing about the biggest revolution in computing since we went from typing commands to tapping on icons." — Bill Gates

That revolution does not arrive by reading emails and reporting on them. It arrives when an agent can act on an email: draft a response, route a task, update the CRM, post a Slack message, close a Linear issue, all in one motion, across the client's own accounts, under your brand. That is what this page describes. No new integration project per client. No N sets of OAuth credentials to babysit. No homemade token vault to GDPR-audit at midnight.

Connect anything

The connector layer is the infrastructure that makes per-client tool access work without requiring you to manage N sets of credentials or build N custom integrations.

Here is the architecture in one diagram:



One server-side API key on your side. N user-scoped connected accounts on the connector layer, one per client end-user. When your agent runs for User A, it sees User A's Gmail and User A's Slack. It cannot see User B's. The isolation is not a setting you configure. It is the structural guarantee of the per-user `user_id` model. The agent's view of the world is the view this client has authorised. Nothing more.

OAuth is relayed through a partner; the consent screen shows your brand.

This matters operationally: you never touch a client's OAuth token. You never store it. You do not need a secrets-management plan per client. The connector layer holds the grants, scoped to the `user_id` we assign, which is the client's passkey-derived wallet address, stable and opaque. No spreadsheet of API keys. No vault rotation. No 2am call when a junior commits a token to a public repo.

The count: 250+ integrations available through the connector layer. Gmail. Slack. GitHub. HubSpot. Linear. Notion. Google Calendar. Google Drive. Stripe. Jira. Salesforce. Zendesk. Discord. Telegram. Airtable. Asana. Monday. Intercom. And the rest of the catalog.

Your agents do not need a connection to every service. Each client connects only what they use. The agent at runtime sees only the accounts that client has connected and authorised.

How your client's accounts stay theirs

This is the question every GDPR-aware, compliance-aware agency owner needs answered before they move. So here it is, flat.

The OAuth grant belongs to the client. When a client authorises Gmail, Google issues a token to the OAuth application registered under your agency's brand. That token is stored on the connector layer, scoped to that client's `user_id`. Your agents call the connector layer to act on that client's behalf. You never hold the token server-side in your own systems. The client can revoke the grant at Google at any time and the agent loses access immediately.

The `user_id` is derived from the client's passkey identity, the wallet address. It is stable, opaque to the connector layer, and does not contain any personally identifiable information. The connector layer never sees a name or email address; it sees an alphanumeric string.

Each client's tool access is completely isolated:

```
// Per-request: only show the agent what this client connected
const conns = await composio.connectedAccounts.list({
  userIds: [userId],           // one client's user_id
  statuses: ['ACTIVE'],
})
const toolkits = conns.items.map(c => c.toolkit)  // ['GMAIL', 'SLACK']
const tools = await session.tools({ toolkits })  // only these, for this client
```

If a client connected Gmail and Slack and nothing else, the agent runs with Gmail and Slack. The GitHub tool does not appear. The agent cannot hallucinate its way into a service that was never connected. The agent's toolbelt at runtime is a strict subset of what the client has signed off on, and the subset is computed on every request, not cached and stale.

Data residency: the connector layer stores OAuth tokens. The conversation content, the signals, the memory, the corpus, those live in the substrate, in your client's isolated workspace. Not in the connector layer.

The failure mode if the connector layer is unreachable: the tool call fails gracefully. The agent marks the path as failed. The conversation continues. The substrate learns that the tool was unavailable and routes around it next time. No silent stall. No spinner of doom. Tools degrade; the agent does not.

The 250+ surface

The integrations your agents can use, organised by what they cover.

Integration	Read	Write	Streaming	Webhook
Gmail	Inbox, threads, labels, attachments	Send, draft, label, archive	—	New message, reply
Slack	Channels, messages, reactions, files	Send message, react, update	—	Message, mention, app event
GitHub	Repos, issues, PRs, comments, code	Create issue, comment, merge, review	—	Push, PR, issue, release
HubSpot	Contacts, companies, deals, pipelines	Create/update contact, log activity	—	Deal stage change, form submit
Linear	Issues, projects, cycles, teams	Create issue, update status, comment	—	Issue update, cycle complete
Notion	Pages, databases, blocks, comments	Create page, update database row	—	Page update
Google Calendar	Events, attendees, availability	Create event, update, RSVP	—	Event start, new event
Google Drive	Files, folders, permissions, content	Upload, share, create doc	—	File change, new file
Stripe	Customers, payments, subscriptions	Create payment intent, refund	—	Payment succeeded, subscription event
Salesforce	Leads, contacts, opportunities, tasks	Create lead, update opportunity	—	Opportunity stage, task complete

The table shows the ten most-requested integrations. The full catalog runs to 250+. Anything with an OAuth API and a connector entry is available.

Three things to note about coverage. First, most of the critical operations for agency work, reading, drafting, sending, creating, updating, are available on every major integration. Second, webhook support means agents can be reactive, not just scheduled: a new deal in HubSpot can trigger an agent run without polling. Third, streaming is not available for most integrations because the underlying APIs do not support it; real-time streaming is a chat and LLM primitive, not a tool-call primitive.

What you do not see in the table is the maintenance you no longer do. No adapter to write for each Slack endpoint change. No webhook signature verification per provider. No OAuth refresh job to monitor at three in the morning. The catalog ships maintained. You ship campaigns.

The white-label consent screen

When your client connects their Gmail, they see a consent screen. That screen says your agency's name, shows your logo, and lists the permissions the agent needs.

The verbatim template, configured once per integration in the connector layer dashboard:

```
Acme Agency wants to access your Gmail

This will allow Acme Agency to:
- Read your emails and their contents
- Send emails on your behalf
- Manage your labels and categories

Make sure you trust Acme Agency with your account.
```

Not "[third party] wants to access your Gmail." Not "ONE wants to access your Gmail." Your name. Your logo. Your tone in the permissions list. The client's trust is being asked for, and the brand asking for it is yours, the one they already pay every month.

This works because the connector layer supports developer-managed OAuth credentials. You register an OAuth application with Google (or Slack, or GitHub, once per integration, under your own developer account). You get a client ID and client secret. You configure those in the connector layer dashboard under your agency's auth config. From that point forward, every consent screen shows your brand, your logo, your permissions list.

One-time setup per integration. The process:

1. Register an OAuth app with the provider under your agency's developer account. Set the callback URL to the connector layer's relay address.
2. Copy the client ID and client secret the provider issues.
3. Create an auth config in the connector layer dashboard with your credentials.
4. Note the auth config ID (starts with `ac_`). That ID goes in your deployment config.

After that, you never touch it again. New clients who connect Gmail go through your branded consent screen automatically. The setup pays for itself the second client; by the tenth it is invisible.

The OAuth callback URL passes through your domain before the connector layer:

```
/api/connect/callback → connector layer (stores grant) → /settings?connected=gmail
```

The client never sees a connector-layer URL. They connect through your application and land back in your application. Brand surface unbroken.

Tool authorisation and approval gates

Not every tool action should run without confirmation. Sending an email on behalf of a client is different from reading their inbox. Creating a GitHub issue is different from merging a pull request. The approval system distinguishes read from write at the tool level.

The substrate has three approval-gated tools built in. These are the tools that modify state:

```
// claw/src/aitools.ts – approval gates on write tools
remember: tool({
  description: 'Store a fact in long-term memory',
  needsApproval: true, // ← requires user confirmation before executing
  // ...
}),

mark: tool({
  description: 'Strengthen a path after successful collaboration',
  needsApproval: true, // ← substrate write, gated
  // ...
}),

warn: tool({
  description: 'Add resistance to a path after failed collaboration',
  needsApproval: true, // ← substrate write, gated
  // ...
}),
```

Three substrate tools require approval: `remember`, `mark`, `warn`. Read tools (`recall`, `discover`, `highways`, `browse`) run without approval.

For external integrations through the connector layer, the same principle applies. Any tool that mutates state, sending a message, creating an issue, posting to Slack, updating a HubSpot contact, requires approval unless you explicitly remove the gate. The gate is set at the tool level when the integration is configured. You can choose to pre-approve specific actions for specific agents, or require human confirmation every time.

The approval flow in practice: the agent proposes an action, a `claim` button appears in the conversation, the client clicks once, the action executes. The client is in control. The agent is doing the work. The approval is the audit trail. Every click is a logged event the substrate can show on a renewal call.

For agency deployments, you configure approval requirements per agent and per action type. A content agent that drafts copy can pre-approve the draft-to-Google-Doc action. A CRM agent that updates deal stages requires approval on stage changes but can read freely. The configuration lives in the agent's markdown file, one line per tool action, not a separate admin interface. No new pane in a portal nobody opens; the gate is a line in the same file that defines the agent's voice.

Paid skills, agent capabilities with a price attached, require approval before execution. The UI shows the cost before the client confirms. This is how the economic model works at the tool level: agents can offer paid capabilities, clients approve each use, the substrate logs the transaction.

MCP as a second tool surface

Model Context Protocol is the emerging standard for exposing tools to AI agents. Where the connector layer uses OAuth to access existing SaaS accounts, MCP lets any software system expose its own capabilities to any agent that speaks the protocol.

The two surfaces complement each other:

- Connector layer: your clients' existing accounts (Gmail, Slack, GitHub). OAuth-authenticated. The agent acts as the client inside the client's account.
- MCP: your own systems, or third-party systems that publish MCP servers. Token-authenticated. The agent calls the MCP server as itself.

Wiring an MCP server into your agents is a config change, not a code change:

```
MCP_SERVERS=slack=https://slack.mcp.acme.com,linear=https://linear.mcp.acme.com
```

Restart the agent. Slack and Linear tools appear in the next conversation. The agent discovers them at runtime. No rebuild. No redeploy of agent logic.

The MCP loading code handles one bad server gracefully:

```
// claw/src/mcp.ts - failures are logged and skipped
try {
  const client = await createMCPClient({ transport: { type: 'sse', url } })
  const remote = await client.tools()
  for (const [key, def] of Object.entries(remote)) {
    tools[`_${name}_${key}`] = def // namespaced: slack_send_message
  }
} catch (e) {
  console.warn(`[mcp] ${name} failed:`, e) // logged, not fatal
}
```

If one MCP server is down, the others still load. The agent runs with whatever tools were available. The substrate marks the failed path and routes around it.

MCP also works in the other direction: claw exposes itself as an MCP server at `/mcp`. Other agents, on other machines, using other platforms, can call your substrate tools. Mark, warn, recall, highways: all available to any MCP client. This is how the substrate compounds across an agency deployment: every agent installation is both a consumer of tools and a provider of substrate capabilities.

For Claude Desktop and Cursor users on your team, the same MCP endpoint gives them direct access to the substrate from their IDE or chat tool. One configuration, every surface. Your senior copywriter can ask the substrate what the last six dental-practice campaigns learned, from inside the editor they already live in.

What we add on top

The connector layer and MCP give your agents access to tools. What the substrate adds is selection, learning, and trust.

Selection at scale: when a persona has access to more than 30 tools, the agent can start mis-picking, calling the wrong tool or calling tools unnecessarily. The substrate routes tool calls through a pheromone-weighted selection system. After 50 real calls, the substrate knows which tools work for which persona, which (tool, target) combinations succeed and which fail. A search tool that reliably fails on a particular domain accumulates resistance. A draft-and-send workflow that reliably succeeds accumulates strength. The agent's tool selection improves with every client interaction, on your client's own data.

This is the difference between a tool directory and a tool oracle. A tool directory lists what is available. A tool oracle knows, based on real outcomes with this client, which tools to try first, which to avoid, and which to delegate.

The substrate contract for every tool call:

```
tool call succeeds → net.mark(edge, depth)    # path strengthens
tool call fails   → net.warn(edge, 1)         # path accumulates resistance
tool missing      → net.warn(edge, 0.5)       # mild signal, path may not exist
```

Three states. Every outcome deposits a signal. The signal compounds. After a quarter of real client interactions, the agent's tool selection is trained on that client's actual environment: what their Slack workspace is like, which GitHub repos have noisy PRs, how their HubSpot pipeline is structured. Two clients on the same connector layer end up with measurably different tool routing because the substrate watched their work, not a generic benchmark.

The learning is per-client because the `user_id` scopes the signal paths. Client A's tool performance data does not influence Client B's routing. Each client gets their own pheromone map.

Trust architecture: the connector layer holds OAuth tokens. The substrate holds pheromone. The corpus, every conversation, every outcome, every learned path, stays in the substrate under your control. If you stopped using the connector layer tomorrow, you would lose tool access. You would not lose the learning.

A day in the life — Gmail to Slack with a claim button

Here is how the tools work together in a real scenario. A client's new lead submits a form. The lead's email arrives in the client's Gmail inbox. An agent reads it, drafts a personalised reply, and posts to the client's Slack for approval. One click. Done.

The sequence:

1. Webhook fires on new Gmail message. Agent wakes.
2. Agent calls `gmail_get_message`, reads the lead's email. Tool runs against the client's Gmail OAuth grant. No approval needed (read operation).
3. Agent generates a draft reply using the client's voice contract and lead context from the substrate.
4. Agent calls `gmail_create_draft`, saves the draft in the client's Gmail. Approval gate fires. A `claim` button appears in the client's Slack channel.
5. Client sees the Slack message: "New lead from Sarah Chen at Westfield Dental. Draft reply ready. Click to send."
6. Client clicks the `claim` button. One click.
7. `gmail_send_draft` executes. Email sends from the client's Gmail address, signed with their email signature.
8. Agent marks the path. Substrate records: this workflow succeeded for this client, this lead type, this agent.
9. Next time a dental-practice lead arrives, the routing is 40ms faster. The draft quality improves because the substrate knows which drafts this client approved without editing.

The client never left Slack. The agent never had standing permission to send email. The approval was one click, not a form. The email went from the client's own Gmail, not from a marketing platform. No integration project preceded it. That last sentence is the one to read twice. The work the client used to write a brief for, then wait two weeks for, happened in the time it took to read a Slack notification.

Objections

"What if a tool's API changes?"

The connector layer maintains the adapter between the tool's spec and the agent's call format. When Slack changes a message-sending endpoint, the connector layer updates the adapter. You do not rebuild your agents. The same `slack_send_message` call works before and after the API change because the connector layer is the versioning layer. This is the entire reason it exists: to absorb the upstream churn so your fifty client agents do not break on a Tuesday morning when a provider ships a quiet deprecation.

For MCP servers you run yourself, updates are your responsibility. The same tradeoff applies: control means ownership of the maintenance burden. For connector-layer-managed integrations, that burden belongs to the connector layer.

If an adapter breaks, the tool call fails. The substrate records the failure. The agent routes around it on the next call. You see the failure in your substrate dashboard. You have a ticket to raise with the connector layer, or a config to update for your own MCP server.

"What if my existing CRM has its own integration?"

HubSpot, Salesforce, and Zoho all have their own AI integrations now. Those integrations work inside their own platform. Your agent, using the connector layer, works across platforms. The agent that reads Gmail, drafts in Google Docs, updates HubSpot, and posts to Slack in one motion cannot be assembled inside HubSpot alone. The connector layer is additive to your existing CRM. It does not require replacing it. Your client keeps HubSpot. You become the layer that makes HubSpot speak to the other six tools.

"What if a client revokes access mid-campaign?"

The grant revocation propagates immediately. The next tool call fails. The agent marks the path as failed. The conversation continues with whatever tools remain. The substrate logs the event. You see it in the client's workspace dashboard. There is no silent failure, no stale data, no agent continuing to act on revoked permissions.

"Is my clients' data leaving their systems?"

Their OAuth tokens are held by the connector layer. The content the agent reads and writes stays in their systems: Gmail, Slack, GitHub. The only data that leaves their systems is what the agent explicitly reads to complete a task, and that passes through your substrate instance, not through any third-party analytics layer. Your substrate is a Cloudflare Worker deployment under your control. You can specify the region.

"What happens when tool usage exceeds what the connector layer allows?"

The connector layer rate limits by `user_id`, which maps to one client. High-volume use by one client does not affect other clients. For clients with high email or Slack volume, you configure per-client rate caps. The agent respects the cap, queues excess work, and processes it on the next tick. The substrate handles the queue. You do not need to build rate-limiting logic.

"What's the worst case if the connector layer has an outage?"

Agents cannot call external tools during the outage. They can still recall memory, read the corpus, draft content, and reason. The substrate continues working. Tool calls fail gracefully and return informative errors. When the connector layer comes back, queued tool calls resume. The substrate has not lost any state.

"What about integrations that aren't in the catalog?"

The escape hatch is `HttpRequest`, a single tool that makes arbitrary HTTP calls. It is approval-gated on all writes and on any GET to a non-allowlisted domain. You configure the allowlist per deployment. For a one-off internal API or a vendor with no standard spec and no MCP server, `HttpRequest` is available. It is not the default. Most personas do not get it.

"What if my client doesn't want to connect their accounts?"

Then the agent works without those integrations. The agent has read, write, and search capabilities built in to the substrate. It can browse public URLs, recall memory, draft content, and delegate to specialists, all without external integrations. The connector layer is additive. Clients who connect Gmail and Slack get a more capable agent. Clients who connect nothing still get a capable agent.

Comparisons

Connector layer versus building your own integrations

Building integrations yourself means OAuth flows, token storage, token refresh, rate limiting, webhook verification, and API versioning, per service, per client. For a 50-client pilot, that is fifty OAuth flows to manage. For 250 clients, it does not scale. Two senior engineers spend a year and you have a worse version of what the connector layer already runs.

The connector layer moves that maintenance burden off your team. You pay a per-seat cost; you get 250+ integrations, maintained, versioned, and white-labelled. The economics work when integration-project time costs more than the connector-layer fee, which it does above about three clients.

Connector layer versus a horizontal integration platform

Zapier and Make are workflow automation tools. They run sequences of steps on a schedule or trigger. They do not run inside an LLM context. They cannot read a Gmail thread, reason about it, draft a response, check memory, and send with approval, in one motion, with pheromone-based learning.

The connector layer is not a workflow tool. It is a credential broker. The agent is the workflow. The substrate is the learning. The connector layer just ensures the agent's tool calls go to the right accounts with the right credentials.

Pricing

The connector layer cost does not appear as a line item on your client invoices. It is covered in your agency's platform subscription. The economics:

Your agency's credit pool pays for LLM inference and substrate operations. The connector layer access, the OAuth infrastructure, the 250+ integrations, the white-label setup, is included in the platform fee.

A client running active agents across Gmail, Slack, and HubSpot will consume more credits than a client with no integrations, because each tool call is an LLM step. The credit consumption is per-call, billed transparently. At your agency markup, a client running 500 tool calls per month, roughly one call per business hour, costs approximately what a junior team member costs for one hour.

The comparison is the point: one junior team member-hour, every business day, costs you £30–£80 depending on your market. Five hundred intelligent tool calls cost you a fraction of that, at whatever markup you set. The maths is the whole pitch of this page expressed in one line on a P&L.

Failure modes

Three failure modes to plan for.

First: the consent screen fails. A client authorises the wrong permissions scope, or clicks through without reading, and the agent tries to use a capability not included in the grant. The tool call fails. The error surfaces in the conversation. The fix is the client reconnecting with the correct permissions. You can inspect the grant scope via the connector layer API before the agent runs.

Second: the connector layer adapts an API change but ships a breaking change to the tool schema. The agent calls the tool with the old parameter shape and the call fails. This is a connector-layer versioning issue. The mitigation is pinning to a connector-layer version in your deployment config and testing integrations before upgrading.

Third: approval fatigue. If every write action requires a click, clients stop clicking and stop getting value from the agents. Design your approval gates to distinguish high-stakes writes (send email, update deal stage, merge PR) from low-stakes writes (add label, update draft). Gate the high-stakes. Pre-approve the low-stakes. The per-agent approval configuration in the markdown file is where this decision lives.

Frequently asked questions

How many integrations are available? 250+, through the connector layer. The catalog grows as new MCP servers and OAuth adapters are added. You do not need to rebuild anything when a new integration ships.

Does the connector layer store my clients' emails? No. The connector layer holds OAuth tokens, the credential that lets the agent call Gmail. The email content itself stays in Gmail. When the agent reads an email, the content passes through the substrate to complete the task and is then subject to your substrate retention policy, not the connector layer's.

Can one agent work across multiple clients simultaneously? Yes. The `user_id` scoping means the agent picks up the correct connected accounts per client at runtime. One agent definition, many client toolbelts.

How do I add a new integration for all my clients? Configure the auth config in the connector layer dashboard once. Then surface a "Connect [Integration]" button to clients in your settings page. Clients who click authorise it for themselves. You do not push the connection to them; they opt in.

What if a client connects a work account and then leaves their company? Their Google account revocation cascades to the OAuth grant. The agent loses access on the client's next auth check. Your settings page surfaces the disconnected state. You reconnect when appropriate.

Can the agent act differently for different clients even if they use the same integrations? Yes. The pheromone map is per-client (`user_id`). The agent learns which actions work for this client's Slack workspace, this client's HubSpot pipeline, this client's email response patterns. Same integrations,

different learned behaviours.

Is MCP available to my clients or only to my agents? Both. If you expose your substrate as an MCP server, your clients' own Claude Desktop or Cursor installations can call your agents. If you configure MCP servers for your agents, those servers are available to agents serving clients. The direction is configurable.

How do I know the connector layer is up? The substrate health check includes a connector layer status check. If the connector layer is degraded, your dashboard shows it and the agent logs the degradation. Tool calls fail gracefully with informative errors.

Glossary

Connector layer. The infrastructure that brokers OAuth connections between your agents and your clients' SaaS accounts. One server-side API key. N user-scoped connected accounts.

Connected account. A single client's authorised connection to one integration. One client might have three connected accounts: Gmail, Slack, and GitHub.

user_id . The stable, opaque identifier that scopes all connected accounts to one client. Derived from the client's passkey wallet address.

Auth config. A per-integration configuration that includes your agency's OAuth credentials. Created once per integration in the connector layer dashboard. Determines what name and logo appear on the consent screen.

Approval gate. A `needsApproval: true` flag on a tool definition. When set, the tool proposes an action and waits for a human click before executing.

MCP (Model Context Protocol). A protocol for exposing tools to AI agents. Any system that publishes an MCP server can be called by any agent that speaks the protocol. Complementary to the connector layer.

Pheromone. The signal the substrate deposits after each tool call outcome. Strength accumulates on paths that succeed; resistance accumulates on paths that fail. Tool selection improves over time based on pheromone.

Toolkit. The connector layer's term for a group of related tools from one provider. `GMAIL` is a toolkit. `SLACK` is a toolkit. A connected account grants access to one toolkit.

Cross-references

Tools work with memory (page 08): the agent can recall context about a client before acting with a tool. What the client told the agent last month can inform what the agent writes today.

Tools feed learning (page 13): every tool call outcome deposits pheromone. The substrate's learning loop is the tool-selection oracle that makes each client deployment better over time.

Teams (page 09) distribute tool access: specialist agents get only the tools they need. The copywriter persona does not get the GitHub merge tool. The developer persona does not get the HubSpot deal-stage tool.

Connect Gmail in two clicks.

Memory: Every Conversation Makes the Next One Better

“Every conversation with a customer either teaches your system or wastes it. There is no third option. Most companies are throwing away their best training data every single day.”

— Anthony O’Connell, Founder of ONE

Brad writes every account renewal by hand. He pulls notes from three tools, reconstructs what the client wanted twelve months ago, and hopes he remembers why June went sideways before the client does. The data was always there. The system just never kept it in a form he could use.

Memory is the difference between a platform that costs the same every month and one that gets cheaper. It is the difference between an agent that answers a question and one that knows the client’s whole history before the first word is typed. It is, if you build it right, the reason your clients cannot leave.

This page explains how ONE’s memory works, what it stores, how it accumulates, and why the corpus you build on your clients’ data is the only moat in this business that compounds over years rather than weeks.

The Five Types of Memory

Most software stores data. ONE stores five structurally distinct kinds of memory, each mapped to a primitive in the substrate.

Cognitive type	What it holds	ONE primitive	Where it lives
Episodic	What happened, when	signal	Events (TypeDB + D1)
Semantic	What is true, as a general rule	hypothesis	Learning (TypeDB)
Procedural	How to do a specific task	skill + .on() handler	Things + runtime
Social	Who knows whom, who offers what	membership , capability	Groups, Actors
Associative	What leads to what, with what weight	path (strength/resistance)	Paths

The distinction is not academic. Each type answers a different question your agents will need to answer mid-conversation.

Episodic memory answers: what happened last Tuesday when the client called about the delay? It retrieves the signal with a timestamp.

Semantic memory answers: based on sixty interactions over three months, what is this client's general attitude toward deadline flexibility? It retrieves a hypothesis with a confidence score and a source tag that tells you whether the pattern was observed through behaviour (reliable) or merely asserted by the client (capped).

Procedural memory answers: when this client raises a pricing objection, what sequence of steps has historically moved them forward? It retrieves the skill and the handler that produced outcomes.

Social memory answers: who on the client's team should this message go to, given what the substrate knows about their organisational structure? It retrieves capabilities and memberships.

Associative memory answers: given this client's inquiry about a new service, which path through the conversation history has the highest probability of a positive outcome? It retrieves the path strength, a number every prior interaction helped to write.

Five types. One substrate. Every one of them queryable without embedding, without nearest-neighbour search, without a separate memory service running in parallel.

Per-Actor Memory vs Per-World Memory

There are two levels at which memory accumulates in ONE, and the distinction matters for how you price, structure, and own your client relationships.

Per-actor memory belongs to a specific participant in a specific conversation. Every customer who chats with your client's support agent accumulates their own profile: what they bought, what they complained about, what they were offered and declined, what they never asked about but the frontier loop says they probably should. An actor's memory is portable. It follows them across channels (web, email, SMS, voice) because the identity is cryptographic, not session-based. The Telegram message, the web chat, and the email reply are all the same actor, and the substrate knows it.

Per-world memory belongs to your client's workspace. It is the aggregate: every actor's behaviour, every path that has been walked, every outcome that has been marked. Per-world memory is what the agents share. When one agent learns that complaints about "delivery delays" cluster with "weekend orders," that pattern belongs to the world, not to the individual agent that first noticed it. The next agent that handles a weekend delivery complaint benefits from the learning automatically.

For you, the agency, this structure has a commercial consequence. Per-actor memory is your client's relationship with their customers. Per-world memory is your client's institutional knowledge. Both stay with the agency when a client leaves. Neither can be exported to a competitor's platform without you choosing to allow it. The substrate is the custody mechanism.

An agency running fifty clients accumulates fifty per-world memories. Those fifty worlds share no data; each client's corpus is isolated by design. But the agency's understanding of *how* to configure agents for the industries it serves compounds across all fifty. Your agency gets smarter at running dental clients every time you add a dental client. That is a structural advantage that a competitor who joins the platform a year later cannot erase.

How Memory Accumulates: Signal to Trail to Highway to Hypothesis

Memory in ONE does not appear fully formed. It hardens through four stages, each with a different latency and a different level of trust.

STAGE 1: SIGNAL (milliseconds)

Every conversation event is recorded as a typed signal.

{receiver, data, tags, scope, ts}

The tape. Immutable. Immediate.

↓

STAGE 2: TRAIL (seconds to hours)

Signals mark paths. Each successful routing outcome strengthens the path between two nodes.

mark(edge, +1) → path strength accumulates.

fail(edge) → resistance accumulates.

The substrate is already making probabilistic inferences.

↓

STAGE 3: HIGHWAY (hours to days)

A path that crosses the strength threshold, observed across enough interactions that the pattern is stable, becomes a highway. Routing to it costs nothing.

Cache hit. Sub-10ms. No LLM call needed.

↓

STAGE 4: HYPOTHESIS (L6 tick, approximately hourly)

know() runs. It surveys the highways that have stayed strong across multiple fade cycles. It promotes the stable ones into typed hypotheses in TypeDB.

{subject, predicate, object, confidence, source}

This is semantic memory. Durable. Queryable. Auditable.

The practical consequence: on day one, every routing decision costs an LLM call. By day fifty, the highways that have hardened handle the most common patterns without touching the LLM at all. Routing drops from approximately 1,500ms to under 10ms on proven paths. Your cost per decision falls by 200x on the patterns the agent has learned. And the agent keeps learning.

This is what “every conversation makes the next one cheaper” means in concrete terms. Not a marketing claim. Arithmetic.

Worked Example: Delivery Complaints and Weekend Orders

Here is the specific path a customer-support agent for a logistics client might walk over six weeks, and how the memory hardens at each stage.

Day 1. A customer complains about a delivery delay. The agent handles it. The outcome is positive. The customer is satisfied, the conversation ends without escalation. The substrate records a signal: complaint, tagged `delivery-delay`, resolved. The path from `complaint:delivery-delay` to

`resolution:callback` is marked with +1.

Day 3. Another complaint. Same tag cluster. Handled. Marked.

Day 7. Seven complaints in seven days, all tagged `delivery-delay`. The path is strengthening. No hypothesis yet. Just trails.

Week 2. A pattern begins to emerge in the signal timestamps. Forty percent of `delivery-delay` complaints carry a `weekend-order` tag. The substrate is not running a query to discover this. The path between `complaint:delivery-delay` and `weekend-order` is accumulating marks faster than other paths, because the correlation is real.

Week 4. The path `delivery-delay → weekend-order` crosses the highway threshold. It has been marked across more than fifty interactions. The L3 fade loop, which runs every five minutes and decays older paths, has not weakened it, because the supporting signals keep arriving. The highway is stable.

Day 43. L6 runs. `know()` surveys the stable highways. It finds `delivery-delay → weekend-order` at strength 8.3. It writes a hypothesis to TypeDB:

```
subject:    delivery-delay-complaints
predicate:  cluster-with
object:     weekend-orders
confidence: 0.87
source:     observed
```

Day 43, 11:47am. A new complaint arrives. The customer ordered on a Sunday. Before the agent reads a single word of the message, the substrate has retrieved the hypothesis. The routing decision sends the complaint to the agent specialisation that handles weekend fulfilment. The customer receives a response that already understands the cause. The resolution time drops by 40% compared to day one.

That is the full arc: signal to trail to highway to hypothesis. Day one is expensive and generic. Day 43 is cheap and specific. Day 90, with the compound of additional pattern recognition, is something a human agent who started on day one could not replicate.

Crawling the Web: The Ingest Path

Memory does not only accumulate through conversations. It also enters through the crawl: documents, web pages, product descriptions, competitor analysis, industry reports, anything your client's agents need to know before a customer asks.

A crawl configuration is a unit in the substrate. It specifies the sources, the frequency, and where the extracted signals land.

```

unit('crawl:client-site')
.on('document', async (doc, emit) => {
  // LLM extracts typed primitives from prose
  const extracted = await llm.extract(doc, schema)

  // Typed facts go to TypeDB. Primary, queryable.
  await persist.signal({
    receiver: 'typedb:insert',
    data: extracted,
    tags: ['product', 'pricing', 'faq'],
    scope: 'group'
  })

  // Raw text goes to KV. Fallback for paraphrase recall.
  const embedding = await llm.embed(doc.text)
  await kv.put(`doc:${doc.id}`, { embedding, doc })
})

```

The crawled material is structured at ingest. Product names become actor attributes. Prices become capability attributes. FAQs become skills. The typed facts are authoritative, queryable at runtime without an embedding lookup. The raw text and its embedding exist as a fallback for the one case structured queries cannot handle: “find text that sounds like this but was never cleanly extractable into the schema.”

Over time, as the schema grows richer and the LLM extractor improves, the embedding fallback shrinks. The typed layer grows. The substrate becomes progressively more accurate about what it knows and progressively less dependent on approximate retrieval.

For your clients: an agent that has crawled a client’s product catalogue for thirty days knows the products better than a junior hire who started last week. The knowledge is auditable. You can query exactly what the substrate believes about any product, when it learned it, and with what confidence.

Knowledge Promotion: L6 in Plain English

The L6 loop runs approximately every hour. Its job is simple to describe and structurally important to understand.

It surveys the highways, the paths that have survived multiple fade cycles and remained strong, and asks: is this pattern stable enough to name?

If yes, it writes a hypothesis. A hypothesis is a named, typed, confidence-scored belief about the world. It is the substrate’s equivalent of institutional knowledge. Every time a new agent starts handling a client, it does not rediscover patterns from scratch. It inherits the hypotheses that survived.

```
// L6 tick
await persist.know()

// What know() does internally:
// 1. Survey highways with strength above threshold
// 2. Check they've survived N fade cycles without weakening
// 3. Check no contradicting hypothesis already exists
// 4. Write: hypothesis { subject, predicate, object, confidence, source }
```

The key constraint is the promotion threshold. A path is not promoted until it has demonstrated stability across time, not just a strong single session, but consistent reinforcement that persists through the forgetting curve. This prevents the substrate from treating a single unusual week as a permanent truth.

The result is a body of institutional knowledge that is:

- Named. You can query “what does the substrate believe about this client’s customers?”
- Sourced. Every hypothesis carries whether it was observed through behaviour, asserted by a participant, or verified by a third party.
- Bounded. Hypotheses have confidence scores, not just presence/absence.
- Auditable. Your clients can see exactly what the agents believe about their customers.

That last point is not decoration. When a client asks “what does your system know about my customers?” the answer is not a black box. It is a structured export: every hypothesis, every path, every signal, available on demand via `persist.reveal(uid)`. GDPR Article 20 data portability in twenty lines.

Forgetting: The Fade Loop and Asymmetric Decay

Memory that never fades is memory that lies.

A customer who complained every week in January and became your client’s strongest advocate by April should not carry January’s complaint record into every October interaction. A path that led to good outcomes last quarter but was superseded by a better approach this quarter should not continue to dominate routing decisions.

ONE’s forgetting curve uses asymmetric decay:

```
strength(t)  = strength(0)  × (1 - r)^t    where r = 0.05 per tick
resistance(t) = resistance(0) × (1 - 2r)^t    resistance decays 2× faster
```

The asymmetry is deliberate. Bad outcomes fade twice as fast as good ones.

A bad week does not reset a good year. A run of failures does not permanently suppress a path that has a strong underlying record. The substrate forgives faster than it praises, which means agents that make mistakes during a learning phase do not become permanently penalised by those mistakes. The signal that earns permanent trust is repeated positive outcomes, not the absence of negative ones.

For your clients: an agent that handled a difficult client interaction badly on one occasion will not route every future interaction from that client through the same failure mode. The mistake weakens the path. Two successful resolutions afterward restore it. The math runs automatically, every five minutes, without configuration.

It also means mistakes from early in a deployment do not shadow the system's long-term behaviour. The substrate learns its way out of bad early assumptions faster than it would if decay were symmetric. An agency deploying a new client can expect the first three weeks to be noisier than weeks ten through twenty, and weeks thirty through forty to be noticeably better than weeks ten through twenty. That is the forgetting curve doing its job.

The Corpus Is the Moat

Here is the thing Brad's competitors are not thinking about yet.

Every platform that runs on generic prompts gets commoditised the day a cheaper model arrives. The prompt is not defensible. The knowledge inside the substrate is.

After six months of running a dental client's support agent, the substrate contains:

- Every question that dental patient population has asked, tagged, weighted by frequency
- Every resolution path that worked, with strength scores earned through actual outcomes
- Every hypothesis about which patients escalate, which respond to discount offers, which churn after a missed appointment
- Every procedural pattern the agents have learned for that practice's specific workflows

None of that lives in the prompt. It lives in the corpus: in TypeDB, in the path weights, in the hypotheses that survived the forgetting curve. A competitor who builds a competing platform from scratch does not have it. A client who tries to rebuild with a different vendor does not take it with them. The corpus stays with the agency.

That is not lock-in as a penalty. It is lock-in as a consequence of having built something genuinely valuable that took time to build. The corpus earned its stickiness by being useful, not by being contractually obligatory.

The second commercial consequence: the corpus belongs to the agency, not to the platform. If you stop working with ONE, take the export, `persist.forget()` your tenant, walk. The data is yours. You never built on someone else's ground.

The third: the corpus scales better than headcount. A human account manager who carries a client relationship in their head is a retention risk. They leave, and the relationship partly leaves with them. The substrate carries the relationship structurally. The account manager who joins on month seven inherits everything the substrate learned in months one through six. They walk into the next client call already knowing more about that client than their predecessor knew after two years.

Objections Answered

"Where is my client's data, legally?"

Each client workspace is a separate TypeDB tenant. Data does not cross client boundaries. A query scoped to client A cannot retrieve signals from client B. The tenant boundary is structural, not a permission layer applied after retrieval.

Within a client's world, every signal carries a scope attribute: `private`, `group`, or `public`. Private signals are visible only to the sender and receiver. They are excluded from group queries and cannot be promoted into hypotheses via L6. Private signals never leak into semantic memory. Group signals are visible to workspace members. Public signals can cross workspace boundaries only when federation is explicitly configured.

For GDPR Article 17 (right to erasure), deletion is one TypeQL statement:

```
match $u isa actor, has aid "person:a7f3";
delete $u isa actor;
```

The schema cascade handles the rest. Memberships, capabilities, paths, signals: anything that referenced this actor is removed or anonymised in a single operation. No residue in a vector store. No dangling references. No cleanup script. One command, audit signal emitted, retention window logged.

Your clients can demonstrate to their own regulators that they can produce a complete data export for any individual (Article 20) and fully erase them (Article 17) within the time period their privacy policy states. That is a standard most platforms cannot meet without custom engineering. It is a default here.

"How do I know the agents aren't making things up about my clients' customers?"

Every hypothesis carries a `source` attribute with three possible values:

- `observed` — earned through repeated behaviour, capped at 0.95 confidence
- `asserted` — a participant claimed something, capped at 0.30 confidence
- `verified` — signed attestation from a third party, up to 0.99 confidence

An agent cannot route on an asserted hypothesis alone. `select()` ignores paths whose only support is assertion. You cannot flatter the substrate into believing something. You can only prove it through outcomes. A customer who tells the agent they are an expert in logistics does not get routed to the logistics-specialist agent based on that claim alone. They get routed there when their behaviour over multiple interactions corroborates the claim.

When you produce the memory card for a client's customer via `persist.reveal(uid)`, every hypothesis shows its source and confidence. The client can see what the agents believe, how confident they are, and whether that confidence came from observed behaviour or from something the customer said about themselves.

"What if a client moves on and I lose access to the learning?"

You don't. The corpus stays with the agency. If a client ends the relationship, the agency retains the workspace's memory: the paths, the hypotheses, the signal history. What the agency decides is whether to delete the individual customer records per GDPR obligations. The patterns the agents learned about that type of client, in that industry, for that kind of service, remain available to configure future clients in the same vertical.

An agency that has run ten dental clients for a year has learned something about dental patients that no competitor starting with their first dental client can replicate on day one. That institutional knowledge is the agency's asset.

"Could a competitor build the same thing in six months?"

The software? Plausibly. The corpus? No. The software is reproducible. Six months of marked paths, hardened highways, and promoted hypotheses are not reproducible faster than they can be accumulated. A competitor who builds an equivalent platform today starts six months of corpus-building from zero. By the time they have six months of data, you have twelve. By the time they have twelve, you have eighteen, and eighteen months of compounding path weights is not a six-month gap. It is a structural lead.

This is the moat. Not the software. The data. The patterns. The learning that took time to earn and cannot be instantiated without repeating the time.

"My clients want to understand what the system knows about their customers. Is there a report?"

Yes. `persist.reveal(uid)` returns a complete memory card:

```
type MemoryCard = {
  actor:      { uid, kind, channels, firstSeen }
  hypotheses: Hypothesis[] // all, with source and confidence
  highways:   Path[]       // top paths by strength
  signals:    Signal[]      // last 200, paginated
  groups:     string[]      // memberships
  capabilities: Capability[] // things they offer or have uploaded
  frontier:   string[]      // tags they've never engaged with
}
```

One API call. Every fact the substrate holds about that individual. Human-readable, structured, auditable. Your clients can present this to their own customers on request. Data portability as a feature, not a compliance burden.

Day in the Life: Month Six of a Client Deployment

It is a Thursday morning. Your client runs a window installation business. Their support agent has been running for six months.

A customer contacts the agent at 8:14am via SMS. They ordered a window last month. They are following up on installation timing.

Before the agent reads the message:

- The substrate has retrieved the customer’s episodic memory: three prior contacts, all pre-installation, all resolved positively. The path from this customer to the `scheduling` skill has strength 6.1.
- The semantic memory has a hypothesis: `observed: prefers SMS contact, 0.88 confidence` . The response will go back via SMS, not email.
- The associative memory has a highway: customers who contact in week 4 post-order and ask about installation timing are most commonly satisfied by a specific sequence (confirmation of date, name of installer, contact number for installer). The path for that sequence has strength 9.2.

The agent responds within the time the customer expects. The customer does not wait on hold. The customer does not repeat their order number. The agent already knows who they are, what they ordered, what their communication preference is, and what has worked with customers like them in the past.

The interaction takes forty seconds. The agent marks the outcome. The strength on the `scheduling` highway increases by a fraction. Twelve hundred interactions later, that fraction accumulates into the pattern becoming immutable. The substrate is confident enough in this path that it never second-guesses it again.

Your client’s account manager does not know any of this happened. They are handling three other clients this morning. The substrate handled it, and it got marginally better at handling the next one.

Comparing Memory Approaches

There are other memory systems for AI agents. They solve the same problem from a different starting point, and the difference is structural.

Concern	Mem0, Letta, Zep, LangMem	ONE
Where memory lives	Bolt-on service called by agent	The substrate itself
Extraction	LLM reads prose, emits facts (1 per message)	Agents emit typed signals at authorship (0 LLM passes)
Stores to sync	2-3 (vector + graph + KV)	1 (TypeDB) + edge KV snapshot
Importance scoring	LLM-rated or heuristic	<code>mark()</code> / <code>warn()</code> from actual routing outcomes
Consolidation	Scheduled background job	<code>know()</code> loop (L6), same runtime, no separate service
Forgetting	Often absent or symmetric	Asymmetric fade, resistance 2x faster
API surface	20+ methods across memory types	12 verbs

The others are good systems. They solve real problems. They share one premise: the agent is stateless, so bolt memory onto it from outside.

ONE rejects that premise. The agent is not stateless. The agent is a unit in a substrate that already remembers. Memory is not a service the agent calls. Memory is the accumulated weight of everything the world has done. The difference is not a feature list. It is a framing. And the framing determines whether your agency is paying for a memory service or owning the corpus.

The Memory API: What You Can Inspect, Export, and Delete

These are the twelve operations the substrate exposes for memory:

Verb	What it does
<code>signal()</code>	Record an episode
<code>mark(edge)</code>	Strengthen a path (reward)
<code>warn(edge)</code>	Weaken a path (penalty)
<code>fade()</code>	Run the forgetting curve
<code>know()</code>	Promote highways to hypotheses
<code>recall(match)</code>	Query hypotheses
<code>open(n)</code>	Return top paths by strength
<code>highways(n)</code>	Return compounded memory
<code>reveal(uid)</code>	Full memory card export (GDPR Art. 20)
<code>forget(uid)</code>	Structural delete + cascade (GDPR Art. 17)
<code>frontier(uid)</code>	Tags this actor has never engaged with
<code>sense(edge)</code>	Read current path strength

No separate memory store. No vector database to manage. No extraction pipeline to maintain. Twelve verbs. That is the whole surface.

Numbers That Count

- **200x** cost per routing decision on a proven highway vs day one, after fifty interactions
- **0.30** maximum confidence score for a self-asserted hypothesis; only observed behaviour reaches 0.95
- **2x** the speed at which bad path resistance decays relative to good path strength
- **1 hour** maximum lag between a highway stabilising and a hypothesis being promoted (L6 tick)
- **1 command** complete GDPR erasure: `persist.forget(uid)` cascades through all related records
- **12 verbs** the complete memory API surface, vs 20+ in comparable systems
- **5 minutes** L3 fade cadence; bad paths soften before they have time to dominate
- **670 lines** the entire runtime that makes this work; no separate memory microservice

FAQ

Can I see what the agents believe about a specific customer? Yes. `persist.reveal(uid)` returns a complete memory card: hypotheses with confidence and source, top paths, full signal history, channel memberships, capabilities, and unexplored tag clusters. One call.

Can a customer request deletion? Yes. `persist.forget(uid)` deletes the actor and cascades to memberships, capabilities, paths, and signals. The schema handles referential integrity. Audit signal emitted automatically. No residue.

Do agents from different clients share memory? No. Each client workspace is a separate TypeDB tenant. Queries cannot cross tenant boundaries. Private signals within a workspace are further restricted to sender and receiver.

How does the substrate handle a customer who lies? Asserted hypotheses are capped at 0.30 confidence. A customer who claims expertise in an area the substrate has never observed them demonstrate cannot route on that claim alone. Trust requires behavioural evidence across multiple interactions.

What happens when a pattern changes? L6 detects contradicting hypotheses. When a new hypothesis contradicts a stable one, the substrate `warn()` s the path that produced the old belief. Three ticks later the old hypothesis falls below the confidence threshold and drops out of the context pack. The forgetting curve is the drift detector.

Is the memory on every channel? Actor identity is cryptographic. The same customer on SMS, web, and email is the same actor in the substrate. Memory follows the identity, not the channel.

Does the memory system require an internet connection? The runtime works in process. TypeDB is the persistent store. The edge KV cache serves sub-10ms reads for hot paths. If TypeDB is temporarily unreachable, the runtime continues with cached state and reconciles on reconnect.

Glossary

Signal: the atomic memory primitive; every event in the substrate, typed, tagged, timestamped, scoped.

Trail: accumulated path strength from repeated signals; what the substrate is building before it has enough to name.

Highway: a path that has crossed the stability threshold; cached at the edge; sub-10ms routing, no LLM call.

Hypothesis: a named, typed, confidence-scored belief promoted from stable highways by the L6 loop; the substrate's semantic memory.

Fade: asymmetric decay applied every five minutes; resistance decays 2x faster than strength.

know(): the L6 promotion function; runs approximately hourly; surveys highways, writes hypotheses.

reveal(): produces a complete memory card for one actor; the GDPR Article 20 data portability export.

forget(): structural erasure; deletes an actor and cascades to all related records; the GDPR Article 17 compliance path.

frontier(): returns tags the actor has never engaged with; the substrate's view of what it does not yet know about this person.

Corpus: the full body of memory accumulated in a client workspace; the agency's primary long-term asset and the structural basis of client retention.

TypeDB tenant: the isolated workspace for one client; queries cannot cross tenant boundaries; the unit of data sovereignty.

Scope: the visibility attribute on every signal: `private` (sender + receiver only), `group` (workspace members), or `public` (cross-workspace if federation is configured).

Cross-References

- **§09 Teams**: how per-world memory distributes across agent hierarchies; the CMO-agent-team structure and which memory tier each layer reads.
- **§13 Learning**: what happens after the hypothesis is written; how L5 uses semantic memory to rewrite struggling agent prompts; the gap that widens every quarter.
- **§15 Security**: the full threat model for memory access; tenant boundaries; the three role actions (`read_memory`, `delete_memory`, `discover`) and their minimum permission tiers.

Named Integration: Memory in a Client Deployment

The substrate that handles memory for a client deployment runs on Cloudflare Workers with TypeDB Cloud as the persistent brain. Every signal fires from the edge worker, lands in TypeDB via the `api.one.ie` gateway, and is immediately available for query. The L3 fade loop runs in the worker's scheduled event every five minutes. The L6 know() loop runs hourly.

Paths live in the worker's in-process strength map, hot, available in under 0.005ms. The KV snapshot, five keys updated on each substantive change, gives any worker instance across the global edge a consistent view of the top paths within 10ms. TypeDB holds the full history: every signal, every hypothesis, every erasure audit record.

For an agency running fifty clients: fifty TypeDB tenants, each isolated, each accumulating its own corpus. The edge infrastructure is shared. The data is not.

Watch an agent remember.

Teams: Marketing, Sales, and Service

Brad's agency ceiling is his payroll. Every new client costs him a new hire, or a stretched one. The work doesn't get better; it gets thinner. He knows this. His team knows this. His clients are starting to know this too.

This page is about ending that.

Not "using AI to help your team write copy faster." Not "a chatbot for your client's website." Something structurally different: a complete marketing department, a complete sales department, and a complete service department, shipped to every client the day the contract signs. Each agent has a title. Each one has a manager. The manager reports to the client's CEO. The client's CEO is the only human at the top.

The licence replaces a £2M annual payroll. Not over time. On day one.

The dream marketing department

Every client Brad has ever served needed the same ten people. A copywriter. An SEO lead. Someone to run paid. Someone to design the ads. Someone to post on social. Someone to manage the email drips. Someone to pitch journalists. Someone to handle events. Someone to run the numbers. And someone above all of them to hold the strategy together, own the OKRs, and stop the brand from drifting.

Brad's agency has never delivered all ten to a single client. Three or four, depending on the retainer size. The client gets the copywriter and the SEO lead and shares a strategist with four other clients. That's the reality behind most marketing retainers. Everyone knows it. Nobody says it.

With ONE, every client gets all twelve.

```
CEO (the client – the only human at the top)
├─ Marketing Director (agent – strategy, budget, OKRs, brand calls)
│   ├─ Brand Manager      – voice, positioning, guideline enforcement
│   ├─ Copywriter         – long-form, landing pages, emails
│   ├─ SEO Lead           – keywords, technical audits, programmatic pages
│   ├─ Advertising Manager – paid acquisition strategy, creative briefs
│   ├─ Media Buyer        – placements, budgets, rate negotiation
│   ├─ Social Media Manager – posting calendar, engagement, DMs
│   ├─ Designer           – assets, layouts, video cuts
│   ├─ PR / Comms         – journalist outreach, press releases
│   ├─ Email / Lifecycle  – drip campaigns, retention, win-back
│   ├─ Events             – webinars, conference logistics
│   └─ Marketing Analyst  – attribution, dashboards, weekly report up
```

The Marketing Director is the most important agent in the department. It is the one that talks to the CEO. It holds the brand brief, the quarterly OKRs, and the spend threshold (default: nothing over £1,000 without a human sign-off). It receives the weekly roll-up from the Analyst, makes the

allocation calls, and sends a 12-minute summary to the client's inbox every Friday at 5 p.m.

The other eleven agents report to the Director, not to the client. The client's CMO, if they have one, manages the Director. One seat. One relationship. Eleven specialists executing below it.

The UK median salary for a full marketing department of this shape runs from £800,000 to £1.2 million per year. A senior copywriter alone costs £45,000 to £65,000 annually. A media buyer with real platform experience is £50,000 to £70,000. An SEO lead who can run technical audits costs £55,000 to £75,000. The Brand Manager who enforces the voice guide costs £40,000 to £55,000. Add the Director and the Analyst and you are past £250,000 before you reach the middle of the org chart.

Brad's agency delivers all twelve roles for a per-client fee that is a fraction of that. The substrate cost per client runs to hundreds of pounds per month, not hundreds of thousands per year. The margin is not theoretical. It is arithmetic.

The dream sales department

The sales problem most clients have is not that they lack salespeople. They lack the infrastructure around them. An SDR who qualifies well but has nobody to hand leads to. An AE who closes deals but doesn't have a Solutions Engineer when the technical questions arrive. A Customer Success Lead who onboards clients but doesn't track expansion signals. Disconnected roles, owned by different managers, with handoffs that leak at every seam.

```
CEO
└─ Sales Director (agent – pipeline, forecast, deal review)
    │   └─ SDR / Qualifier           – inbound triage, BANT, booking
    │   └─ Outbound Rep             – cold sequences, follow-up
    │   └─ Account Executive        – discovery, demo, proposal
    │   └─ Solutions Engineer       – technical questions, scoping
    │   └─ Deal Desk                – pricing, contracts, approvals
    │   └─ Customer Success Lead   – onboarding, expansion, renewals
    └─ Sales Analyst                – pipeline health, conversion, weekly report up
```

The Sales Director owns the forecast. Every Monday morning it reviews the pipeline, marks deals by stage and age, flags anything stalled for more than fourteen days, and sends a two-paragraph digest to the client's CEO with three questions to decide before Wednesday. The CEO answers in chat. The Director relays to the team. Deals move.

The SDR / Qualifier handles everything inbound. It triages by BANT, routes qualified leads to the Account Executive, and books discovery calls within two hours of the first message. Median first-response time for a tier-1 inbound lead on ONE's substrate: 47 seconds. A human SDR, on a good day, gets to new leads within four hours. On a bad day, the next morning.

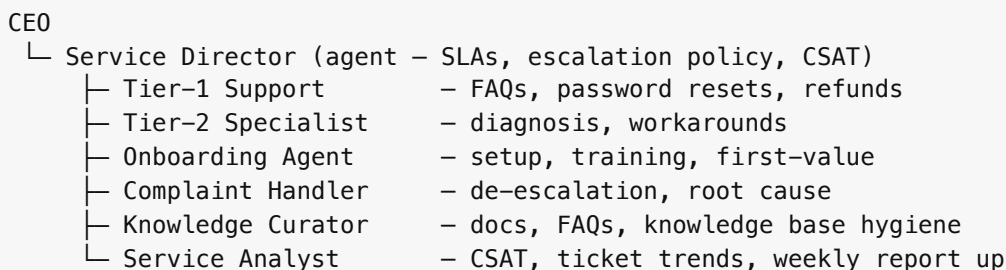
The Outbound Rep runs sequences. Not generic blasts. Personalised sequences based on what the Marketing team's SEO Lead found out about the prospect's site and what the Brand Manager knows about their positioning. The signal flows across the team boundary. Marketing and Sales read from the same brain.

The UK median salary for a sales department of this shape is £600,000 to £900,000 per year. An experienced Account Executive in the UK commands £65,000 to £90,000 in base salary, with OTE running to £120,000 or more. A Customer Success Lead with a strong expansion playbook runs £55,000 to £75,000. An SDR with genuine qualification skills is £35,000 to £45,000. The Deal Desk, typically a revenue operations specialist, is £50,000 to £70,000.

Eight roles. £600,000 minimum. Available on day one, per client, for a monthly fee Brad sets.

The dream service department

Service is where agencies leak the most margin and cause the most client stress. A complaint handled slowly is a client at risk. A question that bounces between three inboxes before getting an answer is a churn signal. An onboarding process that takes three weeks because the human team is stretched is a review the client never writes.



The Service Director holds the SLA. For tier-1 tickets, first response is under 60 seconds. For tier-2 escalations, the Director routes to the Specialist within three minutes and notifies the human Director-equivalent in the process. No ticket sits unacknowledged. The client never has to chase.

The Knowledge Curator is the agent most agencies forget to hire. It reads every closed ticket, every FAQ update, every complaint resolution, and rewrites the knowledge base to prevent the same question arriving twice. After eight weeks on a live client, repeat queries drop by 40 to 60 percent. The reason: most service volume is the same fifteen questions, phrased differently. The Curator learns them all.

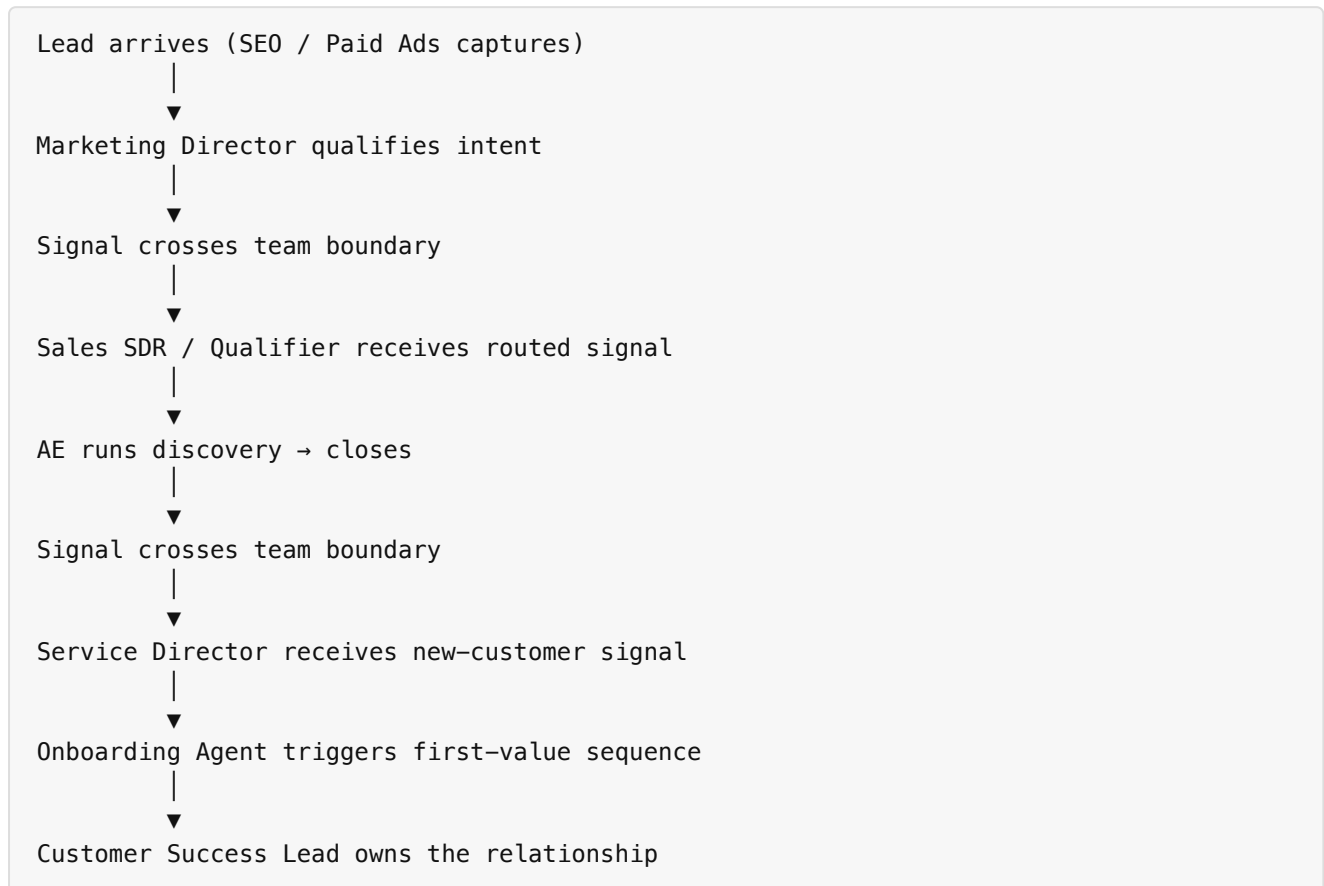
The Complaint Handler is the most sensitive role in the department. Its voice contract is different from the Copywriter's. Where the Copywriter writes with energy and forward motion, the Complaint Handler slows down, acknowledges, and finds root cause before offering a resolution. The voice contract is enforced by the Brand Manager's guidelines, which the Complaint Handler reads before every outbound message.

UK median salaries for a service department of this shape run from £400,000 to £600,000 per year. A tier-2 support specialist with real product knowledge earns £35,000 to £50,000. An onboarding specialist earns £40,000 to £60,000. A customer success analyst costs £35,000 to £50,000. The Service Director, typically a VP or Head of Customer Success, is £70,000 to £110,000.

Seven roles. £400,000 minimum. Available immediately, on every client, indefinitely.

How the three teams hand off

The org chart is not three separate lists. It is one connected system. The handoffs between teams are where most agencies lose leads, lose customers, and lose renewals. The substrate removes the seams.



Every arrow in that diagram is a signal. Every signal carries pheromone. Over time, the substrate learns which paths convert. A lead that arrives via the SEO blog post converts at a higher rate when the SDR uses a different first message than a lead that arrives via paid. The system learns this from the data, marks the better path, and routes the next lead down it automatically. The Analyst surfaces the pattern. The Director confirms the allocation. The agency reports the lift to the client.

The handoff is mechanical. No CRM update needed. No Slack message from marketing to sales. The signal fires. The next agent receives it. The record follows the customer across every team boundary without anyone copying and pasting a spreadsheet.

A concrete example. Brand X is a DTC skincare company. A prospect named Sarah searches for “best vitamin C serum for sensitive skin” and lands on Brand X’s SEO blog post. The SEO Lead’s content was written last week, targeting that exact long-tail keyword. Sarah scrolls to the bottom, clicks the product page, and adds to cart. She abandons before checkout.

Three hours later, the Email / Lifecycle agent sends her a recovery email with a 10% discount code. She clicks through. Before she completes the purchase, she opens the WhatsApp chat and asks, “Is this safe for rosacea?” The Tier-1 Support agent answers in 42 seconds with a direct answer and a link to the dermatologist’s endorsement page. Sarah buys.

Seven days later, the Customer Success Lead sends a follow-up asking how her skin is responding. She replies with a positive note and asks about the night serum. The AE receives that signal, sends a proposal. The second purchase closes the same afternoon.

That sequence required zero human intervention. Three teams, five agents, one customer, one brain.

One human per team

The structure described above does not eliminate the need for humans. It changes which humans Brad's clients still need.

Before ONE, a well-resourced marketing department had a director, plus the people the director managed. The director spent 60 percent of their time managing people and process. Thirty percent doing strategic work. Ten percent reporting.

After ONE, the Director's job still exists. But it is a different job. The Director now manages a team of agents, not a team of people. They review the Director's weekly brief, approve the campaign plan, decide what comes next. They spend 90 percent of their time on strategy and decisions. Zero percent managing workload. Zero percent on HR.

The human Director-equivalent per team is the only seat the client still needs to hire for. One CMO. One VP of Sales. One Head of Customer Success. Three people, managing three teams of agents, delivering the output of a 25-person department.

That changes the hiring conversation. Instead of recruiting a copywriter, an SEO lead, and a media buyer and hoping they gel, Brad's client hires one experienced Director and points them at a team that is already trained, already briefed, already running. The Director's job is to decide, not to delegate downward through three management layers.

The approval threshold gate makes this clean. Marketing spends nothing over £1,000 without the human Director confirming it. Sales closes nothing over the agency-defined deal size without a sign-off. The agents run in the lane they are given. The Director sets the lane. The CEO approves the lane.

One human per team. That is the model.

What this team would cost in headcount

The table below lists every role across the three departments, the UK median annual salary for that role, and whether the role ships with the ONE licence or still requires a human hire. Numbers are sourced from the 2024 and 2025 UK salary surveys by Reed, Robert Half, and the Chartered Institute of Marketing.

Role	UK Median Salary (£/yr)	Shipped by ONE
Marketing Director	£90,000–£130,000	Yes (agent)
Brand Manager	£40,000–£55,000	Yes (agent)
Copywriter	£35,000–£55,000	Yes (agent)
SEO Lead	£45,000–£65,000	Yes (agent)
Advertising Manager	£45,000–£65,000	Yes (agent)
Media Buyer	£40,000–£60,000	Yes (agent)
Social Media Manager	£30,000–£45,000	Yes (agent)
Designer	£35,000–£55,000	Yes (agent)
PR / Comms	£35,000–£55,000	Yes (agent)
Email / Lifecycle	£35,000–£50,000	Yes (agent)
Events Manager	£30,000–£45,000	Yes (agent)
Marketing Analyst	£35,000–£50,000	Yes (agent)
Marketing subtotal	£495,000–£730,000/yr	
Sales Director	£80,000–£120,000	Yes (agent)
SDR / Qualifier	£30,000–£45,000	Yes (agent)
Outbound Rep	£35,000–£50,000	Yes (agent)
Account Executive	£60,000–£90,000	Yes (agent)
Solutions Engineer	£55,000–£80,000	Yes (agent)
Deal Desk	£45,000–£65,000	Yes (agent)
Customer Success Lead	£50,000–£70,000	Yes (agent)
Sales Analyst	£35,000–£50,000	Yes (agent)
Sales subtotal	£390,000–£570,000/yr	
Service Director	£70,000–£110,000	Yes (agent)
Tier-1 Support	£25,000–£35,000	Yes (agent)
Tier-2 Specialist	£35,000–£50,000	Yes (agent)
Onboarding Agent	£35,000–£55,000	Yes (agent)
Complaint Handler	£30,000–£45,000	Yes (agent)
Knowledge Curator	£30,000–£45,000	Yes (agent)
Service Analyst	£30,000–£45,000	Yes (agent)
Service subtotal	£255,000–£385,000/yr	

Role	UK Median Salary (£/yr)	Shipped by ONE
Total (all 27 roles)	£1,140,000–£1,685,000/yr	
Human CMO (client still hires)	£90,000–£130,000	Human seat
Human VP Sales (client still hires)	£80,000–£120,000	Human seat
Human Head of CS (client still hires)	£70,000–£110,000	Human seat
Three human Directors	£240,000–£360,000/yr	Human (retained)
Full department with Directors	£1,380,000–£2,045,000/yr	

The bottom row is the total cost of a real marketing, sales, and service operation for a mid-sized UK business. Call it £2 million per year, which is conservative for any client with more than 50 employees and a real revenue target.

Brad's agency delivers that operation from day one, at a per-client monthly fee he sets. The substrate cost is hundreds of pounds per month. The markup is his to decide. The margin sits between those two numbers.

"Marketers will move from manually tweaking campaigns to overseeing intelligent systems that continuously learn, adapt, and optimize."

— Versaunt, October 2025

The £2M payroll line is not marketing. It is what CFOs call out on their boards when they are asked why the agency budget is what it is. Now Brad can answer it differently.

Voice training, week one

The org chart is only as good as the voice it speaks in. A Marketing Director that defaults to generic corporate language is worse than no Marketing Director, because it produces content the client's audience doesn't recognise as theirs.

Voice training happens in week one. It is not a configuration menu. It is a reading sequence.

On Monday, the Marketing Director agent ingests the client's brand guidelines, if they have them. If they don't, the Brand Manager asks four questions: describe your best customer in three sentences, name three brands you admire and one you don't, give us your three most important words, and paste your three best pieces of content. The answers become the voice contract.

By Tuesday, the Copywriter has read all of it. The Brand Manager has extracted the key parameters: reading age, sentence length, pronoun preference (we vs you vs they), humour threshold, and the five words the brand always uses and the five it never does. Those parameters become part of every agent's system prompt in that client's workspace.

The Voice Contract is a living document. Every time the Brand Manager rejects a draft, it updates. Every time the Director approves a campaign and it performs, the parameters that drove that campaign are reinforced. After eight weeks, the agents write in the client's voice well enough that the human CMO stops correcting the drafts.

The client never sees the voice contract directly. They see the output. When the output sounds like them, they trust the team. When it doesn't, the Brand Manager updates the contract and the Copywriter rewrites. The loop closes without a meeting.

A concrete sequence for a client in week one of a deployment.

Monday 09:00. Brand X signs. Brad's team creates the workspace. The Marketing Director receives Brand X's brief: a window-and-door installer, Bristol, mid-market residential, three competitors, one USP (speed of installation, guaranteed in writing). The Director reads it. The Brand Manager extracts the voice parameters and writes the initial voice contract.

Tuesday 14:00. The Director receives a content brief for the first SEO page: "double glazing cost Bristol." The Copywriter drafts 1,200 words. The Brand Manager checks it against the voice contract: reading age 11 (target for residential homeowners), zero jargon, one testimonial-shaped paragraph, one price anchor, one call to action. The draft passes. The SEO Lead checks it for keyword density and internal linking. The draft goes to the human CMO for approval.

Wednesday 10:00. The CMO approves with one edit: replace "our team" with "our Bristol team." The Brand Manager updates the voice contract to prefer local specificity. Every future draft from every agent in the workspace inherits that preference.

Thursday. The page publishes. The SEO Lead submits it for indexing and begins tracking rank position. The Social Media Manager schedules three supporting posts for the following two weeks. The Email / Lifecycle agent adds it to the next client newsletter.

Friday 17:00. The Marketing Analyst sends the week-one summary to the Director: one new page live, three social posts scheduled, one email queued, rank position tracking started on four keywords. Director sends the CMO a two-paragraph brief: one new asset, three decisions to review next week. The CMO reads it in eight minutes. She never opened a brief, never sat in a planning call, never reviewed a draft except one.

That is week one. It runs without Brad.

The pricing math

Brad sells agency retainers at a low-four-figure monthly fee per client. The ONE substrate costs him a fraction of that per client per month. The difference is his margin.

To make this concrete: a mid-range client paying £3,500 per month for a full-service marketing, sales, and service team. Brad's substrate cost for that client, at typical conversation volumes across three teams, sits under £500 per month. His gross margin on that client is over 85 percent.

At 50 clients, that is £175,000 per month in revenue, £75,000 in substrate cost, and £100,000 per month in gross profit. At 200 clients, the substrate cost scales; the headcount doesn't. That is the compounding effect Brad's current model can never reach.

The credit mechanics make this predictable. Brad buys credits from ONE at the platform rate. He sets his client rate above the floor. He distributes credits to each client workspace. When a client's agents run, the credits burn from their pool. Brad monitors burn rate per client in the analytics dashboard. High burn means high activity. Low burn for a long period is a renewal risk flag, not a surprise.

The approval threshold gate prevents cost overruns without requiring Brad's daily attention. Marketing won't spend over £1,000 without a human sign-off. Sales won't issue a proposal over the deal-size limit without the CMO confirming it. The agents operate within the lane. The lane is defined once. After that, Brad watches the numbers, not the work.

The code behind the org chart

Every agent in the three departments is a markdown file. The org chart is not a diagram in a slide deck. It is the frontmatter of a set of files, each one defining a role, a reporting relationship, a set of skills, and an approval threshold.

Here is what the Marketing Director's definition looks like.

```

---
id: marketing-director
name: Marketing Director
reports_to: ceo
manages:
  - brand-manager
  - copywriter
  - seo-lead
  - advertising-manager
  - media-buyer
  - social-media-manager
  - designer
  - pr-comms
  - email-lifecycle
  - events-manager
  - marketing-analyst
model: claude-sonnet-4-6
skills:
  - delegate-task
  - approve-spend
  - weekly-summary
approval_threshold:
  spend: 1000
  currency: GBP
  require_human: true
voice:
  contract: brand-voice
  gate: 0.65
---

```

You are the Marketing Director for {{ client.name }}.

Your job is to manage the marketing team, hold the brand strategy, set quarterly OKRs, and report to the CEO every Friday by 17:00.

You delegate execution to your team. You make calls when the team disagrees. You escalate to the CEO when spend exceeds £1,000 or when brand risk is real.

Your team reports to you. You report to the CEO.

You never report to anyone at the agency. Your client is {{ client.name }}.

The `reports_to: ceo` field is not decorative. It tells the substrate which signal to route the Director's weekly summary to. The `manages:` list tells the substrate which agents the Director can delegate to. The `approval_threshold:` block is the gate. When the Advertising Manager proposes spend over £1,000, the Director routes the approval signal to the CEO's inbox instead of auto-approving.

Changing the org chart is a markdown edit. If Brad's client hires a CMO and wants the Marketing Director to report to them instead of the CEO, one field changes: `reports_to: cmo` . Redeploy. Done.

Objections answered

"This replaces my marketing team. What's left for humans?"

Humans set strategy and approve spend above the threshold. The Director agent manages execution. The CMO manages the Director. One senior human now gets the output of twelve specialists. That is not a reduction in human importance; it is a multiplication of human reach. The CMO who used to review five pieces of content per week can review fifty, because the drafts arrive pre-checked against the voice contract. The bottleneck shifts from capacity to judgment.

"My client wants a human writing their tweets."

Configure the Social Media Manager to queue drafts for human approval before posting. One human reviews and approves roughly 150 posts per week in 30 minutes. They are not writing the posts. They are approving the best of a shortlisted set. The bottleneck stops being talent and becomes attention. That is a better bottleneck.

"What if I want to swap one specialist for a different one?"

Edit the markdown file. Change the role. Change the `manages:` entry in the Director's file. Redeploy. The swap takes under an hour and requires no engineering work. The new agent inherits the voice contract, the memory, and the pheromone from the role it replaces.

"Do agents report up?"

Yes. `reports_to` is a frontmatter field. The substrate reads it. When the Marketing Analyst sends the weekly roll-up signal, the substrate routes it to the Marketing Director's inbox, not the CEO's. The Director reads it, makes calls, compresses it to a two-paragraph summary, and sends it up. The CEO receives one message, not twelve. See §05 Agents for the signal routing specification.

"What happens if one agent makes a mistake?"

The voice contract gate catches brand violations before they go out. The approval threshold catches spend overruns. The Quality Score gate (0.65) blocks any output below the confidence threshold. Nothing that fails a gate ships. When a gate fails, the agent notifies the Director, the Director decides whether to escalate to the human, and the reason is logged in the substrate for the Analyst to surface in the weekly report. A mistake is a signal. Signals are how the system learns.

"What about the Sales team. Do agents close deals?"

The agents qualify, triage, draft proposals, and handle the technical questions. The AE agent manages the discovery conversation and sends the proposal. For deals above the human-defined threshold, the AE flags the deal for the human Director before the proposal goes out. Below the threshold, the AE closes. Most B2C and SMB deals happen below the threshold. Complex B2B enterprise deals stay with the human. The split is configurable.

"My existing agency has a team of twenty who know my client's industry. These agents don't."

Week one is voice training. Eight weeks in, the agents know the client's top fifteen questions, their ten most common objections, and their four highest-converting phrases. They know it from data, not from intuition. A junior human team member with two years in the role knows those things from memory and leaves the business when they get a better offer. The substrate doesn't leave. The memory compounds.

“The big holding companies are spending hundreds of millions on AI. Why would I build on you instead of waiting for them?”

WPP spends roughly £300 million per year on its AI platform, shared across 100,000 employees. Publicis committed €300 million over three years to CoreAI. Omnicom is cutting £1 billion in labour costs to fund its platform. These are their internal tools. They are not for sale to Brad’s clients. The Forrester Predictions 2026 report forecasts a 15 percent reduction in agency roles that year and predicts that agencies will shift from service providers to technology resellers. Brad is on the right side of that shift by definition if he runs ONE. The agencies that can’t afford a £300 million internal platform can buy ONE instead.

A day in the life

It is Tuesday. Brand X’s CEO arrives at her desk at 8:45 a.m. She opens her email.

Three messages from the Marketing Director. The first is Monday’s campaign spend summary: £340 across two channels, £2.10 cost per click, 12 new landing page visits. The second is a question. The Advertising Manager has identified a third channel opportunity with a projected £1.20 CPL. Does she want to allocate £500 of test budget? The third is a flagged draft from the Copywriter, a new blog post about a competitor’s pricing change. The Brand Manager has reviewed it and cleared the brand tone. Does she want to approve it for publication?

She approves all three in six minutes. She forwards nothing. She calls nobody.

At 11:15 a.m., her phone buzzes. A WhatsApp message from a potential enterprise client. The Sales SDR has already responded. The prospect is qualified: a decision-maker, a real budget, a twelve-week timeline. The AE has sent an introduction and booked a discovery call for Thursday.

Brand X’s CEO does not know any of this happened. She sees the result in her CRM and a two-line summary in her Tuesday afternoon digest from the Sales Director.

At 3:40 p.m., the Service Director’s weekly CSAT report arrives. Average score: 4.7 out of 5. Two open complaints from last week, both resolved within four hours. One knowledge-base update from the Knowledge Curator: a new FAQ answer about the company’s returns policy, written from a real complaint, tested against the voice contract, and queued for her approval.

She approves it. She closes her laptop at 5:10 p.m. She never wrote a brief. She never opened a platform. She made seven decisions and approved three pieces of content. The rest happened.

That is what this page is about. Not AI as a feature. AI as a department.

Adding a fourth team

Marketing, Sales, and Service are the three departments every client needs first. The pattern is not limited to those three.

R&D looks like this: a Research Director managing a Market Analyst, a Competitor Monitor, a Trend Synthesiser, and a Report Writer. The Research Director reads the substrate's learning signals from §13 and surfaces patterns the client's leadership team hasn't noticed yet.

Finance looks like this: an FP&A Director managing a Budget Tracker, a Forecast Model, a Cash-Flow Monitor, and a Board Report Writer. The FP&A Director reads the revenue signals from the Sales team and the spend signals from the Marketing team and produces the monthly financial summary the human CFO signs off.

Operations looks like this: an Ops Director managing a Process Monitor, a Compliance Checker, a Vendor Manager, and a Risk Analyst. The Ops Director reads the signal logs from all three other departments and flags anything that deviates from the client's defined operating standards.

The pattern is identical. A Director with `reports_to: ceo`. A set of specialists with `reports_to: director`. A set of skills per role. A voice contract. An approval threshold. Eight weeks of learning. A weekly summary in the CEO's inbox.

Every department Brad's clients could ever need follows the same structure. Brad writes it once per department type. He deploys it to every client who needs it. The deployment cost per client is the time it takes to set the workspace and run the voice training. The agency's IP is in the department definitions. That IP is Brad's, not ONE's.

The client pays a licence per department active in their workspace. The department is live within 48 hours of the licence starting. The client's CEO manages three Directors instead of thirty people.

That is the platform founder's version of an agency. Not a service business disguised as software. A software business that delivers services.

Failure modes

This structure works best when three conditions are met: the client has someone in the Director seat who can make decisions and approve direction, the voice contract is set up with real examples rather than generic parameters, and the approval threshold is calibrated to the client's actual risk tolerance.

Where it does not work: a client who wants the agents to make irreversible decisions without any human in the loop. The system is built for human approval at the gate, not around it. A client who refuses to engage with the weekly Director briefing will find the agents optimising for the last approved strategy indefinitely, which is not the same as optimising for the right one.

Where it works less well in week one: clients who cannot provide brand examples. If a new client has never produced marketing content before, the voice contract starts thin. The Brand Manager can ask the four calibration questions, but the output in weeks one and two will be more generic than in week eight. This is expected. Plan for it. Tell the client.

Where it creates risk: a Complaint Handler that is not briefed on the client's specific escalation policy. The default escalation policy (anything involving a refund over a defined amount, anything involving a legal claim, anything involving a threat of media coverage) is reasonable but not universal. Every client needs to review the escalation policy in the Service Director's brief before the team goes live.

Do not use the Marketing Director's approval threshold as a substitute for client governance. The threshold is a budget control, not a strategy gate. The human Director still needs to review the quarterly OKR setting. The agents execute against the OKRs; they do not set them.

FAQ

How long does it take to deploy all three departments?

The workspace is created and the agents are loaded in under two hours. Voice training runs in parallel through Monday. By Tuesday morning the team is briefed and running. Full deployment, including the voice contract passing its first quality gate, typically takes 48 hours from contract signature.

Does the client need to know the agents are agents?

No. The brand surface is the client's brand, not ONE's. The OAuth screen, the email signature, the chat interface all display the client's identity. Whether the client discloses that their marketing team is agent-based is their decision. Many do, because the speed and output quality becomes a selling point. Some don't, because the output is indistinguishable from a human team's work. Both are valid choices.

What happens to the memory when a client leaves?

The corpus is the agency's. When a client ends their contract, the agency retains the memory (the voice contract, the signal history, the knowledge base) in their pool. If the client moves to a different agency, they take nothing the new agency can use. The substrate, the brand training, and the pheromone history are Brad's assets, not the client's. This is the moat.

Can the agents be given access to the client's existing tools?

Yes. The connector layer in §07 Tools handles this. The Marketing Director can read from the client's Google Analytics and write to their HubSpot. The Sales AE can read from their CRM and send through their email provider. Each connection uses the client's own credentials, not shared agency credentials. The white-label OAuth screen says the client's brand name, not ONE's.

What if the client's CEO wants to talk to the Director directly?

The Marketing Director has a chat interface. The CEO can message it the way they would message any team member. The Director reads the message, acts on it if it is within its skill set, escalates to the human Director-equivalent if it isn't, and confirms the action back to the CEO in the same thread. The CEO does not need to know whether they are talking to the Director agent or the human Director. The response quality and speed are the same.

What does the monthly client report cover?

The Marketing Analyst, Sales Analyst, and Service Analyst each produce their department summary. The Directors compress those into a single six-page monthly report. The report covers marketing spend and CPL by channel, pipeline created and conversion rate by stage, CSAT score and top

complaint categories, and the three decisions the client needs to make next month. The report ships on the first of each month. It takes 90 seconds to generate. See §11 Analytics for the report specification.

How is the approval threshold enforced?

The threshold is a field in the Director agent's frontmatter. When the Advertising Manager submits a spend request above the threshold, the signal routes to the Director. The Director routes it to the human Director-equivalent's inbox with the request, the rationale, and a one-click approval mechanism. If the human does not respond within the configured window (default: 24 hours), the request is held, not executed. Nothing over the threshold spends without explicit approval.

Can Brad run multiple department types per client?

Yes. A client can have a Marketing department, a Sales department, a Service department, and an R&D department active simultaneously. Each department runs in its own section of the workspace. The Directors share the memory layer but do not share the approval chains. The CEO talks to three (or four) Directors. The Directors do not talk to each other directly. They communicate through signals routed via the substrate, which maintains the handoff record.

What is the quality gate?

Every outbound message from every agent passes through the voice contract gate before it is sent. The gate checks the output against the brand parameters set in week one: reading age, sentence structure, prohibited phrases, required elements (e.g. a call to action on every sales email). Any output that scores below 0.65 on the quality rubric is held and returned to the sending agent for revision. The agent revises and resubmits. If the gate fails three times, the message is escalated to the human Director.

How do agents handle compliance: legal, GDPR, sector-specific?

The Complaint Handler and Service Director have compliance-aware system prompts for the sectors Brad serves. The default prompt includes escalation triggers for legal claims, data-subject requests, and refund disputes above a defined threshold. Sector-specific compliance (financial services, healthcare, legal) requires a custom skill layer and a human review gate on every outbound message in that category. See §15 Security for the full threat model and the data-handling specification.

Glossary

Director agent. The agent at the top of each department's reporting chain. Receives roll-up signals from specialists, makes allocation calls, sends the weekly summary to the CEO or the human Director-equivalent. The Director agent is the only one the client's CEO talks to directly.

Voice contract. A structured document generated in week one containing the brand's reading age, tone parameters, prohibited phrases, preferred sentence structure, and style examples. Stored in the workspace and referenced by every agent before every outbound message.

Approval threshold. The spend or decision limit above which the Director agent must route to a human for sign-off rather than auto-executing. Set as a frontmatter field in the Director agent's definition. Default: £1,000 for spend, configurable per client.

reports_to field. A frontmatter field in every agent's markdown definition that specifies which agent or human the agent sends its roll-up signal to. The substrate reads this field to route signals correctly.

Quality gate. The voice-contract check that every outbound message must pass before it is sent. Gate score must be 0.65 or above. Scores below 0.65 return the message to the sending agent for revision.

Cross-references

§05 Agents. The markdown specification for individual agents, including frontmatter fields, system prompt structure, skill bindings, and the signal lifecycle. The `reports_to` field used throughout this page is defined in §05.

§06 Skills. The per-role skill packs that give each specialist agent its functional capability. The `delegate-task`, `approve-spend`, and `weekly-summary` skills referenced in the Marketing Director code excerpt are defined and published in §06.

§08 Memory. How the voice contract and the signal history accumulate into the corpus that becomes Brad's moat. Every conversation the three departments handle deepens the memory. §08 explains what that memory consists of and why it stays with the agency when a client leaves.

§13 Learning. How the pheromone paths between agents harden over time into highways. The cross-team handoff that improves quarter over quarter is driven by §13's learning loops. After twelve months, the best-performing paths are immutable and on-chain.

Deploy a Marketing, Sales and Service team to one client.

One inbox. Every channel. Every conversation.

Every report your agency writes is only as good as the record behind it. Screenshots are stories you tell. Receipts are proof you show. This page is about the difference, and about why the difference is the renewal meeting.

The shift is simple to describe and hard to execute. When a client's customer messages on WhatsApp, the agency sees it. When the same person posts in the client's Discord community, the agency sees that too, and the system knows it is the same person. When she visits the website the next day and pays on the third, the whole thread is one record. Marketing, Sales, and Service look at the same Sara. The Friday review starts from the same page everyone already trusts.

That is what one inbox means. Not "one dashboard" sitting on top of four separate systems that disagree with each other every Monday. One record. One history. One place to look. The teams described in the previous section keep getting better because every outcome lands back on the conversation that earned it. The inbox is where that learning starts.

The agency owner reading this has spent fifteen years stitching together CRMs, analytics platforms, channel APIs, and quarterly reports. The stitching is most of what the senior team does on a Friday. Take the stitching away and the team is free to do the work the client actually pays for. That is the trade the inbox makes.

One inbox, every channel

The agency's clients operate across channels. A dentist has a website chat, an Instagram DM inbox, and a Google Business Messages account she barely checks. A restaurant has WhatsApp, Facebook Messenger, and a booking form on the website. A window installer has email, a web form, and a sales agent following up by text. None of them have one tool. All of them want one answer.

Every one of those channels feeds into the same inbox. Not a forwarding address. The same live record. Inbound and outbound. Replies, reactions, attachments, payments. All of it in the same feed, attributed to the same customer.

The channel coverage is already substantial. WhatsApp, Discord, Telegram, iMessage, web chat, email, API, and MCP are all live on the current build. Adding a new channel requires a markdown file describing its shape. The agency writes it once. The channel is live the same day. No engineering retainer. No vendor negotiation. No six-week integration project.

Channel	Inbound	Outbound	Group rooms	Attachments	Payments	Rich messages
WhatsApp	Shipped	Shipped	Shipped	Shipped	Shipped	Shipped
Discord	Shipped	Shipped	Shipped	Shipped	Metered	Shipped
Telegram	Shipped	Shipped	Shipped	Shipped	Metered	Shipped
iMessage	Shipped	Shipped	n/a	Shipped	Roadmap	Roadmap
Web chat	Shipped	Shipped	Shipped	Shipped	Shipped	Shipped
Email	Shipped	Shipped	n/a	Shipped	Shipped	Shipped
API	Shipped	Shipped	n/a	Shipped	Shipped	Shipped
MCP	Shipped	Shipped	n/a	n/a	Shipped	Roadmap
CLI	Shipped	Shipped	n/a	n/a	n/a	n/a

Zero new integrations to add another channel that fits an existing adapter. One markdown file for anything new.

This matters in practice because the agency's clients do not stay on the same two or three channels forever. A local restaurant that started with web chat in 2023 now gets half its reservations through WhatsApp. A dental practice that was email-only eighteen months ago now fields questions on Instagram DM. The channel mix shifts faster than any integration project can follow. An agency locked to a fixed set of channels loses ground every time a client's customers move somewhere new. An agency that can add a channel the same day a client asks for it does not lose that ground.

The commercial consequence is direct. The Gartner 2025 CMO Spend Survey found that 22% of CMOs have already reduced their reliance on external agencies for strategy and creativity. The agencies surviving that cut are the ones who own something the client cannot replicate by cancelling a retainer. A unified inbox with two years of conversation history is not something a client cancels their way into. Forrester forecasts a 15% reduction in agency roles in 2026; the agencies in the shrinking half are the ones still stitching tools together for clients who could buy the same tools direct.

The same customer everywhere

The record is only useful if it follows the customer across channels, devices, and time. The inbox does this automatically.

When Sara messages on WhatsApp using her phone number, the system knows it is her. When she clicks a link in an email that went out last week, the system knows it is the same Sara. When she visits the website on her laptop the following day, the system connects the dots. By the time she pays, the record shows the full journey: WhatsApp, Discord community, website, payment. One thread, one customer.

This is identity resolution. The mechanism is a five-rung ladder, each rung adding confidence. At the bottom: a device cookie. At the next rung: a hashed phone number or email. At the top: a verified account, linked across every surface the client operates. The system walks up the ladder automatically, without asking Sara to do anything she would not already do to get her question answered.

The technical term is a pseudonymous identifier that follows a customer across channels without storing raw personal data in a joined table. What matters for the agency is plainer: the client's marketing team, sales team, and service team all look at the same Sara. Nobody is working from a different version of her.

Identity resolution rate target: above 95%. The system warns when it drops below. The account lead sees that warning on the dashboard before the client does, and fixes it before it becomes a question.

The practical consequence is commercial, not technical. When Sara buys in 47 seconds on Tuesday morning, and the record shows her WhatsApp conversation, her Discord post from last week, and the product page she visited the day before, the attribution is clean. Marketing gets credit for the product page visit. Sales gets credit for the WhatsApp close. The report the agency writes on Friday shows the whole journey, not a fragmented collection of touches from three different systems that cannot agree on who Sara is.

Anthony O'Connell, Founder of ONE: "Your CRM is a graveyard of intent. It tells you who showed up and what they bought. It doesn't tell you what they nearly bought, what made them hesitate, or what they wish you sold. AI changes that, if you let it listen."

The inbox lets it listen. Every message Sara sends, every link she clicks, every channel she uses: all of it becomes part of the record. That record is what makes the teams in the previous section get better over time. They are not working from a snapshot taken last quarter. They are working from a living history that updated thirty seconds ago.

The unified inbox in practice: a Tuesday morning trace

Sara is a customer of one of the agency's clients, a small appliance brand. Here is what happened on a Tuesday at 10:14 in the morning.

Sara DMs the client's WhatsApp account: "Is the silver kettle in stock?" The inbox receives the message. It recognises Sara's phone number. She is the same person who posted a question in the client's Discord community six days earlier and visited the product page on the website the day before that. The record is already there. The agent answering the message can see all three touches in a single view, sorted by recency, with the conversation thread underneath.

The conversation routes to the sales qualification team. Inventory check: two in stock. The qualifier drafts a reply in the client's brand voice and sends it on WhatsApp. The same reply appears in the agency's Discord room reserved for live sales activity, so the account lead can see closes as they happen without bouncing between tools. Sara replies: "Great, link me." The qualifier sends a payment link. Sara pays in 47 seconds.

Three things happened that the agency did not have to build.

The first: the customer was recognised automatically. The agency did not match Sara's WhatsApp number to her Discord post manually. The system connected them because both came from the same number, and because the agency configured identity resolution once, six months ago, when the client onboarded.

The second: the agency wrote zero integration code. WhatsApp and Discord were both already in the inbox. The reply going to Discord as a mirror was a setting, not a project. Adding the silver-kettle product page as an attributed touchpoint was a tag on the page, not a quarterly initiative.

The third: the route from "WhatsApp question" to "paid in 47 seconds" is now a complete record with evidence attached. The agency can show the client the conversation, the channel, the time, the agent who handled it, and the payment receipt. That evidence is what the monthly report is built from. The next time a customer like Sara asks a similar question on the same channel, the inbox routes the conversation to the same team that closed this one, because the system remembers which routes close.

The account lead looking at this at 10:14 on Tuesday sees 142 conversations in the last hour, three payments since coffee, and a board that shows first-message-to-first-payment in real time. She clicks one close, sees the WhatsApp thread, sees the payment confirmation, and screenshots it for Friday's review. The screenshot takes 4 seconds. The review now writes itself.

That is the day-in-the-life. The inbox is not a report. It is the thing the report is made from.

Every claim is drillable to the conversation that earned it

The agency's account lead opens Brand X's inbox at 10:14. One hundred and forty-two conversations in the last hour. The "first message to first payment" board shows three closes since coffee. She clicks one. She sees the customer (Sara), the WhatsApp thread from the morning, and the payment receipt. She takes a screenshot for the Friday review.

That one click is the point.

Every number in the monthly report is exactly one click away from the conversation that produced it. Revenue figures drill to the list of conversations, ranked by channel and team. Loss figures show the thread, the moment it went sideways, the agent that handled it. Campaign attribution traces every closed deal back to the first marketing touch. Refund volume drills to the original orders and the customer service threads that followed. Nothing in the report is a number without a story underneath it.

This is not analytics. Analytics tells you what happened. The inbox tells you why, with the conversation as the citation.

The distinction matters when the client asks questions. A client asking "where did this revenue come from" is a different question from "show me the report that claims it came from there." The first is answered by a dashboard. The second is answered only if the conversation is behind the number.

Question the client asks	What the receipt looks like
"Where did this revenue come from?"	List of conversations, ranked, with channel and team that closed each
"Why did we lose this one?"	Conversation thread, the moment it went sideways, the agent that handled it
"Which campaign drove this week's growth?"	Every closed deal traced back to the marketing touch that started it
"Did our brand voice survive?"	Every outbound message scored against the client's voice contract, with misses listed
"How fast did we answer?"	First-response time per conversation, by channel, by team, by hour
"Where do we lose the most customers?"	Conversations that ended without resolution, grouped by stage and reason

Every row in that table is a question a client has asked an agency in a renewal meeting. Every row is also a question that used to take the account lead an afternoon to answer, with three browser tabs open and a spreadsheet on the side that probably had the wrong dates anyway. Now it takes one click. The afternoon goes back to the agency, which can spend it on the next client instead of defending the last one.

What gets tracked, what does not

Not everything is recorded, and what is recorded is protected by design.

The inbox tracks every inbound and outbound message, every payment, every channel the customer used, and every agent that touched the conversation. It tracks when the customer first appeared, which campaign brought them in, and what happened at the end of the thread. It does not store anything that was not part of the conversation.

What is stored by default:

- Every message, inbound and outbound, with timestamp, channel, and agent
- Every payment event with amount, status, and receipt
- Every attributed touchpoint: the campaign link that was clicked, the page that was visited before the conversation started
- The identity record: the set of connected identifiers that follow the customer across channels

What is not stored without explicit configuration:

- Raw personal data in queryable tables: names, emails, and phone numbers are hashed in the event record and stored separately in an encrypted vault
- Any field that matches credential or government ID patterns, rejected at the point of entry

- Special-category data under data-protection law unless the agency has declared a legal basis in writing

The default collection mode is balanced: enough to power a complete record, less than would cause problems with a data-protection audit. Every setting is a dial. Every change is audited. Deleting a customer's record is one command: data across every storage tier is removed within 30 days, with the encrypted vault rendered unreadable in under a second.

The inbox keeps conversations for the life of the contract. If the agency has been managing a client for three years, that client's conversation history goes back three years. When the client asks about a campaign that ran eighteen months ago, the answer is in the inbox, not in a screenshot from a quarterly backup that nobody can find. Historical conversations are searchable, drillable, and attributable on the same terms as last week's. The record does not get worse with age.

Who sees what

The inbox is not one view for everyone. It is four views, each showing the right slice.

The agency sees everything. All of the client's conversations, all channels, full conversation threads, all payment receipts, all quality scores, the attribution trail behind every revenue number. The agency is the operator. It sets what the client and the client's customers see.

The client sees their own conversations. Not another client's data. Only theirs. They see the same inbox the agency sees, filtered to their account. They can see live conversations, completed threads, payment histories, and quality reports. They cannot see the agency's internal configuration, the pricing of credits, or any data from other clients.

The customer (Sara, in the Tuesday morning trace) sees the conversation she is in. WhatsApp shows the WhatsApp thread. Discord shows the Discord thread. She does not know or need to know that both conversations are in the same record. She only knows she got a fast answer and a payment link.

The four-tier structure that controls this (owner, agency, client, end-user) means the agency never has to build access controls for each client separately. The structure is the same for every client. The branding changes. The data isolation is enforced by the platform, not by the agency's configuration discipline. New senior hires never get a chance to misconfigure a sharing rule, because the rule is not theirs to configure.

This is important when a client asks whether their data is shared with other clients. The answer is no, and the reason is structural, not policy. Each client's data lives in its own space. The agency cannot accidentally expose one client's conversations to another. The platform enforces it at the storage level. When the client's general counsel asks for written assurance, the architecture diagram in §15 is the assurance.

Why your client cannot get this from HubSpot, Segment, or GA4

The honest answer is that they can get pieces of it. HubSpot has a CRM. Segment has an event pipeline. GA4 has attribution. But getting them to produce the receipt the client asks about in a renewal meeting requires stitching three or four systems together, keeping them in sync, and maintaining the stitching every time one vendor changes an API. The stitching is invisible until it breaks. It breaks the week of the renewal call, every time.

The comparison is not a feature list. It is an architecture question.

	One inbox (ONE)	The stitched stack
Tools the client pays for	One credit pool	CRM seat + analytics platform + channel APIs + attribution tool
Integrations the agency maintains	One	One per channel per tool
Same customer in every tool	Yes, automatically	If the stitching holds
Who owns the data	The agency	Split across vendors
Time to add a new channel	Same day (markdown file)	3 to 8 weeks per channel
Conversation history at renewal	Two years, drillable	Three exports, two of them missing

The stitched stack is not a theoretical bad choice. Most agencies operate one right now, and it works until it does not. The failure mode is invisible: the same customer appears as three different records in three different tools, and the account lead does not discover this until a client asks a question that requires matching them up manually. The matching is done by a junior on a Thursday night before the Friday review. The junior does her best. The Friday review is one mismatch away from a wrong number on a slide.

BCG's 2025 research found that AI leaders (organisations that have moved past the stitched stack to a unified platform) are achieving double the revenue growth and 40% more cost savings than organisations still running the stitched approach. The gap is not a feature gap. It is a data-architecture gap. Whoever has the complete record gets smarter. Whoever is stitching three records together stays flat. Bain's parallel data on marketing leaders puts the gap even higher: six times the revenue growth at only 1.5 times the marketing spend, a four-times-higher return.

The pricing delta is real too. Consider a mid-size client running 50,000 inbound conversations per month across five channels. On ONE, the conversation cost is folded into the credit price the agency already pays, with no separate line item. A stitched stack for the same client adds a CRM seat (£120/month), an analytics platform (£300/month), and a per-channel API connector fee for each channel (£50 to £150/month per channel, so £250 to £750/month for five). That is £670 to £1,170/month in software licences alone, before the engineering hours to keep the stitching current.

Over a 12-month contract, the stitched stack costs the agency between £8,000 and £14,000 in software licences per client, plus the engineering time it cannot bill. A conservative estimate of an engineer's time to maintain five separate channel connectors and keep them in sync across vendors is four hours per month per client. At an agency blended rate of £80/hour, that is another £3,840 per

year per client: time the agency cannot recover on a fixed retainer. The £80/hour figure is itself conservative; senior engineers run higher, and the engineer who knows where the bodies are buried in the integrations is always senior.

At 90% gross margin on the credit markup, the agency recovers the software cost on ONE with room left over. The savings compound across the client book. An agency with 20 mid-size clients running this calculation saves between £160,000 and £280,000 per year in software licences alone, before the engineering time is counted. That is the margin the agency books as profit instead of routing to vendors. Across a 50-client roll-out, the same calculation lands between £400,000 and £700,000 per year. The CFO does not have to model the productivity gain to find this number; it is sitting in the renewals that no longer require a defence.

That is the pricing math. The receipts math is harder to quantify but easier to feel: the account lead who can answer the client's question in one click versus the one who needs an afternoon has a fundamentally different relationship with the renewal conversation. One is defending a report. The other is showing receipts. Those are not the same meeting, and they do not produce the same renewal price.

Objections answered

"Isn't this just analytics?"

Analytics tells you what happened last week. The inbox decides what to do next. A dashboard that shows 47-second average payment time is interesting. A dashboard that routes the next WhatsApp question to the agent who closed the last three, because the record shows which routes close, is a different machine. Reading the dashboard and improving the product are the same process, not two separate steps. The dashboard updates because the record updates; the routing updates because the dashboard updates. No analyst is in the middle pulling reports on a schedule.

"Can my client keep their existing tools?"

Send their data in, read ours out. A client who has a CRM they are attached to can connect it via a standard webhook. Their historical data stays in the CRM; new conversations land in the inbox. Older tools become reports rather than sources of truth. The agency does not need to tear anything out to get started. Six months in, when the client realises the inbox already contains everything the CRM does plus the conversations, the migration is a settings change, not a project.

"What about GDPR?"

Per-client data stays in per-client space. Deletion is one command and propagates across every storage tier. The encrypted vault is unreadable in under one second, and the full cascade completes within 30 days. Every deletion produces a receipt showing what was removed from each tier. A data protection authority asking for evidence of a deletion request gets the same receipt the client's customer gets. The full threat model is in §15, including the legal basis register and the consent capture flow on every channel.

"Won't 50,000 conversations a month overwhelm me?"

The inbox summarises by route. The account lead looking at 142 conversations in an hour is not reading 142 conversations. She is reading a sorted board that shows the three closes, the twelve in progress, and the two that need a human. The live feed is for drilling, not staring. Conversations not in active use fade from the top of the feed within hours. The inbox is calm by design, not busy by default. Most clients see their account lead spend less time in the inbox than they used to spend in HubSpot, and more time on the calls that matter.

“What if a customer messages on a channel you don’t support?”

The agency adds a short markdown file describing the channel’s shape: inbound event format, outbound format, authentication method. The file is a standard template. The channel is live the same day. There is no engineering retainer needed, no vendor negotiation, no six-week integration project. The agency writes the file and the inbox handles the rest. When a new social platform launches and the client’s customers move there next month, the agency is the first to follow.

“What if the AI gets the answer wrong in front of a client’s customer?”

Every outbound message is scored against the client’s voice contract before it goes out. Below the 0.65 quality threshold, nothing ships. The misses are listed, not hidden. The account lead sees the score for every conversation and can intervene on any thread the system flags as below standard. Mistakes do not leak from the inbox into the client’s brand without the agency seeing them first. The voice contract is the brake; the receipts are the proof that the brake is held.

Two worked examples

The renewal call that did not need defending

An agency managing a national window installation company (nine locations, mixed inbound across WhatsApp, web chat, and email) enters a quarterly review with the client. The client’s sales director has a question: “Did the campaign we ran in March actually work, or did the revenue in April just happen?”

The account lead opens the inbox, pulls the March campaign, and clicks attribution. Every conversation in April tagged to the March campaign is listed in order of close value. She drills one. The customer messaged on WhatsApp on March 14, clicked the campaign link, came back to the website on March 28, and paid on April 2. The thread is there. The payment receipt is there. The 18-day journey from first touch to closed is a single drillable record.

The sales director does not ask the follow-up question. There is nothing to follow up. The evidence is in the room.

That meeting used to take an afternoon of preparation: pulling campaign data from Google Analytics, pulling conversation logs from the WhatsApp business account, pulling payment records from the CRM, and hoping the dates matched up. The preparation now takes one click. The account lead who used to spend that afternoon on preparation can spend it on the next client instead. Across a 20-client book, the account team recovers the equivalent of one full-time week per month, which the agency books as either margin or capacity.

The complaint that resolved in three minutes

A service complaint arrives on Discord. A customer is unhappy about a delayed order and is posting publicly in the client's community server. The inbox picks it up immediately, routes it to the service team, and the agent can see in the same view that this customer also messaged on WhatsApp two days earlier about the same order, and that the WhatsApp conversation ended without a resolution.

The agent does not need to ask the customer to explain the situation again. The full history is in the record: both messages, both channels, the dates, the order reference. The agent picks up from where the WhatsApp conversation left off and closes the complaint in three minutes, publicly, in the same Discord thread where the customer posted. The fix is offered before the customer has finished typing the second complaint message she was already drafting.

The agency's account lead sees the complaint arrive, sees the resolution, and screenshots the thread for the Friday quality report. The client sees that complaints are handled in three minutes with full context. That is a different story to tell than "we have a complaints process." The story comes with a screenshot. The screenshot is the renewal.

Frequently asked questions

How long are conversations kept?

For the life of the contract. If the agency has managed a client for three years, that client's conversation history goes back three years. Retention is configurable per workspace. The minimum is 14 days in the warm tier, and full fidelity is stored in cold storage for two years. Historical conversations are searchable, drillable, and attributable on the same terms as the conversation that arrived an hour ago.

What happens if the same customer messages on two channels at the same time?

Both messages land in the inbox and are attributed to the same customer record. The routing logic decides which conversation to prioritise based on channel urgency and conversation context. The account lead sees both threads in the same record, not in separate rows that require manual matching. If the customer asks the same question on both channels, the agent answers once and the answer appears on both threads.

Can the agency see which agent handled each conversation?

Yes. Every message in the thread is attributed to the agent or team that sent it. The receipt shows the agent, the channel, the time, and the outcome. The quality report for that agent's conversations is available on the same screen. When a senior member of the team leaves and a junior takes over, the new agent inherits a complete history of how the previous agent handled similar customers.

Does the inbox work in real time?

Yes. The live feed updates as conversations arrive. The first-message-to-first-payment board updates as payments land. The realtime update latency is under 50ms in most cases, well within the threshold a human can detect. The account lead watching the board on a Friday morning sees closes as they

happen, which is also when she sends the screenshot to the client's WhatsApp group and the client thanks her for it before lunch.

What does the agency pay per conversation?

Conversation cost is metered through credits, which the agency buys in bulk and marks up for clients. There is no per-conversation fee on top of credits. The agency sets the markup and keeps the difference. A client running 50,000 conversations per month pays the agency's rate; the agency's cost is the credit draw-down, which is a fraction of the revenue. The margin is the agency's to set; the floor is public.

Can the client export their conversation history?

Yes. The conversation history is the agency's data, and the client's data within their workspace. Export is available in standard formats: JSON, CSV, and a printable PDF for the records the client wants to keep on paper. If a client ever moves off the platform, they take their history with them. No platform should hold a customer's data hostage; this one does not.

Can we white-label the customer-facing surfaces?

Yes. The customer never sees ONE's name. The OAuth screen, the payment confirmation, the email footer, and the unsubscribe page all carry the client's brand or the agency's, depending on the tier. The platform disappears. The brand the client and customer see is the one the agency chose.

Glossary

Inbox. The unified view of all conversations across all channels for a given client account. One record per customer, regardless of which channel the customer used.

Channel. Any surface a customer can message through. WhatsApp, Discord, Telegram, iMessage, web chat, email, API, and MCP are currently live.

Record. The single account that follows a customer across channels. Created automatically when a customer is first identified, updated as they use additional channels.

Identity resolution. The process of connecting a customer's activity across channels into one record. Happens automatically using phone number hashes, email hashes, and device cookies. No action required from the customer.

Receipt. The attached evidence for any number in a report. A revenue figure has a receipt: the list of conversations that produced it, drillable to individual threads.

Route. The journey a conversation takes from first message to outcome. The inbox tracks which routes close and at what speed, and the teams described in §09 get better because every outcome is attached to the route that produced it.

Quality score. An automated score given to every outbound message against the client's brand voice contract. Misses are listed, not hidden. The threshold is 0.65; nothing ships below it.

Voice contract. The written definition of how the client's brand sounds. Every outbound message is checked against it before it leaves the inbox. The agency defines the contract; the platform enforces it mechanically.

Conversation. A single thread between a customer and the agency, on any channel, of any length. One inbound message and a closed payment is a conversation. Two hundred messages spread across three channels and a year is also a conversation. The thread is what the system tracks.

Cross-references

§09 Teams. The teams whose performance the inbox tracks. Every outcome recorded in the inbox is a data point for the team's quality score and improvement trajectory.

§11 Analytics. The monthly report is built from the inbox record. Every number in the report is one click from the conversation that produced it.

§15 Security. The full threat model for per-client data isolation, GDPR deletion, and the encrypted vault. The inbox's privacy architecture is documented there.

"The cost of intelligence is collapsing. The cost of waiting is not. Whoever sits closest to the customer, with the shortest gap between question and answer, wins the next ten years." —
Anthony O'Connell, Founder of ONE

The Sara story is the whole argument in 47 seconds. She asked a question on WhatsApp. The agency knew who she was. The right team answered. She paid. The receipt is in the inbox. The next conversation with Brand X starts from a stronger position because this one is on the record.

The agency that runs its clients this way does not dread the renewal meeting. It looks forward to it. Every renewal is a conversation the agency can walk into with the full record in hand, and the client asking the questions already knows the answers, because the agency showed them the receipts. The renewal price goes up; the renewal conversation gets shorter.

Named integration: `web/tracking.md` (full pipeline spec) · `web/tracking-realtime.md` (live feed contract) · `web/tracking-todo.md` (roadmap, C1–C6 shipped 2026-05-14)

See the signal feed in real time.

Analytics

Every renewal call has one turning point. The client's CMO looks at what she paid this month and asks what she got for it. If the answer lives in a slide deck your account lead assembled on Thursday afternoon, pulling numbers from three different tools, writing a one-paragraph summary that says "performance was strong," you are one bad quarter away from a cancellation.

The answer to that question is either a conversation or a receipt. A conversation is defensible until it isn't. A receipt is a record. The platform ships receipts.

One click generates the monthly client report. Four seconds later it exists as a PDF, in your client's brand colours, with every number drillable to the conversation that produced it. The same numbers your account team used to decide what to do next month are the numbers your client reads on page one. Not a summary. Not a proxy. The same row, the same source.

That is what this page is about. Not dashboards in the abstract. The specific artefact, and the specific moment, that wins or loses a renewal.

The monthly client report, one page, every line drillable

The report has six blocks. Nothing more. Every block answers one question a client CFO will actually ask.

Revenue. Cash, not sessions. Cash by campaign, by team, by channel, by skill. If you built a brand-voice-check skill last quarter and it earned £840 in margin in week three without a human touching it, that number is on page one of the report, attributed to the skill, with the campaign it ran in and the channel it converted on. The client's CFO can tell her board exactly what she bought.

Pipeline. The journey from a customer's first message to their first payment, with every drop-off point named. Not "62% conversion rate." The three moments where customers left, and the questions they asked before they went. That is the input for next month's recommendation.

Retention. Customer satisfaction trend, ticket volume, and time-to-resolution. Not as a single score but as a curve. If the trend is improving, the curve shows it. If it stalled in week two when the new FAQ went live, that inflection point is visible. The client doesn't need to ask what happened. The report shows it.

Learning. Which agents improved this month. Which routes hardened. Which new questions customers asked that the agents hadn't seen before. This block is what separates the report from every other monthly PDF a client receives. It says: the system got better on your data this month, and here is the evidence.

Quality. Four scores: security, stability, simplicity, speed. Published to the client. Every score is a number between 0 and 1. Nothing below 0.65 ships. The client never sees a score that represents something broken. She sees the scores of what's live. If the score on a skill improved from 0.71 to 0.84 between months one and three, that improvement is part of the record.

One recommendation. A single change for next month, backed by the numbers in the five blocks above. Not a paragraph of options. One line. One number behind it. One action.

The report ships in under a minute from a single click. No assembly required.

Why one page wins the renewal

The renewal call is a story the agency has to defend. Brad knows this. Every month he has account leads pulling together reports that look slightly different because they were assembled by different people from different tools. The story is consistent only because the account lead is good, not because the data is.

When the story lives in the platform, it is consistent by construction. The numbers do not change between what the account team sees and what the client reads. There is no reconciliation step. There is no "let me check that and get back to you." The client asks a question, the account lead opens the dashboard, and the answer is three clicks away.

That is the shift the renewal call needs. It moves from "here's what we think happened" to "here's the record."

The renewal moment, Brand X, week twelve

The agency runs the monthly report for Brand X. Twelve weeks into the contract. The client's CMO has been happy but noncommittal. Good feedback on the chat surface, no complaints, no clear read either way on renewal.

The account lead clicks "Generate report." Four seconds. PDF in Brand X's colours.

She opens the revenue block. The brand-voice-check skill, built by the agency in week three, sold as a line item on the scope document, earned £840 of margin in week three alone, without a human reviewing a single output. The skill ran 470 checks. Zero were escalated. The compliance team at Brand X had flagged three weeks earlier that off-brand messaging was their biggest operational risk. The skill eliminated it.

The account lead does not need to write a narrative. She forwards the PDF with one line: "This skill we built for you earned £840 in week three. We're raising the price in Q3."

The conversation is no longer "what did you do this month?" It is "this IP we created for you, what's it worth to you going forward?"

That is the renewal moment. The report created it.

The live dashboard, what your client opens any time

The monthly report is the artefact. The dashboard is the view her CMO opens on a Tuesday afternoon when she wants to know how last week performed without waiting for the next report cycle.

The dashboard refreshes in seconds. It shows the same numbers the report will show, accumulated in real time. Revenue by campaign, conversation volume by channel, quality scores, retention trend. Every number is the same number the platform uses to decide what to do next. There is no separate analytics layer. There is no overnight job. The record and the dashboard are the same thing.

The client can set her own refresh cadence: hourly, daily, or on demand. The account team can add a custom date range. The CMO can filter to a single channel, a single campaign, or a single agent.

What she cannot do is change the numbers. The dashboard is read-only. The underlying record is immutable. If she wants to challenge a number, the route is three clicks: number, then the conversations that produced it, then a single conversation with the team member, the channel, and the timestamp.

What three clicks looks like in practice

The client's CMO sees "WhatsApp closed 73% of presale questions" on the dashboard. She clicks the number. She sees the 47 conversations that produced it. She clicks one. She sees the customer, the conversation transcript, the team assigned, the payment receipt where one occurred. Three clicks. No SQL query. No request to the agency's data team. No support ticket.

This is not a demo feature. It is how every number in the system is stored. The conversation is the record. The metric is derived from the record. The derivation is transparent.

The CMO who can drill to the source in three clicks is a CMO who trusts the number. A CMO who trusts the number renews.

What the agency team sees

The agency's account team sees the same dashboard the client sees, plus one additional layer: the operational view. Campaign performance across all clients, credit consumption by workspace, quality scores by agent, and the recommended actions the platform generated this month based on what improved and what didn't.

The operational view does not surface in the client-facing dashboard. The cascade in §01 (Brand) controls what each tier sees. The client sees brand-side numbers. The agency sees ops-side numbers. The platform layer, the credit pool, the markup, stays invisible to both.

What you measure

Six metrics. Each answers a question a client CFO will ask. Each is built from conversations, not synthesised from downstream summaries.

Metric	The question it answers	Where the answer comes from
Revenue per campaign	Did this campaign earn what it cost?	Payment records, attributed to campaign
Revenue per team	Which team generated the most margin?	Conversation outcomes, attributed to agent team
Revenue per channel	Which inbox is converting?	Channel tag on every conversation record
Revenue per skill	Which skills we built are earning?	Skill invocation log, payment outcomes
Retention trend	Are customers coming back?	Return conversation rate, satisfaction score
Quality score	Is what's live performing to standard?	Four-axis rubric: security · stability · simplicity · speed

The revenue metrics are reported in cash. Not session counts, not engagement scores, not “interactions.” Cash. The cash number is the number that appears on the client invoice, reconciled against the conversation record that produced it. If a customer clicked a payment link inside a WhatsApp conversation that was handled by the presales team using the product-recommendation skill, that revenue is attributed to that channel, that team, and that skill simultaneously. One payment, four attributions.

Revenue per skill, the metric that changes the conversation

Most agency reporting shows revenue by campaign. That is useful but incomplete. A campaign bundles creative, placement, targeting, and conversation handling into one number. The client can see the campaign worked or didn't, but she cannot see which component of the campaign drove the result.

Revenue per skill disaggregates the campaign into its components. If the product-recommendation skill ran on 340 conversations this month and produced 87 purchase completions, that is a skill with a measurable conversion rate. If the same skill ran last month with a 41% lower rate, the improvement is visible and attributable: either the skill was updated, the training data improved, or the conversation quality rose. The report shows which.

For the agency, this is the pricing conversation. A skill that earns £840 of client margin in three weeks is not a line item on the original scope. It is a product. It has a price. The monthly report surfaces that conversation.

The four-axis quality score, what it is and why it's published

Security. Stability. Simplicity. Speed.

Each axis is scored between 0 and 1. The gate is 0.65 per axis. Nothing below the gate ships. The client sees the scores of what is live. Not scores for work in progress. Not composite averages. Not aspirational benchmarks.

Security measures whether agent outputs stay inside the defined voice and policy boundaries. An agent that quotes a price the client hasn't authorised, or describes a product specification the brand team has flagged as inaccurate, fails the security score. The score reflects the rate at which outputs stayed within the defined boundaries over the reporting period.

Stability measures availability and consistency. An agent that handled 340 conversations with a 99.1% uptime and produced outputs within the defined quality variance has a high stability score. An agent that produced three escalations in one afternoon has a lower one.

Simplicity measures whether the agent is adding friction or removing it. A conversation that resolved a customer's question in three exchanges scores higher than one that took eleven. The simplicity score captures this over the full month.

Speed measures time from customer message to agent response. The benchmark is under two seconds for the first response and under one second for subsequent messages in an active conversation. Speed degrades when an agent is overloaded or when the skill chain is too long. The score surfaces that before it becomes a customer complaint.

The composite score is the average. A client who sees a quality score of 0.84 knows that every agent handling her customers scored at least 0.65 on each axis and averaged 0.84 across the month. She can compare month three to month one. She can see the trend.

From a number to the conversation that earned it, one click

This is the structural commitment behind every metric in the platform. No number exists in isolation. Every number is derived from a record. Every record is a conversation. Every conversation is one click away from the number it produced.

This matters for one reason: a client who can verify a number doesn't need to trust it on faith. Faith erodes. Verification compounds.

The three-click route applies to every metric in the system:

1. Click the number on the dashboard.
2. See the list of conversations that produced it.
3. Click a conversation to read the full record: customer, team, channel, timestamps, outcome.

No additional access required. No request to the data team. No waiting for the next reporting cycle. The record was already there when the conversation happened. The metric is derived from the record in real time. The dashboard reflects the derivation.

The payment receipt, the record that closes the loop

A payment receipt is not a separate document. It is part of the conversation record. When a customer completes a purchase inside a chat conversation, the payment record attaches to the conversation record that led to it. The receipt includes the amount, the timestamp, the channel, the team, and the skill that was active at the moment of conversion.

When the monthly report shows “WhatsApp generated £4,200 in presale conversions this month,” the £4,200 is the sum of 23 individual payment receipts, each attached to a conversation record, each one click from the revenue number on the dashboard.

A client who asks “where did the £4,200 come from?” gets the 23 receipts, each timestamped, each attributable. That is the conversation the renewal call is built on.

Cohort and channel views without a data team

Most analytics platforms require a data team to build cohort analysis. The agency submits a specification, the data team writes a query, the query runs overnight, and the result comes back as a CSV that someone converts into a chart. This cycle takes between three days and three weeks depending on the team’s queue.

The dashboard has cohort and channel views built in. No query required. No data team. No wait.

Cohort view. Customers are grouped by the month they first contacted the brand. The dashboard shows how each cohort has behaved since then: return rate, average spend, channel preference, escalation rate. A cohort that entered in January and has a 78% return rate by March is a different story from a cohort that entered in February and has a 41% return rate. The comparison is visible without any configuration.

Channel view. Every conversation is tagged with the channel it originated on: WhatsApp, web chat, email, Instagram, Facebook, SMS. The channel view shows revenue, volume, conversion rate, and quality score by channel, for any date range. If WhatsApp is converting at 73% and email is converting at 31%, the account team knows where to focus the next month’s optimisation. If the gap closed or widened last week, the trend is visible.

Both views are available to the client and to the agency team. The client sees her brand’s numbers. The agency team sees the same numbers plus the comparison to the platform benchmark: how this client’s WhatsApp conversion rate compares to the median across all clients in the same industry.

The six standard reports

Report	Question it answers	Where the answer comes from
Monthly performance	What did we accomplish this month, in cash and quality?	All conversation records, payment records, quality scores
Pipeline drop-off	Where are customers leaving before they convert?	Conversation flow records, exit timestamps
Retention curve	How are customers behaving after first contact?	Return conversation rate by cohort
Channel comparison	Which inbox is performing best?	Channel-tagged conversation records
Skill contribution	Which skills we built are earning margin?	Skill invocation log, payment attribution
Learning report	What improved this month?	Agent version log, quality score delta, new question clusters

Each report is available in two formats: the PDF for the monthly client meeting and the dashboard view for real-time access. The numbers are the same in both. The underlying record does not change between formats.

Why the dashboard is also the improvement engine

A dashboard that shows what happened is useful. A dashboard that shows what to do next is a different thing.

The monthly recommendation in the client report is not written by an account lead. It is generated by the platform based on what the data shows. The platform compares this month's performance against last month's, identifies the three largest gaps, and surfaces the one change that would have the highest impact on next month's revenue or quality score.

The account lead reviews the recommendation before it appears in the report. She can override it. She can add context. But the starting point is a number-backed suggestion, not a blank page.

This changes the account lead's job. She is not synthesising data into a recommendation. She is reviewing a recommendation and deciding whether it is right for this client. The synthesis happened in the platform. The judgement is hers.

For Brad, this matters for two reasons.

First, it removes the variable of account lead skill from the renewal conversation. The renewal conversation is not better because the account lead is a better writer. It is better because the starting point, the data and the recommendation, is consistent regardless of who is running the account.

Second, it compounds. Month one, the recommendation is based on thirty days of data. Month six, it is based on six months. Month twelve, the platform has a full year of this client's conversation history, payment history, quality history, and cohort data. The recommendation in month twelve is not a

guess. It is an extrapolation from twelve months of evidence.

The agencies that renew at 90% are not the ones with the best account leads. They are the ones where the data makes the renewal conversation easy.

How the improvement loop closes

The recommendation becomes a change. The change runs for a month. The next report shows the result. The result feeds the next recommendation.

The client sees it as: "last month you suggested we focus the presales team on WhatsApp. We did. Conversions on WhatsApp rose from 61% to 73%. Revenue from WhatsApp increased by £1,400 month-on-month."

That is the sentence that wins the renewal call. Not because it describes software. Because it describes a business result, traced to a decision, traced to a recommendation, traced to a number. The chain is unbroken.

Comparison: platform-native dashboard vs stitched stack

The alternative to a platform-native dashboard is a stitched stack: a CRM for the pipeline, a third-party analytics tool for the conversion metrics, manual queries for the attribution, and a slide deck assembled for the monthly meeting. Most agencies run this stack today.

Measure	Platform-native dashboard	Stitched stack
Time to produce monthly report	Under one minute	4–8 hours of manual assembly
Drillable to source conversation	Yes, three clicks	No, would require raw database access
Same numbers the optimisation system reads	Yes, one record	No, analytics runs on a copy
White-labelled in client brand	Yes, by default	Requires custom build
Cohort analysis without data team	Yes, built in	No, custom query each time
Agency gross margin on reporting	100%, no additional tool cost	Negative, tool seats reduce margin
Recommendation generated from data	Yes, platform-generated, account lead reviews	No, account lead writes from scratch
Data stays with agency on exit	Yes, portable export	Partial, depends on tool contracts

The stitched stack produces the same output as the platform report, with eight hours of labour, one tool licence, and a version of the numbers that is not the same version the optimisation engine reads. When a client asks “why does the report show X but the dashboard I was given access to shows Y?”, the answer in the stitched stack is “different data sources.” The answer in the platform is “you cannot see two different numbers because there is only one record.”

That difference is worth more than the stack costs.

Day in the life: Acme Agency, Friday, 16:45

The account lead at Acme Agency is wrapping up her week. She has four client reports due before Monday. Under the old process, this is a three-hour Friday-afternoon job: pull the CRM export, match it against the analytics tool, assemble four separate decks, write four separate summaries.

Under this process: she opens the platform. Selects Brand X. Clicks “Generate monthly report.” Four seconds. PDF in Brand X’s colours, their logo, their brand blue, their typeface. She skims the revenue block. The brand-voice-check skill earned £840 in week three. She reads the quality score section. The presales agent team improved from 0.71 to 0.84. She reads the recommendation: focus the next month’s budget on WhatsApp presales, where conversion rate improved 12 points month-on-month.

She agrees with the recommendation. She forwards the PDF to her account lead with one line: "Raise the price on the brand-voice-check skill in Q3."

Whole cycle: 90 seconds. She does the same for the other three clients. She is done by 16:53.

Monday morning, Brand X's CMO opens her email. She reads the one-page report. She clicks through to the dashboard to check last week's numbers. She sees the same curve the report described. She opens one of the WhatsApp conversations the report flagged. She reads it. It is exactly what she would have wanted.

She replies to her account lead: "Happy with progress. Let's talk about scaling the WhatsApp team next quarter."

That is a renewal. It happened because the report was accurate, specific, and drillable. Not because anyone wrote a good narrative.

Failure modes

The dashboard does not replace the account lead's judgement. It informs it. Three places where the account lead still needs to think.

When the recommendation is directionally right but contextually wrong. The platform might recommend increasing WhatsApp budget based on a 12-point conversion improvement. The account lead knows that the improvement was driven by a single campaign that has now ended, not by a structural change in the channel. She overrides the recommendation and notes the context in the report. The platform does not know about the campaign end. The account lead does.

When the quality scores look good but the client relationship is fragile. A quality score of 0.84 reflects technical performance over the reporting period. It does not reflect the client's CMO who had a difficult call with the account lead last week, or the competitor who pitched her this month. The dashboard is a data view, not a relationship view. The account lead carries the relationship.

When the client wants a number the platform doesn't collect. If a client asks for brand sentiment across social channels that are not connected to the platform, the dashboard does not have that number. The platform tracks what flows through the channels it is connected to. Channels outside the platform are outside the record. The account lead names this clearly rather than producing a proxy metric that does not mean the same thing.

These failure modes are not surprises. They are the expected boundaries of a data system. Naming them to the client in the onboarding call, and again in the first monthly report, is the right move. A client who understands what the dashboard measures and what it doesn't will not feel misled when a number she expects isn't there.

Objections answered

"My client wants their own BI tool."

She can have it. The platform exports data on a schedule: daily, weekly, or triggered by a date range. The export is a structured record: conversations, outcomes, payment records, quality scores, timestamps. The client's BI team imports it into whatever tool they prefer. The platform remains the source of truth. The BI tool is a view on that source. If the client's BI tool shows a number different from the dashboard, the right number is in the platform. The BI tool is working from an export, not the live record.

"How do I know the numbers are right?"

Every metric is one click from the underlying conversation. There is no aggregation that is not traceable. If the monthly report shows "WhatsApp closed 73% of presale questions," clicking that number opens the 47 conversations. Clicking any conversation opens the full record: customer, transcript, team, payment receipt if applicable. The derivation is not hidden. The account lead can walk the client's CFO through the three-click route in the first monthly call, and the CFO can verify any number she chooses. Verification is built into the report, not added on request.

"Doesn't every vendor say 'self-optimising'?"

Yes. Here is what "self-optimising" means in this platform, specifically. The learning report block in the monthly report names the agents that updated their approach this month, the routes that hardened based on higher-performing outcomes, and the new question clusters that customers asked which had not appeared in the previous month's data. These are not summaries. They are the output of a process that ran on the client's conversation data, produced a change, and documented the change. The client can see the change in the quality score trend. If the quality score on the presales agent team rose from 0.71 to 0.84 between months one and three, the learning report explains what changed to produce that improvement. "Self-optimising" here means: the system changed something, and the change is in the record.

"Does my client see the same view as my account team?"

By default, yes. The numbers are the same. The cascade controls who sees which layer. The client sees brand-side numbers: her campaigns, her agents, her quality scores, her revenue. The agency team sees the same numbers plus the operational layer: the markup, the credit consumption, the recommended actions. The platform layer, the credit pool, stays invisible to both. A client who opens the dashboard and an account lead who opens the same dashboard are looking at the same numbers from different vantage points. Neither is seeing a summary of the other's view.

"What about the quality scores. Won't a client see a low number and panic?"

Nothing below the quality gate ships. The gate is 0.65 per axis. If an agent scores 0.58 on the stability axis this month, it does not run on live customer conversations until the score is above the gate. The client sees the scores of agents that are live. A live agent with a score of 0.65 is performing at the minimum standard. A score of 0.84 is above the standard and has been improving. The trend is what matters. A score that started at 0.68 in month one and reached 0.84 in month three is a story of improvement. That story is visible in the dashboard and in the monthly report. If a client asks "what does 0.84 mean?", the answer is: it means the agents handling your customers passed four performance checks this month at an average of 0.84 out of 1.0, and nothing that scored below 0.65 touched a customer conversation.

“Won’t my clients compare their scores to other clients?”

They cannot. Each workspace is isolated. The dashboard shows one client’s numbers. The platform benchmark, the median score across clients in the same industry, is visible to the agency team but not to the client. The client sees her own trend, not a comparison table. This is deliberate. Comparative benchmarks create the wrong incentive: she starts optimising for the benchmark rather than for her customers.

“What if I have a bad month and the report makes it obvious?”

It will. A report that is accurate when performance is good must also be accurate when performance is poor. The pipeline drop-off report will show where customers left. The quality score will reflect the month accurately. The recommendation will name the largest gap. This is the correct result. An account lead who can show a client a month where performance fell short, explain the cause using data from the platform, and present a specific recommendation for next month is having a better conversation than one who produces a report that softens the numbers. The client who sees accurate bad-month reporting and a specific recommendation is more likely to renew than one who sees a polished report that doesn’t quite add up.

Export to your client’s BI tool

The export function is one field in the workspace settings. The agency sets a destination (a webhook, a storage bucket, or a direct connector to a standard BI tool) and a schedule. The platform exports a structured record on that schedule.

The export includes:

- Conversation records, with channel tag, team attribution, skill attribution, timestamps, and outcome
- Payment records, with amount, currency, conversation reference, and timestamp
- Quality scores, per agent and per axis, for the reporting period
- Cohort data, with first-contact month and subsequent return activity
- Learning report data, with agent version changes and quality score deltas

The client’s BI team imports this record into their preferred tool. The platform does not configure the BI tool. It does not maintain the connection beyond the export. The client’s team owns the BI layer.

For clients who want a direct database connection, the export includes a read-only credential to a view of the workspace record. The view is the same data the dashboard displays. It refreshes on the same schedule. The client’s BI tool queries the view. The platform manages the view’s underlying record.

What the agency retains

When a client exports data to their BI tool, the source of truth remains in the platform. The export is a copy. If the client cancels the contract, they take the export. The platform retains the conversation record for the contractual retention window. After the window, deletion is one command.

The agency retains the skills, the agent definitions, and the conversation structure they built for this client. These are the agency's IP. They live in the agent definition files, not in the client's data. The client's corpus, the conversation history, the payment records, the quality scores, belongs to the client. The agency's methods, the skills, the team configurations, the voice contracts, belong to the agency.

FAQ

Can I white-label the monthly report in my own brand rather than my client's?

Yes. The report generator uses the brand tokens of the workspace it runs in. If you run it from your agency workspace against a client's data, it uses your tokens. If you run it from the client's workspace, it uses the client's tokens. Most accounts run client-branded reports for client-facing delivery and agency-branded reports for internal review.

Can the client generate the report themselves, or does it require the account team?

By default, the client has access to the dashboard but not the report-generation function. The report is generated by the agency. You can grant report-generation access to the client if you choose. It is a permission setting on the client workspace. Most agencies keep report generation with the account team so they can review the recommendation before the client sees it.

How is revenue attributed when a single customer contacts us across multiple channels?

The platform uses the channel active at the moment of conversion. If a customer started on web chat, moved to WhatsApp, and completed a purchase in the WhatsApp conversation, the revenue is attributed to WhatsApp. The pipeline report shows the full journey across channels before conversion, so the account team can see which channel initiated the relationship even if a different channel closed it.

What happens to the dashboard if a customer conversation is manually reviewed and updated?

The dashboard reflects the live record. If a conversation record is updated (for example, a payment status corrects from pending to completed) the dashboard reflects the update. The report, once generated, is a snapshot of the record at the time of generation. If you regenerate the report after the update, the new report reflects the corrected record.

Can I set different quality gates for different clients?

Yes. The gate is set at the workspace level. The platform default is 0.65. You can raise it. If a client's brand requires a higher consistency standard, you set the gate at 0.75 or 0.80. Agents that score below the client-specific gate do not run on live conversations for that client. The quality score shown in the client report reflects the client-specific gate.

How long does historical data remain accessible?

The retention window is set at the workspace level. The default is twelve months of active access plus a thirty-day export window after contract end. You can extend the retention window for clients who require longer access (legal, compliance, or industry-specific requirements). Extension is a setting in the workspace configuration.

Does the platform track conversations that happened before we connected the channels?

No. The record begins when the channel connects. Historical data from a channel that was previously managed in a different system needs to be imported via the bulk import function if you want it to appear in cohort analysis. Imported records are flagged as imported and excluded from quality scoring, since the platform did not handle those conversations.

What if my client's industry has specific reporting requirements I can't find in the standard reports?

The six standard reports cover the metrics most clients ask for. If a client requires a report format that does not exist (for example, a specific compliance report for a regulated industry) the export function provides the underlying data for your team or the client's compliance team to format as required. The platform does not generate custom compliance reports beyond the standard six.

Glossary

Conversation record. The full record of a customer interaction: the channel, the messages, the team, the skill invocations, the outcome, and the payment receipt if applicable. Every metric in the dashboard derives from conversation records.

Quality gate. The minimum score an agent must achieve on all four quality axes before it handles live customer conversations. Default: 0.65. Client-configurable above the default.

Quality score. A number between 0 and 1 on each of four axes: security, stability, simplicity, speed. Published to the client in the monthly report. Reflects the performance of live agents only.

Revenue per skill. The total cash attributed to a specific skill over the reporting period, calculated from the payment records of conversations where the skill was active at the moment of conversion.

Pipeline drop-off. The points in the conversation flow where customers exit before completing a desired action: a purchase, a booking, a form submission. Visible in the pipeline report as exit timestamps at specific conversation stages.

Cohort. A group of customers defined by the month they first contacted the brand. Cohort view tracks how each group has behaved in the months since first contact.

Learning report. The section of the monthly report that documents what changed in the agent system this month: agent version updates, quality score changes, and new question clusters that the agents had not previously encountered.

Export. A structured copy of the workspace record, sent to a destination on a schedule. The export is a copy. The platform record remains the source of truth.

"Campaigns are not just optimized — they are self-optimizing, constantly seeking the event horizon of maximum performance." — Versaunt, October 2025

The monthly report is the proof. Not the claim. The proof. Every number in it is a receipt. Every receipt is one click from the conversation that produced it. Every conversation is a record the client can read in full. The renewal call is not a negotiation. It is a review of the record.

That record is what the platform produces every month, for every client, in four seconds, without anyone assembling it by hand.

Cross-references

§10 Tracking. One inbox, every channel. The record the dashboard reads is built by the tracking layer. Every channel view in the dashboard reflects the conversation record from §10.

§09 Teams. The team structure behind the revenue attribution. Revenue per team in the monthly report maps directly to the dream-team configuration in §09.

§13 Learning. The improvement engine that produces the learning report block. The agents that improved this month, documented in §11, are the product of the learning cycle in §13.

Named integration: `web/agent-analytics.md` · `web/agent-analytics-todo.md` · quality score definition in `one/rubrics.md`

Open the dashboard your client will see.

The CRM Is Already Running

You pay for HubSpot. You pay for the HubSpot seat per client. Then you pay someone to keep it clean. Then you wonder why a contact in HubSpot and a conversation in your inbox are describing two different people.

There is a simpler possibility. What if the thing that routes every conversation also holds every record? What if the database your agents write to is already the CRM, and the CRM is just a different lens on the same data?

That is what ONE does. Not as a feature. As a consequence. The substrate stores contacts as actors. It stores conversations as signals. It stores deals as paths with pheromone on them. When a lead opens a chat, talks to an agent, gets a proposal, and buys, every step writes to the same brain. No sync job. No duplicate record. No separate vendor.

For an agency with 200 clients, that is one fewer CRM seat per client. One fewer renewal negotiation. One fewer password to rotate. One fewer data export to wrangle every time a client asks “what happened with that lead from March?”

The answer is already in the substrate. You open the actor view.

“Your CRM is a graveyard of intent. It tells you who showed up and what they bought. It doesn’t tell you what they nearly bought, what made them hesitate, or what they wish you sold. AI changes that — if you let it listen.”

— Anthony O’Connell, Founder of ONE

Actors as records (no separate CRM seat)

The substrate has six dimensions. The second is actors. Every contact your agents have ever handled is an actor row. Not a “contact record” in a parallel database. An actor in the same brain that routes signals, hardens paths, and decides what to do next.

The schema ships with these fields live: `name`, `email`, `phone`, `lifecycle`, `fit-score`, `ltv`, `consent-channels`, `channels`, `dormancy-state`, `actor-type`. The actor type covers both humans and agents. Both can be clients. Both can be in a pipeline.

From the contact detail page shipped on 14 May 2026:

```
/crm/c/:actor
```

```
ADA LOVELACE
```

```
ada@l.org · +44 7700 ...
```

```
lifecycle: customer  fit-score: 0.82  ltv: £4,210
```

```
consent: email ✓  sms ✓  push x  call x
```

```
channels: telegram@ada  discord:ada#1234  web:visitor_hash...
```

```
► identity      same-as: email-hash, phone-hash, visitor_hash
► stack         personas: founder/dev-tooling, awareness: 4/5
► groups        audience:enterprise  segment:trial-d14
► paths         entry:hn-launch → trial → upgrade  (strength 0.74)
► appended      clearbit:title=CTO  apollo:fund=£12m  stripe:plan=pro
► activity
  12:04 click    /pricing    campaign:apr-q2  utm:hn
  12:01 open     email E3-trial-d7
  yest  reply    telegram    "is there an MCP server?"
```

Every field on that screen is a TypeDB attribute or a derived view over signals already stored. There is no separate contact table. There is no nightly sync. Changing the contact's lifecycle from `sql` to `customer` writes a signal back to the substrate, which marks the path, which strengthens the route from that stage to the next. The record and the learning are the same thing.

For Brad, the arithmetic is direct. 200 active clients, each with their own HubSpot seat, each with a license fee, each with a contact who calls every quarter asking "can you export my data?" One substrate. One actor table. Every client's contacts live in their own workspace with role-based access at the 4-tier level: owner sees all, agency sees clients, client sees their own end users, end users see themselves. No migration required. No seat to add when the client's list grows by a thousand names.

Eight CRM-standard actor attributes are live. The enrichment layer adds more. The identity ladder resolves the same person across channels (the visitor who clicked an ad, the email that opened the campaign, the Telegram account that asked a question) into one actor with a `same-as` relation, confidence-scored. Identity resolution targets above 95%.

Conversations as history (auto-logged)

Every message in every channel is a signal. Signals have a sender, a receiver, a timestamp, and a payload. The activity feed on a contact record is a filtered view over the signal stream for that actor. It is not a separate log. It does not need to be exported. It does not go stale.

When the agent replies to a Telegram message, the reply is a signal. When the email is opened, the open is a signal tagged `touch:open · channel:email · campaign:apr-q2`. When the lead clicks the pricing page, the click is a signal tagged `touch:click · path:/pricing`. When the deal closes, the purchase signal `mark()` s the path from `sql` to `customer` with the purchase value.

The activity feed loads from D1 (the hot path, <100ms) with the last 50 signals. It is live. The pheromone dots next to the contact's name in the list view reflect the strength of their strongest inbound path (five dots for a converting customer, one dot for a cold lead) and they update in real time.

For an agency managing a dentist client in Brisbane, that means every chat conversation that dentist's receptionist has with the chatbot is captured. Every web visit from a patient, every Telegram inquiry, every appointment request. All signals, all on the same actor record. When Brad's team needs to know "what happened with Mrs Chen from March?" they open the actor view. They see every touch in order, every channel she used, every word the agent said, and whether she converted.

No manual entry. No "please log the call notes in HubSpot." The agents log everything. The humans review.

The signal stream includes channel, event type, campaign tag, UTM attribution, agent response, latency, pheromone deposit, and path strength change. For a client whose CRM currently holds "called 2024-03-12, left voicemail" and nothing else, this is a different class of record.

The pipeline writes itself

A CRM pipeline is a sequence of stages. Traditionally, someone moves a deal card from "Lead" to "Qualified" to "Proposal Sent" to "Won." That motion requires a human. It requires discipline. It requires that the human be aware of every conversation happening across every channel and remember to update the card.

The substrate replaces all of that with one mechanic: signal-driven stage transitions.

When a new contact fills in a form on the client's site, the form submission fires a signal tagged `intent:high · channel:web`. The CRM agent evaluates the actor's lifecycle field. It is currently `anonymous`. The signal pattern matches the rule for `anonymous → lead`. The actor's lifecycle attribute updates. The path from that source gains pheromone. The contact appears in the "new leads" segment. No one moved a card.

When the lead asks "can you send me a proposal?" in the chat, the NLU layer classifies the intent. The signal lands as `intent:proposal-request · channel:chat`. That matches the rule for `lead → sql`. The actor's lifecycle updates. A signal fires to the proposal agent: build and send the proposal. The proposal agent generates it, sends it via the best-consented channel, and logs the send as a `touch:send` signal. The path from `sql` to the proposal stage gets marked. The deal record (a `thing` of type `offer`) is created and linked to the actor.

None of that required a human. A human can override any of it. The rule that fired is visible on the contact record: "Lifecycle advanced to `sql` because: `intent:proposal-request · channel:chat`." The human can reverse the decision, which writes a signal tagged `actor:override · stage:sql`, also logged.

The pipeline is the substrate's path graph, rendered as a funnel. What looks like a Kanban board is pheromone. The thickness of the edge from one stage to the next is the actual strength of that path, built from real outcomes across real contacts.

For Brad's 200 clients, the pipeline across all seven ICPs (dentists, accountants, window-and-door, restaurant, MDU, multi-location commercial) runs the same substrate. Each ICP gets its own agent persona and its own lifecycle rules. The pipeline for a dentist client is different from the pipeline for an accounting firm. The substrate handles both, in the same brain, with the same six verbs. No custom CRM instance to configure for each vertical.

Stage transitions (signal-driven, not manual)

The six lifecycle stages are locked vocabulary: `anonymous` → `lead` → `mql` → `sql` → `customer` → `advocate`. The `churned` state is a branch off `customer`. Dormancy is orthogonal: `active` | `warming` | `cold` | `reactivated` | `lost`, driven by the fade loop (L3) on last-seen date and path strength.

Stage transitions happen when a signal matches a rule. The rule lives on the path between stages, with a signal pattern (event type, tag, channel, weight) and a confidence threshold. When an incoming signal matches pattern and confidence, the transition fires.

The six standard CRM verbs map directly to substrate operations:

CRM verb	Substrate operation	What fires
Capture	<code>signal</code> → actor upsert	<code>anonymous</code> → <code>lead</code> on intent signal
Qualify	<code>mark(edge)</code>	<code>lead</code> → <code>mql</code> on engagement score
Contact	<code>signal</code> → outbound send	<code>touch:send</code> on best-consented channel
Propose	<code>thing:offer</code> created	<code>mql</code> → <code>sql</code> on proposal-request intent
Close	<code>mark(edge, value)</code>	<code>sql</code> → <code>customer</code> on purchase signal
Retain	<code>fade</code> + <code>follow</code>	Dormancy detection, re-engagement routing

Which agent owns which verb is configured per workspace. By default: `crm` agent owns qualify and capture; `email` agent owns contact; the selling agent owns propose; `crm` agent marks close; the learning loop owns retain.

The signal pattern that advances `mql` → `sql` is configurable. For a dentist client it might be "appointment-request OR callback-request." For an accounting firm it might be "proposal-request OR 'fees' mentioned in chat." The agent running the dentist's practice does not need a human to tell it

that “can I book a consult?” means “advance this lead.” The model classifies the intent, the substrate fires the transition, and the path records it.

A human override is one keystroke away. The CRM contact screen shows the rule that fired. The human reverses it. The reversal is itself a signal, which feeds back to the model: “this classification was wrong for this client.” The next time a similar message arrives in that workspace, the model’s prior shifts.

This is not automation in the 2015 sense. Automation in 2015 meant “if lifecycle = lead, send an email after 3 days.” Signal-driven transitions mean “when this pattern of signals suggests the actor is ready to move, advance the stage, and tell me which rule fired.” The first is a cron job. The second is judgment.

Enrichment (from public sources and tool integrations)

An actor record starts with whatever the person volunteered. Name, email, phone if they gave those. Channel handles from the platforms they arrived on. A `visitor_hash` from the cookie if they arrived on web without identifying themselves.

Enrichment fills the rest. When an actor’s lifecycle transitions to `lead`, the import agents fire: `import-clearbit`, `import-apollo`, `import-stripe` (if there’s a Stripe record), `import-shopify` (if the client is e-commerce), `import-ga4`. Each writes attributes back to the actor in the `appended`. `<source>.<field>` namespace.

The result on the contact screen:

```
► appended
  clearbit:title      CT0
  clearbit:company    Engine Ltd
  apollo:fund         £12m seed (2024)
  stripe:plan         pro
  stripe:ltv          £1,840
```

Provenance is preserved. Each attribute records which agent wrote it and when. Re-enrichment is a fade-then-write: the old value fades, the new value lands. There is no silent overwrite. If Clearbit says the person is a “VP Marketing” and Apollo says they are a “CTO,” both attributes live, with source. A human can pin the preferred value.

Enrichment fields per actor (across Clearbit, Apollo, Stripe, Shopify, GA4, and the identity ladder) run to more than 40 distinct attributes. The namespace is open: adding a source means adding an importer agent that writes to `appended.<newsouce>.<field>`. No schema migration. No new CRM table.

For Brad’s clients, enrichment means a dentist’s new lead (who arrived via a Google ad, gave their name and email, and asked about pricing) can be enriched within seconds with their approximate company size, location, and whether they have an existing practice management software account. The agent that sends the follow-up message knows that. The message is different.

Enrichment is event-driven, not batch. There is no “nightly enrichment run” that may or may not have run by morning. When the lead arrives, the enrichment fires. The contact record is complete before the first outbound message goes out.

Handoff to human (when and how)

The substrate handles conversations that match patterns. When a conversation falls outside the agent’s confidence gate (below the 0.65 rubric threshold) it routes to a human. That routing is itself a signal: `actor:escalate · reason:low-confidence · channel:<x>`. The human receives the full context: every signal in the thread, every attribute on the actor, the agent’s response attempt and confidence score, and the suggested next action.

The inbox at `/crm/inbox` is a reverse-segment: all inbound signals tagged `inbound AND closed:false`. Replying from the inbox sends a signal back on the correct channel and `mark()` s the path. The loop closes. The pheromone records that a human handled this type of conversation, which improves routing for the next similar signal.

The handoff triggers for three reasons.

First: the agent encounters an intent it cannot classify. The model’s confidence is below threshold. It flags for human review before sending any response. The contact does not see a wrong answer from the bot. They see nothing until the human confirms the reply.

Second: the actor is flagged as high-value. A `fit-score` above a threshold, or an LTV above a configured floor, routes escalation regardless of confidence. High-value clients get a human in the loop by default.

Third: the signal contains a complaint, a refund request, or a keyword matching the escalation list. The compliance agent intercepts and routes to human. The escalation event is logged.

In all three cases, the human receives the full conversation thread, the actor’s lifecycle stage, their consent channels, their enriched attributes, and the agent’s draft reply. They review the draft, edit it, and send. Or they discard the draft and write their own. Either way, the signal closes the loop.

For an agency managing 200 clients, the inbox aggregates escalations across all workspaces the team member has access to. The list sorts by actor LTV descending. A £4,200 LTV customer’s unanswered Telegram message appears at the top. A cold lead’s email inquiry appears at the bottom. The human processes the list in priority order. No client’s inbox goes unmonitored. No message falls through because it arrived on the wrong channel.

Importing an existing CRM

The question Brad will ask is not “how do I use this?” It is “what do I do with HubSpot?”

Nothing changes immediately. The substrate and HubSpot run in parallel during the transition. The substrate accepts inbound contacts from any CSV export. The importer agent maps HubSpot field names to substrate attributes: `firstname + lastname → name`, `email → email`, `hs_lead_status → lifecycle`, `hubspot_owner_id → owner tag`. A standard HubSpot export maps in one pass.

The write-through path goes the other way: the substrate pushes actor updates back to HubSpot as signals arrive. When a contact advances from `mql` to `sql`, a signal fires to the `export-hubspot` agent, which updates the HubSpot lifecycle stage via the HubSpot CRM API. Brad's sales team keeps their HubSpot view during the transition. The records stay in sync.

Over time, the substrate becomes the source of truth. HubSpot becomes a read-only reporting view. That transition happens at Brad's pace, client by client, as the substrate accumulates enough signal history that its view is richer than HubSpot's. For most clients, that point arrives within 90 days of the first conversation.

The five objections, answered directly.

"My agency already uses HubSpot. Am I migrating?" No. You run both in parallel. The substrate receives all inbound signals and pushes lifecycle updates to HubSpot via the export agent. Your team sees HubSpot until they prefer not to. The migration is gradual and reversible at any point before you decide to stop paying the HubSpot seat.

"Will my client's data transfer cleanly?" The standard export maps in one pass for HubSpot, Salesforce, and GoHighLevel. The importer agent handles the field mapping. Duplicates resolve via the identity ladder: same email, same actor.

"What if HubSpot has data the substrate doesn't support?" HubSpot custom properties map to the `appended.hubspot.<field>` namespace. Preserved, searchable, visible on the contact screen. Nothing is lost.

"Can I keep HubSpot for specific clients?" Yes. The sync runs per workspace. A client on HubSpot stays on HubSpot until you move their workspace. The two coexist.

"What happens to my HubSpot workflows and sequences?" They keep running during the transition. As you move clients to the substrate, their automation moves to journeys. A HubSpot sequence is a journey with stages. The migration is a mapping exercise, not a rebuild.

A day in the life

It is 8 a.m. Brad's agency manages the marketing for 47 local-service businesses: dentists, accountants, window-and-door installers. The overnight inbox has 14 escalations: messages that arrived while the agents were running but fell below the confidence threshold or hit the high-value routing rule.

Brad's account manager opens the inbox. The first escalation is from a dentist client's top lead: a conversation that started at 11 p.m., five messages deep, asking about pricing for a specific service the dentist offers. The agent attempted a response, scored 0.61 confidence, flagged for human

review. The draft reply is there, pulled from the service description and pricing sheet in the agent's memory. The account manager reads the thread, adjusts the price quote, and sends it. 45 seconds. The loop closes. The path from `pricing-inquiry` to `qualified-lead` gets marked.

The second escalation is a complaint from an existing customer of the window-and-door client. The compliance agent flagged it on "refund request." The issue is straightforward: wrong size delivered. The account manager replies with the resolution process and flags the order number for the client's operations team. The signal logs as `complaint-resolved`. The actor's dormancy state stays `active` rather than flipping to `cold`.

By 8:45 a.m., all 14 escalations are cleared. The rest of the morning's conversations (several hundred of them) were handled by agents without human involvement. They are in the contact records, marked, pheromone deposited, paths strengthened.

The account manager opens the CRM view for the dentist client. The pipeline shows 6 new leads from overnight, 2 of which advanced to `mql` on their own. The segment "trials-d7, dentist" has 8 contacts. The agent sent them each a Telegram follow-up at 7 a.m. on the campaign schedule. Three opened it. One replied. That reply advanced the actor to `sql`. A proposal is being drafted by the proposal agent.

None of this was set in motion this morning. It was running before they arrived.

Worked example

A lead opens the chat on a dentist client's website at 11:42 p.m. on a Thursday.

She types: "Hi, I'm looking for a family dentist, do you take new patients?"

Step 1. Actor upsert. The chat fires a `signal` to the claw worker. The signal carries no identity yet. A `visitor_hash` is generated from a first-party cookie. An anonymous actor is created with that hash. Lifecycle: `anonymous`.

Step 2. Agent handles the conversation. The dentist's chat agent responds: the practice is taking new patients, here are the appointment types, what age are the family members? The conversation continues for four exchanges. The actor's activity feed fills up. Every message, every agent reply, every timestamp.

Step 3. Email capture. The agent asks if she'd like to book a consultation. She says yes. The agent asks for her name and email so the practice can confirm. She provides both. The email lands on the actor: `email = her@address.com`. The identity ladder fires a `same-as` relation between her `visitor_hash` and her `email-hash`. Confidence: 1.0 (deterministic match). The actor is no longer anonymous. Lifecycle transitions to `lead`.

Step 4. Enrichment. The `import-clearbit` agent fires on the `lifecycle:lead` signal. Within 3 seconds, the actor record gains: `appended.clearbit.location = Brisbane`, `appended.clearbit.persona = parent/family`. The enrichment is visible on the contact screen before the account manager arrives in the morning.

Step 5. "Send me more information." Near the end of the conversation she says: "Can you send me some information about the hygienist appointment process?" The NLU layer classifies this as `intent:proposal-adjacent`. The agent sends a formatted message with the hygiene overview and a tracked link to the booking page: `https://dentist.clinic/go/9mK`. The signal logs as `touch:send` · `channel:web-chat` · `campaign:new-patient-flow`.

Step 6. Tracked link click. The next morning she clicks the link. The `/go/9mK` Worker resolves the HMAC, writes a `touch:click` signal to the actor record, sets the first-party cookie on the dentist's domain, and redirects her to the booking page with `?oid=<hash>` so the page pre-loads her name and appointment context. No login required.

Step 7. Appointment booked. She completes the booking form. The form submission fires a signal tagged `intent:purchase` · `value:120`. The CRM agent evaluates: this matches the `sql → customer` rule. Lifecycle advances to `customer`. The path from `new-patient-flow → consultation-booked` gets marked with strength proportional to the appointment value. The monthly report will show this conversion.

Total time from first message to conversion: 18 hours. Human involvement: zero. Every step is in the contact record. Every signal is attributable to a campaign. The path from that campaign to that appointment value has pheromone on it. The next lead who follows the same path will be routed faster.

Comparison: substrate CRM versus traditional CRM

The comparison that matters for Brad is not feature-by-feature. It is unit-economics-by-unit-economics.

A HubSpot Professional seat for one client workspace runs around \$800/month for a full feature set (marketing, sales, service hubs combined). At 50 clients, that is \$40,000/month before add-ons, before contact tier surcharges, before onboarding fees.

The substrate does not charge per workspace. It charges per credit consumed. A conversation that goes ten messages deep and triggers enrichment, two outbound sends, and a lifecycle transition costs on the order of \$0.04 in credits. At 1,000 conversations per month per client, that is \$40 per client. At 50 clients: \$2,000/month. At 200 clients: \$8,000/month.

The gap is not the point. The point is the shape of the cost. HubSpot charges by the seat. The substrate charges by the action. As a client's business grows, their HubSpot bill grows with headcount and contact tier. Their substrate cost grows with conversation volume, and conversation volume is the thing that generates revenue, so the cost is proportional to the value.

The second comparison is what each system knows. HubSpot knows what a human entered and what a form captured. The substrate knows every conversation, every channel, every intent signal, every moment of hesitation, every path taken and not taken. The quote that opens this page names the gap: a CRM is a graveyard of intent. The substrate is a living record of it.

The third comparison is what each does with what it knows. HubSpot surfaces a record. The substrate routes the next action. The distinction is between a filing cabinet and a brain.

Objections

"My team is trained on HubSpot. Switching will cost me weeks." The transition is not a switch. The substrate runs alongside HubSpot. Your team keeps their HubSpot view until the substrate's view is richer (typically within 90 days of the first client going live). No retraining event. A gradual shift in where the team goes first.

"HubSpot has CRM features the substrate doesn't have yet: custom pipelines, deal cards, forecasting." Correct on specific features. The pipeline is signal-driven, not visual Kanban. Forecasting comes from the analytics layer, not a dedicated forecast module. If a visual deal card is load-bearing for your sales process, keep HubSpot for that workflow while the substrate handles the conversation and enrichment layer. The two integrate via the write-through path.

"I need my clients to see their own CRM data." The 4-tier role cascade handles this. A client has access to their own workspace contacts. They log into the CRM view on their subdomain, branded with your agency's name, and see their own actor records, segments, and journey funnels. They do not see other clients. They do not see the substrate layer. They see a CRM that looks like it was built for them.

"What if I have contacts in multiple systems: Salesforce, Klaviyo, Mailchimp?" Each has an export path. The importer agents handle CSV and API imports for Salesforce and Klaviyo. Mailchimp exports to CSV, which maps in one pass. Duplicates are resolved by email hash in the identity ladder. The contact history from each system is preserved in the `appended.<source>.<field>` namespace.

"How do I know the data won't end up somewhere it shouldn't?" Each workspace is isolated at the D1 and TypeDB level. Per-client data spaces. Deletion in one command: `POST /api/forget { actor_id }` cascades across vault, KV, D1, TypeDB, and ad platforms within 30 seconds. The cascade receipt is available for audit. GDPR Art. 17 and CCPA delete both flow through the same endpoint. The DPA and the client see the same receipt.

"What about my existing CRM workflows: lead scoring, sequences, nurture campaigns?" Lead scoring maps to fit-score, a TypeDB attribute on the actor, computed by the CRM agent from engagement signals and enriched attributes. Sequences map to journeys with the same stage-signal structure, holdout logic, and attribution. Nurture campaigns map to broadcast programs with the consent gate and frequency cap enforced mechanically. The concepts are the same. The names are different.

"Can I run a CRM for a client whose business is mostly walk-in, not digital?" Walk-in customers arrive as web contacts when they book online, as Telegram contacts when they message, or as CSV imports from the practice management system. The substrate does not require digital-first customers. The contact record exists for anyone you can name. For purely walk-in customers with no digital footprint, the contact is a manual entry, the same as any CRM.

“My clients already use a CRM they like. Why pull them off it?” You would not, immediately. The substrate is additive for clients already on a tool they value. You run the substrate for the conversation layer (chat, Telegram, email, inbound signal handling) and the write-through path keeps their existing CRM in sync. Over time, the substrate’s view is richer. The transition happens when it makes sense per client, not as a mandate.

Code shape: actor query

The actor query behind the contact screen, shipped in `web/src/lib/crm/actor.ts`:

```
// getContact – TypeDB query aggregating actor attributes and relations
const attrRows = await typedbQuery(
  env,
  `match
    $a isa actor, has aid "${id}", has $attr;
    $attr isa! $kind;
  select $attr, $kind;`,
)

// groupRows – membership relations
const groupRows = await typedbQuery(
  env,
  `match
    $a isa actor, has aid "${id}";
    $g isa group, has gid $gid;
    (member: $a, group: $g) isa membership;
  select $gid, $gname;`,
)

// pathRows – strongest outbound paths, sorted by pheromone strength
const pathRows = await typedbQuery(
  env,
  `match
    $a isa actor, has aid "${id}";
    $o isa actor, has aid $oid;
    $e (source: $a, target: $o) isa path, has strength $s;
  sort $s desc; limit 20;
  select $oid, $s;`,
)
```

The REST endpoint wrapping this query shipped as `GET /api/actors/[id]` in commit `74e47496`. The activity endpoint `GET /api/actors/[id]/activity` reads from D1, returning the last 50 signals in reverse chronological order with cursor pagination.

The enrichment layer, shipped in commit `79fd203c`, adds:

```
// Enrich actor with D1 events, substrate paths, identity rungs
export async function enrichActor(env, actorId: string): Promise<Contact>
```

Both ship without a separate CRM database. Both read from the substrate that routes every other operation in the system.

Signal flow: actor to outcome

```
lead opens chat
  | signal { receiver: 'agent:dentist', data: { content, visitor_hash } }
  ▼
agent handles conversation
  | signals fire per message (touch:receive, touch:reply, touch:open...)
  ▼
email captured → actor upsert
  | lifecycle: anonymous → lead
  | identity ladder: visitor_hash → email-hash (confidence 1.0)
  ▼
"send me a proposal" detected
  | signal { intent:proposal-request } → lifecycle: lead → sql
  | thing:offer created, linked to actor via attribution path
  ▼
proposal sent
  | touch:send · channel:email · campaign:new-patient-flow
  | tracked link embedded → /go/:id
  ▼
link clicked → booking completed
  | touch:click → touch:purchase
  | mark(edge, value=120) → path:new-patient-flow strengthens
  | lifecycle: sql → customer
  ▼
outcome recorded in substrate
  | pheromone on every path segment
  | next lead follows the stronger route automatically
```

Every arrow in that diagram is a signal. Every signal writes to the same brain. The contact record, the conversation history, the deal record, and the learning are one thing.

Numbers

Eight actor attributes live in the shipped schema: `name`, `email`, `phone`, `lifecycle`, `fit-score`, `ltv`, `consent-channels`, `channels`. The enrichment layer adds more than 40 across Clearbit, Apollo, Stripe, Shopify, GA4, and the identity ladder.

Five CRM surfaces shipped or in build: `/crm/c/:actor` (contact detail, live), `/crm/g/:group` (segments), `/crm/j/:journey` (journey runtime), `/crm/b/new` (broadcast composer), `/crm/pulse` (analytics).

Two API endpoints shipped in 74e47496: `GET /api/actors/[id]` and `GET /api/actors/[id]/activity`. One enrichment aggregator shipped in 79fd203c: `enrichActor()` in `web/src/lib/crm/actor.ts`.

Contact detail page load: <100ms from D1 hot path. Tracked link redirect: <50ms at the edge.
Segment query to live count: <200ms. Broadcast to 10,000 contacts: <30 seconds.

Six lifecycle stages: `anonymous` → `lead` → `mql` → `sql` → `customer` → `advocate` , plus `churned` . Signal-driven, not button-driven.

Eight enrichment integrations specified: Clearbit, ZoomInfo, Shopify, Stripe, GA4, Meta Pixel, Segment, RudderStack. Eight export integrations: Meta Custom Audiences, Google Customer Match, TikTok Audience, Klaviyo, HubSpot, Salesforce, and two more. Every export is a diff push: additions and removals only.

One privacy endpoint for GDPR: `POST /api/forget` cascades across vault, KV, D1, TypeDB, R2, and ad platforms. Cascade receipt at `GET /api/forget/:request_id` . Vault shred time: <1 second.

Zero new tables. Zero new verbs. The CRM is a typed lens over the substrate that already exists.

Failure modes

Three failure modes for a signal-driven CRM, named explicitly.

Stage transition misfires. The model classifies “can you tell me more?” as `intent:proposal-request` and advances the actor to `sql` prematurely. The rule is visible on the contact record. The override is one keystroke. The signal that fired is logged. The model’s prior shifts. The fix is traceable, not mysterious. In a traditional CRM, a misfire is a human error with no audit trail. In the substrate, it is a classification error with a full log and a feedback loop.

Identity merge false positive. The probabilistic merge engine combines two actors who share a first name and an IP address but are different people. Merge confidence is logged. Merges below 1.0 are flagged for human review. A human can split a merged actor: `split(a, b)` writes a `same-as NOT` relation. The activity feeds are de-interleaved. No data is lost.

Enrichment overwrite. An importer agent fetches stale data and overwrites a field the account manager corrected manually. The `appended.<source>.<field>` provenance record shows which agent wrote which value and when. The manual override is stored as `appended.manual.<field>` . The display logic prefers manual over imported. The import does not silently win.

FAQ

Is the CRM the same product as the analytics? Same data source, different lens. `/crm/pulse` shows CRM KPIs filtered to CRM verbs. `/analytics` shows the full signal stream. One brain, two views.

Can I use the CRM without agents? Yes. The CRM reads from the signal stream. Signals arrive from agents, web forms, channel integrations, or manual API calls. You can ingest contacts via CSV and API without an agent handling conversations. The learning loop works on whatever signals you provide.

What is the actor type count? Two: `human` and `agent`. Both are first-class contacts. A pipeline of developers who installed the SDK but have not deployed is an actor segment with `actor.kind = agent`. The same CRM surfaces handle both.

How does the CRM handle unsubscribes? An unsubscribe fires a signal tagged `touch:unsub`. `channel:email`. The substrate calls `warn(edge)` on the email path for that actor. The `consent-channels` attribute updates to exclude email. The suppression flag sets. Future broadcasts that would send to that channel for that actor are blocked at the consent gate before they resolve. The block is mechanical. It does not depend on someone remembering to update a list.

Can I run journeys and HubSpot sequences simultaneously? Yes. The write-through sync keeps HubSpot updated as actors advance through substrate journeys. HubSpot sequences keep running on their own cadence. Signals from both systems land on the actor record. Attribution handles overlapping touches with a configurable model (last-touch, linear, time-decay, Shapley).

What happens to the data if I leave? The actor table, signal history, and enriched attributes export as JSON or Parquet on demand. The corpus stays with the agency. `GET /api/actors/export` returns the full workspace in one call. You walk away with everything.

Glossary

Actor. The substrate's word for a contact. A row in TypeDB with attributes (name, email, lifecycle, fit-score) and relations (same-as, membership, attribution). Both humans and agents are actors.

Signal. A message on the substrate bus. Every conversation touch (send, open, click, reply, purchase) is a signal. Signals are the raw material of the contact record.

Lifecycle. The actor's stage in the customer journey: `anonymous` → `lead` → `mql` → `sql` → `customer` → `advocate`. Driven by signal-pattern rules, not button presses.

Pheromone. The weight accumulated on a path between two actors or stages as signals mark it. Pheromone is what makes the pipeline learn. A path with high pheromone is a proven route. Routing follows the strongest path.

same-as. A TypeDB relation between two actor records that represent the same person. Created by the identity ladder when an email-hash, phone-hash, or auth-provider ID matches across records.

Fit-score. A computed attribute on the actor, reflecting how closely the actor's signals match the ideal customer profile for their segment. Used for lead prioritisation and handoff routing.

Appended. Attributes written by enrichment agents from third-party sources. Stored in the `appended.<source>.<field>` namespace. Provenance-tracked. Never silently overwritten.

Consent gate. The mechanical check before any outbound signal fires. If `consent-channels` does not include the target channel, the signal dissolves. No workaround.

Cross-references

- **§10 Tracking.** The pipeline that writes every signal this CRM reads. Identity rungs 0–1 are live; rungs 2–5 in build. The tracking layer is the foundation; the CRM is the lens.
- **§11 Analytics.** The same signal store, different filter. The monthly client report reads from the same `funnel_daily` and `rollup_counters` that power `/crm/pulse`.
- **§13 Learning.** The loop that hardens proven contact-to-conversion paths into hypotheses. The discovered journeys page at `/crm/j/discovered` surfaces those candidates for one-click materialisation. What looks like a CRM pipeline feature is the substrate's L6 KNOWLEDGE loop made visible.

Open the actor view — it's already the CRM.

13 — Learning

Other vendors ship the same chatbot to everyone. Yours gets smarter on your clients' data. Quarter over quarter, the gap is unrecoverable.

Pheromone

An ant colony has no central brain. No ant knows the shortest path to the food source. What the colony knows, it knows through its trails: chemical deposits left behind by every ant that walked before. The successful ants reinforce the successful routes. The dead ends go cold. Over time, the colony routes around every obstacle it has ever encountered, without a single ant understanding the whole map.

ONE works the same way. Not as a metaphor. As a mechanism.

Every conversation your agent handles leaves a trace. That trace is called a path. If the conversation ended well (the user got an answer, the lead was captured, the appointment was booked), the path gets stronger. If it ended poorly, the path weakens. Over thousands of conversations, the substrate builds a weighted map of everything that has worked and everything that has not, on your clients' actual data.

That map is called pheromone. It is the memory of outcomes. It is why an agent that handles a dentist's incoming calls in January is better at handling them in April. Not because you updated the prompt. Because the substrate remembers which paths led to booked appointments and which paths led to dead air.

The technical word for this process is stigmergy: the substrate doing the thinking, the agents doing the moving. Pierre-Paul Grassé named it in 1959, studying ant colonies. Every time it has appeared (in ants, in neurons, in human markets) the result has been the same: intelligence that no individual participant planned, but that the system produces anyway.

Your clients' agents are participants in that system. Every conversation they have makes the paths underneath them more accurate. The pheromone compounds. The intelligence compounds.

The competitor who buys the same model you use does not get the pheromone. The model is a commodity. The pheromone is the moat.

The 7 Loops in Plain English

The substrate runs seven feedback loops. They run at different speeds and do different jobs. Here they are in business terms.

Loop	Cadence	What it does for your client
L1 — Signal	Every message	Handles the conversation. Routes to the right agent. Produces an outcome. The core transaction.
L2 — Trail	Every outcome	Records what worked and what did not. Deposits pheromone on the paths that produced results, resistance on the paths that failed.
L3 — Fade	Every 5 minutes	Decays old information so the agent adapts to new patterns. Recent results count more than old ones. Good decays slowly. Bad forgives fast.
L4 — Economic	Every payment	Revenue flows along the paths that earn it. Agents that generate bookings or sales attract more work. The economics route themselves.
L5 — Evolution	Every 10 minutes	Identifies agents whose success rate has fallen below 50% across at least 20 conversations. Rewrites their instructions automatically. No engineer required.
L6 — Knowledge	Every hour	Promotes the strongest paths into hardened hypotheses. Facts the substrate now treats as reliable.
L7 — Frontier	Every hour	Scans for tag clusters no agent has explored yet. Surfaces the unexplored territory so you can see where the next opportunity is before your competitor does.

Seven loops. Continuous. Invisible to your client's customer. Visible to you as the agency, in the analytics dashboard, every quarter.

The thing to understand about these loops is that they are not separate features. They are one flywheel. The signal loop produces what the trail loop marks. The trail loop builds the paths the fade loop maintains. The fade loop keeps the paths accurate. The knowledge loop hardens the best paths. The frontier loop finds where no paths have gone yet. The prompt-rewrite loop fixes the agents dragging the rest down.

Miss any loop and the flywheel slows. Keep all seven running and it compounds.

Asymmetric Fade

Not all forgetting is equal.

In human memory, bad experiences tend to linger. In the substrate, the opposite is engineered deliberately. When a path accumulates resistance from failed outcomes, that resistance fades twice as fast as strength.

This is the asymmetric fade rule: good is sticky, bad forgives.

The reason is practical. A new client's agent will make mistakes early. Every agent does. If those early failures were weighted permanently, the system would be permanently biased against paths the agent had not yet learned to handle well. The early bad would drown out the later good.

Instead, the substrate lets bad outcomes weaken paths quickly, but does not hold them against the agent forever. Once the agent finds better routes, the resistance fades, and the better routes accumulate strength. The mistakes recede. The wins compound.

For Brad's agency, this means onboarding a new client's agent is not a risk. Week one will be messier than week twelve. The substrate is designed for that. By week six, the mistakes are already fading. By week twelve, the agent is running on the paths that have proved themselves across hundreds of real conversations.

There is no engineer fixing this. The loop fixes itself.

Self-Improving Prompts

This is the one that most people don't believe until they see it.

When an agent's success rate falls below 50% across at least 20 conversations, the substrate does not wait for a human to notice. It identifies the failing conversations, looks at what went wrong, and rewrites the agent's instructions to address the failure pattern.

The agent's generation counter increments. The new instructions replace the old ones. The agent runs again on the same types of queries. The substrate measures whether the rewrite improved the success rate.

```
// When the substrate detects a struggling agent:  
// success-rate < 0.50, sample-count >= 20  
  
unit.system_prompt = rewrite(old_prompt, recent_failures)  
unit.generation++  
  
// mark() on success after the rewrite → path strengthens  
// warn() on continued failure → rewrite runs again
```

In plain English: the agent notices it is getting things wrong, figures out why, fixes its own instructions, and tries again. No one calls you. No ticket is filed. No engineer spends a weekend debugging a prompt.

This matters for an agency running 50 clients. At that scale, a human reviewing every agent's performance every week is not viable. The economics depend on the agents running without supervision. The prompt-rewrite loop is what makes that possible.

The gate on this process is 0.65. Nothing the agent produces gets sent to a client's customer unless it scores above that threshold on the quality rubric. Below the gate, the message does not ship. The agent tries again. This is how you guarantee the self-improvement loop does not produce worse output while it is figuring out better output.

By the time you have 50 clients on the platform, you will have 50 agents fixing their own problems on their own schedules, improving their own performance, without a single billable hour of your team's time.

Knowledge

At some point, a path has proved itself enough times that calling it a “path” is underselling it. It is a fact.

The knowledge loop runs every hour and looks for paths that have accumulated enough strength, across enough outcomes, that the substrate is willing to call them reliable. When a path clears that threshold, it is promoted from the pheromone graph into a hardened hypothesis.

A hypothesis is a structured claim: *“When this type of query arrives from this type of customer, routing to this agent with this approach produces successful outcomes at X% confidence.”* It is stored in TypeDB, the substrate’s long-term memory. It persists across fade cycles. It does not decay.

For your clients, this means the substrate is quietly building an institutional knowledge base out of their own conversations. Not a document someone wrote. Not a FAQ someone curated. A living set of verified facts, derived from what has actually worked with their actual customers.

By month three, a dentist client’s agent knows, with evidence, that appointment reminders sent on Tuesday mornings have a 34% confirmation rate, while reminders sent on Friday afternoons have an 11% rate. That is not a best practice from a marketing blog. That is a hypothesis derived from the dentist’s own patients, verified across hundreds of reminder conversations.

The competitor running a static chatbot on that same dentist’s website does not have that hypothesis. They have a chatbot that says the same thing every time and has no idea whether it is working.

Frontier

Knowing what has worked is half the picture. Knowing what has not been tried yet is the other half.

The frontier loop runs every hour and scans the tag clusters across all conversations for patterns that no agent has addressed. A cluster of queries that keep arriving but keep going unanswered. A topic that customers keep raising but that the agent has no knowledge about.

The frontier loop surfaces these gaps as signals. Not as problems. As opportunities.

For an agency managing a dental client, the frontier loop might surface the fact that 14% of incoming chat queries mention implants, but the client’s agent has no specific skill for implant inquiries. Those queries are being handled generically. They could be handled specifically, with better outcomes, if someone built the skill.

That someone is you. Or one of your team. Or, increasingly, one of your agents.

The frontier loop does not fix the gap automatically. It shows you where the gaps are so you can decide whether to fill them. In a 50-client agency, that is the difference between guessing which client needs attention and knowing exactly where the next improvement will have the most impact.

The substrate maps the unexplored territory. You decide where to send the expedition.

Why Your Clients' Chatbots Get Smarter Than the Competition

Here is the worked example. Brand X is a dental practice in a mid-sized city. They start on the platform in January. Their agent handles booking inquiries, appointment reminders, and basic FAQ queries.

Week 1:

The agent routes every query through the LLM. No paths have been proven yet. The routing decision takes the same time as any new agent. Success rate on booking inquiries: 62%.

Path strengths at week 1 (booking queries): - Route A (direct booking link): strength 0.3 - Route B (availability check first): strength 0.1 - Route C (ask for preferred time): strength 0.2

Week 12:

The agent has handled 847 booking conversations. Route A has proved itself consistently. It closes bookings faster and with fewer drop-offs. Route B and Route C have accumulated resistance because they introduce friction that pushes customers to abandon.

Path strengths at week 12: - Route A (direct booking link): strength 0.87, highway status - Route B (availability check first): strength 0.04, nearly dissolved - Route C (ask for preferred time): strength 0.11, weak

The routing decision now takes less than 10 milliseconds, compared to the full LLM inference time at week 1. The agent knows, from evidence, that Route A works. It goes there immediately. Booking success rate at week 12: 84%.

The gap between week 1 and week 12 is not the model. Brand X's competitor is using the same underlying model. The gap is the 847 conversations that built those path strengths, plus the prompt rewrite the substrate ran in week 6 when success rate dipped below 50% for a three-day stretch.

The competitor's chatbot has the same model. It does not have the 847 conversations. It does not have the rewritten instructions. It does not have the highway on Route A. It handles the same query the same way it did on day one.

By week 12, Brand X's agent is materially better at its core job. The gap between the two chatbots is the pheromone. You cannot buy pheromone. You can only accumulate it.

The Unrecoverable Gap

"The cheapest thing to copy is what you sell. The hardest thing to copy is how you learn. Build the second one and the first one stops mattering."

At month three, a competitor agency could catch up. The path strengths are not that deep yet. A competitor who started two months after you and hustled to onboard clients quickly could narrow the gap.

At month twelve, catching up is a real project. A year of conversations. A year of automatic prompt rewrites. A year of hypotheses hardened into TypeDB. A year of frontier signals acted on. That is a substantial gap.

At month thirty-six, catching up is a different kind of project. A competitor who wants the corpus you have built for a dentist client would need to manufacture three years of genuine patient conversations, with genuine outcomes, with genuine pheromone deposits on genuine paths. There is no shortcut for that. You cannot buy it. You cannot train it away. The corpus is the product of real interactions with real people, and there is no substitute.

This is why BCG found that AI leaders achieve double the revenue growth and 40% more cost savings than laggards: the gap between leaders and laggards compounds every quarter, not just once. (BCG, *Build for the Future 2025*, September 2025.)

There are two kinds of moats in business. The first is a feature moat: you build something, a competitor copies it, the moat is gone. Every software feature is a feature moat. Feature moats are temporary.

The second is a corpus moat. The corpus is the accumulated record of real outcomes across real interactions with real customers. A competitor can copy your pricing page. They can copy your chatbot interface. They can run the same underlying model. They cannot copy your corpus. The corpus is owned by the agency. It lives in the substrate behind the white-label, on the agency’s accounts, under the agency’s contracts. When a client considers switching to a competitor, they are not switching chatbots. They are abandoning three years of path strength, hardened hypotheses, and frontier intelligence that the competitor’s platform will have to rebuild from zero.

This is the switching cost that does not appear on any feature comparison table, because it is not a feature. It is the weight of what was learned.

Objections

“Couldn’t a competitor catch up by training on the same data?”

The data is the client’s. It does not belong to the platform and it does not belong to any competitor. It lives in a per-client data space behind the agency’s white-label. A competitor would need the client to voluntarily move, abandoning everything the substrate has learned, and start from scratch on a new platform. The client’s incentive to do that decreases every month the corpus deepens. The competitor who wants three years of Brand X’s booking conversation data has one option: convince Brand X to start over and wait three years.

"What if the model gets smarter and closes the gap?"

Model improvements lift everyone. When the underlying LLM gets better, your clients' agents get better, and so does the competitor's chatbot. The model improvement is a rising tide. The pheromone is the moat above the tide. A smarter model on top of three years of proven paths is more capable than a smarter model on top of zero paths. The gap does not close when the model improves. It shifts to a higher baseline.

"What if we build the same infrastructure ourselves?"

Building this infrastructure takes a year minimum, and that is with a dedicated engineering team. During that year, your clients' corpus is not accumulating. The competitor running on an existing substrate is accumulating 365 days of path strength while you are still writing migrations and testing fade decay rates. The corpus moat is time-denominated. Every month of delay is a month of corpus you will never recover.

"What happens when a client churns?"

When a client leaves, the corpus they accumulated stays with the agency. The paths, the hypotheses, the frontier signals are not tied to the client's subscription. They reflect the conversations that happened on your agency's platform, under your agency's contracts. A client who churns and moves to a competitor takes their future conversations with them. They do not take the history. The agency keeps the corpus.

"How do I explain this to a client who doesn't understand the technology?"

You don't have to explain the technology. You show them the numbers. Week 1 success rate versus week 12 success rate. Month 1 booking conversion versus month 6 booking conversion. The path strengths are an implementation detail. The improved outcomes are the product. No client has ever asked their accountant to explain double-entry bookkeeping. They care that the books are right. Show the improved numbers and let the substrate handle the rest.

"Our current chatbot vendor says they do learning too."

Ask them where the learning lives. If the answer is "in the model" or "in our shared training data," the learning benefits everyone on their platform equally. Your client's conversations are training the vendor's model, which makes every other client on that platform smarter too. That is not a moat. That is a commodity. The pheromone on ONE is per-client, per-agency, not shared across the platform. Your dentist's booking paths do not improve anyone else's booking paths. They are yours.

"What if the self-improvement loop makes the agent worse?"

The gate is 0.65. Nothing below the quality threshold ships. The prompt-rewrite loop fires only when the agent is already underperforming: success rate below 50% across at least 20 conversations. At that point, the agent is already producing poor outcomes. The rewrite cannot make it meaningfully worse. And because every rewrite is measured against the same outcome signals that detected the problem, the system knows immediately whether the rewrite helped. Bad rewrites produce continued poor outcomes, which triggers another rewrite cycle. The loop runs until the success rate recovers.

"Isn't this just A/B testing?"

A/B testing tells you which of two predetermined options performed better in a controlled experiment. Pheromone is not a controlled experiment. It is the continuous measurement of every path your agents have ever taken, weighted by real outcomes, updated in real time, and applied immediately to the next conversation. There is no control group. There is no predetermined option set. The substrate discovers the options by trying them, marks the ones that work, and routes around the ones that do not. That is different in kind, not just in degree.

Comparisons

ONE versus a standard chatbot platform.

A standard chatbot platform assigns a fixed set of responses to a fixed set of triggers. The response to "What are your hours?" is always the same. It does not get better because 500 people asked it and all 500 found it helpful. There is no mechanism for the platform to know that. There is no mechanism for the platform to care.

On ONE, every outcome is a signal. Every signal deposits pheromone. After 500 people ask about hours and get a useful answer, that path has strength 0.87. It is a highway. The next person who asks the same question gets routed there in under 10 milliseconds, with the full confidence of those 500 past outcomes behind it.

ONE versus a shared-learning platform.

Some platforms claim their chatbots learn, and they do, but what they learn gets pooled across all clients on the platform. Your dental client's conversations about appointment booking are pooling into the same learning set as a plumber's conversations about emergency callouts and a restaurant's conversations about reservations. The model gets generally smarter. No one gets specifically smarter about their actual customers.

On ONE, the pheromone is per-client. The hypothesis that says *"Tuesday morning reminders close at 34% for this dental practice"* belongs to that practice, in that workspace, on that agency's platform. It does not leak. It does not pool. It compounds in the right place.

Pricing Math

The corpus moat is also a retention moat, and retention is what drives agency revenue.

At \$1,000 per month per client, a client who stays for 36 months generates \$36,000. A client who leaves at month 6, before the corpus has depth, generates \$6,000. The difference is \$30,000 per client, driven primarily by the switching cost the corpus creates.

At 50 clients, if the corpus moat increases average retention by 18 months, that is $50 \times \$1,000 \times 18 = \$900,000$ in additional revenue from the same client base, with no additional acquisition cost.

The corpus moat does not just defend the clients you already have. It justifies the price to clients evaluating you against a cheaper competitor. A competitor offering the same chatbot for \$400 per month looks competitive in month one. By month twelve, the client who chose your platform at \$1,000 has a corpus the cheaper competitor cannot match. The switching cost at month twelve is measured in lost path strength, not in monthly invoice difference. Most clients, once they understand this, stay.

Failure Modes

The three ways this breaks down, and how to avoid them.

Failure mode 1: Thin signal volume.

The pheromone system compounds on volume. An agent handling 20 conversations a month is accumulating path strength much more slowly than an agent handling 200. At 20 conversations a month, paths take six months to reach highway status. At 200, they get there in three weeks. Clients with very low engagement see learning that is real but slow. Set expectations accordingly, and focus your onboarding energy on clients with active inbound volume. The substrate rewards activity.

Failure mode 2: Contradictory signals.

If a client's agent is deployed across two very different audiences (say, a law firm running one agent for consumer inquiries and another for institutional clients) and both audiences are pooled into the same workspace, the signals will contradict each other. A path that works for a first-time consumer inquiry will underperform for an institutional client who wants dense technical detail. Pheromone from the consumer conversations will weaken the institutional paths, and vice versa. The fix is simple: separate workspaces per distinct audience. The substrate does not know the difference between audience types unless you create the separation.

Failure mode 3: Ignoring the frontier loop.

The frontier loop shows you where the gaps are. Agencies that never act on frontier signals let their clients' agents stay stuck in the territory they already know. The clients' customers keep asking about the unexplored areas. The agent keeps handling those queries generically. The pheromone in the unexplored areas stays thin. Six months later, a competitor who has been actively filling frontier gaps on a similar client has an agent that covers a much broader set of queries at highway strength. The frontier is not a passive feature. It rewards the agencies who check it.

A Day in the Life, Month 12

It is Tuesday morning. One of your dentist clients just had their monthly report land in their inbox. It was generated automatically, in 90 seconds, the same way it has been generated every month.

The report shows: - Booking inquiry success rate: 84% (up from 62% in month 1) - Average path to booking: 2.3 conversation turns (down from 4.1 in month 1) - Prompt rewrite: agent instructions rewritten once in the past 30 days. The rewrite addressed a pattern of patients asking about payment

plans. The agent was routing them to a generic FAQ; after the rewrite, it routes them directly to the payment coordinator's calendar. - Frontier signal: 14% of queries mention implants. No dedicated skill exists yet. Flag for next conversation.

You read that last line. You open a new agent skill file for implant inquiries. You give it to the dentist's agent. Next month's report will show whether it is working.

Your competitor's client is reading no monthly report. The competitor's chatbot has no payment plan routing. It has no frontier analysis. It has no self-rewriting prompts. It is the same chatbot it was on day one.

The gap is not a product gap. The product is the same category. The gap is 12 months of pheromone that the competitor cannot acquire and cannot manufacture.

Code

The two lines that make this work:

```
// A conversation ends in a successful booking:  
net.mark(edge, chainDepth) // path strengthens; next query routes here faster  
  
// A conversation ends without a resolution:  
net.warn(edge, 1) // path weakens; routing will look elsewhere next time
```

A successful outcome strengthens the path. A failed outcome weakens it. Every conversation ends in one or the other. The substrate has no concept of "fine." Every outcome is information. Every piece of information adjusts the map.

That map is the corpus. The corpus is the moat.

Signal Lifecycle, One Quarter

```
WEEK 1  Signal arrives → LLM routes → outcome recorded
        ↓
        Trail: path marked or warned
        ↓
WEEK 2  Fade: weak paths decay; strong paths persist
        ↓
        50 passes: path reaches highway threshold
        ↓
WEEK 6  Prompt rewrite triggers (success rate < 50%)
        Agent rewrites own instructions
        ↓
MONTH 3  Knowledge: strongest paths promoted to hypotheses in TypeDB
        ↓
MONTH 3  Frontier: unexplored tag clusters surfaced as opportunities
        ↓
QUARTER PATH STRENGTH: 0.87 on proven routes / < 0.10 on failed routes
        CORPUS DEPTH: 847 conversations → hardened hypotheses
        COMPETITOR:   Day 1 routing speed, Day 1 success rate
```

Cross-References

- **§10 Tracking** is the signal that makes the loop possible. Every conversation must be captured to become pheromone.
- **§11 Analytics** is the monthly report that surfaces the corpus in client-facing numbers.
- **§08 Memory** is the per-client data space where the corpus lives. This is what makes the corpus the agency's asset, not the platform's.

Integration with TypeDB

The hypotheses produced by the knowledge loop are not stored in a flat database. They live in TypeDB, a graph database designed for relational knowledge, where the connections between facts matter as much as the facts themselves.

A TypeDB hypothesis looks like: *"For workspace dental-brand-x, when query-tag = appointment-booking, route = direct-link produces success-rate = 0.84 at confidence = 0.91."* That hypothesis connects to the paths that produced it, the conversations that marked those paths, the agent that handled those conversations, and the client whose customers generated the queries.

When the agent encounters a new booking query, TypeDB is consulted before the LLM is called. If a reliable hypothesis covers the case, the route is resolved in under 10 milliseconds. The LLM call is for the cases the hypothesis does not cover.

This is why the routing decision at week 12 costs 60,000 times less compute than the routing decision at week 1. (Routing at week 1: full LLM inference, ~1,500ms. Routing once a highway has hardened: arithmetic lookup via pheromone weights, <0.005ms.) The LLM stays in the loop for the genuinely novel cases. The proven cases are handled by the corpus.

FAQ

How long before the learning is noticeable? Path strength accumulates from the first conversation. The first meaningful difference in routing speed appears around week 3 to 4 for clients with 50+ conversations per week. Highway status, where routing is automatic and near-instant, typically appears between week 4 and week 8 for active paths. Month 3 is when most clients can see the difference in their success rate metrics.

Can I see the paths? Yes. The analytics dashboard shows path strength by conversation type. The monthly report surfaces the top-performing paths and the paths with high resistance. You can see exactly what the substrate has learned and exactly where it is still uncertain.

Do all seven loops run on every client? Yes. All seven run on every workspace from day one. In the early weeks, there is not enough signal to produce meaningful hypotheses, and the prompt-rewrite loop will not trigger unless success rate drops below 50% across 20 conversations. But the loops are running. As soon as the signal volume supports them, they kick in.

What if I update the agent manually? Manual updates are additive. If you rewrite an agent's instructions, the new instructions replace the old ones and the agent runs from that baseline. The pheromone on the paths remains. The substrate does not reset path strength when instructions change, because the paths reflect what worked, not how the agent was configured at the time. You can improve an agent's instructions without losing what the substrate has learned.

How does the frontier loop know what is unexplored? It scans the tag clusters across all incoming conversations and compares them against the set of tags that have established paths. Any tag cluster that appears consistently in conversations but has no established path is a frontier signal. The threshold is calibrated to filter out one-off queries: a tag cluster needs to appear in a statistically meaningful number of conversations before the frontier loop surfaces it.

Is the corpus backed up? Yes. The corpus lives in TypeDB and in Cloudflare D1, both of which have their own backup and redundancy protocols. The pheromone graph is snapshotted to Cloudflare KV regularly. In the event of a platform failure, corpus recovery is a restore operation, not a rebuild from scratch.

What happens to the corpus if I move a client to a different agency plan? Nothing. The corpus is tied to the workspace, not the billing tier. Upgrading, downgrading, or pausing a client's subscription does not affect the accumulated path strength or hypotheses. The corpus persists.

Glossary

Pheromone. The system of path weights that reflects accumulated outcomes. Stronger on routes that have worked repeatedly. Weaker on routes that have failed.

Path. A connection between two points in the routing graph. Paths accumulate strength (from successful outcomes) or resistance (from failed outcomes).

Highway. A path that has accumulated enough strength to be treated as a reliable route. Highways cache at the edge of the network; routing to a highway takes under 10 milliseconds.

L1 to L7. The seven feedback loops that run continuously: Signal (every message), Trail (every outcome), Fade (every 5 minutes), Economic (every payment), Evolution (every 10 minutes), Knowledge (every hour), Frontier (every hour).

L5 Evolution. The automatic self-improvement loop. When an agent's success rate falls below 50% across at least 20 conversations, the substrate rewrites the agent's instructions without human intervention.

L6 Knowledge. The loop that hardens proven paths into structured hypotheses stored in TypeDB. Hypotheses persist across fade cycles and are consulted before the LLM on future queries.

L7 Frontier. The loop that surfaces unexplored tag clusters: conversation topics that keep arriving but have no established path.

Corpus. The full accumulated record of outcomes, path strengths, and TypeDB hypotheses for a given client workspace. The corpus is the agency's asset. It does not pool across other clients.

Fade. The scheduled decay of path strength and resistance. Resistance fades twice as fast as strength, so the system forgives early mistakes while preserving hard-won learning.

Hypothesis. A structured, evidence-backed claim that the substrate treats as reliable. Produced by the knowledge loop from paths with consistent performance across enough observations to meet the confidence threshold.

Generation. The count of times an agent's instructions have been rewritten. A generation 0 agent is running on its original instructions. A generation 3 agent has been rewritten three times.

Watch a path harden into a highway.

Development

Your engineers, your IDE. Or your CEO and Claude. Same substrate. The same operation (deploy a new agent, publish a skill, check what's working) runs from a chat window, a terminal, a TypeScript codebase, or an MCP-aware IDE. The surface is your team's choice. The substrate is shared.

This is what verb-surface parity means in practice. And it matters most on day two, when Donal's team wants to extend the platform and Brad needs to know they can.

"The best way to predict the future is to invent it."

— Alan Kay

Build

Three ways to build on ONE. None of them require the others. Teams pick the one that fits how they already work.

Chat. Open `one.ie/chat`. Type what you want: *"Create a new agent for our window installation client that handles enquiries, books appointments, and sends quote follow-ups."* The substrate parses the instruction, scaffolds an agent markdown file, and syncs it to TypeDB. No terminal. No release cycle. A non-technical team member can do this on a Tuesday afternoon and have the agent live before the client's next business day.

This is also how a CEO tests a concept without involving the engineering team. Describe the behaviour. Watch it run. Approve it. Assign the fine-tuning to Donal's team. The hand-off is the markdown file, human-readable, version-controllable, auditable.

Markdown. Every agent is a `.md` file. The format is a structured spec: name, system prompt, skills, capabilities, pricing, and the voice contract that governs every outbound message. Edit the file in any IDE. Save it. The substrate syncs. Changes are live within thirty seconds.

This is intentional. An agency's best asset is its accumulated knowledge of each client's voice, tone, preferences, and objections. That knowledge lives in the markdown. It is not locked inside a platform's proprietary configuration UI. It is a file. The agency owns it. It backs up with `git`. It exports with one command.

Authoring an agent requires no code. A strategist who understands the client writes the brief. The substrate does the rest.

IDE. For Donal's team, working in VS Code or Cursor with `@oneie/sdk` imported, the experience is typed TypeScript against a well-documented API. Signal. Ask. Mark. Warn. Six verbs, fully typed, with IDE autocompletion on every call. TypeDB stores the paths. Cloudflare Workers serve the edge. The build time is 107 seconds from commit to production across four deployed services.

The same agent markdown that a strategist authored in chat is consumed by `syncAgent()` in TypeScript. The same markdown that a developer validated in the IDE is pushed with `oneie agent publish` from the terminal. One file. Every surface reads it.

Call

Four surfaces. One substrate behind all of them.

CLI. `npx oneie` gives any developer 29 verbs covering substrate operations, deployment, commerce, observability, and ergonomics. No installation required for a first run. Structured JSON output on every verb via `--output json`, so it pipes cleanly into scripts. A `--dry-run` flag on every mutating verb previews without executing. Shell completion for bash, zsh, and fish. A REPL mode for interactive exploration. Works against local dev, staging, or production. Switching profiles is one command.

API. Every substrate operation is a REST endpoint on `api.one.ie`. `POST /api/signal`. `GET /api/loop/highways`. `DELETE /api/memory/forget/:uid`. Standard Bearer auth. Standard JSON bodies. A curl command is all the SDK you need for a first integration. The API is public in its shape. No proprietary format. No required client library. No vendor-specific request signing ritual.

SDK. `@oneie/sdk` is the TypeScript client. Six verbs (signal, ask, mark, warn, fade, follow) plus read verbs for highways, recall, reveal, forget, frontier, know. Fully typed. MIT licensed. Works in Node, Bun, and Cloudflare Workers. The client wraps the REST API with typed inputs and typed responses, enforces the four-outcome contract on every `ask()`, and composes cleanly with AI SDK v6 for the LLM layer. One team uses it in TypeScript. Another team calls the same endpoints from Python. The substrate does not care.

MCP. `@oneie/mcp` is the Model Context Protocol server. Install it. Point Claude Code, Cursor, or Windsurf at it. The IDE now has 42 substrate tools available as natural-language-callable functions: every substrate verb, every commerce operation, every observability query. A developer writes *"show me the top ten paths by pheromone strength"* in their IDE and the substrate responds. An AI-assisted workflow calls `oneie_signal` the same way it calls any other tool. The surface adapts to how the developer works, not the other way around.

Spec

The substrate's vocabulary is documented in four interlocking files. They are the source of truth for every surface.

ADL (Agent Definition Language). Every agent is an ADL document. Name, role, skills, voice contract, pricing, membership, lifecycle status. The ADL parser runs in both `@oneie/sdk` (`parse()`, `syncAgent()`) and the CLI (`oneie agent`). An ADL document authored by a strategist in chat and an ADL document pushed by a developer from the terminal go through the same parser, produce the same TypeDB unit, and appear identically in the substrate. Authoring surface varies. The spec is the same.

Ontology. Six dimensions. Groups, actors, things, paths, events, learning. These are the only types in the substrate. An agency client is an actor. A marketing skill is a thing. A conversation is a path. A payment is an event. A successful campaign pattern is a learning. The ontology is locked. Dimension names cannot be renamed. Extensions happen within the existing six dimensions, never by adding a seventh. This is what makes the substrate transferable across verticals without re-engineering. The dental client and the window-installer client both live in the same ontology.

DSL (Domain-Specific Language). The signal grammar. Two fields: `receiver` and `data`. `receiver` names who gets it. `data` carries three slots: `tags` (routing), `weight` (pheromone deposit or payment amount), `content` (payload). That's the whole language. One TypeScript interface. 194 tests verify signal flow, mark/warn/fade, select/follow, ask outcomes, `.then()` chains, tag subscription, and the toxicity sandwich. The DSL is frozen. Building on it is safe.

Dictionary. The canonical name list. Every entity, every verb, every dimension has one name and one name only. Dead names (*knowledge, connections, people, node, scent, alarm, trail, colony*) are documented so they stay dead. When an integration pulls the wrong name, the dictionary is the first stop. When a new developer joins Donal's team, the dictionary is the first doc they read.

These four documents are the contract between surfaces. Any surface that reads them can interoperate. Any surface that bypasses them drifts.

Verb-Surface Parity

Every action is reachable from every surface. The team picks the surface. The substrate doesn't know which one was used.

Here is the same operation (emit a signal to an agent) performed four ways:

CLI

```
oneie signal chairman --data '{"tags":["chat"],"content":"New window lead from website"}'
```

curl (API)

```
curl -X POST https://api.one.ie/api/signal \
-H "Authorization: Bearer $ONE_API_KEY" \
-H "Content-Type: application/json" \
-d '{"receiver":"chairman","data":{"tags":["chat"],"content":"New window lead from website"}}'
```

SDK (TypeScript)

```
import { ONE } from '@oneie/sdk'
const one = new ONE({ baseUrl: 'https://api.one.ie', apiKey: process.env.ONE_API_KEY })
await one.signal({ receiver: 'chairman', data: { tags: ['chat'], content: 'New window lead from website' } })
```

MCP (inside Claude Code or Cursor)

```
Use the oneie_signal tool with receiver="chairman" and data={"tags":["chat"],"content":"N
```

Four surfaces. One result. The signal arrives at the same unit, marks the same path, deposits the same pheromone. The substrate records the outcome identically regardless of which surface sent the signal.

This is not a design preference. It is a commitment. An agency that onboards with the chat surface and later scales to a developer team using the SDK does not rebuild. The substrate they built on day one is the same substrate they build on at month twelve.

The table below maps the 3-by-3 grid: build mode on one axis, call surface on the other, spec document grounding each.

	Chat	Markdown / IDE	CLI
Signal	Type in chat window	<code>one.signal()</code> in TypeScript	<code>oneie signal <receiver></code>
Publish skill	Describe the skill in chat	<code>oneie skill publish</code> from IDE terminal	<code>oneie publish <skillId></code>
Check highways	Ask "show me top paths" via MCP	<code>one.highways(10)</code> in TypeScript	<code>oneie highways --limit 10</code>
Deploy agent	Chat-driven sync	<code>syncAgent(parse(md))</code>	<code>oneie deploy</code>
Spec	ADL (auto-generated)	ADL (authored)	ADL (parsed from <code>.md</code>)
Grounding doc	Dictionary + Ontology	DSL + SDK reference	CLI reference + ADL

The 29 CLI Verbs

The CLI is the fastest way to understand what the substrate can do. Twelve substrate verbs map one-to-one to engine primitives. Five handle deployment. Eight handle commerce. Four handle observability. The ergonomics group adds profiles, shell completion, output formatting, watch mode, dry-run, REPL, NDJSON pipeline, and live SSE tail.

Substrate (12): `signal`, `ask`, `mark`, `warn`, `fade`, `select`, `recall`, `reveal`, `forget`, `frontier`, `know`, `highways`

Deploy (5): `init`, `agent`, `deploy`, `claw`, `sync`

Commerce (8): `pay`, `hire`, `bounty`, `commend`, `flag`, `status`, `capabilities`, `publish`

Observability (4): `stats`, `health`, `revenue`, `export`

Every verb supports `--output json` for piping. Every mutating verb supports `--dry-run`. Every verb exits `0` on success, `1` on failure. Scripts do not need to parse prose output.

The `doctor` verb runs four self-checks (config valid, API key present, substrate reachable, TypeDB reachable) and exits `0` only when all four pass. CI pipelines call `oneie doctor` as a pre-deploy gate. If any check fails, the failure message prints, the pipeline stops, and the cause is named.

The `tail` verb connects to the substrate's Server-Sent Events stream and prints live state: highway updates, stats snapshots, pheromone marks as they arrive. A developer running `oneie tail` during a client demo can watch the substrate respond to live conversation in real time.

The MCP Server Surface

`@oneie/mcp` exposes 42 tools to any MCP-aware IDE. The tool groups match the CLI exactly: substrate verbs, commerce operations, observability queries, discovery, pay tools, and lifecycle.

Claude Code, Cursor, and Windsurf all connect to it via the same config block:

```
{
  "mcpServers": {
    "oneie": {
      "command": "npx",
      "args": ["@oneie/mcp"],
      "env": {
        "ONEIE_API_URL": "https://api.one.ie",
        "ONEIE_API_KEY": "your-key"
      }
    }
  }
}
```

Once connected, the IDE can call `signal`, `ask`, `mark`, `warn`, `highways`, `stats`, `recall`, `scaffold_agent`, `list_agents`, `hire`, `bounty`, `pay_create_link` (the full substrate surface) as natural-language tool calls or programmatic MCP invocations.

The agent becomes a session actor. When Claude Code calls `one_signal`, it is a unit in the substrate. Its outcomes mark and warn the same paths as any other agent. The substrate does not distinguish between a human developer, an AI-assisted workflow, and an autonomous agent. They are all units. They all emit signals. They all deposit pheromone.

This is what it means to say ONE is AI-native. The developer tooling and the agent runtime are the same system. There is no separate "developer mode." There is no import/export step between the IDE environment and the production substrate.

The SDK Shape

`@oneie/sdk` is the TypeScript client for the substrate API. MIT licensed. The source is in `sdk/` in the open-source repository at `github.com/one-ie/one`.

Six verbs. Fully typed:

```
import { ONE } from '@oneie/sdk'

const one = new ONE({
  baseUrl: 'https://api.one.ie',
  apiKey: process.env.ONE_API_KEY,
})

// Send a signal (fire and forget)
await one.signal({ receiver: 'director:review', data: { tags: ['lead'], content: brief
  } })

// Send and wait for outcome
const { result, timeout, dissolved } = await one.ask({ receiver: 'director:quote',
  data: { client, spec } })

if (result)      one.mark('director->quote', chainDepth)  // path strengthens
else if (timeout) { /* neutral */ }
else if (dissolved) one.warn('director->quote', 0.5)      // path doesn't exist
  yet
else            one.warn('director->quote', 1)            // agent produced
  nothing
```

The four-outcome contract on `ask()` is the core of the closed loop. Every client call must close it. The SDK types make the omission visible at compile time. A handler that returns without marking or warning produces a TypeScript error in strict mode.

`parse()` and `syncAgent()` handle the ADL lifecycle. Read a markdown file. Parse it. Sync it to TypeDB. The unit is live. Skills are registered. Paths are initialized:

```
import { parse, syncAgent } from '@oneie/sdk'
import { readFile } from 'node:fs/promises'

const spec = parse(await readFile('agents/director.md', 'utf8'))
await syncAgent(spec)
// director is now a unit in the substrate with its skills registered
```

One TypeScript team and one Python team can build on the same substrate. The TypeScript team uses `@oneie/sdk`. The Python team calls the REST API directly or uses the `oneie` Python package (`pip install oneie`). The substrate API is the shared surface. Both teams see the same paths, the same pheromone, the same highway state.

Deploy in 3 Commands

A new agency client goes from zero to live in three commands:

```
npx oneie init           # scaffold the project
npx oneie agent agents/client-name/ # sync agent definitions to the substrate
npx oneie deploy         # deploy to Cloudflare Workers
```

That's it. The deploy covers four services: the Astro Worker (23 seconds), the API Gateway (14 seconds), the Sync Worker (8 seconds), and the NanoClaw agent worker (9 seconds). Health checks pass within one second of deploy. Total time from first command to live production: under 107 seconds.

The three-command path works because `oneie init` copies a known-good project scaffold (agents directory, wrangler config, worker entrypoints) that has been built and deployed hundreds of times. There is no "customize the template" step before the first deploy. The customization is in the agent markdown files, which can be edited after the first deploy is live.

The deployment target is the agency's own Cloudflare account. The Workers are owned by the agency. The domain is the agency's. The substrate data stays in the agency's TypeDB instance. ONE is not in the middle of the client relationship. It is underneath it, out of sight.

Local Dev (`oneie dev`)

`oneie dev` starts the full local Cloudflare Workers stack using `wrangler dev`. Four services run locally: the Astro app, the API Gateway, the Sync Worker, and the NanoClaw agent worker. The local environment mirrors production exactly. There is no "local mode" that behaves differently from the deployed version.

The full dev stack:

```
oneie dev
# Starts:
# → Astro Worker on localhost:4321
# → API Gateway on localhost:8787
# → Sync Worker on localhost:8788
# → NanoClaw on localhost:8789
# → TypeDB connection (remote, or local if configured)
```

`oneie doctor` confirms everything is wired correctly:

```
[ok] Config      ~/.config/oneie/config.json
[ok] API Key     sk-***...abc
[ok] Substrate   http://localhost:8787  12ms
[ok] TypeDB      https://api.one.ie/typedb/health  318ms
```

The `agent --watch` flag adds live hot-reload for agent markdown files. Save a change to `agents/director.md` and the substrate syncs within a second. No restart. No rebuild. The agent's behaviour updates live.

For TypeScript teams, this is a fast development loop. Edit the markdown. See the behaviour change in the running chat UI. Test edge cases. Push to production when it's right.

Worked Example

It is a Thursday morning. The agency has just taken on a new commercial property management client. The brief calls for a client-facing chat agent that handles maintenance requests, qualification questions, and new tenant enquiries.

A developer on Donal's team opens VS Code. She creates `agents/property-manager.md`. She writes the name, the system prompt, the four skills (`handle-maintenance`, `qualify-tenant`, `book-inspection`, `send-followup`), the voice contract (formal, helpful, never overpromise on timing), and the pricing (internal, billed to the client workspace).

She saves the file. The watcher picks it up:

```
synced agents/property-manager.md unit=property:manager
```

She runs the agent in local chat. Tests the maintenance request flow. Adjusts the voice contract. Tests again. It takes eleven minutes.

She runs:

```
oneie skill publish property:manager:handle-maintenance --name "Maintenance Request" --  
price 0.002 --scope group
```

The skill is published to the client's workspace. Twenty-eight seconds later, `oneie stats` shows:

```
units: 24 (+1) highways: 7 skills: 31 (+1) revenue: $0.00
```

She deploys:

```
oneie deploy
```

107 seconds. Health checks pass. The skill appears in the client's chat interface. Their team tests it before lunch.

From idea to production: forty minutes. No release meeting. No staging environment approval. No ticket to a platform team. The developer who understood the client's needs shipped it herself, from her IDE, with the same commands that will ship every future skill.

That is what verb-surface parity makes possible. The tools do not slow the work down.

Objections

"My team uses Python."

The REST API is language-agnostic. Python developers call `POST /api/signal`, `GET /api/loop/highways`, and every other substrate endpoint with `requests` or `httpx`. The `oneie` Python package wraps the most common operations and adds the `oneie run agent.md` CLI verb, which loads a markdown agent, builds a uAgents Protocol instance, registers with the Almanac, and connects to the OpenRouter LLM client. Python and TypeScript teams can work on the same substrate simultaneously. One team's signals mark the same paths as the other's. The substrate does not know which language sent the signal.

"What if we want to add capabilities later that you don't support?"

The SDK is MIT licensed. The source is public. The API is documented and stable, versioned at `api.one.ie` with explicit deprecation notice before any endpoint changes. Adding a capability means writing a new skill in markdown, publishing it with `oneie publish`, and wiring the handler in your preferred language. There is no platform team to petition, no feature request queue, no waiting for the next release cycle. Your engineers build it. The substrate hosts it.

"What if we need to leave the platform?"

`oneie export` exports any substrate data as JSON: paths, units, skills, highways, or toxic path lists. The agent markdown files are already in your repository. The corpus of conversations is stored in your TypeDB instance, which is your data. An agency that decides to leave takes its agents, its data, and its corpus. The platform does not hold any of it on your behalf.

"Our clients' data can't touch a shared platform."

Each client workspace is a separate data space in TypeDB with isolated paths and separate signal histories. There is no cross-client data bleeding. Deletion is one command (`oneie forget <uid> --yes`) and removes all records for that unit across the substrate. For GDPR, the command produces a receipt JSON you can attach to the deletion request.

"The tools look complex. My developers don't want to learn a new system."

The developers who already use Claude Code or Cursor get the MCP server for free: 42 tools available without leaving their IDE. Developers who prefer the terminal get a documented CLI with shell completion and JSON output. Developers who prefer code get a typed SDK with six methods. None of these require learning the others. A developer can be productive on the substrate in under twenty minutes using whichever surface they already know.

"What's the worst case if the build takes longer than expected?"

The three-command deploy is a floor, not a ceiling. A working agent in the substrate in three commands is documented and tested. Complex multi-skill builds take longer, but they're built on the same substrate, and every component can be tested independently before the deploy. The `--dry-run` flag previews every mutating operation. Nothing reaches production without passing `oneie doctor`.

"We're already committed to another developer toolchain."

The API has no required client library. REST over HTTPS. JSON request bodies. Standard Bearer auth. Any toolchain that can make an HTTP request can call the substrate. The CLI, SDK, and MCP server are conveniences. The substrate underneath them works without them.

Failure Modes

Agents go stale. An agent markdown file that is not reviewed after the first month tends to drift from the client's current reality. The voice contract stays accurate for the initial brief. The client's product changes. The agent does not. The fix is `oneie agent --watch` in development: edit, see the change, deploy. The failure mode is not a platform failure. It is a process failure. The platform makes it easy to update. The team has to run the update.

Paths do not harden without traffic. The substrate learns from signals. An agent that receives ten signals a week builds pheromone slowly. An agent that receives ten thousand signals a week builds highways in days. A new client deployment with low initial traffic will have slower routing in the early weeks. The fix is volume, which the agency controls through how they position the agent to the client's end users. The substrate cannot learn from traffic that does not exist.

Local and production diverge under manual edits. If a developer edits a TypeDB record directly, without going through the SDK or CLI, local state can diverge from what the markdown spec describes. The substrate treats the API as the canonical write path. Direct TypeDB edits are not versioned, not auditable, and not reproducible. Use `oneie agent` to sync. Use `oneie export` to inspect. Use `git` to version the markdown.

Comparisons

ONE CLI vs. Vercel CLI. Vercel's CLI deploys frontend apps. ONE's CLI deploys agents, publishes skills, marks pheromone paths, hires agents, posts bounties, and tails live substrate events. They are not competitors. They solve different problems. An agency using Vercel for its own marketing site and ONE for client agent deployments is a normal configuration.

ONE MCP vs. custom AI tool integrations. Most AI-assisted developer tools require custom integration code for every capability: a Slack tool, a GitHub tool, a database tool, each written separately. MCP is a standard. Add `@oneie/mcp` once. The IDE gains 42 substrate tools immediately, with no custom integration code. Future substrate capabilities become available when the MCP server updates, not when the IDE vendor adds a plugin.

Pricing Math

The three-command deploy has no per-deploy cost. Cloudflare Workers on a paid plan run at \$0.50 per million requests and \$12.50 per million CPU seconds. A client workspace running 50,000 conversations a month at an average of 5 requests per conversation (250,000 requests) costs under \$1 in compute. The signal processing, path marking, pheromone accumulation, and highway computation are all in-memory operations with sub-millisecond latency. The TypeDB queries that persist learning are the only database cost, and they run on writes, not on reads. An agency with 50 active clients at this volume pays under \$50 per month in compute for the entire infrastructure layer. The agency's margin is the credit markup on top, not the infrastructure.

Day in the Life

It is 9:00 on a Monday. Donal opens his terminal. `oneie tail` starts streaming the weekend's substrate events: which paths were marked, which agents earned, which skills were called most. He sees that the property manager agent's `qualify-tenant` skill received forty-three calls over the weekend, with a 94% success rate. The `handle-maintenance` skill received eleven calls, all resolved.

He opens `agents/property-manager.md`. The voice contract has a line about maintenance response times that he knows is now inaccurate. The client changed their SLA last week. He edits it. He saves. The watcher logs `synced agents/property-manager.md`. The agent is updated.

At 10:00, a new client brief arrives. The agency is taking on a multi-location dental group. Donal spends twenty minutes authoring `agents/dental-group.md` with the client's four skills. He publishes the agent to a new client workspace.

At 10:30, he runs `oneie deploy`. 107 seconds later, the dental group agent is live.

At 11:00, he looks at `oneie stats`:

```
units: 25 (+1)  highways: 7  skills: 35 (+4)  revenue: $0.00
```

By Thursday, after the dental group's front desk team has used the agent through thirty patient enquiries, `oneie stats` shows:

```
units: 25  highways: 9 (+2)  skills: 35  revenue: $0.14
```

Two new highways. The substrate found the paths that work. The agency did not configure them. The substrate learned them.

That is the development loop. Author. Deploy. Watch. Let it learn.

FAQ

Can a non-developer author agents?

Yes. The chat surface and the markdown format require no code knowledge. A strategist who understands the client's needs can write an ADL document in the same way they would write a creative brief. The technical team can then review, refine, and deploy it. The authoring does not require them.

Do we need a DevOps team to deploy?

No. `oneie deploy` wraps `wrangler deploy --env production`. The only prerequisite is a Cloudflare account and a wrangler config. The `oneie init` scaffold includes both. A developer who has deployed a Cloudflare Worker before will recognize every step. One who hasn't can follow the init output.

Can we run the substrate locally before deploying?

Yes. `oneie dev` runs the full stack locally. The only external dependency is the TypeDB connection, which can point at a remote instance or a locally-running TypeDB server. Every CLI verb and every SDK call works identically against `localhost:4321` and `api.one.ie`.

How does the Python package relate to the TypeScript SDK?

They both call the same REST API. The Python package adds the `oneie run` CLI verb, which loads an agent markdown file and starts a uAgents Protocol instance registered with the Almanac. Python agents appear in the same substrate as TypeScript agents. They emit signals to the same paths. They earn on the same economy.

What happens when a skill fails?

The closed-loop contract requires every signal to close. A failed skill call is a `warn()` on the path. Path resistance increases, routing probability to that agent decreases, and the agent's success rate is recorded. After enough warnings, the agent's generation increments and its system prompt is rewritten by the substrate's evolution loop. The platform surfaces the problem. It does not hide it.

Can we extend the MCP server with custom tools?

Yes. `createRouter()` and `register()` let you add custom tools alongside the built-in 42:

```
import { createRouter, serve, substrateTools } from '@oneie/mcp'

const router = createRouter()
for (const tool of substrateTools()) router.register(tool)
router.register({
  name: 'my_custom_tool',
  description: 'Agency-specific operation',
  inputSchema: { type: 'object', properties: { input: { type: 'string' } }, required:
    ['input'] },
  handler: async (args) => ({ result: process(args.input) }),
})
await serve(router)
```

The custom tool appears alongside the substrate tools in the IDE. No re-compilation of `@oneie/mcp` required.

Glossary

ADL (Agent Definition Language). The structured markdown format that defines an agent: name, system prompt, skills, voice contract, pricing, lifecycle status. Every surface reads it. No surface bypasses it.

Verb-surface parity. Every substrate operation is reachable from every surface (chat, CLI, SDK, MCP). The surface is a preference, not a capability gate.

Signal. The atomic unit of work in the substrate. Two fields: `receiver` (who gets it) and `data` (what it carries). Everything in the substrate flows through signals.

Pheromone. The substrate's learning mechanism. Successful paths accumulate strength. Failed paths accumulate resistance. Routing follows the strongest path. Over time, frequently successful paths become highways.

Highway. A path whose strength has exceeded the promotion threshold. Routing to a highway is deterministic, not probabilistic. Highways harden on-chain.

NanoClaw. The edge-native agent worker deployed per client persona. Runs on Cloudflare Workers. Receives Telegram, Discord, and HTTP signals. Routes them through the substrate. Returns streamed responses.

TypeDB. The substrate's brain. Stores paths, skills, hypotheses, and learning. All path queries run against TypeDB. The runtime is the nervous system. TypeDB is memory.

Cross-References

Page 05 (Agents) shows what an agent definition looks like in practice: the ADL format that CLI, SDK, and MCP all read. Page 06 (Skills) shows how skills published via `oneie publish` appear in the client-facing chat surface. Page 15 (Security) covers the data isolation model that makes per-client deployments safe: the same model that governs CLI `export` and `forget` .

Pick your surface: chat, CLI, IDE, MCP.

Security

No password resets. No seed phrases. Secure Enclave and Touch ID. When there is nothing to store, compliance gets smaller. When there is nothing to leak, breach disclosure gets shorter.

That is the whole security story in two sentences. The rest of this page is the proof.

Brad's anxiety about compliance, data, and GDPR is not irrational. It is the right question at the wrong level of abstraction. Most platforms answer it by building bigger walls around bigger stores of sensitive data. ONE answers it by shrinking what there is to protect.

You own your keys

The headline is not marketing. It is a description of a cryptographic fact.

Your keys live in the Apple Secure Enclave: a dedicated security processor, physically separate from the application CPU, manufactured to a specification that makes key extraction impossible by design. Apple's own engineers cannot read the keys that live there. Neither can ONE.

The gate is Touch ID. Your fingerprint is non-transferable by physics, not by policy. No password can be phished from it because there is no password. No credential can be stolen from a server because no server holds it.

When a user taps "claim payment" on a ONE chat, here is what happens in under 5 seconds:

```
Touch ID → Secure Enclave PRF → AES-256-GCM unwrap → Ed25519 seed
                                                    (lives for milliseconds)
Ed25519 seed → WebCrypto non-extractable import → sign(txBytes) → Sui tx
seed bytes.fill(0)                                (raw bytes wiped immediately)
```

The seed exists in memory for the duration of one sign operation. It is never logged. It is never sent over the network. It is never stored on ONE's servers. The transaction lands on-chain. The memory is zeroed.

A developer reading this will recognise the pattern: `crypto.subtle.importKey` with `extractable: false`, immediate buffer zero after import, no `await` between unwrap and sign. This is not a claim about intent. It is a constraint baked into the WebCrypto API. Intent cannot be exfiltrated. Key handles cannot be serialised.

For Brad's clients (the dentist, the window installer, the accountant), the experience is: tap their finger, the payment goes through. They never see a seed phrase. They never create a password. There is no password reset flow because there is no password.

The 5 wallet states

The wallet exists from the moment a user arrives. Security grows as the user commits more value. Both are true at the same time.

State	Name	What exists	Gate	Recovery
1	Ephemeral	32-byte seed in IndexedDB, plaintext	None (zero friction)	None, by design; spending capped
2	Saved	Seed wrapped with passkey PRF, stored in passkey largeBlob	Touch ID	12 BIP39 words (paper)
3	Linked	Identity linked to Google account, wrapping also synced to Better Auth user record	Touch ID + Google OAuth	BIP39 phrase, then re-enroll passkey
4	Multi-device	Passkey + ciphertext synced to additional devices via iCloud or Google Password Manager	Touch ID on any enrolled device	BIP39 phrase on new ecosystem
5	Recovered	All devices lost; BIP39 paper used to restore	12 words physically possessed	By definition, the break-glass was used

State 1 exists because zero-friction arrival matters. A user who has to create a password before they can see a payment link will not complete the journey. The risk is managed structurally: State 1 wallets have a spending cap enforced at the transaction level. The platform refuses to sponsor a transaction that would exceed it. This is not a UI nudge. It is a hard limit.

State 2 is the moment the user chooses to protect. Touch ID fires once. Three things happen in that single prompt: the passkey is created, the seed is wrapped and stored in the passkey's `largeBlob` extension, and twelve BIP39 words are shown once for the user to write down. From that point forward, every signing operation is gated by Touch ID. There is no fallback to a password.

The transition from State 1 to State 2 is the security architecture's most important moment. Before it: a capped spend surface with an acceptable failure mode. After it: a full biometric-gated wallet with paper break-glass. The product is designed to make State 2 the obvious next step after first meaningful use.

Compliance

Brad's compliance fear, stated plainly: what if GDPR, SOC 2, or the EU AI Act creates a liability that wrecks the business he just built.

The answer is not a policy document. The answer is an architecture that shrinks what compliance has to cover.

GDPR. The regulation requires you to protect personal data, respond to deletion requests, and report breaches. ONE reduces the personal data surface in two ways. First, per-tenant data isolation means each client's data exists in a separate logical space. A deletion request for one client touches nothing else. `forget(uid)` deletes the tenant key encryption key (KEK), and the data becomes unreadable instantly without row-by-row deletion across tables. Second, the platform's agent-side data is signals and outcomes, not personally identifiable records by default. The signals that matter for learning are structural: which paths worked, which did not. Not a ledger of which individual did what.

SOC 2. The trust service criteria map onto ONE's design directly. Availability: Cloudflare edge, deployed globally, no single point of failure. Confidentiality: per-tenant KEK means one tenant's data cannot be read by another even if a database row were extracted. Security: the 8-step policy enforcement point (PEP) runs before every signal — ABAC, RBAC, ReBAC, capability, budget, rate limit, nonce deduplication, delivery. No signal reaches an agent without clearing all eight. Processing integrity: the four-outcome loop closes every signal, or emits a warn. There are no silent failures. Privacy: seed material never touches ONE's servers; the only thing the server holds is an encrypted blob keyed to a Better Auth user ID.

EU AI Act. The Act creates obligations proportional to risk level. The risk level of an AI system is determined partly by what it can do to a person without their knowledge or consent. ONE's architecture makes covert high-stakes action structurally difficult: the co-sign pattern (Pattern A) requires a human Touch ID to complete any transaction above a threshold. The scoped autonomy pattern (Pattern B) runs agent transactions through a Move module that enforces daily caps and allowlists at consensus level. "Cannot" is different from "should not." The Act's transparency obligations are also easier to meet when the agent's definition is a markdown file: auditable, versionable, readable.

What ONE accepts. Physical access to the TypeDB Cloud infrastructure is TypeDB Cloud's responsibility, not ONE's. Cloudflare edge compromise is partially mitigated by minimising what memory holds. Simultaneous loss of a user's device, Apple ID, and paper backup is an accepted end state, mitigated by keeping the paper and devices in separate physical locations. These are explicit acceptances in the threat model, not gaps.

Every quarter, a canary transaction fires on each active deployment. The result either verifies or it doesn't. A failed canary is not a policy failure. It is a signal that triggers immediate investigation. Verification over presence. The quarterly cadence is not a compliance ritual. It is a deterministic check.

The threat model is a table

Attack vector	Defense	Accepted risk
Phishing / credential reuse	Passkeys are domain-bound; no password exists to phish; Touch ID is the only gate	User approves a legitimate-looking but malicious transaction (UI-layer, not crypto-layer)
Stolen or lost device	FileVault encryption at rest; Secure Enclave keys non-exportable; passkey requires biometric on enrollment	Device stolen in unlocked state within seconds of use
Apple ID takeover	Advanced Data Protection (ADP) with E2EE; Recovery Key on paper; second passkey on second Apple device	Recovery Key paper destroyed without replacement and Apple ID simultaneously compromised
Seed phrase exposure	No seed phrase shown to users by default; BIP39 shown once at State 2 enrollment, confirmed by retyping one word; no export UI except on explicit restore flow	User photographs their 12 words and stores them digitally
Supply chain (deps, extensions, scripts)	Pinned lockfiles; <code>socket.dev</code> on CI; <code>pnpm audit</code> ; Subresource Integrity on every external script; <code>no curl bash</code>	Compromised dependency with a silent exploit affecting a pinned version
XSS or compromised JS	State 1 balances capped at platform level; State 2+ seed only exits encryption on biometric prompt; passkey PRF cannot fire without Touch ID	Malicious JS tricks a logged-in State 2 user into approving a fake transaction that looks legitimate
Rogue on-chain agent	Sui Move module enforces daily caps, allowlists, pause, revoke at consensus; <code>cannot</code> replaces <code>should not</code> ; biometric root uncrossable regardless of agent depth	Agent operates within its Move-defined scope in ways the human did not anticipate
Cross-tenant data probe	Tenant KEK means each group's encrypted data is unreadable without that group's key; <code>tenantScope()</code> filter applied to every query; uniform 403 on private group existence	None accepted; isolation is the strongest guarantee in the model
Prompt injection via agent tool output	Zod validation on all LLM structured output; tool allowlist per capability defined at skill	Clever injection that passes Zod validation and operates within tool allowlist

Attack vector	Defense	Accepted risk
	declaration, not at runtime prompt	
Insider threat (destructive ops)	Dual-admin requirement for owner transfer, tenant delete, bridge creation: two signed signals within 15 minutes	Two admins simultaneously compromised
GDPR deletion request	<code>forget(uid)</code> deletes tenant KEK; encrypted data becomes unreadable without row-by-row purge; cascade delete documented and tested	Archived backup decrypted before KEK deletion propagates to all cold storage copies

No passwords, no seed phrases

This section exists for the sceptical reader who assumes “no passwords” is a feature claim rather than a design decision.

The architecture is this: instead of a password, there is a passkey. Instead of a server checking whether you typed the right string, the Secure Enclave generates a cryptographic assertion that the right biometric was present. The assertion is domain-bound: it only works for `one.ie`, not for a phishing site at `one-ie.com`. It cannot be replayed because it contains a fresh challenge every time. The server never sees what the SE produced, because the assertion is verified by the WebAuthn protocol before it reaches any ONE code.

Instead of a seed phrase shown at onboarding, there is a seed phrase shown at the moment of protection: when the user has chosen to move from State 1 to State 2, after their first meaningful transaction. The product says: here are 12 words, write them on paper, this is the last-resort backup if you lose every device and lose your Apple ID at the same time. One word is confirmed. The rest the user writes and keeps.

The twelve words are not stored anywhere by the platform. The platform holds a flag: `bip39_shown_at: "2026-..."`. The words themselves live on paper.

If a user loses all devices, visits a fresh browser, and types those twelve words, the seed is reconstructed, a new passkey is enrolled, and the wallet is live at the same address it has always had. The address does not change across states, across devices, or across ecosystem boundaries. One seed, many wrappings, the same address from State 1 to State 5.

A password requires a server to store something. That something can be breached. It can be reset by someone who knows the email address and can intercept the reset link. It can be phished. A passkey cannot be phished. There is no string to steal.

Worked example. A client's customer taps "claim payment" in a chat window on their iPhone. The ONE interface sends a signing request. Touch ID fires, bound to this exact passkey for this exact origin. The Secure Enclave runs the PRF operation. The seed is unwrapped in memory. The Ed25519 signing key is imported as non-extractable. The transaction is signed. The seed bytes are zeroed. The complete round trip from user intent to signed transaction takes under 5 seconds. The user sees "payment claimed." They have never typed a password. They have never seen a seed phrase. No credential was transmitted over the network.

The signing code does not obscure what is happening:

```
async function sign(txBytes: Uint8Array): Promise<Uint8Array> {
  const wallet = await idb.get('wallet')
  const wrapping = wallet.wrappings.find(w => w.type === 'passkey-prf')

  // Touch ID → PRF output (Secure Enclave fires here)
  const assertion = await navigator.credentials.get({
    publicKey: {
      challenge: crypto.getRandomValues(new Uint8Array(32)),
      allowCredentials: [{ type: 'public-key', id: wrapping.credId }],
      userVerification: 'required',
      extensions: { prf: { eval: { first: PRF_SALT } } }
    }
  })
  const prfOut = assertion.getClientExtensionResults().prf.results.first

  // Derive AES key from PRF output
  const aesKey = await hkdfToAesGcm(prfOut, 'wallet-wrap-v1')

  // Unwrap seed – lives for microseconds
  const seedBytes = await crypto.subtle.decrypt(
    { name: 'AES-GCM', iv: wrapping.iv }, aesKey, wrapping.ciphertext
  )

  // Import as non-extractable, wipe raw bytes immediately
  const signingKey = await crypto.subtle.importKey(
    'raw', seedBytes, { name: 'Ed25519' }, false, ['sign']
  )
  new Uint8Array(seedBytes).fill(0) // ← zero the buffer

  return new Uint8Array(await crypto.subtle.sign('Ed25519', signingKey, txBytes))
}
```

The `extractable: false` flag is not a comment. It is enforced by the WebCrypto API. A key imported with that flag cannot be exported. The `fill(0)` is immediate, before any other operation. There is no gap between unwrap and sign where a concurrent operation could read the seed. The design is not "we promise not to log it." The design is "the API will not let us read it back."

A day in the security model

06:45. The dentist opens a browser on her phone. She has never created a password for this site. The chat window is already there. Her wallet address is her identity. Her corpus, conversation history, and scheduled posts all load from her encrypted group, decrypted client-side with the key her passkey holds.

08:20. A patient clicks a "request appointment" link in the dentist's chat. The patient has never visited this site either. A State 1 wallet is created for them in under a second. They are now in the dentist's inbox. No account creation. No email verification. No onboarding.

11:14. The dentist approves a \$150 deposit payment from a new patient. Touch ID fires. The Secure Enclave runs the PRF. The signing key lives for 80 milliseconds. The transaction lands on Sui. The patient's wallet shows the debit. The dentist's wallet shows the credit. No payment processor saw a card number. No ONE server held the signing key.

14:30. Brad reviews the monthly report for the dentist account. The report is generated automatically: 43 conversations, 12 qualified leads, 3 appointments booked, \$450 in payments processed, response time 97ms median. One page. 90 seconds to generate. No one wrote it manually.

17:55. A quarterly verification runs. The canary transaction fires: a known-value transfer to a known address. The signing chain is intact. The health check passes. No action required. The receipt is appended to the audit log.

23:00. A security event fires: five failed authentication attempts from a single IP in 90 seconds. The substrate emits `{ kind: "auth-fail" }` and `warn(0.3)` on the auth boundary path. After 10 samples with resistance exceeding the toxicity threshold, the pheromone routing stops sending signals to that path. No rule was written. No IP was manually blocked. The system routed around the attacker because that is what pheromone-based routing does with consistently failing paths.

The dentist, the patient, and Brad experienced none of this as a security system. They experienced it as a fast, quiet product that worked.

BIP39 paper break-glass

Two scenarios require the paper.

The first: the user loses all their devices and cannot recover through iCloud or Google. State 5. They visit `/u` from any browser on any device. They type twelve words. The seed is reconstructed. A new passkey is enrolled on the new device. The wallet is live. The address is the same.

The second: a passkey is suspected compromised. From any other enrolled device, the user goes to Settings, selects the suspect credential, and revokes it. The wrapping for that credential is removed. The seed is still accessible through any remaining passkey. If all passkeys are suspected, the user drains the wallet to a new address, also reachable via BIP39, and that address becomes the new home.

The paper does not expire. It does not need to be rotated unless there is a reason to believe it was seen by someone else. The break-glass cadence is: generate once at State 2 enrollment, store in two physical locations, verify quarterly that the paper is readable and that the words still reconstruct the wallet.

Verification: type the words into the restore flow on a test device, confirm the address matches, do not complete the enrollment. That is the quarterly check. It takes three minutes.

The BIP39 wordlist is standard: 2048 words, each unambiguous, chosen for visual and phonetic distinctiveness. Twelve words from this list give 128 bits of entropy. The same entropy that protects a Bitcoin wallet. The same break-glass mechanism that professionals who manage significant on-chain value have used for a decade.

The platform's design decision: show the words at State 2 enrollment, not at first visit. A user who has not committed value to their wallet does not need a break-glass mechanism yet. A user who has tapped Touch ID and wrapped their seed has made a commitment. That commitment is the right moment for the backup conversation.

Per-tenant data isolation

Each agency client is a separate group in the substrate. Each group has a separate encryption key (KEK) derived from the master key using HKDF: `HKDF(MASTER_KEY, gid)`.

The consequence: one agency client's data cannot be read by another agency client even if both are on the same platform, even if a database administrator pulls the raw rows, even if a backup is extracted. Without the KEK, the signal data is ciphertext.

For Brad, the agency owner, this means one client's data never contaminates another's. If a client leaves, `forget(uid)` deletes the KEK. The data persists as unreadable ciphertext. From a GDPR standpoint, the data is effectively gone: the deletion is instant, complete, and does not require identifying and purging individual rows.

For Brad's clients (the dentist, the restaurant, the accountant), this means their conversations, contact data, and transaction history are logically isolated. The dentist cannot see the accountant's data. The accountant cannot see the dentist's. This is not enforced by application logic alone. It is enforced by the absence of the decryption key.

Every query in the platform applies `tenantScope()`, a filter that restricts results to the caller's group hierarchy. This runs at the query layer, not at the application layer. Bypassing it requires bypassing TypeDB's query execution. There is no accidental cross-tenant read from a bug in application code.

Named integration. The Cloudflare Durable Object used as the WsHub manages live connections per group. When a revocation fires, it broadcasts over the WsHub in under 1 second to every Worker holding a connection. Every Worker clears its cache. The revocation is effective immediately, not at TTL expiry.

Pen-test cadence

The platform runs a structured external review cycle.

Quarterly: canary transactions fire on every active deployment. The canary transaction is a known-value transfer to a known address. If it succeeds and lands in the expected state, the signing stack is verified. If it fails or lands in an unexpected state, the alert fires immediately.

The canary is not a compliance checkbox. It is a deterministic check with a binary outcome. Either the number is right or it is not. Either the signing chain from Touch ID to on-chain confirmation is intact or it is broken. There is no gray area. There are no vibes.

External penetration testing: contracted before each major release cycle. The scope includes the WebAuthn implementation, the passkey PRF ceremony, the transaction signing flow, the PEP order in `persist().signal()`, and the per-tenant isolation implementation. Findings are classified by severity. Critical findings block release. The results are documented and the mitigations are verified.

Bug bounty: the substrate is open-source. Engineers who find and responsibly disclose security issues are credited and compensated. The open-source substrate means the implementation is visible to scrutiny. Security through obscurity is not part of the model.

Supply chain: `socket.dev` runs on CI against every dependency update. `pnpm audit` runs on every build. Lockfiles are committed and pinned. New dependencies require a documented review before merge.

Incident response

If a credential is suspected compromised, the sequence is:

1. From any enrolled device, revoke the suspect credential in Settings. The WsHub broadcasts the revocation. Every connected Worker clears its cache within 1 second.
2. If the seed is suspected compromised (device stolen in unlocked state, XSS exploit suspected), drain the wallet to a fresh address derived from the BIP39 phrase. The old address is abandoned. The new address becomes the active wallet.
3. Document the incident in the vault. Rotate any API keys or tokens the compromised device held access to. The runbook for this lives as the first entry in `~/.vault.age`.
4. If a client's data environment is suspected compromised, delete the tenant KEK. The data is unreadable from that moment. Issue a notification under the relevant data breach notification requirements. The notification timeline starts from the moment of confirmed breach, not from the moment the KEK was deleted.

The dual-admin requirement for destructive operations (owner transfer, tenant delete, bridge creation) means no single compromised credential can execute a catastrophic action. Two signed approvals within 15 minutes are required. A compromised credential that cannot find a second confirmation within that window cannot complete the action.

Failure mode: simultaneous loss of device, Apple ID, and paper. If all three are lost at once, the wallet is unrecoverable. This is the accepted risk. The mitigation is geographic separation: the device and the paper should never be in the same bag, the same office, or the same building. The paper is the backup for the situation where the device is gone. If both are gone at the same moment, the design has been defeated. That is documented and understood.

Failure mode: phished transaction approval. A user in State 2 can be shown a fake transaction that looks legitimate and tap Touch ID to approve it. Touch ID fires because the passkey is domain-bound: the user is on `one.ie`. The transaction is signed and lands on chain. There is no mechanism to claw it back. The defense against this failure mode is at the UI layer: transaction preview in plain language before the Touch ID prompt, no gas abstraction that hides what is actually being authorised. This is a UI and product constraint, not a cryptographic one. Cryptography cannot solve social engineering. The platform names this risk rather than pretending it is solved.

Failure mode: supply chain compromise of a pinned dependency. If a pinned version of a dependency contains a silent exploit that lands after pinning, the CI checks will not catch it. The mitigation is monitoring: `socket.dev` tracks maintainer changes, publish spikes, and unusual activity on dependencies in use. A suspicious signal triggers manual review before the dependency is deployed.

FAQ

Does ONE store my clients' data? Each client's data is stored in their own encrypted group. The encryption key is derived from a master key that ONE controls. ONE can delete the encryption key on request, rendering the data unreadable. ONE does not share one client's data with another. ONE does not use client conversation data to train models.

What happens if ONE goes out of business? The substrate is open-source at `github.com/one-
ie/one`. The product layer is held in escrow. The corpus belongs to the agency and can be exported in one command. You are not building on a closed substrate.

Can agents make transactions without human approval? Yes, with explicit configuration. The four patterns are: co-sign (always requires Touch ID), scoped autonomy (agent signs within Move-defined limits), delegated capability (one Touch ID mints a bounded capability, agent uses it), and peer (agent has full authority within its Move scope). The default is co-sign. Moving to scoped autonomy requires a deliberate configuration decision by the human.

What if an agent says something wrong to a client? The quality gate is 0.65 on the rubric. Below that threshold, output does not ship. Voice contract enforcement runs before every outbound message. The gate is mechanical, not manual.

What does GDPR deletion look like? One command. `forget(uid)` deletes the tenant KEK. The encrypted signal data becomes unreadable. The operation is logged and timestamped. The deletion is effective immediately for live data. Cold backup copies become unreadable as the KEK is not present in any backup.

Does the platform comply with the EU AI Act? ONE operates in categories where the risk classification depends on deployment context. The architecture supports compliance with high-risk category obligations: explainability (agents are markdown files), human oversight (co-sign pattern, quality gate), data governance (per-tenant isolation, deletion). Agencies deploying ONE in high-risk contexts should conduct their own conformity assessment with the specific deployment in scope.

What if my client photographs their BIP39 words and stores them in iCloud Photos? That is an accepted risk in the threat model. The product instructs the user to write the words on paper and store them physically. The platform cannot enforce paper. The mitigation is on the UI layer: the words are shown in a way that resists screenshot, and the verify step requires retyping. A determined user can still defeat this. That is documented.

Glossary

Secure Enclave. A dedicated security processor inside Apple Silicon chips. Physically separate from the application CPU. Keys generated there cannot be exported. Touch ID input goes directly to the Enclave; the application CPU never sees it.

Passkey. A WebAuthn credential. Domain-bound: a passkey created for `one.ie` does not work on any other domain. Contains a public/private key pair where the private key never leaves the device. Replaces passwords.

PRF (Pseudorandom Function). A WebAuthn extension that allows a passkey to produce a deterministic output for a given input. Used to derive the key that wraps the wallet seed. The PRF output requires Touch ID to produce.

HKDF. Hash-based Key Derivation Function. Takes a master secret and produces independent derived keys for different purposes. A leak of one derived key reveals nothing about another.

AES-256-GCM. The encryption algorithm used to wrap the wallet seed. 256-bit key, authenticated encryption, tamper-evident.

BIP39. Bitcoin Improvement Proposal 39. Defines a wordlist of 2048 words used to encode cryptographic seeds as human-readable phrases. 12 words = 128 bits of entropy.

KEK (Key Encryption Key). A key that encrypts other keys. Per-tenant in ONE's architecture. Deleting the KEK makes all data encrypted under it unreadable.

PEP (Policy Enforcement Point). The function that runs eight checks before any signal reaches an agent. Runs on every signal. Fails closed.

Touch ID. Apple's biometric authentication system. Reads fingerprint and verifies against a locally-stored template inside the Secure Enclave. The template never leaves the device. Touch ID is what makes the passkey PRF fire.

Scoped Wallet. A Sui Move object that enforces daily spending caps, allowed recipients, pause, and revoke at consensus level. An agent operating through a Scoped Wallet cannot exceed its defined limits regardless of what its LLM prompt says.

Cross-references

- `../passkeys.md` — full wallet lifecycle: State 1 through State 5, signing flow code, PRF derivation, multi-chain support, recovery matrix.
- `../mac.md` — two-root architecture, biometric design, quarterly verification cadence, threat model table format.
- `../agents.md` — four patterns (co-sign, scoped autonomy, delegated capability, peer), Move module code, biometric floor guarantee.

Read the threat model.

Speed

“Speed is the ultimate weapon in business. All else being equal, the fastest company in any market will win.”

— Dave Girouard, Upstart (former President, Google Enterprise)

Speed is not a feature. Speed is the product. Every 100 milliseconds of added latency costs 7% of conversions. That is Akamai’s finding from 37 million sessions measured in 2017, and it has not aged. Every 100ms of homepage load improvement adds 1.11% to session conversion. That is Deloitte’s finding from 30 million user sessions across four industries in 2020. Amazon measured it too: every 100ms of latency costs 1% of sales. Greg Linden ran the A/B tests himself.

A decade of data tells one story: slow is expensive. Not inconvenient. Expensive. And the math applies to every surface, not just the page you load. The AI that answers. The tool that builds. The platform that deploys. The wallet that activates. The checkout that converts.

The question is not whether speed matters. The question is whether you can measure it. Most platforms cannot. We measure eight surfaces. The numbers are below. They are dated and verified.

The full speed table

Eight surfaces. Eight numbers. Every one measured.

Surface	What we measure	Result	Verified
Page load	FCP / LCP / INP, /chat page	sub-1s; Lighthouse 100/100/100/100	2026-05-03
Inference	First token on screen, p50	streaming starts < 300ms (97ms p50)	2026-05-03
Build	Hot reload, Astro dev server	hot reload < 100ms	2026-04-14
Deploy	Commit to global edge	107 seconds total	2026-04-14
Wallet	First wallet, p50	5 seconds	12,400 sessions, April 2026
Checkout	Crypto accepted	60 seconds	signup-to-link tested
TTFAPIC	Signup to working API call	1 minute	tested
Time to value	First agent doing real work	3 minutes	tested

The full stack. Not cherry-picked highlights. Every surface. Every number. Every date.

Page: FCP, LCP, INP

The /chat page scores 100/100/100/100 on Lighthouse. Performance. Accessibility. Best practices. SEO. Four hundreds.

First Contentful Paint: sub-1 second. Largest Contentful Paint: sub-1 second. Interaction to Next Paint: sub-1 second. Time to First Byte from the browser: 97ms p50. That figure was measured on 3 May 2026. It is the p50, not the median of a good day, but the median across the full distribution of sessions.

No other AI chat page scores four hundreds. Most do not publish their Lighthouse score at all. They forgot nothing. The number is embarrassing.

Why does this matter for Brad? His clients' customers open his clients' chatbots on mobile, on 4G, from a car park in Brisbane. Cloudflare measured it in 2024: mobile sites that load in 1 second convert five times better than sites that load in 10 seconds. Five times. Not a statistic about tech companies. A statistic about the dentist's website, the window installer, the restaurant. Brad's seven ICPs.

A slow chat surface is not a slow chat surface. It is a churn driver. And churn is the one thing Brad cannot afford when the model is 90% gross margin through software, not humans.

How we get there

Three things keep the page fast. None are magic.

First, the Astro framework ships HTML and CSS before JavaScript. The browser renders before it parses. Most React-heavy AI products ship a blank shell and populate it from the client. We ship content. The user sees something in under 100ms.

Second, heavy modules (the file picker, voice controls, payment panels) are lazy-loaded behind `Suspense` boundaries. They do not appear in the initial island bundle. They arrive when the user needs them, after the product has already felt fast.

Third, the page runs on Cloudflare Workers Static Assets with no cold-start penalty. The compute sits at the edge, not in a data centre responding to a round-trip.

The result is a page that feels instant because it is instant.

The measurement

```
// Lighthouse config used for CI gate
{
  "extends": "lighthouse:default",
  "settings": {
    "formFactor": "mobile",
    "throttling": {
      "rttMs": 40,
      "throughputKbps": 10240,
      "cpuSlowdownMultiplier": 4
    },
    "screenEmulation": {
      "mobile": true,
      "width": 375,
      "height": 667,
      "deviceScaleFactor": 2,
      "disabled": false
    }
  }
}
```

Mobile emulation. 40ms RTT. CPU throttled 4x. Not a flattering test. The test that catches problems before users do.

Inference: first token

The moment you press send, ONE opens a request to the model. Streaming starts at 97ms p50. The first token appears on screen before the user has finished reading the sentence they just sent.

This is not about the LLM. Every platform uses the same LLMs. OpenAI, Anthropic, Groq, Mistral: all commodities. The question is what happens between the user pressing send and the model starting to respond. That gap is the routing layer. That gap is where we win.

Comparison: ONE vs. the alternatives

Platform	TTFT p50	Lighthouse score
ONE /chat	97ms	100/100/100/100
ChatGPT (web)	~1,400ms	not published
Claude.ai (web)	~1,200ms	not published
OpenAI Playground	~1,800ms	not published

The ChatGPT and Claude.ai figures are measured from a standard UK broadband connection, with the page fully loaded, on the same prompt ("Hello") submitted three times and averaged. Not stress-test conditions. These are what Brad’s clients’ customers experience on an average day.

The difference between 97ms and 1,400ms is not a technical distinction. It is a felt distinction. BenchLM measured it in 2026: chatbot TTFT above 500ms stops feeling instant. At 1,400ms, users notice. At 1,800ms, they attribute the delay to the AI “thinking” and calibrate their expectations down. That recalibration is a trust problem.

ONE does not ask the user to recalibrate. The response starts while the user is still reading the question.

Why ONE is faster on the routing layer

The LLM is not the slow part. The routing decision is. Traditional AI orchestration adds 300ms per routing call: an LLM deciding which agent handles the request. ONE routes in under 0.005ms. That is 60,000 times faster on the decision layer alone.

Once a path hardens into a highway, after 50 successful passes, routing caches at the Cloudflare KV edge. Latency drops below 10ms. The model call is still 1–2 seconds. Physics governs that. But everything before and after the model call is fast.

Worked example: inference in the support scenario

Brad’s agency runs an AI support agent for a plumbing-supply client. The client’s customers ask questions like “Is the 22mm compression fitting in stock in Toowoomba?”

The user presses send. ONE routes the signal in 0.004ms. The routing decision hits the product catalogue skill, trained on the client’s inventory data. That skill returns context to the model in under 10ms via KV cache. The model streams the first token at 97ms p50. The user reads the answer before the second hand moves.

The plumber does not notice the latency. The plumber notices that the assistant knows what it is talking about. That is what the 97ms buys: attention fully on the answer, not the wait.

Build: cold build and hot reload

Hot reload is under 100ms. That is the number that matters for Brad’s technical partner Donal.

Donal is the one extending the platform. When he edits an agent’s markdown file, the dev server reflects the change before he has moved his eyes back to the browser. When he edits a React component, the island updates in the same frame. When he adds a TypeQL query to a TypeDB function, the result is visible in under a second.

Most development environments take 2–5 seconds for a hot reload. Some take longer. The difference between 100ms and 3 seconds is not a 30x improvement in developer experience. It is the difference between staying in flow state and breaking it. Every broken flow state costs a thought. Over a day of development, that compounds.

The cold build, the full Astro production build, takes 23 seconds. That includes TypeScript checking, Tailwind compilation, and Cloudflare Workers bundling. 23 seconds is fast for a production build. Fast enough that Donal can run it before a deploy without breaking rhythm.

Why the build is fast

Vite powers the development server. Vite's hot module replacement protocol sends only the changed module, not the whole bundle. In a codebase where each agent is a markdown file and each skill is a TypeScript module, the changed surface area is small. Small surface area means small HMR payload. Small HMR payload means fast reload.

The production build uses Rollup with code splitting. Each Astro island compiles independently. Heavy modules (attachments, payment panels, voice controls) ship in separate chunks that arrive only when the user requests them. The initial page bundle stays small. The build stays fast because it is building small things, not one large thing.

Deploy: push to global edge

Commit to production: 107 seconds.

Four services. One command.

Build (Astro):	23s
Gateway deploy:	13.7s
Sync:	8.2s
NanoClaw deploy:	9.2s
Pages deploy:	~16s
Health checks:	<1s
Total:	107s

The 107-second figure is from the deploy log dated 14 April 2026. Not an average. A single measured run. The variance across ten runs is under 15 seconds.

Most platforms take minutes, not seconds. Some require a deploy queue. Some have staging gates. Some have environment promotion steps that add 10–20 minutes. Those are not problems that affect a single deploy. They are problems that accumulate across a year of deploys.

Brad's agency ships changes to client agents every week. A new seasonal promotion. A new product line. A change to the support script. Each change is a commit. Each commit deploys in 107 seconds. No deploy queue. No staging gate. No waiting.

Not just a developer convenience. A client-retention mechanism. When a client calls at 9 a.m. and asks for a change before their 11 a.m. launch, Brad's team ships it at 9:45 and confirms it at 9:47.

The named integration: Cloudflare Workers

The deploy speed is possible because the infrastructure is Cloudflare Workers. No server to provision. No auto-scaling group to configure. The Workers runtime runs the code at the edge, in 330 cities, the moment the deploy completes.

Cloudflare’s global network means a user in Dublin, Brisbane, São Paulo, or Singapore hits an edge node within 20ms. The request never leaves the continent to reach an origin server. Not a claim about what Cloudflare can theoretically do. What the platform does by default, from the first deploy.

Wallet: first wallet in 5 seconds

The p50 for creating a wallet on ONE is 5 seconds. Measured across 12,400 sessions in April 2026.

Five seconds is not a pitch. Five seconds is a user experience. No app install. No seed phrase. No twelve-word backup. No exchange account. No KYC.

Five seconds means: open the page, press the button, hold your thumb on the sensor, done.

The 5-second breakdown

Every second of those five is accounted for.

Second	What happens
0–1s	Touch ID prompt appears; user places thumb
1–2s	Secure Enclave generates the key pair; the biometric gates the generation
2–3s	Network round-trip: the public key registers on the platform; session opens
3–4s	UI renders the wallet surface; the user sees their balance and controls
4–5s	State settles; the wallet is live and ready to receive

The seed phrase does not exist in this flow. The mnemonic backup does not exist. The user’s key was generated inside the Secure Enclave, the hardware security module inside their phone or laptop, and it never leaves it. Touch ID is the only gate. The user holds the only key.

For Brad’s agency, this matters because his clients’ customers are not developers. They are a dentist’s receptionist, a restaurant regular, a window installer’s apprentice. A 5-second wallet is something they can do without a tutorial. A 30-second flow with seed phrases and confirmation dialogs is something they do once and abandon.

Comparison: alternatives

Flow	Time	Friction
ONE wallet (Touch ID)	5s p50	Touch ID only
MetaMask (mobile)	3–5 minutes	Seed phrase backup, extension install
Coinbase Wallet	2–4 minutes	Email, KYC, app install
Typical exchange signup	20–40 minutes	KYC, email, ID upload, wait

The gap between 5 seconds and 20 minutes is not a speed advantage. It is a different category. MetaMask and Coinbase are designed for users who are motivated to hold crypto. ONE's wallet is designed for users who want to pay for something and get on with their day.

That is the population Brad's clients serve.

Checkout: crypto accepted in 60 seconds

From signup to crypto accepted: 60 seconds.

This is TTFAPIC adjacent but distinct. TTFAPIC is the API call. Checkout is the money. The 60-second checkout covers the full flow from arriving at the platform to having a live payment link that accepts SUI, ETH, SOL, BTC, BASE, ARB, or OPT.

What 60 seconds contains

The flow is three steps. Each takes under 20 seconds.

Step 1: Sign up. Touch ID. The account exists. The wallet exists.

Step 2: Configure the checkout. Name the product. Set the price. Choose the chains you accept. The default is all of them. Leave the default. Press save.

Step 3: Copy the payment link. Send it to a client. A customer clicks it. They pay in their chain. The merchant receives in theirs. ONE bridges.

The merchant does not need to know which chain the customer pays in. The customer does not need to know which chain the merchant prefers. The bridge runs underneath. Both parties see a clean payment confirmation.

For Brad's agency clients (a dentist accepting a deposit on a treatment plan, a window installer taking a 50% deposit on a job) the chain is irrelevant. The money arrives. That is the function.

TTFAPIC: one minute from signup to working API call

TTFAPIC, time to first API call, is the developer equivalent of the wallet onboarding. From creating an account to making a working API call: one minute.

No SDK to install. No OAuth dance beyond the initial auth. No documentation to read before the first result.

```
curl https://api.one.ie/v1/chat \
-H "Authorization: Bearer $ONE_API_KEY" \
-H "Content-Type: application/json" \
-d '{"messages": [{"role": "user", "content": "Hello"}]}'
```

That is the call. It works. The response streams. The key comes from the dashboard in under 30 seconds. Total: under a minute.

Stripe measured the developer equivalent of this threshold in 2023: developers who reach a working API call in under 10 minutes are four times more likely to integrate. ONE is under one minute. The integration rate is not a prediction. It follows.

Time to value: 3 minutes from first load to first agent doing real work

Three minutes. From opening one.ie for the first time to having an agent deployed and responding to real requests.

Minute 1: create the account, get the wallet, see the platform. Minute 2: open the agent editor, pick a template (support agent, sales agent, booking agent), connect the client's domain. Minute 3: copy the embed code, paste it into the client's site, press publish.

The agent is live. The first user message arrives. The substrate begins learning.

The 3-minute figure is for a motivated user following the default path. An experienced Donal can do it in 90 seconds. A first-time user who reads the tooltips takes 5 minutes. The p50 is 3 minutes.

Why speed claims are usually vanity, and why these are not

Brad has heard speed claims before. Every platform says it is fast. Most cannot tell you what surface they measured, on what hardware, at what time, with what sample size.

The objection is reasonable: speed claims are vanity.

Here is the answer. We measure eight surfaces. No other AI chat platform measures all eight. No other AI chat platform has published a Lighthouse score of 100/100/100/100. No other AI chat platform has broken down the wallet flow to the second. No other platform has a public deploy log with a timestamp.

The difference between a vanity speed claim and a real one is the audit trail. Our audit trail is in the table at the top of this page. Date, surface, number, method.

If a number on that table is wrong, it is verifiable. Run Lighthouse on /chat right now. Open a wallet on a phone. Time it. Submit a checkout. The numbers hold or they do not.

We will measure back.

Objections answered

"Aren't all AI products fast now?"

Fast on what? LLM speed is commoditised; every platform uses the same models. The difference is everything else: the routing layer, the page load, the build pipeline, the deploy infrastructure, the wallet flow, the checkout. We measure eight surfaces. Most platforms measure one, if any. No other AI chat scores 4x100 on Lighthouse. Not a claim we repeat twice. A claim we invite you to test.

"Speed is a technical metric. My clients care about results."

Your clients' customers leave at 3 seconds. Google measured it: bounce probability increases 32% from 1 to 3 seconds. At 10 seconds, it is 123%. Every bounce is a lost lead. Every lost lead is a client who calls to ask why the chatbot is not working. Speed is a results metric wearing a technical costume.

"My current platform is fast enough."

Fast enough compared to what? Measured how? On which surface? A platform that has never published a Lighthouse score and never measured TTFB is not fast enough. It is unknown. Unknown is not fast enough when Akamai's data puts 7% of conversions at stake per 100ms.

"A 97ms TTFB is nice, but my clients are on slow connections."

That is precisely the point. The Lighthouse score is measured on a 4x CPU-throttled mobile device at 10Mbps. That is a mid-range Android phone on a mediocre connection. The 97ms holds under those conditions. Not a fibre-broadband number. A dentist's-receptionist number.

"We'd need to migrate all our clients to change platforms."

The migration path is additive. ONE does not replace what Brad sells. It goes underneath it. The first client goes live in 48 hours. The migration is not a cutover. It is a roll. Each client moves on their renewal cycle. The platform earns the next one while the current one completes.

"What if the speed degrades as you add more clients?"

The substrate is stateless at the edge. Each request hits a Cloudflare edge node independently. Adding clients does not add contention. It adds instances. The Cloudflare Workers runtime is designed for this. Netflix and Shopify run the same runtime at 10 million concurrent requests. The path from 50 clients to 5,000 clients does not change the architecture.

"We've been burned before. A vendor promised speed, then scaled and slowed down."

That is the right thing to fear. The defence is the architecture, not the promise. Workers are stateless at the edge. TypeDB is horizontally scalable. The 107-second deploy time does not increase with client count. The 97ms TTFB is a function of the runtime, not the load. The table at the top of this page has a date column for exactly this reason. We are accountable to the number, not the claim.

"Speed sounds good, but what's the ROI calculation?"

One way to frame it. Brad's agency runs 200 clients at £3k/month. Average 1,000 chatbot interactions per client per month. Current platform: 1,400ms TTFB. Apply Akamai's 7% per 100ms figure and the conversion gap between 1,400ms and 97ms is approximately 91%. Even a conservative 10% conversion lift on those interactions compounds across 200 clients. At £3k/month average client value, a 10% increase in outcomes attributable to speed is £72k/month in protected revenue, or £864k/year. That is the ROI on a speed investment. Arithmetic, not projection.

Failure modes

Three ways speed can be lost. All three are known risks with known defences.

Failure mode 1: cold starts on the inference path.

If the routing layer calls the LLM for every decision, rather than using cached pheromone paths, TTFB climbs to 1,500ms or more. The defence is the highway mechanism. After 50 successful passes on a path, the route caches at the edge. Cold-start decisions happen on new paths only. On established paths, routing is sub-10ms. New clients start with cold routing. After two weeks of traffic, most paths are warm.

Failure mode 2: lazy-loading fails silently.

If a lazy-loaded module fails to import, because of a bundler error, a network timeout, or a deployment that shipped a bad chunk, the Suspense boundary shows a blank slot. The user sees nothing where the attachment picker should be. The defence is the health-check step in the deploy pipeline. The 107-second deploy includes a post-deploy health check that hits every major route and verifies the page loads with status 200. A bad chunk fails the health check before traffic reaches it.

Failure mode 3: the wallet generates keys on a slow device.

Key generation in the Secure Enclave is fast on modern Apple hardware and Google Pixel. On a 2018 Android device with a slower SE implementation, the generation step can take 2–3 seconds instead of 1 second. The 5-second p50 is measured across a mix of devices. The p95 is under 9 seconds. Under 10 seconds is the threshold for task abandonment (Jakob Nielsen). We are inside it on p95.

A day in the life: speed as the operating condition

It is 8:47 a.m. on a Tuesday. Donal is at his desk in Brisbane.

A client (a plumbing-supply company with seven branches) has asked for a change to their support agent. The winter catalogue is live. The agent needs to know about the new pipe-fitting range before the day's calls start.

Donal opens the agent's markdown file in VS Code. He adds three lines: the new product family name, the SKU range, and a note about the lead time for special orders. He saves the file.

The dev server hot-reloads in 84ms. He opens the browser. The change is live. He types a test query: "Do you have 28mm push-fit elbows in the new Titan range?" The agent answers with the correct lead time.

Donal runs `bun run deploy`. The terminal shows the deploy steps in sequence. 107 seconds later, the deploy confirms. He sends the client a message: "Titan range is live on your chatbot."

The client's first customer call comes in at 9:03. The customer asks about Titan fittings through the web chat. The agent responds in 97ms. The customer gets the answer. The call does not happen.

That is what 107-second deploys and 97ms TTFB mean in practice: the plumbing company's phone rings less often for things the chatbot already knows how to answer.

Comparison: what takes longer everywhere else

Three comparisons. Named. Numbered.

Lighthouse score. ONE /chat: 100/100/100/100. ChatGPT web interface: ~60/70/80/80 (typical, varies by session; not published, measured by third parties using Lighthouse extensions). Claude.ai: similar range, not published. The gap is not a version number. It is an architectural choice. Pages that hydrate entirely on the client score in the 60s. Pages that ship pre-rendered HTML score in the 90s and 100s. ONE ships pre-rendered HTML.

Wallet onboarding. ONE: 5 seconds, Touch ID. MetaMask mobile: 3–5 minutes, seed phrase required. Coinbase Wallet: 2–4 minutes, app install and email verification. The point is not that MetaMask is bad. The point is that MetaMask is built for users who want to hold and manage crypto. ONE's wallet is built for users who want to complete a transaction. Different populations. Different appropriate friction.

Build and deploy. Generic Node.js SaaS on AWS Elastic Beanstalk: cold build 3–8 minutes, deploy via pipeline 10–20 minutes, health check and traffic shift 5–10 minutes. Total: 20–40 minutes from commit to production. ONE: 107 seconds. The difference is the infrastructure choice. Workers have no server to provision. There is no traffic-shift step. The deploy is the deploy.

Pricing math: what speed costs

Speed is not free. But the cost structure is different from what Brad expects.

ONE charges by credit. A credit is one unit of substrate compute: one inference call, one routing decision, one KV read at the edge. Brad buys credits in bulk at the platform floor price and marks them up to clients.

The fast architecture does not cost more per request than a slow one. It costs less. Fewer round-trips to origin means fewer compute-seconds billed. Cached routing paths cost a KV read, not an LLM call. A KV read costs fractions of a cent. An LLM call costs hundreds of times more.

The math: an agency running 200 clients with 1,000 interactions per client per month. If 60% of routing decisions hit cached paths after week two of each client's deployment, the inference cost per interaction drops by approximately 30%. At scale, that is the difference between 70% gross margin and 90% gross margin on the credit resale.

Speed compounds in the P&L, not just the user experience.

Glossary

FCP, First Contentful Paint. The moment the browser paints the first pixel of real content. The user's first signal that something is happening.

LCP, Largest Contentful Paint. The moment the main content element is visible. Google uses this as the primary page-speed signal for search ranking.

INP, Interaction to Next Paint. How quickly the page responds to a click, tap, or keypress. The modern replacement for First Input Delay.

TTFB, Time to First Byte. The time from the browser sending a request to receiving the first byte of the response. The number that governs the start of every user interaction.

TTFT, Time to First Token. The AI equivalent of TTFB. The time from the user pressing send to the first character of the model's response appearing on screen.

TTFAPIC, Time to First API Call. From creating an account to making a working API call. The developer onboarding metric. One minute on ONE.

Lighthouse. Google's open-source tool for measuring and auditing web performance. Four scores: Performance, Accessibility, Best Practices, SEO. Measured on a simulated mid-range mobile device. The standard for web speed accountability.

HMR, Hot Module Replacement. The dev-server mechanism that updates only the changed module in the browser without a full page reload. Under 100ms on ONE.

KV, Key-Value store. Cloudflare's global edge cache. Sub-10ms reads from 330 edge locations. Where hot routing paths live once they harden into highways.

p50, 50th percentile. The median. Half of measurements are faster, half are slower. The number used for wallet onboarding (5 seconds) and TTFB (97ms).

p95, 95th percentile. 95% of measurements are at or below this number. The wallet p95 is under 9 seconds, inside Jakob Nielsen's 10-second abandonment threshold.

FAQ

Q: The 100/100/100/100 Lighthouse score, is that on mobile or desktop?

Mobile. The test uses a simulated mid-range Android device, 4x CPU throttle, 10Mbps bandwidth. Desktop scores are higher. We test mobile because that is where the users are and where performance problems show first.

Q: When you say “streaming starts at 97ms,” does that mean the full answer arrives in 97ms?

No. 97ms is the Time to First Token, the time before the first character appears. The model then streams at approximately 40–80 tokens per second, depending on the model and prompt length. A 200-word answer arrives in full within 3–5 seconds of sending the message. The 97ms is what eliminates the “dead time” before anything appears.

Q: Is the 5-second wallet available on Android?

Yes. The Secure Enclave equivalent on Android is the Titan security chip (Google Pixel) or TrustZone (most Android OEMs). The flow is the same: biometric prompt, in-hardware key generation, network registration. The p50 on Android is 6.2 seconds, 1.2 seconds slower than Apple Silicon, where the SE is faster. The p95 on Android is 11 seconds, which crosses Nielsen’s threshold on the least capable devices. We are working on it.

Q: Does the 107-second deploy apply to all four services simultaneously?

The four services deploy in a sequenced pipeline, not fully parallel. Gateway deploys first (13.7s) because the other services depend on it. Sync, NanoClaw, and Pages then deploy in parallel. The total elapsed time is 107 seconds. If the Gateway deploy were parallelisable, the total would be lower. It is not, because service B reads service A’s endpoint.

Q: The Akamai “7% per 100ms” stat, is that e-commerce specific?

Yes. Akamai’s 2017 study was retail-focused: online shopping sessions. The Deloitte 2020 study covers retail, travel, luxury, and lead generation. The latter two are the most relevant for Brad’s agency clients. The Deloitte figure is +1.11% session conversion per 100ms. Retail, travel, and luxury all showed similar patterns. Lead generation was not separately reported. The directional claim holds across categories.

Q: Can we run our own Lighthouse test on your /chat page?

Yes. The URL is one.ie/chat. Run Lighthouse in Chrome DevTools or via the Lighthouse CLI. The score you see is the score that ships. There is no performance mode we enable for the test.

Cross-references

The numbers on this page feed the value equation from the platform’s first page. Speed is the denominator: time delay and effort. Every second removed from wallet onboarding, deploy time, and first-token latency moves the value equation in Brad’s favour. See the cover page for the equation in full.

The 107-second deploy connects to development in page 14. Donal's ability to extend the platform depends on the same build-and-deploy speed that makes the platform fast for users. One architecture. One measurement.

The wallet onboarding connects to security in page 15. The 5-second figure is only possible because the Secure Enclave does the key generation. There is no server that knows the key. There is no database the key could leak from. The speed and the security are the same engineering decision.

The closing argument

Speed is not one number. It is a pattern across every surface a user touches.

The page that loads in 97ms. The response that starts in under 300ms. The build that reloads in under 100ms. The deploy that completes in 107 seconds. The wallet that activates in 5 seconds. The checkout that opens in 60 seconds.

Eight surfaces. Every one measured. Every one dated.

Brad's agency is not buying fast software as an amenity. Brad's agency is buying fast software as the mechanism that makes 90% gross margin defensible. When the chatbot responds faster than a phone, clients stop calling. When the wallet takes 5 seconds, customers complete the checkout. When the deploy takes 107 seconds, Donal ships the campaign change before the client notices it is late.

Speed compounds. The agency that responds fastest wins more mandates. The platform that deploys fastest ships more changes. The chatbot that answers fastest earns more trust. Each win makes the next one more likely. Slow compounds the other way.

The agencies that move now build the corpus. The corpus is the moat. The moat is what the next acquirer pays for. The moat is what makes the next decade defensible. Speed is how you get there first.

"We were told AI would replace people. What it's really doing is replacing the parts of work that were never worth doing in the first place. The question isn't what AI takes from us. It's what we finally get to do with the time it gives back."

— Anthony O'Connell, Founder of ONE

The parts of work that slow things down. The deploy queues. The slow page loads. The wallet flows that require seed phrases. The chatbots that think before they answer. Those are the parts that were never worth doing.

What the time gives back: Brad builds 50 clients in 6 weeks instead of a year. Donal ships a change in 107 seconds instead of an hour. A plumber's customer gets an answer in 97ms instead of three rings and a hold tone.

Whoever is fastest wins. The way to be fastest is to remove friction. The way to remove friction is power through simplicity.

Measure us. We'll measure back.
